

Volume 21, Number 2
Second Quarter 2021
ISSN 1531-636X

IEEE Circuits and Systems MAGAZINE

Special Issue FPGA Evolution

4 Architectures

30 Accelerators

48 Robotic Applications

75 Domain-Specific Deep Learning

97 High-Level Synthesis



IEEE CASS Seasonal School on Circuits and Systems for IoT

Hybrid mode

September 20-24, 2021, Bordeaux, France

<http://www.iot-bordeaux.org>

Crédit Yann Texier

General Co-chairs

Francois Rivet, Univ. Bordeaux, France
Guillaume Ferre, Univ. Bordeaux, France

Technical Program Committee Chair

Antoine Frappé, Junia, Lille, France
Ricardo Reis, UFRGS, Brazil

Local Arrangement Co-chairs

Marwane Rezzouki, Univ. Bordeaux, France
Romane Dumont, Univ. Bordeaux, France

Liaison chair

Patricia Desgreys, Telecom ParisTech, France

The Internet of Things domain touches many themes, some of which are in line with humanity priorities for the coming decades. Thus, the **3rd edition of the IEEE CASS Seasonal School on Circuits and Systems for IoT** will cover the whole scope of what is the IoT, from radiocommunications and electronics to data processing and Artificial Intelligence, with hybrid half days of technical, scientific and industrial presentations delivered by international experts of each domain and a in-person workshop to imagine and design your own connected object based on LoRa technology.

FREE FOR IEEE CAS STUDENTS



IEEE CASS
France
Chapter

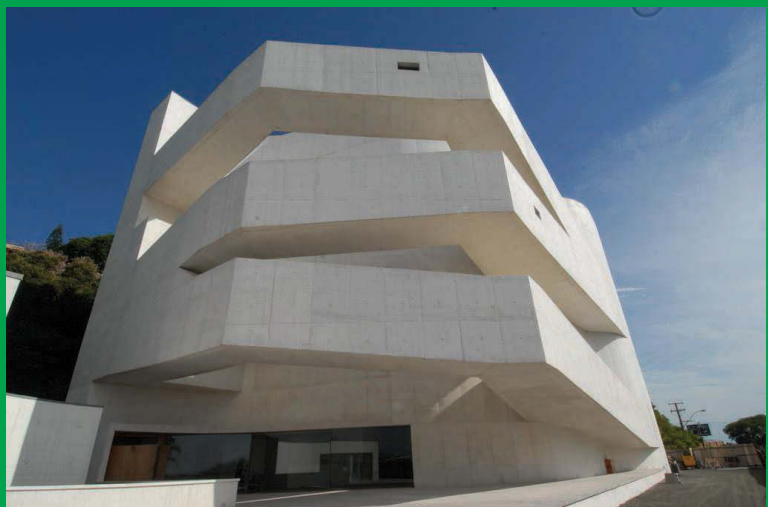


IEEE
France Section



IEEE CASS Seasonal School on Monolithic 3D Circuits

July 26-30, 2021
Porto Alegre, Brazil



www.ufrgs.br/cass/mono3d



IEEE Circuits and Systems MAGAZINE

Volume 21, Number 2 Second Quarter 2021

Features



©SHUTTERSTOCK.COM/EVGENIY JAMART



IEEE Circuits and Systems Magazine (ISSN 1531-636X) (ICDMEN) is published quarterly by the Institute of Electrical and Electronics Engineers, Inc. Headquarters: 3 Park Avenue, 17th Floor, New York, NY, 10016-5997 USA. Responsibility for the contents rests upon the authors and not upon the IEEE, the Society, or its members. IEEE Service Center (for orders, subscriptions, address changes): 445 Hoes Lane, Piscataway, NJ 08854 USA. Telephone: +1 732 981 0060, +1 800 678 4333. Individual copies: IEEE members US\$20.00 (first copy only), nonmembers US\$189 per copy; US\$7.00 per member per year (included in Society fee) for each member of the IEEE Circuits and Systems Society. Subscription rates available upon request. **Copyright and Reprint Permission:** Abstracting is permitted with credit to the source. Libraries are permitted to photocopy beyond the limits of the U.S. Copyright law for private use of patrons: 1) those post-1977 articles that carry a code at the bottom of the first page, provided the per-copy fee indicated in the code is paid through the Copyright Clearance Center, 222 Rosewood Drive, Danvers, MA 01923; and 2) pre-1978 articles without fee. For other copying, reprint, or republication permission, write to: Copyrights and Permissions Department, IEEE Service Center, 445 Hoes Lane, Piscataway, NJ 08854 USA. Copyright © 2021 by the Institute of Electrical and Electronics Engineers, Inc. All rights reserved. Periodicals postage paid at New York, NY, and at additional mailing offices. **Postmaster:** Send address changes to IEEE Circuits and Systems Magazine, IEEE Operations Center, 445 Hoes Lane, Piscataway, NJ, 08854 USA. Printed in U.S.A.

Digital Object Identifier 10.1109/MCAS.2021.3056246

SECOND QUARTER 2021

4 FPGA Architecture: Principles and Progression

Andrew Boutros and Vaughn Betz

Since their inception more than thirty years ago, field-programmable gate arrays (FPGAs) have been widely used to implement a myriad of applications from different domains. As a result of their low-level hardware reconfigurability, FPGAs have much faster design cycles and lower development costs compared to custom-designed chips. The design of an FPGA architecture involves many different design choices starting from the high-level architectural parameters down to the transistor-level implementation details, with the goal of making a highly programmable device while minimizing the area and performance cost of reconfigurability. As the needs of applications and the capabilities of process technology are constantly evolving, FPGA architecture must also adapt. In this article, we review the evolution of the different key components of modern commercial FPGA architectures and shed the light on their main design principles and implementation challenges.

30 FPGA Acceleration for Big Data Analytics: Challenges and Opportunities

Joost Hoozemans, Johan Peltenburg, Fabian Nonnenmacher,

Ákos Hadnagy, Zaid Al-Ars, and H. Peter Hofstee

The big data revolution has ushered an era with ever increasing volumes and complexity of data requiring ever faster computational analysis. During this very same era, CPU performance growth has been stagnating, pushing the industry to either scale their computation horizontally using multiple nodes in datacenters, or to scale vertically using heterogeneous components to reduce compute time. However, networking and storage continue to provide both higher throughput and lower latency, which allows for leveraging heterogeneous components, deployed in data centers around the world. Still, the integration of big data analytics frameworks with heterogeneous hardware components such as GPGPUs and FPGAs is challenging, because there is an increasing gap in the level of abstraction between analytics solutions developed with big data analytics frameworks, and accelerated kernels developed with heterogeneous components. In this article, we focus on FPGA accelerators that have seen wide-scale deployment in large cloud infrastructures. FPGAs allow the implementation of highly optimized hardware architectures, tailored exactly to an application, and unburdened by the overhead associated with traditional general-purpose computer architectures. FPGAs implementing dataflow-oriented architectures with high levels of (pipeline) parallelism can provide high application throughput, often providing high energy efficiency. Latency-sensitive applications can leverage FPGA accelerators by directly connecting to the physical layer of a network, and perform data transformations without going through the software stacks of the host system. While these advantages of FPGA accelerators hold promise, difficulties associated with programming and integration limit their use. This article explores the existing practices in big data analytics frameworks, discusses the aforementioned gap in development abstractions, and provides some perspectives on how to address these challenges in the future.

48 A Survey of FPGA-Based Robotic Computing

Zishen Wan,* Bo Yu,* Thomas Yang Li, Jie Tang, Yuhao Zhu,

Yu Wang, Arijit Raychowdhury, and Shaoshan Liu

Recent researches on robotics have shown significant improvement, spanning from algorithms, mechanics to hardware architectures. Robotics, including manipulators, legged robots, drones, and autonomous vehicles, are now widely applied in diverse scenarios. However, the high computation and data complexity of robotic algorithms pose great challenges to its applications. On the one hand, CPU platform is flexible to handle multiple robotic tasks. GPU platform has higher computational capacities and easy-to-use development frameworks, so they have been widely adopted in several applications. On the other hand, FPGA-based robotic accelerators are becoming increasingly competitive alternatives, especially in latency-critical and power-limited scenarios. With specialized designed hardware logic and algorithm kernels, FPGA-based accelerators can surpass CPU and GPU in performance and energy efficiency. In this paper, we give an overview of previous work on FPGA-based robotic accelerators covering different stages of the robotic system pipeline. An analysis of software and hardware optimization techniques and main technical issues is presented, along with some commercial and space applications, to serve as a guide for future work.

75 The Evolution of Domain-Specific Computing for Deep Learning

Stephen Neuendorffer, Alireza Khodamoradi, Kristof Denolf,

Abhishek Kumar Jain, and Samuel Bayliss

With the continued slowing of Moore's law and Dennard scaling, it has become more imperative that hardware designers make the best use of domain-specific information to improve designs. Gone are the days when we could rely primarily on silicon process technology improvements to

IEEE CIRCUITS AND SYSTEMS MAGAZINE

1

provide faster and more efficient computation. Instead, architectural improvements are necessary to provide improved performance, power reduction, and/or reduced cost. Nowhere is this more apparent than when looking at Deep Learning workloads. Cutting-edge techniques achieving state-of-the-art training accuracy demand ever-larger training data-sets and more-complex network topologies, which results in longer training times. At the same time, after training these networks, we expect them to be deployed widely. As a result, executing large networks efficiently becomes critical, whether that execution is done in a data center or in an embedded system. In this article, we look at trends in deep learning research that present new opportunities for domain-specific hardware architectures and explore how next-generation compilation tools might support them.

97 Synthesizing General-Purpose Code Into Dynamically Scheduled Circuits

Lana Josipović, Andrea Guerrieri, and Paolo Ienne

High-level synthesis (HLS) tools generate hardware designs from high-level programming languages and should liberate designers from the details of hardware description languages like VHDL and Verilog. HLS tools typically build datapaths that are controlled using a centralized controller, which relies on a compile-time schedule to determine the clock cycle when each operation executes. Such an approach results in high-throughput pipelined designs only in cases where memory accesses are provably independent and critical control decisions are determinable during code compilation. Unfortunately, when this is not the case, current tools must make pessimistic assumptions, yielding inferior schedules and lower performance. Recent advances in HLS have explored methods to overcome the conservatism in static scheduling and to remove the inability of HLS tools to handle dynamic events. Dataflow circuits play a significant role in this context: they are built out of units that communicate using point-to-point pairs of handshake control signals and this distributed control mechanism effectively implements a dynamic schedule, adapted at runtime to particular memory and control outcomes. Dataflow circuits can exploit the same optimization opportunities as standard HLS circuits (i.e., pipelining and resource sharing), but also introduce to HLS features similar to those of modern superscalar processors (i.e., out-of-order memory accesses and speculative execution), which are key for HLS to be successful in new contexts and broader application domains.

Departments

3 From the Guest Editors

120 CAS Society News

128 Correction

Scope: The magazine covers the subject areas represented by the Society's transactions, including: analog, passive, switch capacitor, and digital filters; electronic circuits, networks, graph theory, and RF communication circuits; system theory; discrete, IC, and VLSI circuit design; multidimensional circuits and systems; large-scale systems and power networks; nonlinear circuits and systems, wavelets, filter banks, and applications; neural networks; and signal processing. Content also covers the areas represented by the Society technical committees: analog signal processing, cellular neural networks and array computing, circuits and systems for communications, computer-aided network design, digital signal processing, multimedia systems and applications, neural systems and applications, nonlinear circuits and systems, power systems and power electronics and circuits, sensors and micromachining, visual signal processing and communication, and VLSI systems and applications. Lastly, the magazine covers the interests represented by the widespread conference activity of the IEEE Circuits and Systems Society. In addition to the technical articles, the magazine also covers Society administrative activities, as for instance the meetings of the Board of Governors, Society People, as for instance the stories of award winners-fellows, medalists, and so forth, and Places reached by the Society, including readable reports from the Society's conferences around the world.



Editor-in-Chief

Yiran Chen, *Duke University, USA*,
Email: yiran.chen@duke.edu

Deputy Editor-in-Chief

Leibo Liu, *Tsinghua University, China*

Associate Editors

Mohab Hussein Anis, *American University, Egypt*
Marco Carli, *Università degli Studi 'Roma TRE', Italy*
Fan Chen, *Indiana University Bloomington, USA*
Anthony C Davies, *King's College London, UK*
Gordana Jovanovic Dolecek, *Institute INAOE, Mexico*
José E. Franca, *University of Lisbon, Portugal*
Bah-Hwee Gwee, *Nanyang Technological University, Singapore*
Liviu Goras, *"Gheorghe Asachi" Technical University of Iasi, Romania*
Jie Han, *University of Alberta, Canada*
Chih-Cheng Hsieh, *National Tsing Hua University, Taiwan*
André Kaup, *Friedrich-Alexander University Erlangen-Nürnberg, Germany*
Chris Kim, *University of Minnesota, USA*
Alexander S. Korotkov, *Peter the Great St. Petersburg Polytechnic University, Russia*
Zhan Ma, *Nanjing University, China*
Mahesh Mehendale, *Texas Instruments, USA*
Nele Mentens, *Katholieke Universiteit Leuven, Belgium*
Marvin Onabajo, *Northeastern University, USA*
Jongsun Park, *Korea University, Korea*
Qinru Qiu, *Syracuse University, USA*
Ricardo Reis, *Federal University of Rio Grande do Sul, Brazil*
Branimir Reljin, *University of Belgrade, Serbia*
Salil Kumar Sanyal, *Jadavpur University, India*
M. N. S. Swamy, *Concordia University, Canada*
Chao Wang, *Huazhong University of Science and Technology, China*
Wujie Wen, *Lehigh University, USA*
Shigeru Yamashita, *Ritsumeikan University, Japan*
Xuan Zhang, *Washington University in St. Louis, USA*

IEEE Periodicals Editorial & Production Services

Managing Editor — Mark Gallaher
Senior Managing Editor — Geraldine Krolin-Taylor
Senior Art Director — Janet Dudar
Associate Art Director — Gail A. Schnitzer
Production Coordinator — Theresa L. Smith
Advertising Production Mgr. — Felicia Spagnoli
Dir. Production Services — Peter M. Tuohy
Dir. Editorial Services — Kevin Lisankie
Senior Director, Publishing Operations — Dawn M. Melley
Director, Business Development – Media & Advertising — Mark David

Magazine Deadlines

Final materials for *IEEE Circuits and Systems Magazine* must be received by the Editor on the following dates:

Issue	Due Date
First Quarter	December 1
Second Quarter	February 1
Third Quarter	May 1
Fourth Quarter	August 1

IEEE prohibits discrimination, harassment, and bullying. For more information, visit <http://www.ieee.org/web/aboutus/whatis/policies/p9-26.html>.

Digital Object Identifier 10.1109/MCAS.2021.3056247

From the Guest Editors

Zhiru Zhang
Cornell University

Hongbo Rong
Intel Labs

Yu Wang
Tsinghua University



Introduction of Special Issue on FPGA-Based Computing

Over the past three decades, field-programmable gate arrays (FPGAs) have evolved from small chips with a few thousand logic blocks to billion-transistor system-on-chips. Modern FPGAs provide massive fine-grained parallelism, high energy efficiency, and more predictable performance than CPUs and GPUs. More importantly, they can be reconfigured to implement a special-purpose architecture that is specifically customized based on the key characteristics of a target application (or domain). For these reasons, recent years have seen a rapidly increasing use of FPGAs for computing, where a plethora of new architectures, compilers, and FPGA-accelerated applications are being demonstrated and actively developed. This special issue includes five survey papers from renowned experts in the field of FPGAs. The goal is to summarize some of the recent advances on FPGA-based computing, to elucidate the architecture- and application-level trends, and to promote new tools to enable productive FPGA programming.

The first survey walks us through the evolution of the FPGA architectures accompanying ever-changing application requirements and recent trends in technology scaling. Design and analysis of an FPGA architecture is broken down into design decisions at various levels, from transistors to embedded blocks. Major challenges with the existing solutions are analyzed,

raising questions to address for future progress of FPGA architectures.

The second paper presents new views on FPGA accelerators for near-future big-data analytics, their challenges, solutions, and opportunities. The paper also describes a prototype system for seamless integration of FPGAs in cloud infrastructure for big data analytics.

The third paper presents a detailed survey of the state-of-the-art FPGA work in different robotic applications, including sensing, perception, localization, and planning. It also introduces two example systems: one for commercial autonomous vehicles and the other for space robots.

The fourth survey covers the recent trends in domain-specific computing for deep learning, from modern software optimization, prevailing hardware backends, and intermediate representations. It offers commentary on the general tradeoffs and challenges during the design process and application deployment, which leads to further discussion on the applicability of FPGA platforms in this domain and an in-depth introduction to the Xilinx Versal architecture.

The last paper focuses on the compilation for FPGA accelerators. It details a drastically different form of high-level synthesis, which generates dynamically scheduled dataflow circuits out of software programs. Such dataflow circuits achieve out-of-order memory accesses and speculation, similar to some of the features in high-performance microprocessors.

Digital Object Identifier 10.1109/MCAS.2021.3071606
Date of current version: 24 May 2021

FPGA Architecture: Principles and Progression

Andrew Boutros and Vaughn Betz

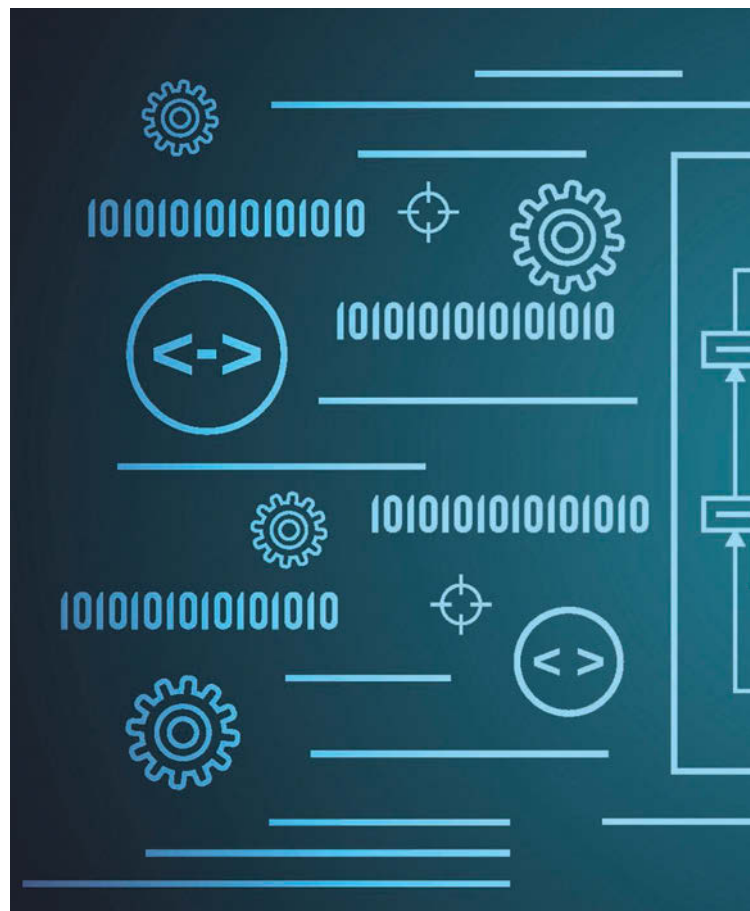
Abstract

Since their inception more than thirty years ago, field-programmable gate arrays (FPGAs) have been widely used to implement a myriad of applications from different domains. As a result of their low-level hardware reconfigurability, FPGAs have much faster design cycles and lower development costs compared to custom-designed chips. The design of an FPGA architecture involves many different design choices starting from the high-level architectural parameters down to the transistor-level implementation details, with the goal of making a highly programmable device while minimizing the area and performance cost of reconfigurability. As the needs of applications and the capabilities of process technology are constantly evolving, FPGA architecture must also adapt. In this article, we review the evolution of the different key components of modern commercial FPGA architectures and shed the light on their main design principles and implementation challenges.

I. Introduction

Field-programmable gate arrays (FPGAs) are reconfigurable computer chips that can be programmed to implement any digital hardware circuit. As depicted in Fig. 1, FPGAs consist of an array of different types of programmable blocks (logic, IO, and others) that can be flexibly interconnected using pre-fabricated routing tracks with programmable switches between them. The functionality of all the FPGA blocks and the configuration of the routing switches are controlled using millions of static random access memory (SRAM) cells that are programmed (i.e. written) at runtime to realize a specific function. The user describes the desired functionality in a hardware description language (HDL) such as Verilog or VHDL, or possibly uses high-level synthesis to translate C or OpenCL to HDL. The HDL design is then compiled using a complex computer-aided design (CAD) flow into the *bitstream* file used to program all the FPGA's configuration SRAM cells.

Digital Object Identifier 10.1109/MCAS.2021.3071607
Date of current version: 24 May 2021

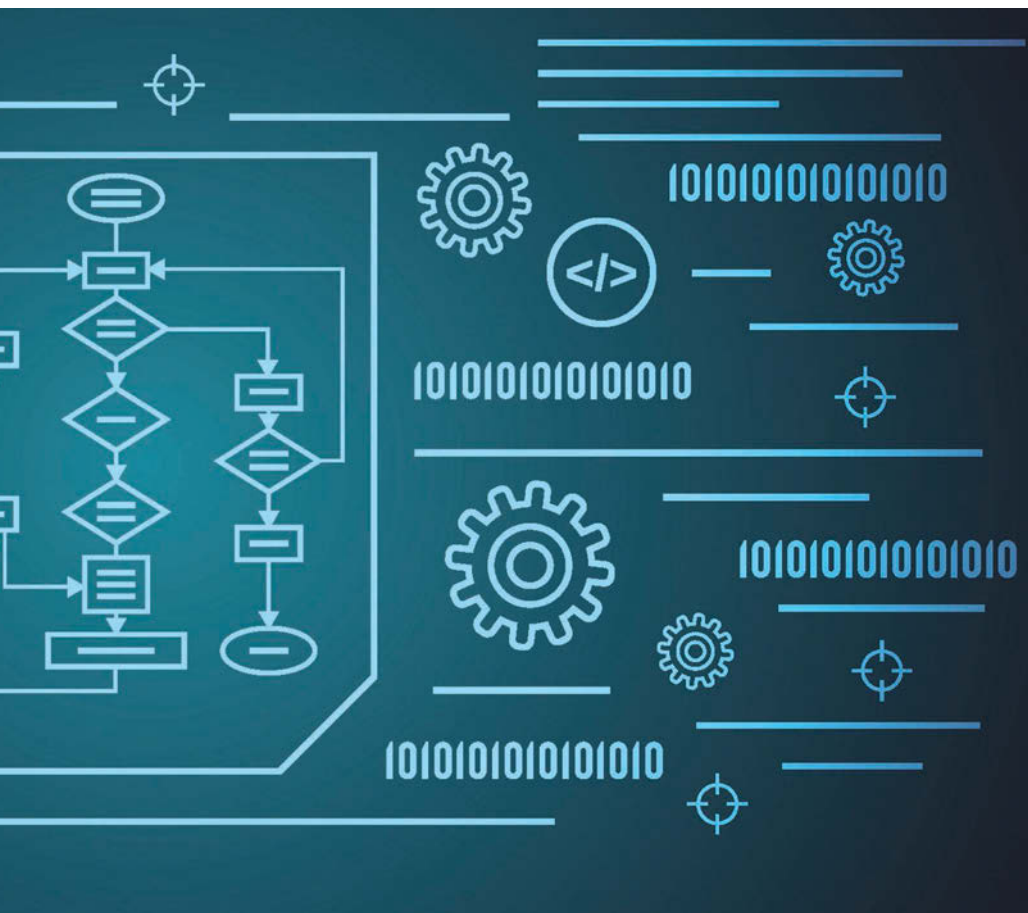


Compared to building a custom application-specific integrated circuit (ASIC), FPGAs have a much lower non-recurring engineering cost and shorter time-to-market. A pre-fabricated off-the-shelf FPGA can be used to implement a complete system in a matter of weeks, skipping the physical design, layout, fabrication, and verification stages that a custom ASIC would normally go through. They also allow continuous hardware upgrades to support new features or fix bugs by simply loading a new bitstream after deployment in-field, thus the name *field-programmable*.

This makes FPGAs a compelling solution for medium and small volume designs, especially with the fast-paced product cycles in today's markets. The bit-level reconfigurability of FPGAs enables implementation of the exact hardware needed for each application (e.g. datapath bit-width, pipeline stages, number of parallel compute units, memory subsystem, etc.) instead of the fixed one-size-fits-all architecture of general-purpose processors (CPUs) or graphics processing units (GPUs). Consequently, they can achieve higher efficiency than CPUs or GPUs by implementing instruction-free streaming hardware [1] or a processor *overlay* with an application-customized pipeline and instruction set [2].

packet processing [9], and machine learning [10] workloads, among others. However, the flexibility of FPGA hardware comes with an efficiency cost vs. ASICs. Kuon and Rose [11] show that circuits using only the FPGA's programmable logic blocks average 35× larger and 4× slower than corresponding ASIC implementations. A more recent study [12] shows that for full-featured designs which heavily utilize the other FPGA blocks (e.g. RAMs and DSPs), this area gap is reduced but is still 9×. FPGA architects seek to reduce this efficiency gap as much as possible while maintaining the programmability that makes FPGAs useful across a wide range of applications.

In this article, we introduce key principles of FPGA architecture, and highlight the progression of these devices over the past 30 years. Fig. 1 shows how FPGAs evolved from simple arrays of programmable logic and IO blocks to complex heterogeneous multi-die systems with embedded block RAMs, digital signal processing (DSP) blocks, processor subsystems, diverse high-performance external interfaces, system-level interconnect, and more. First, we give a brief overview of the CAD flows and methodology used to evaluate new FPGA architecture ideas. We then detail the architecture challenges and design principles for each of the key components of an FPGA. We highlight key innovations in the design and implementation of each of these components over the past three decades along with areas of ongoing research.



©SHUTTERSTOCK.COM/ULVUR

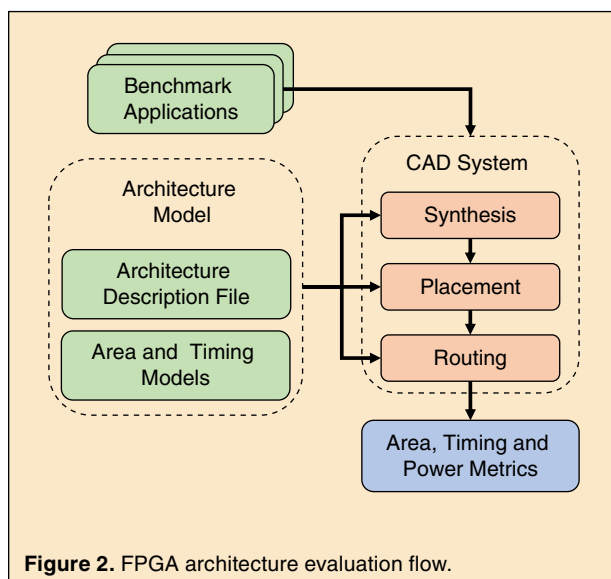
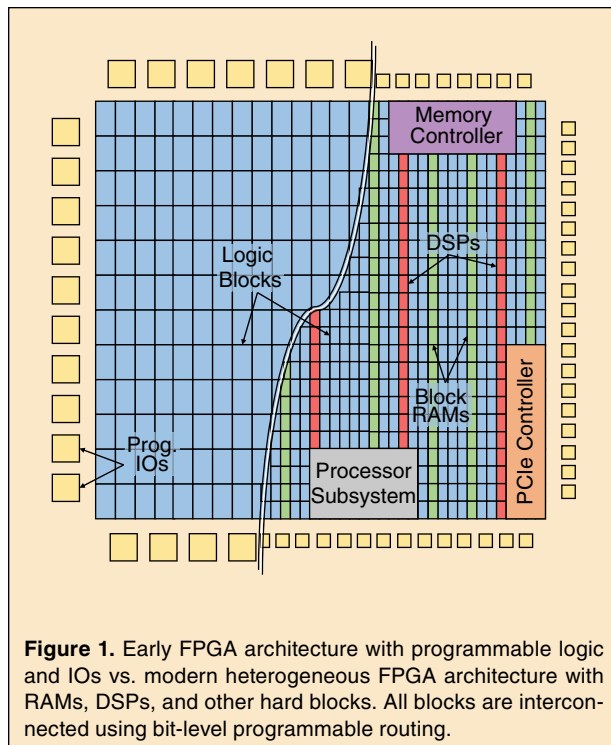
These advantages motivated the adoption of FPGAs in many application domains including wireless communications, embedded signal processing, networking, ASIC prototyping, high-frequency trading, and many more [3]–[7]. They have also been recently deployed on a large scale in datacenters to accelerate search engines [8],

II. FPGA Architecture Evaluation

As shown in Fig. 2, the FPGA architecture evaluation flow consists of three main components: a suite of benchmark applications, an architecture model, and a CAD system. Unlike an ASIC built for a specific functionality, an FPGA is a general-purpose platform designed for many use

Andrew Boutros and Vaughn Betz are with the Department of Electrical and Computer Engineering, University of Toronto and the Vector Institute for Artificial Intelligence. (email: andrew.boutros@mail.utoronto.ca, vaughn@eecg.utoronto.ca).

cases, some of which may not even exist when the FPGA is architected. Therefore, an FPGA architecture is evaluated based on its efficiency when implementing a wide variety of benchmark designs that are representative of the key FPGA markets and application domains. Typically, each FPGA vendor has a carefully selected set of benchmark designs collected from proprietary system implementations and various customer applications. There are also several open-source benchmark suites such as the classic MCNC20 [13], the VTR [14], and the Titan23 [15]



suites which are commonly used in academic FPGA architecture and CAD research. While early academic FPGA research used the MCNC suite of designs, these circuits are now too small (thousands of logic primitives) and simple (only IOs and logic) to represent modern FPGA use cases. The VTR and particularly the Titan suite are larger and more complex, making them more representative, but as FPGA capacity and application complexity continues to grow new benchmark suites are regularly needed.

The second part of the evaluation flow is the FPGA architecture model. The design of an FPGA involves many different decisions from architecture-level organization (e.g. number and type of blocks, distribution of wire segment lengths, size of logic clusters and logic elements) down to transistor-level circuit implementation (e.g. programmable switch type, routing buffer transistor sizing, register implementation). It also involves different implementation styles; the logic blocks and programmable routing are designed and laid out as full-custom circuits, while most hardened blocks (e.g. DSPs) mix standard-cell and full-custom design for the block core and peripherals, respectively. Some blocks (RAM, IO) even include significant analog circuitry. All these different components need to be carefully modeled to evaluate the FPGA architecture in its entirety. This is typically captured using an architecture description file that specifies the organization and types of the different FPGA blocks and the routing architecture, in addition to area, timing and power models obtained from circuit-level implementations for each of these components.

Finally, a re-targetable CAD system such as VTR [14] is used to map the selected benchmark applications on the specified FPGA architecture. Such a CAD system consists of a sequence of complex optimization algorithms that synthesizes a benchmark written in an HDL into a circuit netlist, maps it to the different FPGA blocks, places the mapped blocks at specific locations on the FPGA, and routes the connections between them using the specified programmable routing architecture. The implementation produced by the CAD system is then used to evaluate several key metrics. Total area is the sum of the areas of the FPGA blocks used by the application, along with the programmable routing included with them. A timing analyzer finds the critical path(s) through the blocks and routing to determine the maximum frequencies of the application's clock(s). Power consumption is estimated based on resources used and signal toggle rates. FPGAs are never designed for only one application, so these metrics are averaged across all the benchmarks. Finally, the overall evaluation blends these average area, delay, and power metrics appropriately depending on the architecture goal (e.g. high performance or low power). Other metrics such as CAD tool runtime and whether or not the

CAD tools fail to route some benchmarks on an architecture are also often considered.

As an example, a key set of questions in FPGA architecture is: What functionality should be *hardened* (i.e. implemented as a new ASIC-style block) in the FPGA architecture? How flexible should this block be? How much of the FPGA die area should be dedicated to it? Ideally, an FPGA architect would like the hardened functionality to be usable by as many applications as possible at the least possible silicon cost. An application that can make use of the hard block will benefit by being smaller, faster and more power-efficient than when implemented solely in the programmable fabric. This motivates having more programmability in the hard block to capture more use cases; however, higher flexibility generally comes at the cost of larger area and reduced efficiency of the hard block. On the other hand, if a hard block is not usable by an application circuit, its silicon area is wasted; the FPGA user would rather have more of the usable general-purpose logic blocks in the area of the unused hard block. The impact of this new hard block on the programmable routing must also be considered—does it need more interconnect or lead to slow routing paths to and from the block? To evaluate whether a specific functionality should be hardened or not, both the cost and gain of hardening it have to be quantified empirically using the flow described in this section. FPGA architects may try many ideas before landing on the right combination of design choices that adds just the right amount of programmability in the right spots to make this new hard block a net win.

In the following section, we detail many different components of FPGAs and key architecture questions for each. While we describe the key results without detailing the experimental methodology used to find them, in general they came from a holistic architecture evaluation flow similar to that in Fig. 2.

III. FPGA Architecture Evolution

A. Programmable Logic

The earliest reconfigurable computing devices were *programmable array logic* (PAL) architectures. PALs consisted of an array of and gates feeding another array of or gates, as shown in Fig. 3, and could implement any Boolean logic expression

as a two-level sum-of-products function. PALs achieve configurability through programmable switches that select the inputs to each of the and/or gates to implement different Boolean expressions. The design tools for PALs were very simple since the delay through the device is constant no matter what logic function is implemented. However, PALs do not scale well; as device logic capacity increased, the wires forming the and/or arrays became increasingly longer and slower and the number of programmable switches required grew quadratically.

Subsequently, *complex programmable logic devices* (CPLDs) kept the and/or arrays as the basic logic elements, but attempted to solve the scalability challenge by integrating multiple PALs on the same die with a crossbar interconnect between them at the cost of more complicated design tools. Shortly after, Xilinx pioneered the first lookup-table-based (LUT-based) FPGA in 1984, which consisted of an array of SRAM-based LUTs with programmable interconnect between them. This style of reconfigurable devices was shown to scale very well, with LUTs achieving much higher area efficiency compared to the and/or logic in PALs and CPLDs. Consequently, LUT-based architectures became increasingly dominant and today LUTs form the fundamental logic

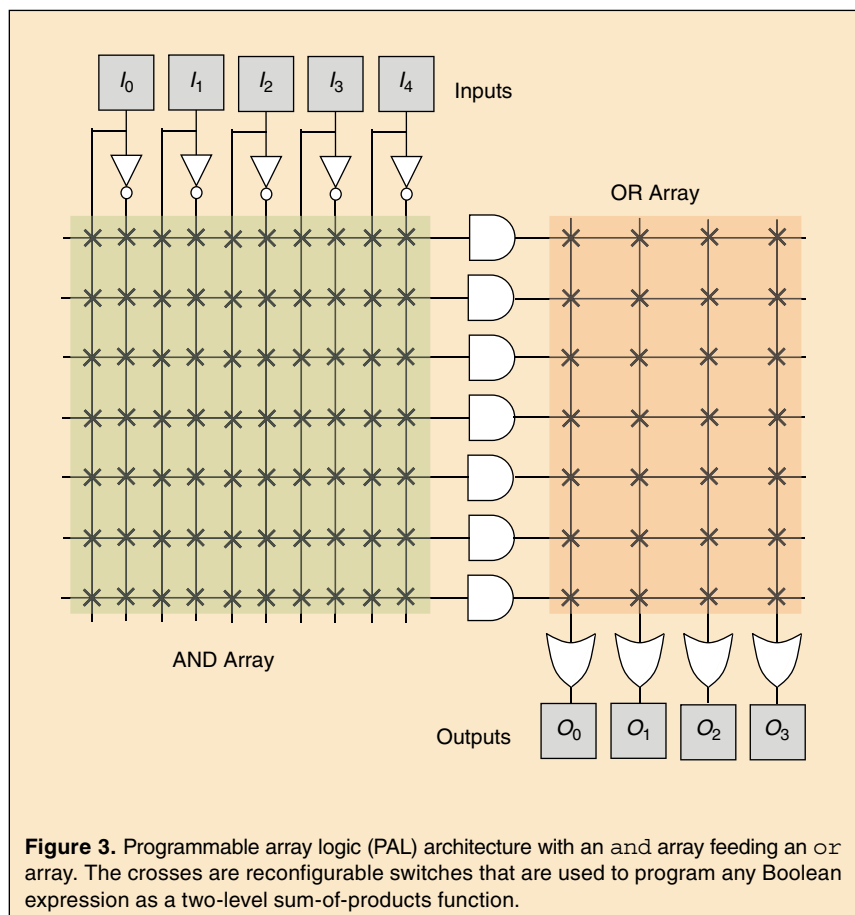


Figure 3. Programmable array logic (PAL) architecture with an and array feeding an or array. The crosses are reconfigurable switches that are used to program any Boolean expression as a two-level sum-of-products function.

element in all commercial FPGAs. Several research attempts [16]–[18] investigated replacing LUTs with a different form of configurable gates: a full binary tree of and gates with programmable output/input inversion known as an *and-inverter cone* (AIC). However, when thoroughly evaluated in [19], AIC-based FPGA architectures had significantly larger area than LUT-based ones,

with delay gains only on small benchmarks that have short critical paths.

A K -LUT can implement any K -input Boolean function by storing its truth table in configuration SRAM cells. K input signals are used as multiplexer select lines to choose an output from the 2^K values of the truth table. Fig. 4(a) shows the transistor-level circuit implementation

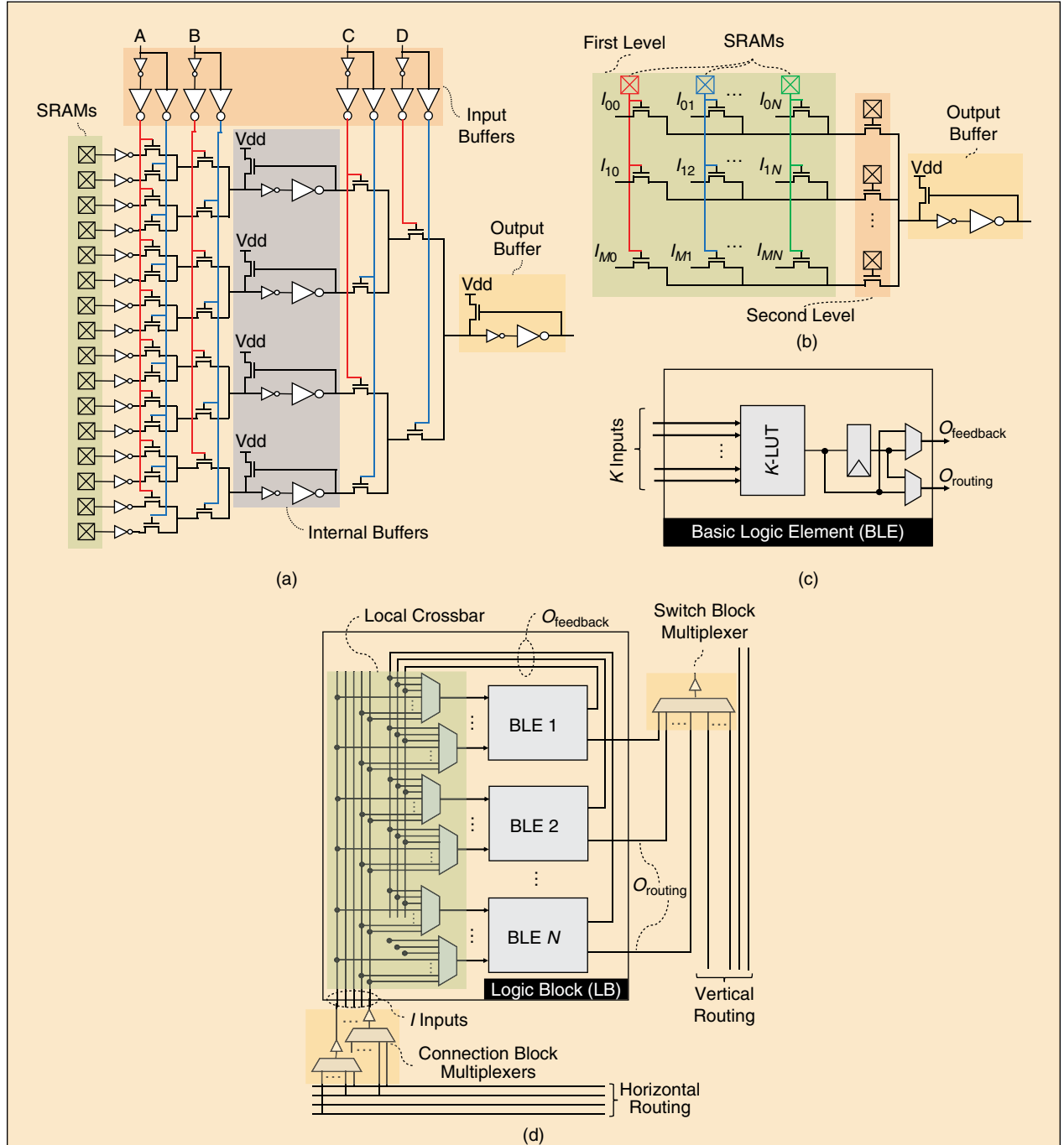


Figure 4. (a) Transistor-level implementation of a 4-LUT with internal buffers between the second and third LUT stages, (b) Two-level multiplexer circuitry, (c) Basic logic element (BLE), and (d) Logic block (LB) internal architecture.

of a 4-LUT using pass-transistor logic. In addition to the output buffer, an internal buffering stage (shown between the second and third stages of the LUT in Fig. 4(a)) is typically implemented to mitigate the quadratic increase in delay when passing through a chain of pass-transistors. The sizing of the LUT's pass-transistors and the internal/output buffers is carefully tuned to achieve the best area-delay product. Classic FPGA literature [20] defines the *basic logic element* (BLE) as a K -LUT coupled with an output register and bypassing 2:1 multiplexers as shown in Fig. 4(c). Thus, a BLE can either implement just a flip-flop (FF) or a K -LUT with registered or unregistered output. As illustrated in Fig. 4(d), BLEs are typically clustered in *logic blocks* (LBs), such that an LB contains N BLEs along with local interconnect. The local interconnect in the logic block consists of multiplexers between signal sources (BLE outputs and logic block inputs) and destinations (BLE inputs). These multiplexers are often arranged to form a local full [21] or partial [22] crossbar. At the circuit level, these multiplexers are usually built as two levels of pass transistors, followed by a two-stage buffer as shown in Fig. 4(b); this is the most efficient circuit design for FPGA multiplexers in most cases [23]. Fig. 4(d) also shows the switch and connection block multiplexers forming the programmable routing that allows logic blocks to connect to each other; this routing is discussed in detail in Section III-B.

Over the years, the size of both LUTs (K) and LBs (N) have gradually increased as device logic capacity has grown. As K increases, more functionality can be packed into a single LUT, reducing not only the number of LUTs needed but also the number of logic levels on the critical path, which increases performance. In addition, the demand for inter-LB routing decreases as more connections are captured into the fast local interconnect by increasing N . On the other hand, the area of the LUT increases exponentially with K (due to the 2^K SRAM cells) and its speed degrades linearly (as the multiplexer constitutes a chain of K pass transistors with periodic buffering). If the LB local interconnect is implemented as a crossbar, its size increases quadratically and its speed degrades linearly with the number of BLEs in the LB, N . Ahmed and Rose [24] empirically evaluated these trade-offs and concluded that LUTs of size 4–6 and LBs of size 3–10 BLEs offer the best area-delay product for an FPGA architecture, with 4-LUTs leading to a better area but 6-LUTs yielding a higher speed. Historically, the first LUT-based FPGA from Xilinx, the XC2000 series in 1984, had an LB that contained only two 3-LUTs (i.e. $N = 2, K = 3$). LB size gradually increased over time and by 1999, Xilinx's Virtex family included four 4-LUTs and Altera's Apex 20K family included ten 4-LUTs in each LB.

The next major change in architecture came in 2003 from Altera, with the introduction of *fracturable* LUTs in their Stratix II architecture [25]. Ahmed and Rose in [24] showed that an LB with ten 6-LUTs achieved 14% higher performance than a LB with ten 4-LUTs, but at a 17% higher area. Fracturable LUTs seek to combine the best of both worlds, achieving the performance of a larger LUT with the area-efficiency of smaller LUTs. A major factor in the area increase with traditional 6-LUTs is under-utilization: Lewis et al. found that 64% of the LUTs in benchmark applications used fewer than 6 inputs, wasting some of a 6-LUT's functionality [26]. A fracturable $\{K, M\}$ -LUT can be configured as a single LUT of size K or can be fractured into two LUTs of size up to $K - 1$ that collectively use no more than $K + M$ distinct inputs. Fig. 5(a) shows that a 6-LUT is internally composed of two 5-LUTs plus a 2:1 multiplexer. Consequently, almost no circuitry (only the red added output) is necessary to allow a 6-LUT to instead operate as two 5-LUTs that share the same inputs. However, requiring the two 5-LUTs to share all their

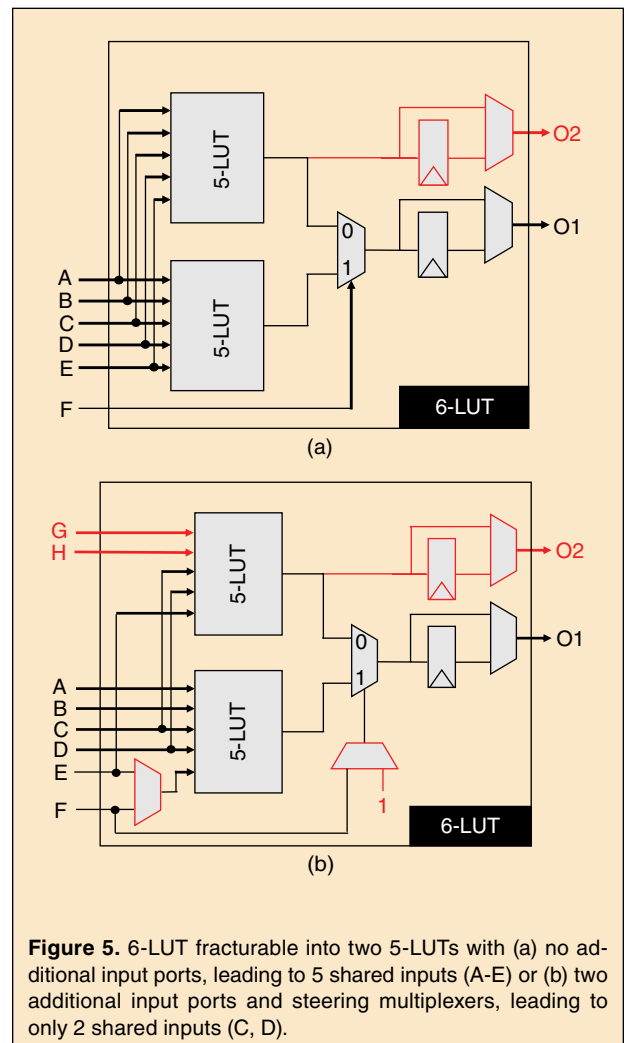


Figure 5. 6-LUT fracturable into two 5-LUTs with (a) no additional input ports, leading to 5 shared inputs (A-E) or (b) two additional input ports and steering multiplexers, leading to only 2 shared inputs (C, D).

inputs will limit how often both can be simultaneously used. Adding extra routing ports as shown in Fig. 5(b) increases the area of the fracturable 6-LUT, but makes it easier to find two logic functions that can be packed together into it. The *adaptive logic module* (ALM) in the Stratix II architecture implemented a {6, 2}-LUT that had 8 input and 2 output ports. Thus, an ALM can implement a 6-LUT or two 5-LUTs sharing 2 inputs (and therefore a total of 8 distinct inputs). Pairs of smaller LUTs could also be implemented without any shared inputs, such as two 4-LUTs or one 5-LUT and one 3-LUT. With a fracturable 6-LUT, larger logic functions are implemented in 6-LUTs reducing the logic levels on the critical path and achieving performance improvement. On the other hand, smaller logic functions can be packed together (each using only half an ALM), improving area-efficiency. The LB in Stratix II not only increased the performance by 15%, but also reduced the logic and routing area by 2.6% compared to a baseline 4-LUT-based LB [26].

Xilinx later adopted a related fracturable LUT approach in their Virtex-5 architecture. Like Stratix II, a Virtex-5 6-LUT can be decomposed into two 5-LUTs. However, Xilinx chose to minimize the extra circuitry added for fracturability as shown in Fig. 5(a)—no extra input routing ports or steering multiplexers are added. This results in a lower area per fracturable LUT, but makes it more difficult to pack two smaller LUTs together as they must use no more than 5 distinct inputs [27]. While subsequent architectures from both Altera/Intel and Xilinx have also been based on fracturable 6-LUTs, a recent Microsemi study [28] revisited the 4-LUT vs. 6-LUT efficiency trade-off for newer process technologies, CAD tools and designs than those used in [24]. It shows that a LUT structure with two tightly coupled 4-LUTs, one feeding

the other, can achieve performance close to plain 6-LUTs along with the area and power advantages of 4-LUTs. In terms of LB size, FPGA architectures from Altera/Intel and Xilinx converged on the use of relatively large LBs with ten and eight BLEs respectively, for several generations. However, the recently announced Versal architecture from Xilinx further increases the number of BLEs per LB to thirty two [29]. The reasons for this large increase are two-fold. First, inter-LB wire delay is scaling poorly with process shrinks, so capturing more connections within an LB's local routing is increasingly beneficial. Second, ever-larger FPGA designs tend to increase CAD tool runtime, but larger LBs can help mitigate this trend by simplifying placement and inter-LB routing.

Another important architecture choice is the number of FFs per BLE. Early FPGAs coupled a (non-fracturable) LUT with a single FF as shown in Fig. 4(c). When they moved to fracturable LUTs, both Altera/Intel and Xilinx architectures added a second FF to each BLE so that both outputs of the fractured LUT could be registered as shown in Fig. 5(a) and 5(b). In the Stratix V architecture, the number of FFs was further increased from two to four per BLE in order to accommodate increased demand for FFs as designs became more deeply pipelined to achieve higher performance [30]. Low-cost multiplexing circuitry allows sharing the existing inputs between the LUTs and FFs to avoid adding more costly routing ports. Stratix V also implements FFs as pulse latches instead of edge-triggered FFs. As shown in Fig. 6(b), this removes one of the two latches that would be present in a master-slave FF (Fig. 6(a)), reducing the register delay and area. A pulse latch acts as a cheaper FF with worse hold time as it latches the data input during a very short pulse instead of a clock edge as in conventional FFs. If a pulse generator was built for each FF, the overall area per FF would increase rather than decrease. Instead, Stratix V contains only two configurable pulse generators per LB; each of the 40 pulse latches in an LB selects which generator provides its pulse input. The FPGA CAD tools can also program the pulse width in these generators, allowing a limited amount of time borrowing between source and destination registers. Longer pulses further degrade hold time, but generally any hold violations can be solved by the FPGA routing algorithm using longer wiring paths to delay signals. Xilinx also uses pulse latches as its FFs in its Ultrascale+ architecture [31].

Arithmetic operations (add and subtract) are very common in FPGA designs: Murray et al. found that 22% of the logic elements in a suite of FPGA designs were implementing arithmetic [32]. While these operations can be implemented with LUTs, each bit of arithmetic in a ripple carry adder requires two LUTs (one for the sum output and one for the carry). This leads to both high

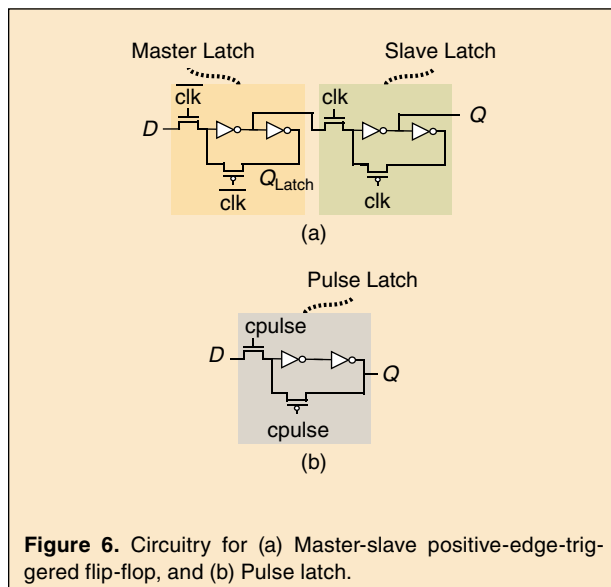


Figure 6. Circuitry for (a) Master-slave positive-edge-triggered flip-flop, and (b) Pulse latch.

logic utilization and a slow critical path due to connecting many LUTs in series to compute carries for multi-bit additions. Consequently, all modern FPGA architectures include hardened arithmetic circuitry in their logic blocks. There are many variants, but all have several common points. First, to avoid adding expensive routing ports, the arithmetic circuits re-use the LUT routing ports or are fed by the LUT outputs. Second, the carry bits are propagated on special, dedicated interconnect with little or no programmability so that the crucial carry path is fast. The lowest cost arithmetic circuitry hardens ripple carry structures and achieves a large speed gain over LUTs ($3.4\times$ for a 32-bit adder in [32]). Hardening more sophisticated structures like carry skip adders further improves speed (an additional 20% speed-up at 32-bits in [33]). The latest Versal architecture from Xilinx hardens the carry logic for 8-bit carry look-ahead adders (i.e. the addition can only start on every eighth BLE), while the sum, propagate and generate logic is all implemented in the fracturable 6-LUTs feeding the carry logic as shown in Fig. 7(a) [29]. This organization allows implementing 1-bit of arithmetic per logic element. On the other hand, the latest Intel Agilex architecture can implement 2-bits of arithmetic per logic element, with dedicated interconnect for the carry between logic elements as shown in Fig. 7(b). It achieves that by hardening 2-bit carry-skip adders that are fed by the four 4-LUTs contained within a 6-LUT [34]. The study by Murray et al. [32] shows that the combination of fracturable LUTs and 2 bits of arithmetic (similar to that adopted in Altera/Intel FPGAs) is particularly efficient compared to architectures with non-fracturable LUTs or 1 bit of arithmetic per logic element. It also concludes that having dedicated arithmetic circuits (i.e. hardening adders and carry chains) inside the FPGA logic elements increases average performance by 75% and 15% for arithmetic microbenchmarks and general benchmark circuits, respectively.

Recently, deep learning (DL) has become a key workload in many end-user applications, with its core operation being multiply-accumulate (MAC). Generally, MACs can be implemented in DSP blocks as will be described in Section III-E; however low-precision MACs with 8-bit or narrower operands (which are becoming increasingly popular in DL workloads) can also be implemented efficiently in the programmable logic [9]. LUTs are used to generate the partial products of a multiplier array followed by an adder tree to reduce the partial products and perform the accumulation. Consequently, multiple recent studies [35]–[37] have investigated increasing the density of hardened adders in the FPGA's logic fabric to enhance its performance when implementing arithmetic-heavy applications such as DL acceleration. The work in [36] and [37] proposed multiple different logic block ar-

chitectures that incorporate 4 bits of arithmetic per logic element arranged in one or two carry chains with different configurations, instead of just 2 bits of arithmetic in an Intel Stratix-like ALM. These proposals do not require increasing the number of the (relatively expensive) routing ports in the logic clusters when implementing multiplications due to the high degree of input sharing in a multiplier array (i.e. for an N -bit multiplier, only $2N$ inputs are needed to generate N^2 partial products). The most promising of these proposals increases the density of MAC operations by $1.7\times$ while simultaneously improving their speed. It also reduces the required logic and routing area by 8% for general benchmarks, highlighting that more arithmetic density is beneficial for applications beyond DL.

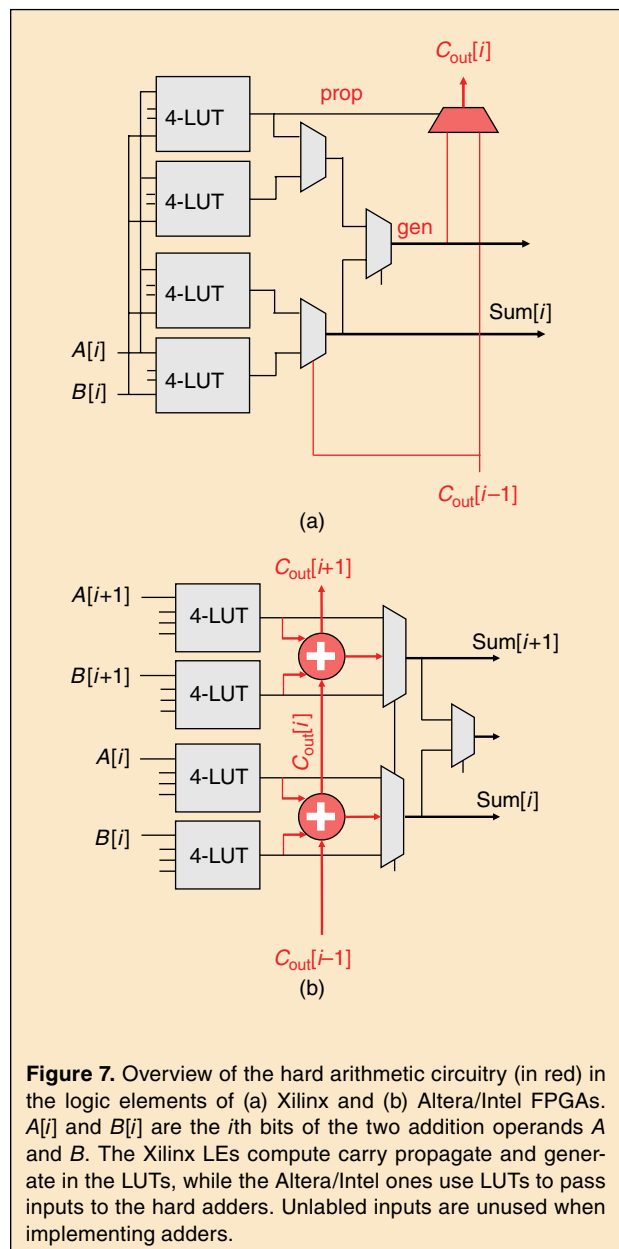


Figure 7. Overview of the hard arithmetic circuitry (in red) in the logic elements of (a) Xilinx and (b) Altera/Intel FPGAs. $A[i]$ and $B[i]$ are the i th bits of the two addition operands A and B . The Xilinx LEs compute carry propagate and generate in the LUTs, while the Altera/Intel ones use LUTs to pass inputs to the hard adders. Unlabeled inputs are unused when implementing adders.

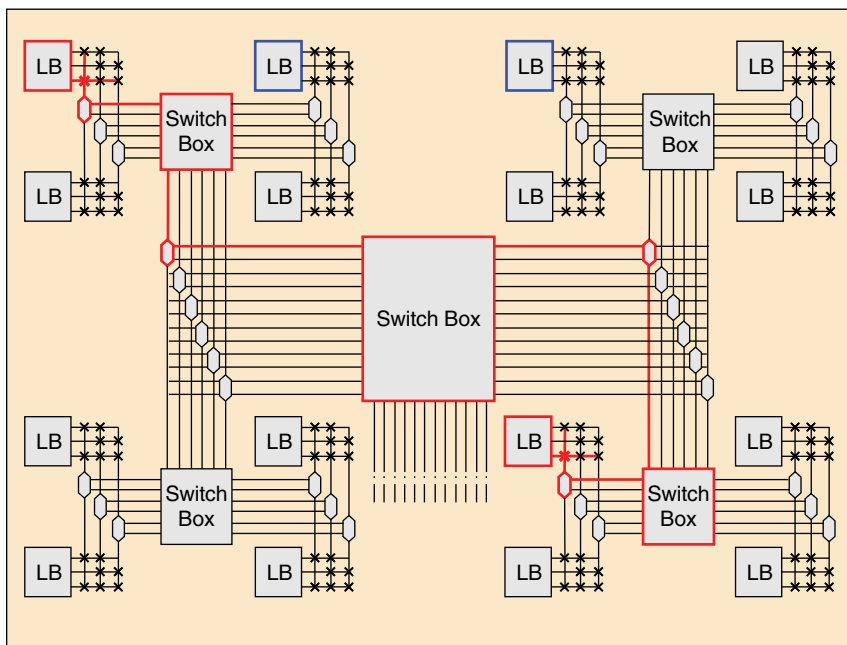


Figure 8. Routing architecture in hierarchical FPGAs.

B. Programmable Routing

Programmable routing commonly accounts for over 50% of both the fabric area and the critical path delay of applications [38], so its efficiency is crucial. Programmable routing is composed of pre-fabricated wiring segments and programmable switches. By programming an appropriate sequence of switches to be on, any function block output can be connected to any input. There are two main classes of FPGA routing architecture. *Hierarchical* FPGAs are inspired by the fact that designs are inherently hierarchical: higher-level modules instantiate lower level modules and connect signals between them. Communi-

cation is more frequent between modules that are near each other in the design hierarchy, and hierarchical FPGAs can realize these connections with short wires that connect small regions of a chip. As shown in Fig. 8, to communicate to more distant regions of a hierarchical FPGA, a connection (highlighted in red) passes through multiple wires and switches as it traverses different levels of the interconnect hierarchy. This style of architecture was popular in many earlier FPGAs, such as Altera's 7K and Apex 20K families, but it leads to very long wires at the upper levels of the interconnect hierarchy which became problematic as process scaling made such wires increasingly resistive. A strictly hierarchical routing architecture also results in some blocks that are physically close together (e.g. the blue blocks in Fig 8) which still require several wires and switches to connect. Consequently it is primarily used today for smaller FPGAs, such as the FlexLogix FPGA IP cores that can be embedded in larger SoC designs [39].

The other type of FPGA interconnect is *island-style*, as depicted in Fig. 9. This architecture was pioneered by Xilinx and is inspired by the fact that a regular two-dimensional layout of horizontal and vertical directed wire segments can be efficiently laid out. As shown in Fig. 9, island-style routing includes three components: routing wire segments, connection blocks (multiplexers) that connect function block inputs to the routing wires, and switch blocks (programmable switches) that connect routing wires together to realize longer routes.

The placement engine in FPGA CAD tools chooses which function block implements each element of a design in order to minimize the required wiring. Consequently, most connections between function blocks span a small distance and can be implemented with a few routing wires as illustrated by the red connection in Fig. 9.

Creating a good routing architecture involves managing many complex trade-offs. It should contain enough programmable switching and wire segments that the vast majority of circuits can be implemented; however, too many wires and switches waste area. A routing architecture should also match the needs of applications: ideally short connections will be made with short wires to minimize capacitance and layout area, while long connections can use longer wiring segments to avoid the extra

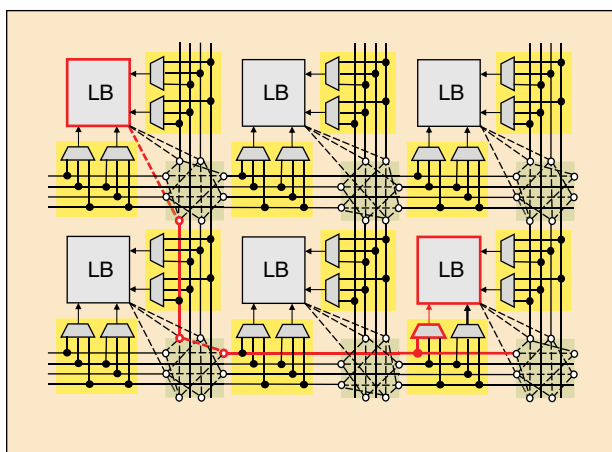


Figure 9. Island-style routing architecture. Thick solid lines are routing wires while dashed lines are programmable switches. Connection and switch blocks are shaded in yellow and green, respectively.

delay of passing through many routing switches. Some of the routing architecture parameters include: how many routing wires each logic block input or output can connect to (F_c), how many other routing wires each wire can connect to (F_s), the lengths of the routing wire segments, the routing switch pattern, the electrical design of the wires and switches themselves, and the number of routing wires per channel [20]. In Fig. 9 for example, $F_c = 3$, $F_s = 3$, the channel width is 4 wires, and some routing wires are of length 1, while others are of length 2. Fully evaluating these trade-offs for target applications and at a specific process node requires experimentation using a full CAD flow as detailed in Section II.

Early island-style architectures incorporated only short wires that traversed a single logic block between programmable switches. Later research showed that this resulted in more programmable switches than necessary, and that making all wiring segments span four logic blocks before terminating reduced application delay by 40% and routing area by 25% [40]. Modern architectures include multiple lengths of wiring segments to better match the needs of short and long connections, but the most plentiful wire segments remain of moderate length, with four logic blocks being a popular choice. Longer distance connections can achieve lower delay using longer wire segments, but in recent process nodes wires that span many (e.g. 16) logic blocks must use wide and thick metal traces on upper metal layers to achieve acceptable resistance [41]. The amount of such long-distance wiring one can include in a metal stack is limited. To best leverage such scarce wiring, Intel's Stratix FPGAs allow long wire segments to be connected only to short wire segments, rather than function block inputs or outputs [42]. This creates a form of routing hierarchy within an island-style FPGA, where short connections use only the shorter wires, but longer connections pass through short wires to reach the long wire network. Another area where hierarchical FPGA concepts are used within island-style FPGAs is within the logic blocks. As illustrated in Fig. 4(d), most logic blocks now group multiple BLEs together with local routing. This means each logic block is a small cluster in a hierarchical FPGA; island-style routing interconnects the resulting thousands of logic clusters.

There has been a great deal of research into the optimal amount of switching, and how to best arrange the switches. While there are many detailed choices, a few principles have emerged. The first is that the connectivity between function block pins and wires (F_c) can be relatively low: typically only 10% or less of the wires that pass by a pin will have switches to connect to it. Similarly, the number of other wires that a routing wire can connect to at its end (F_s) can also be low, but it should be at least 3 so that a signal can turn left, right, or go

straight at a wire end point. The local routing in a logic cluster (described in Section III-A) allows some block inputs and some block outputs to be swapped during routing. By leveraging this extra degree of flexibility and considering all the options presented by the multi-stage programmable routing network, the routing CAD tool can achieve high completion rates even with low F_c and F_s values. Switch patterns that give more options to the routing CAD tool also help routability; for example, the Wilton switch pattern ensures that following a different sequence of channels lets the router reach different wire segments near a destination block [43].

There are also multiple options for the electrical design of programmable switches, as shown in Fig. 10. Early FPGAs used pass gate transistors controlled by SRAM cells to connect wires. While this is the smallest switch possible in a conventional CMOS process, the delay of routing wires connected in series by pass transistors grows quadratically, making them very slow for large FPGAs. Adding some tri-state buffer switches costs area, but improves speed [40]. Most recent FPGAs primarily use a multiplexer built out of pass gates followed by a buffer that cannot be tri-stated, as shown in detail in Fig. 4(b). The pass transistors in this *direct drive* switch can be small as they are lightly loaded, while the buffer can be larger to drive the significant capacitance of a routing wire segment. Such direct drive switches create a major constraint on the switch pattern: a wire can only be driven at one point, so only function block outputs and routing wires near that point can feed its routing multiplexer inputs and hence be possible signal sources. Despite this constraint, both academic and industrial work has concluded that direct drive switches improve both area and speed due to their superior electrical characteristics [42], [44]. The exception is expensive or rare wires such as long wires implemented on wide metal traces on upper metal layers or the interposer-crossing wires discussed later in Section III-G. These wires often have multiple tri-state buffers that can drive them, as the cost of

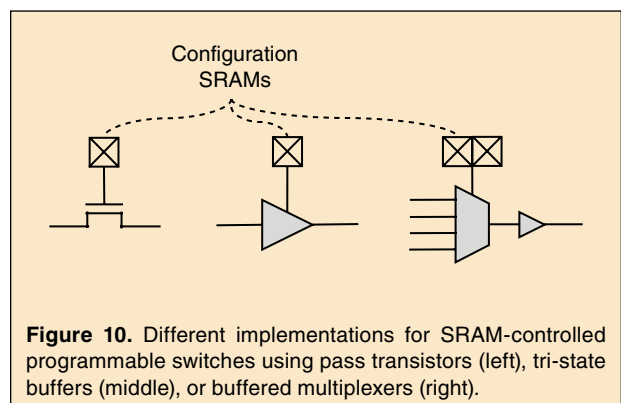


Figure 10. Different implementations for SRAM-controlled programmable switches using pass transistors (left), tri-state buffers (middle), or buffered multiplexers (right).

these larger programmable switches is merited to allow more flexible usage of these expensive wires.

A major challenge for FPGA routing is that the delay of long wires is not improving with process scaling, which means that the delay to cross the chip is stagnating or increasing even as clock frequencies rise. This has led FPGA application developers to increase the amount of pipelining in their designs, thereby allowing multiple clock cycles for long routes. To make this strategy more effective, some FPGA manufacturers have integrated registers within the routing network itself. Intel's Stratix 10 device allows each routing driver (i.e. multiplexer followed by a buffer) to be configured as a pulse latch as shown in 6(b), thereby acting as a register with low delay but relatively poor hold time. This allows deep pipelining of interconnect without using expensive logic resources, at the cost of a modest area and delay increase to the routing driver [45]. Hold time concerns mean that using pulse latches in immediately consecutive Stratix 10 routing switches is not possible, so Intel refined this approach in their next-generation Agilex devices by integrating actual registers (with better hold time) on only one-third of the interconnect drivers (to mitigate the area cost) [34]. Rather than integrating registers throughout the interconnect, Xilinx's Versal devices instead add bypassable registers only on the inputs to function blocks. Unlike Intel's interconnect registers, these input registers are full-featured, with clock enable and clear signals [46].

C. Programmable IO

FPGAs include unique programmable IO structures to allow them to communicate with a very wide variety of other devices, making FPGAs the communications hub of many systems. For a single set of physical IOs to *programmably* support many different IO interfaces and standards is challenging, as it requires adaptation to different voltage levels, electrical characteristics, timing specifications, and command protocols. Both the value and the challenge of programmable IO are highlighted by the large area devoted to IOs on FPGAs. For example, Altera's Stratix II (90 nm) devices support 28 different IO standards and devote 20% (largest device) to 48% (smallest device) of their die area to IO-related structures.

As Fig. 11 shows, FPGAs address this challenge using a combination of approaches [47]–[49]. First, FPGAs use IO buffers that can operate across a range of voltages. These IOs are grouped into banks (commonly on the order of 50 IOs per bank), where each bank has a separate V_{ddio} rail for the IO buffer. This allows different banks to operate at different voltage levels; e.g. IOs in one bank could be operating at 1.8 V while those in a different bank operate at 1.2 V. Second, each IO can be used separately for single-ended standards, or pairs of IOs can be programmed

to form the positive and negative line for differential IO standards. Third, IO buffers are implemented with multiple parallel pull-up and pull-down transistors so that their drive strengths can be programmably adjusted by enabling or disabling different numbers of pull-up/pull-down pairs. By programming some pull-up or pull-down transistors to be enabled even when no output is being driven, FPGA IOs can also be programmed to implement different on-chip termination resistances to minimize signal reflections. Programmable delay chains provide a fourth level of configurability, allowing fine delay adjustments of signal timing to and from the IO buffer.

In addition to electrical and timing programmability, FPGA IO blocks contain additional hardened digital circuitry to simplify capturing and transferring IO data to the fabric. Generally some or all of this hardened circuitry can be bypassed by SRAM-controlled muxes, allowing FPGA users to choose which hardened functions are desirable for a given design and IO protocol. Part ⑤ of Fig. 11 shows a number of common digital logic options on the IO input path: a capture register, double to single-data rate conversion registers (used with DDR memories), and serial-to-parallel converters to allow transfer to the fabric at a lower frequency. Most FPGAs now also contain by-passable higher-level blocks that connect to a group of IOs and implement higher-level protocols like DDR memory controllers. Together these approaches allow the general-purpose FPGA IOs to service many different protocols, at speeds up to 3.2 Gb/s.

The highest speed IOs implement serial protocols, such as PCIe and Ethernet, that embed the clock in data transitions and can run at 28 Gb/s or more. To achieve these speeds, FPGAs include a separate group of differential-only IOs with less voltage and electrical programmability; they can only be used as serial transceivers [50]. Just as for the general-purpose IOs, these serial IOs have a sequence of high-speed hardened circuits between them and the fabric, some of which can be optionally bypassed to allow end-users to customize the exact interface protocol.

Overall, FPGA IO design is very challenging, due to the dual (and competing) demands to make the IO not only very fast but also programmable. In addition, distributing the very high data bandwidths from IO interfaces requires wide soft buses in the fabric, which creates additional challenges as discussed later in Section III-F.

D. On-Chip Memory

The first form of on-chip memory elements in FPGA architectures was FFs integrated in the FPGA's logic blocks as described in Section III-A. However, as FPGA logic capacity grew, they were used to implement larger systems which almost always require memory to buffer and

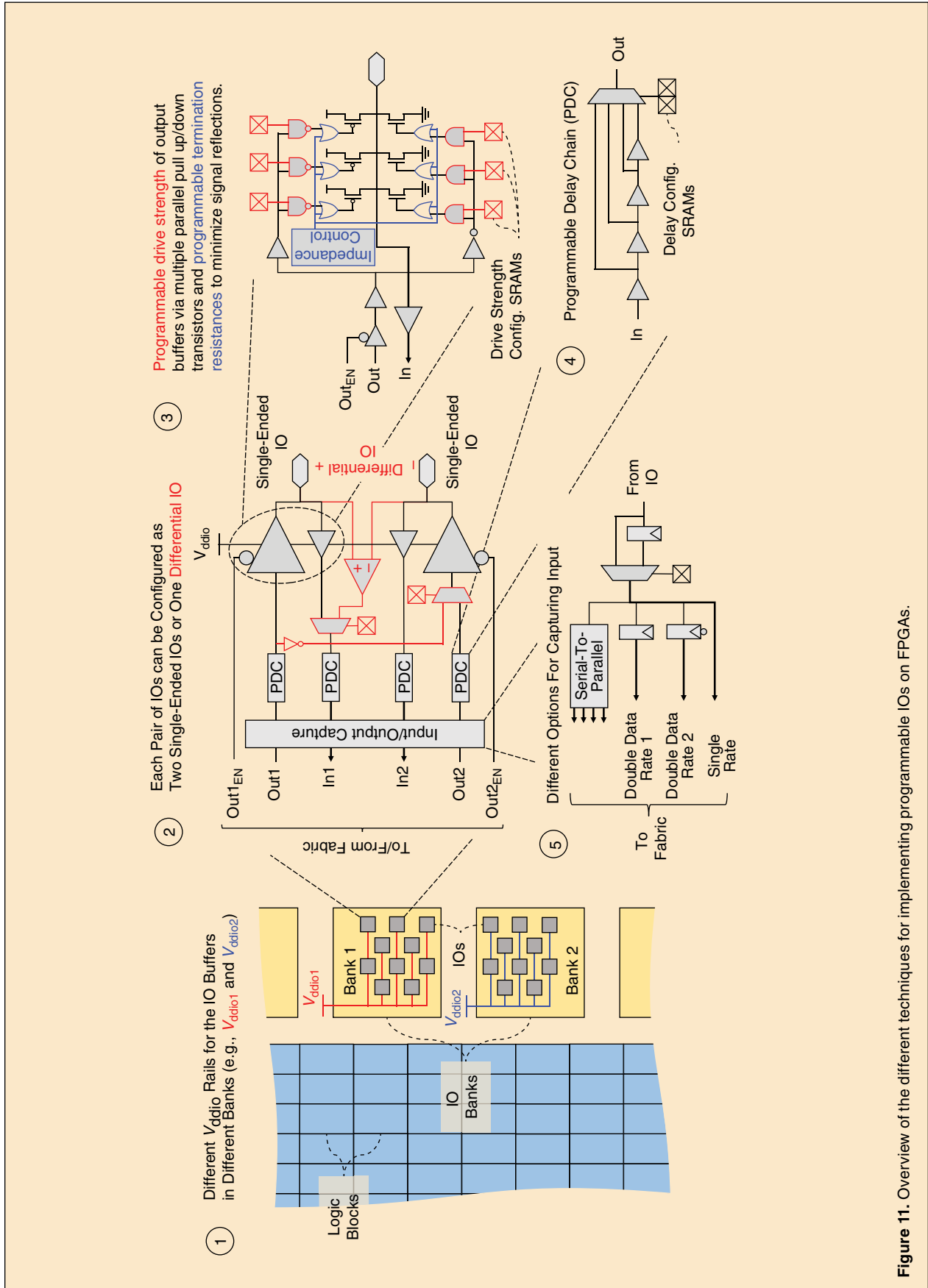
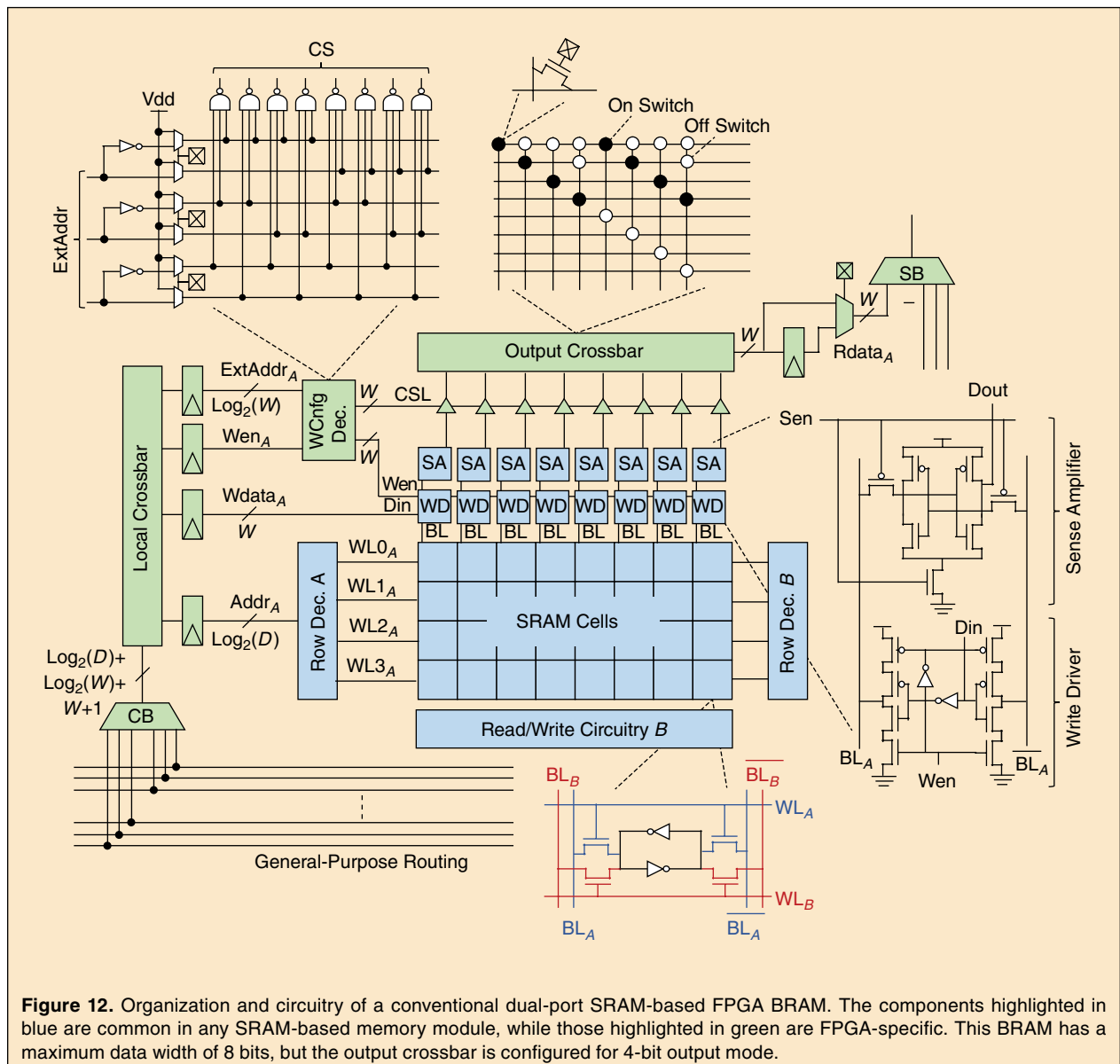


Figure 11. Overview of the different techniques for implementing programmable I/Os on FPGAs.

re-use data on chip, making it highly desirable to have denser on-chip storage since building large RAMs out of registers and LUTs is over $100\times$ less dense than an (ASIC-style) SRAM block. At the same time, the RAM needs of applications implemented on FPGAs are very diverse, including (but not limited to) small coefficient storage RAMs for FIR filters, large buffers for network packets, caches and register files for processor-like modules, read-only memory for instructions, and FIFOs of myriad sizes to decouple computation modules. This means that there is no single RAM configuration (capacity, word width, number of ports) used universally in FPGA designs, making it challenging to decide on what kind(s) of RAM blocks should be added to an FPGA such that they are efficient for a broad range of uses. The first FPGA to

include hard functional blocks for memory (*block RAMs* or *BRAMs*) was the Altera Flex 10 K in 1995 [51]. It included columns of small (2 kb) BRAMs that connect to the rest of the fabric through the programmable routing. FPGAs have gradually incorporated larger and more diverse BRAMs and it is typical for $\sim 25\%$ of the area of a modern FPGA to be dedicated for BRAMs [52].

An FPGA BRAM consists of an SRAM-based memory *core*, with additional *peripheral circuitry* to make them more configurable for multiple purposes and to connect them to the programmable routing. An SRAM-based BRAM is typically organized as illustrated in Fig. 12. It consists of a two-dimensional array of SRAM cells to store bits, and a considerable amount of peripheral circuitry to orchestrate access to these cells for read/



write operations. To simplify timing of the read and write operations, all modern FPGA BRAMs register all their inputs. During a write operation, the column decoder activates the write drivers, which in turn charge the bitlines (BL and $\overline{BL}$) according to the input data to-be-written to the memory cells. Simultaneously, the row decoder activates the wordline of the row specified by the input write address, connecting one row of cells to their bitlines so they are overwritten with new data. During a read operation, both the BL and $\overline{BL}$ are pre-charged high and then the row decoder activates the wordline of the row specified by the input read address. The contents of the activated cells cause a slight difference in the voltage between BL and $\overline{BL}$, which is sensed and amplified by the sense amplifier circuit to produce the output data [52].

The main architectural decisions in designing FPGA BRAMs are choosing their capacity, data word width, and number of read/write ports. More capable BRAMs cost more silicon area, so architects must carefully balance BRAM design choices while taking into account the most common use cases in application circuits. The area occupied by the SRAM cells grows linearly with the capacity of the BRAM, but the area of the peripheral circuitry and the number of routing ports grows sub-linearly. This means that larger BRAMs have lower area per bit, making large on-chip buffers more efficient. On the other hand, if an application requires only small RAMs, much of the capacity of a larger BRAM may be wasted. Similarly, a BRAM with a larger data width can provide higher data bandwidth to downstream logic. However, it costs more area than a BRAM with the same capacity but a smaller word width, as the larger data word width necessitates more sense amplifiers, write drivers and programmable routing ports. Finally, increasing the number of read/write ports to a BRAM increases the area of both the SRAM cells and the peripheral circuitry, but again increases the data bandwidth the BRAM can provide and allows more diverse uses. For example, FIFOs (which are ubiquitous in FPGA designs) require both a read and a write port. The implementation details of a dual-port SRAM cell is shown at the bottom of Fig. 12. Implementing a second port to the SRAM cell (port B highlighted in red) adds two transistors, increasing the area of the SRAM cells by 33%. In addition, the second port also needs an additional copy of the sense amplifiers, write drivers and row decoders (the “Read/Write Circuitry B” and “Row Decoder B” blocks in Fig. 12). If both ports are read/write (r/w), we also have to double the number of ports to the programmable routing.

Because the FPGA on-chip memory must satisfy the needs of *every* application implemented on that FPGA, it is also common to add extra configurability to BRAMs to allow them to adapt to application needs [53], [54]. FPGA

BRAMs are designed to have configurable width and depth by adding low-cost multiplexing circuitry to the peripherals of the memory array. For example, in Fig. 12 the actual SRAM array is implemented as a 4×8 -bit array, meaning it naturally stores 8-bit data words. By adding multiplexers controlled by 3 address bits to the output crossbar, and extra decoding and enabling logic to the read/write circuitry, this RAM can also operate in 8×4 -bit, 16×2 -bit or 32×1 -bit modes. The width configurability decoder (WCnfg Dec.) selects between V_{dd} and address bits, as shown in the top-left of Fig. 12 for a maximum word size of 8 bits. The multiplexers are programmed using configuration SRAM cells and are used to generate column select (CS) and write enable (Wen) signals that control the sense amplifiers and write drivers for narrow read and write operations, respectively. For typical BRAM sizes (several kb or more), the cost of this additional width configurability circuitry is small compared to the cost of a conventional SRAM array, and it does not require any additional routing ports.

Another unique component of the FPGA BRAMs compared to conventional memory blocks is their interface to the programmable routing fabric. This interface is generally designed to be similar to that of the logic blocks described in Section III-A; it is easier to create a routing architecture that balances flexibility and cost well if all block types connect to it in similar ways. Connection block multiplexers, followed by local crossbars in some FPGAs, form the BRAM input routing ports, while the read outputs drive switch block multiplexers to form the output routing ports. These routing interfaces are costly, particularly for small BRAMs; they constitute 5% to 35% of the BRAM tile area for 256Kb down to 8Kb BRAMs, respectively [55]. This motivates minimizing the number of routing ports to a BRAM as much as possible without unduly compromising its functionality. Table I summarizes the number of routing ports required for different numbers and types of BRAM read and write ports. For example, a single-port BRAM (1r/w) requires $W + \log_2(D)$ input ports for write data and read/write address, and W

Table I.
Number of routing ports needed for different numbers and types of BRAM read/write ports (W : data width, D : BRAM depth).

BRAM Ports	BRAM Mode	# Routing Ports
1r	Single-port ROM	$\log_2(D) + W$
1r/w	Single-port RAM	$\log_2(D) + 2W$
1r+1w	Simple dual-port RAM	$2\log_2(D) + 2W$
2r/w	True dual-port RAM	$2\log_2(D) + 4W$
2r+2w	Quad-port RAM	$4\log_2(D) + 4W$

output ports for read data, where W and D are the maximum word width and the BRAM depth, respectively. The table shows that a true dual-port ($2r/w$) BRAM requires $2W$ more ports compared to a simple dual-port ($1r+1w$) BRAM, which significantly increases the cost of the routing interfaces. While true dual-port memory is useful for register files, caches and shared memory switches, the most common use of multi-ported RAMs on FPGAs is for FIFOs, which require only one read and one write port ($1r+1w$ rather than $2r/w$ ports). Consequently, FPGA BRAMs typically have true dual-port SRAM cores but with only enough routing interfaces for simple-dual port mode at the full width supported by the SRAM core (W), and limit the width of the true-dual port mode to only half of the maximum width ($W/2$).

Another way to mitigate the cost of additional BRAM ports is to *multi-pump* the memory blocks (i.e. operate the BRAMs at a frequency that is a multiple of that used for the rest of the design logic). By doing so, a physically single-ported SRAM array can implement a logically multi-ported BRAM without the cost of additional ports as in Tabula's *Spacetime* architecture [56]. Multi-pumping can also be used with conventional FPGA BRAMs by building the time-multiplexing logic in the soft fabric; however, this leads to aggressive timing constraints for the time-multiplexing logic, which can make timing closure more challenging and increase compile time. Altera introduced quad-port BRAMs in its Mercury devices in the early 2000s to make shared memory switches (useful in packet processing) and register files more efficient [57]. However, this feature increased the BRAM size and was not sufficiently used to justify its inclusion in subsequent FPGA generations. Instead designers use a variety of techniques to combine dual-ported FPGA BRAMs and soft logic to make highly-ported structures when needed, albeit at lower efficiency [58], [59]. We refer the interested reader to both [52] and [55] for extensive details about the design of BRAM core and peripheral circuitry.

In addition to building BRAMs, FPGA vendors can add circuitry that allows designers to repurpose the LUTs that form the logic fabric into additional RAM blocks. The truth tables in the logic block K -LUTs are $2^K \times 1$ -bit read-only memories; they are written once by the configuration circuitry when the design bitstream is loaded. Since LUTs already have read circuitry (read out a stored value based on a K -bit input/address), they can be used as small distributed LUT-based RAMs (LUT-RAMs) just by adding designer-controlled write circuitry. However, a major concern is the number of additional routing ports necessary to implement the write functionality to change a LUT to a LUT-RAM. For example, an ALM in recent Altera/Intel architectures is a 6-LUT that can be fractured into two 5-LUTs and has 8 input routing ports,

as explained in Section III-A. This means it can operate as a 64×1 -bit or a 32×2 -bit memory with 6 or 5 bits for read address, respectively. This leaves only 2 or 3 unused routing ports, which are not enough for write address, data, and write enable (8 total signals) if we want to read and write in each cycle (simple dual-port mode), which is the most commonly used RAM mode in FPGA designs. To overcome this problem, an entire logic block of 10 ALMs is configured as a LUT-RAM to amortize the control circuitry and address bits across 10 ALMs. The write address and write enable are assembled by bringing one signal in from an unused routing port in each ALM and broadcasting the resulting address and enable to all ALMs [60]. Consequently, a logic block can implement a 64×10 -bit or 32×20 -bit simple dual-port RAM, but has a restriction that a single logic block cannot mix logic and LUT-RAM. Xilinx Ultrascale similarly converts an entire logic block to LUT-RAM, but all the routing ports of one out of the eight LUTs in a logic block are repurposed to drive the (broadcast) write address and enable signals. Therefore, a Xilinx logic block can implement a 64×7 -bit or 32×14 -bit simple dual-port RAM, or a slightly wider single-port RAM (64×8 -bit or 32×16 -bit). Avoiding extra routing ports keeps the cost of LUT-RAM low, but it still adds some area. Since it would be very unusual for a design to convert more than 50% of the logic fabric to LUT-RAMs, both Altera/Intel and Xilinx have elected to make only half of their logic blocks LUT-RAM capable in their recent architectures, thereby further reducing the area cost.

Designers require many different RAMs in a typical design, all of which must be implemented by the fixed BRAM and LUT-RAM resources on the chip. Forcing designers to determine the best way to combine BRAM and LUT-RAM for each memory configuration they need and writing verilog to implement them would be laborious and also would tie the design to a specific FPGA architecture. Instead, the vendor CAD tools include a *RAM mapping* stage that implements the *logical* memories in the user's design using the *physical* BRAMs and LUT-RAMs on the chip. The RAM mapper chooses the physical memory implementation (i.e. memory type and the width/number/type of its ports) and generates any additional logic required to combine multiple BRAMs or LUT-RAMs to implement each logical RAM. Fig. 13 gives an example of mapping a logical 2048×32 -bit RAM with 2 read and 1 write ports to an FPGA with physical 1024×8 -bit dual-port BRAMs. First, four physical BRAMs are combined in parallel to make wider RAMs with no extra logic. Then, soft logic resources are used to perform depth-wise stitching of two sets of four physical BRAMs, such that the most-significant bits of the write and read addresses are used as write enable and read output mux select signals, respectively. Finally, in this case we require two read ports

and one write port while the physical BRAMs only support a maximum of $2r/w$ ports. To implement the second read port, the whole structure is either replicated (see Fig. 13) or double-pumped as previously explained. Several algorithms for optimizing RAM mapping are described in [61], [62].

Over the past 25 years, FPGA memory architecture has evolved considerably and has also become increasingly important, as the ratio of memory to logic on an FPGA die has grown significantly. Fig. 14 plots the memory bits per logic element (including LUT-RAM) versus the number of logic elements in Altera/Intel devices starting from the 350 nm Flex 10 K devices (1995) to 10 nm Agilex devices (2019). There has been a gradual increase in the memory richness of FPGAs over time, and to meet the demand for more bits, modern BRAMs have larger capacities (20 kb) than the first BRAMs (2 kb). Some FPGAs have had highly heterogeneous BRAM architectures in order to provide some physical RAMs that are efficient for small or wide logical RAMs, and others that are efficient for large and relatively narrow logical RAMs. For example, Stratix (130 nm) had 3 types of BRAM, with capacities of 512 b, 4 kb and 512 kb. The introduction of LUT-RAM in Stratix III (65 nm) reduced the need for small BRAMs, so it moved to a memory architecture with 9 kb and 144 kb BRAM. Stratix V (28 nm) and later Intel devices have moved to a combination of LUT-RAM and a single medium-sized BRAM (20 kb) to

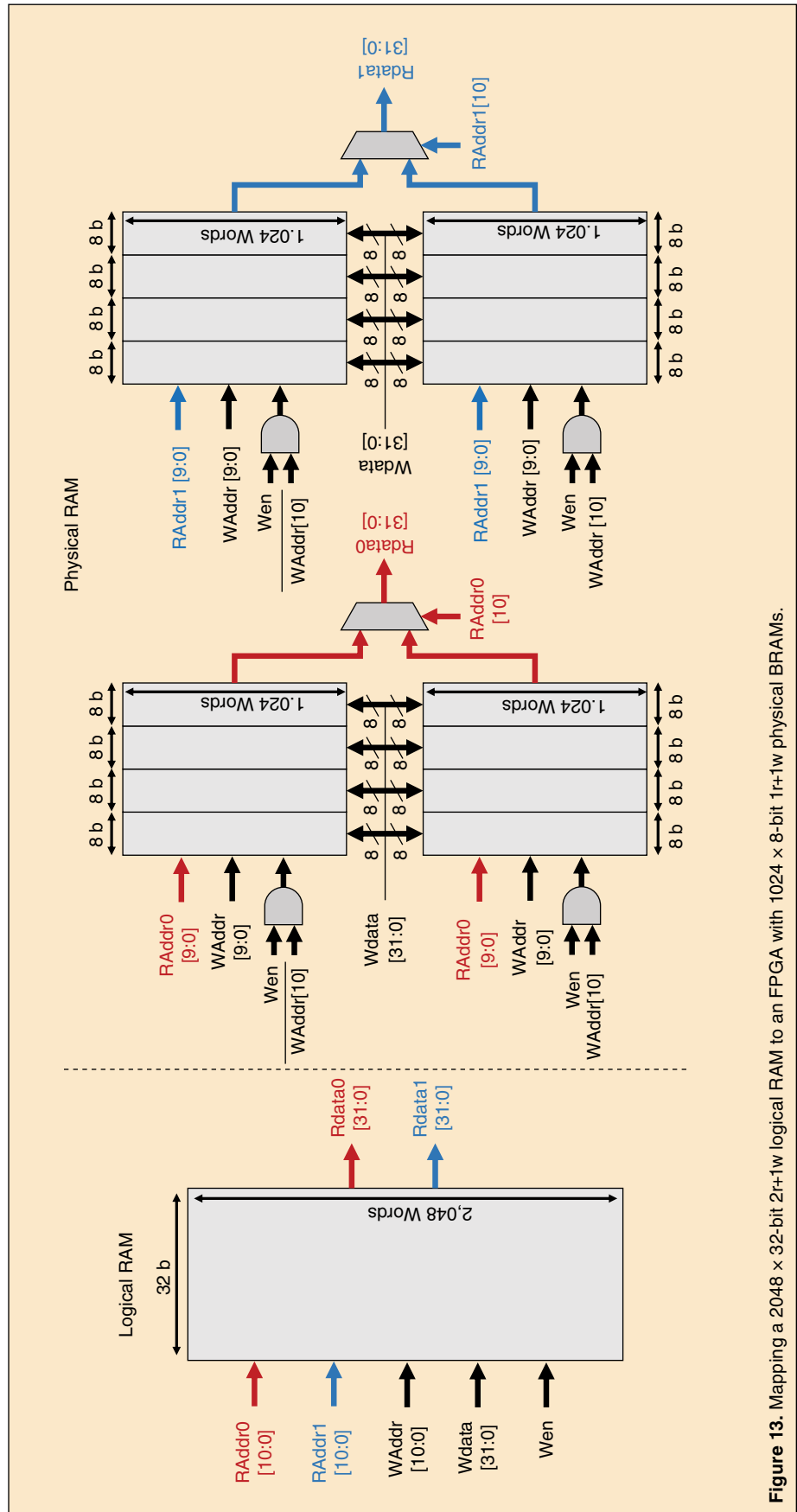
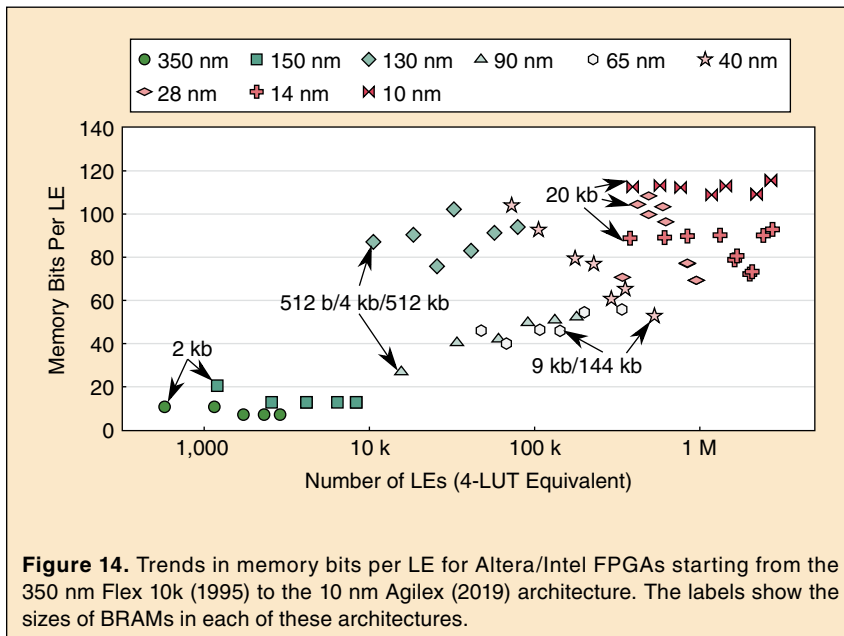


Figure 13. Mapping a 2048 × 32-bit 2r+1w logical RAM to an FPGA with 1024 × 8-bit 1r+1w physical BRAMs.



simplify both the FPGA layout as well as RAM mapping and placement. Tatsumura et al. [52] plot a similar trend for on-chip memory density in Xilinx devices as well. Similarly to Intel, Xilinx’s RAM architecture combines LUT-RAM and a medium-sized 18 kb RAM, but also includes hard circuitry to combine two BRAMs into a single 36 kb block. However, Xilinx’s most recent devices have also included a large 288 kb BRAM (UltraRAM) to be more efficient for very large buffers, showing that there is still no general agreement on the best BRAM architecture.

To give some insight into the relative areas and efficiencies of different RAM blocks, Table II shows the resource usage, silicon area, and frequency of a 2048×72 -bit logical RAM when it is implemented by Quartus (the CAD flow for Altera/Intel FPGAs) in a variety of ways on a Stratix IV device. The silicon areas are computed using the published Stratix III block areas from [63] and scaling them from 65 nm down to 40 nm, as Stratix III and IV have the same architecture but use different process nodes. As this logical RAM is a perfect fit to the 144 kb BRAM in

for this single test case, real designs have diverse logical RAMs, and for small or shallow memories the 9 kb and LUT-RAM options would outperform the 144 kb BRAM, motivating a diversity of on-chip RAM resources. To choose the best mix of BRAM sizes and maximum word widths, one needs both a RAM mapping tool and tools to estimate the area, speed and power of each BRAM [55]. Published studies into BRAM architecture trade-offs for FPGAs include [30], [55], [64].

Until now, all commercial FPGAs use only SRAM-based memory cells in their BRAMs. With the desire for more dense BRAMs that would enable more memory-rich FPGAs and SRAM scaling becoming increasingly difficult due to process variation, a few academic studies (e.g. [52], [65]) have explored the use of other emerging memory technologies such as magnetic tunnel junctions (MTJs) to build FPGA memory blocks. According to [52], MTJ-based BRAMs could increase the FPGA memory capacity by up to $2.95\times$ with the same die size; however, they would increase the process complexity.

E. DSP Blocks

Initially the only dedicated arithmetic circuits in commercial FPGA architectures were carry chains to implement efficient adders, as discussed in Section III-A. Thus multipliers had to be implemented in the soft logic using LUTs and carry chains, incurring a substantial area and delay penalty. As high-multiplier-density signal processing

Table II. Implementation results for a 2048×72 -bit 1r+1w RAM using BRAMs, LUT-RAMs and registers on Stratix IV.

Implementation	half-ALM	BRAMs		Area (mm ²)	Freq. (Mhz)
		9k	144k		
144kb BRAMs	0	0	1	0.22 (1.0 $\times$)	336 (1.0 $\times$)
9kb BRAMs	0	18	0	0.41 (1.9 $\times$)	497 (1.5 $\times$)
LUT-RAM	6597	0	0	2.81 (12.8 $\times$)	134 (0.4 $\times$)
Registers	165155	0	0	68.8 (313 $\times$)	129 (0.4 $\times$)

and communication applications constituted a major FPGA market, designers proposed novel implementations to mitigate the inefficiency of multiplier implementations in soft logic. For example, the multiplier-less *distributed arithmetic* technique was proposed to implement efficient finite impulse response (FIR) filter structures on LUT-based FPGAs [66], [67].

With the prevalence of multipliers in FPGA designs from key application domains, and their lower area/delay/power efficiency when implemented in soft logic, they quickly became a candidate for hardening as dedicated circuits in FPGA architectures. An N -bit multiplier array consists of N^2 logic elements with only $2N$ inputs and outputs. Therefore, the gains of hardening the multiplier logic and the cost of the programmable interfaces to the FPGA's routing fabric resulted in a net efficiency gain and strongly advocated for adopting hard multipliers in subsequent FPGA architectures. As shown at top left of Fig. 15, Xilinx introduced its Virtex-II architecture with the industry's first 18×18 bit hard multiplier blocks [68]. To simplify the layout integration with the full-custom FPGA fabric, these multipliers were arranged in columns right beside BRAM columns. In order to further reduce the interconnect cost, the multiplier block and its adjacent BRAM had to share some interconnect resources, limiting the maximum usable data width of the BRAM block. Multiple hard 18-bit multipliers could be combined to form bigger multipliers or FIR filters using soft logic resources.

In 2002, Altera adopted a different approach by introducing full featured DSP blocks targeting the communica-

tions and signal processing domains in their Stratix architecture [42] (see second block in Fig. 15). The main design philosophy of this DSP block was to minimize the amount of soft logic resources used to implement common DSP algorithms by hardening more functionality inside the DSP block, and enhancing its flexibility to allow more applications to make use of it. The Stratix DSP block was highly configurable with support for different modes of operation and multiplication precisions unlike the fixed-function hard 18-bit multipliers in the Virtex-II architecture. Each Stratix variable-precision DSP block spanned 8 rows and could implement eight 9×9 bit multipliers, four 18×18 bit multipliers, or one 36×36 multiplier.

These modes of operation selected by Altera highlight an important theme of designing FPGA hard blocks: increasing the configurability and utility of these blocks by adding low-cost circuitry. For example, an 18×18 multiplier array can be decomposed into two 9×9 arrays that together use the same number of inputs and outputs (and hence routing ports). Similarly, four 18×18 multipliers can be combined into one 36×36 array using cheap glue logic. Fig. 16 shows how an 18×18 multiplier array can be fractured into multiple 9×9 arrays. It can be split into four 9×9 arrays by doubling the number of input and output pins. However, to avoid adding these costly routing interfaces, the 18×18 array is split into only two 9×9 arrays (colored blue in Fig. 16). This is done by splitting the partial product compressor trees at the positions indicated by the red dashed lines and adding inverting capabilities to the border cells of the top-right

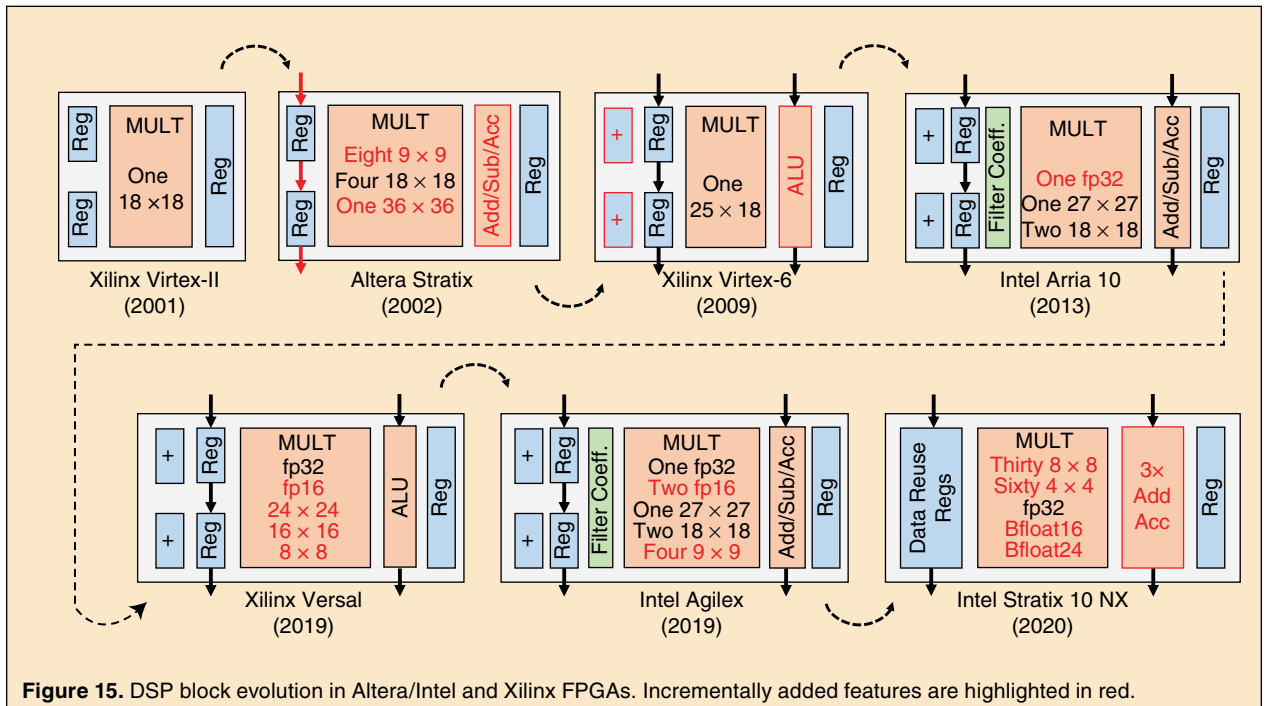


Figure 15. DSP block evolution in Altera/Intel and Xilinx FPGAs. Incrementally added features are highlighted in red.

array, marked with crosses in Fig. 16 to implement two's complement signed multiplication using the Baugh-Woolley algorithm [69] (the bottom left array already has the inverting capability from the 18×18 array).

In addition to the fracturable multiplier arrays, the Stratix DSP also incorporated an adder/output block to perform summation and accumulation operations, as well as hardened input registers that could be configured as shift registers with dedicated cascade interconnect between them to implement efficient FIR filter structures [70]. The latest 28 nm architectures from Lattice also have a variable precision DSP block that can implement the same range of precisions, in addition to special 1 D and 2 D symmetry modes for filter structures and video processing applications, respectively. Xilinx also adopted a full-featured DSP block approach by introducing their *DSP48* tiles in the Virtex-4 architecture [71]. Each DSP tile had two fixed-precision 18×18 bit multipliers with similar functionalities to the Stratix DSP block (e.g. input cascades, adder/subtractor/accumulator). Virtex-4 also

introduced the ability to cascade the adders/accumulators using dedicated interconnects to implement high-speed systolic FIR filters with hardened reduction chains.

An N -tap FIR filter performs a discrete 1 D convolution between the samples of a signal $X = \{x_0, x_1, \dots, x_T\}$ and certain coefficients $C = \{c_0, c_1, \dots, c_{N-1}\}$ that represent the impulse response of the desired filter, as shown in eq. (1).

$$y_n = c_0 x_n + c_1 x_{n-1} + \dots + c_N x_{n-N} = \sum_{i=0}^N c_i x_{n-i} \quad (1)$$

Many of the FIR filters used in practice are symmetric with $c_i = c_{N-i}$, for $i = 0$ to $N/2$. As a result of this symmetry, the filter computation can be refactored as shown in eq. (2).

$$y_n = c_0 [x_n + x_{n-N}] + \dots + c_{N/2-1} [x_{n-N/2-1} + x_{n-N/2}] \quad (2)$$

Fig. 17 shows the structure of a systolic symmetric FIR filter circuit, which is a key use case for FPGAs in wireless base stations. Both Stratix and Virtex-4 DSP blocks can

implement the portions highlighted by the dotted boxes, resulting in significant efficiency gains compared to implementing them in the FPGA's soft logic. Interestingly, while FPGA CAD tools will automatically implement a multiplication (*) operation in DSP blocks, they will generally not make use of any of the advanced DSP block features (e.g. accumulation, systolic registers for FIR filters) unless a designer manually instantiates a DSP block in the proper mode. Consequently, using the more powerful DSP block features makes a design less portable.

The Stratix III DSP block was similar to the Stratix II one, but could implement four 18×18 multipliers per half a DSP block (instead of two) if their results are summed to limit

the number of output routing interfaces [72]. Table III lists the implementation results of both symmetric and asymmetric 51-tap FIR filters, with and without using the hard DSP blocks on a Stratix IV device. When DSP blocks are not used, we experiment with two different cases: fixed filter coefficients, and filter coefficients that can change at runtime. If the filter coefficients are fixed, the multiplier arrays implemented in the soft logic are optimized by synthesizing away parts of the partial product generation logic that correspond to zero bits in the coefficient values. Hence, it has lower resource utilization than with input coefficients that can change at runtime.

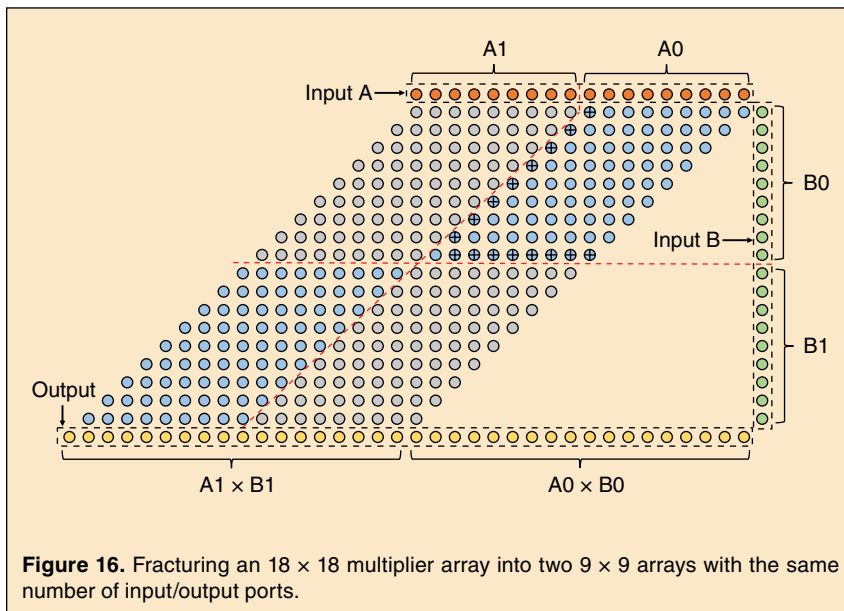


Figure 16. Fracturing an 18×18 multiplier array into two 9×9 arrays with the same number of input/output ports.

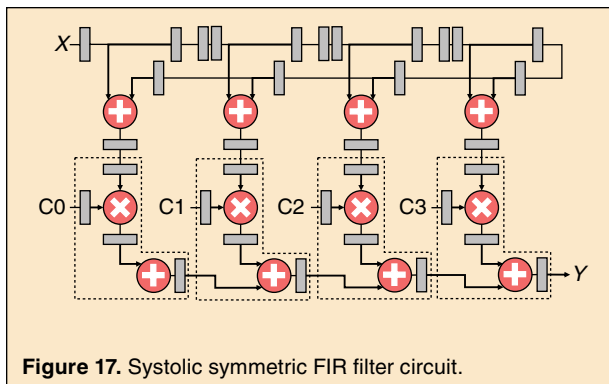


Figure 17. Systolic symmetric FIR filter circuit.

For the symmetric filter, even when using the DSP blocks, we still need to use some soft logic resources to implement the input cascade chains and pre-adders as shown in Fig. 17. Using the hard DSP blocks results in $3\times$ higher area efficiency vs. using the soft fabric in the case of fixed coefficients. This gap grows to $6.2\times$ for filter coefficients that are changeable during runtime. For the asymmetric filter, the complete FIR filter structure can be implemented in the DSP blocks without any soft logic resources. Thus, the area efficiency gap increases to $3.9\times$ and $8.5\times$ for fixed and input coefficients, respectively. These gains are large but still less than the $35\times$ gap between FPGAs and ASICs [11] usually cited in academia. The difference is partly due to some soft logic remaining in most application circuits, but even in the case where the FIR filter perfectly fits into DSP blocks with no soft logic the area reduction hits a maximum of $8.5\times$. The primary reasons for the lower than $35\times$ gain of [11] are the interfaces to the programmable routing and the general inter-tile programmable routing wires and multiplexers that must be implemented in the DSP tile. In all cases, using the hard DSP blocks results in about $2\times$ frequency improvement as shown in Table III.

The subsequent FPGA architecture generations from both Altera and Xilinx witnessed only minor changes in the DSP block architecture. The main focus of both vendors was to fine tune the DSP block capabilities for key application domains without adding costly programmable routing interfaces. In Stratix V, the DSP block was greatly simplified to natively support two 18×18 bit multiplications (the key precision used in wireless base station signal processing) or one 27×27 multiplication (useful for single-precision floating-point mantissas). As a result, the simpler Stratix V DSP block spanned a single row, which is more friendly to Altera's row redundancy scheme. In addition, input pre-adders as well as embedded coefficient banks to store read-only filter weights were added [73], which allowed implementing the whole symmetric FIR filter structure shown in Fig. 17 inside the DSP blocks without the need for any soft logic resources. On the other hand, Xilinx switched from 18×18 multipliers to 25×18 multipliers in their Virtex-5 *DSP48E* tile [74], after which they incorporated input pre-adders and enhanced their adder/accumulator unit to also support bitwise logic operations in their Virtex-6

DSP48E1 tile [75]. Then, they increased their multiplication width again to 27×18 bit and added a fourth input to their ALU in the Ultrascale family *DSP48E2* tile [76].

As illustrated in Fig. 15, up to 2009 the evolution of the DSP block architecture was mainly driven by the precisions and requirements of communication applications, especially in wireless base stations, with very few academic research explorations [77], [78]. More recently, FPGAs have been widely deployed in datacenters to accelerate various types of workloads such as search engines and network packet processing [9]. In addition, DL has emerged as a key component of many applications both in datacenter and edge workloads, with MAC being its core arithmetic operation. Driven by these new trends, the DSP block architecture has evolved in two different directions. The first direction targets the high-performance computing (HPC) domain by adding native support for single-precision floating-point (fp32) multiplication. Before that, FPGA vendors would supply designers with IP cores that implement floating-point arithmetic out of fixed-point DSPs and a considerable amount of soft logic resources. This created a huge barrier for FPGAs to compete with CPUs and GPUs (which have dedicated floating-point units) in the HPC domain. Native floating-point capabilities were first introduced in Intel's Arria 10 architecture, with a key design goal of avoiding a large increase in DSP block area [79]. By reusing the same interface to the programmable routing, not supporting uncommon features like subnormals, flags and multiple rounding schemes, and maximizing the reuse of existing fixed-point hardware, the block area increase was limited to only 10% (i.e. 0.5% total die area

Table III.
Implementation results for a 51-tap symmetric FIR filter on Stratix IV with and without using the hardened DSP blocks.

Symmetric Filter				
Implementation	half-ALMs	DSPs	Area (mm ²)	Freq. (Mhz)
With DSPs	403	$3 \frac{2}{8}$	0.49 (1.0 $\times$)	510 (1.0 $\times$)
Without DSPs (fixed coeff.)	3505	0	1.46 (3.0 $\times$)	248 (0.5 $\times$)
Without DSPs (input coeff.)	7238	0	3.01 (6.2 $\times$)	220 (0.4 $\times$)
Asymmetric Filter				
Implementation	half-ALMs	DSPs	Area (mm ²)	Freq. (Mhz)
With DSPs	0	$6 \frac{3}{8}$	0.63 (1.0 $\times$)	510 (1.0 $\times$)
Without DSPs (fixed coeff.)	5975	0	2.48 (3.9 $\times$)	245 (0.5 $\times$)
Without DSPs (input coeff.)	12867	0	5.35 (8.5 $\times$)	217 (0.4 $\times$)

increase). Floating-point capabilities will also be supported in the *DSP58* tiles of the next-generation Xilinx Versal architecture [80].

The second direction targets increasing the density of low-precision integer multiplication specifically for DL inference workloads. Prior work has demonstrated the use of low-precision fixed-point arithmetic (8-bit and below) instead of `fp32` at negligible or no accuracy degradation, but greatly reduced hardware cost [81]–[83]. However, the required precision is model-dependent and can even vary between different layers of the same model. As a result, FPGAs have emerged as an attractive solution for DL inference due to their ability to implement custom precision datapaths, their energy efficiency compared to GPUs, and their lower development cost compared to custom ASICs. This has led both academic researchers and FPGA vendors to investigate adding native support for low-precision multiplication to DSP blocks. The authors of [84] enhance the fracturability of an Intel-like DSP block to support more `int9` and `int4` multiply and MAC operations, while keeping the same DSP block routing interface and ensuring its backward compatibility. The proposed DSP block could implement four `int9` and eight `int4` multiply/MAC operations along with Arria-10-like DSP block functionality at the cost of 12% DSP block area increase, which is equivalent to only 0.6% increase in total die area. This DSP block increased the performance of 8-bit and 4-bit DL accelerators by $1.3\times$ and $1.6\times$ while reducing the utilized FPGA resources by 15% and 30% respectively, compared to an FPGA with DSPs that do not natively support these modes of operation. Another academic work [85] enhanced a Xilinx-like DSP block by including a fracturable multiplier array instead of the fixed-precision multiplier in the *DSP48E2* block to support `int9`, `int4` and `int2` precisions. It also added a FIFO register file and special dedicated interconnect between DSP blocks to enable more efficient standard, point-wise and depth-wise convolution layers. Shortly after, Intel announced that the same `int9` mode of operation will be added to the next-generation Agilex DSP block along with half-precision floating-point (`fp16`) and brain float (`bfloat16`) precisions [86]. Also, the next-generation Xilinx Versal architecture will natively support `int8` multiplications in its *DSP58* tiles [80].

Throughout the years, the DSP block architecture has evolved to best suite the requirements of key application domains of FPGAs, and provide higher flexibility such that many different applications can benefit from its capabilities. The common focus across all the steps of this evolution was reusing multiplier arrays and routing ports as much as possible to best utilize both these costly resources. However, this becomes harder with the recent divergence in the DSP block requirements of key FPGA

application domains between high-precision floating-point in HPC, medium-precision fixed-point in communications, and low-precision fixed-point in DL. As a result, Intel has recently announced an AI-optimized FPGA, the Stratix 10 NX, which replaces conventional DSP blocks with AI tensor blocks [87]. The new tensor blocks drop the support for legacy DSP modes and precisions that were targeting the communications domain and adopt new ones targeting the DL domain specifically. This tensor block significantly increases the number of `int8` and `int4` MACs to 30 and 60 per block respectively, at almost the same die size [88]. Feeding all multipliers with inputs without adding more routing ports is a key concern. Accordingly, the NX tensor block introduces a double-buffered data reuse register network that can be sequentially loaded from a smaller number of routing ports, while allowing common DL compute patterns to make the best use of all available multipliers [89]. The next-generation Speedster7t FPGA from Achronix will also include a machine learning processing (MLP) block [90]. It supports a variety of precisions from `int16` down to `int3` in addition to `fp24`, `fp16` and `bfloat16` floating-point formats. The MLP block in Speedster7t will also feature a tightly coupled BRAM and circular register file that enable the reuse of both input values and output results. Each of these tightly integrated memory banks has a 72-bit external input but can be configured to have an up-to 144-bit output that feeds the MLP's multiplier arrays, reducing the number of required routing ports by $2\times$.

F. System-Level Interconnect: Network-on-Chip

FPGAs have continuously increased both in capacity and in the bandwidth of their external IO interfaces such as DDR, PCIe and Ethernet. Distributing the data traffic between these high-speed interfaces and the ever-larger soft fabric is a challenge. This system-level interconnect has traditionally been built by configuring parts of the FPGA logic and routing to implement *soft* buses that realize multiplexing, arbitration, pipelining and wiring between the relevant endpoints. These external interfaces operate at higher frequencies than the FPGA fabric can achieve, and therefore the only way to match their bandwidth is to use wider (soft) buses. For example, a single channel of high-bandwidth memory (HBM) has a 128-bit double data rate interface operating at 1 GHz, so a bandwidth-matched soft bus running at 250 MHz must be 1024 bits wide. With recent FPGAs incorporating up to 8 HBM channels [91] as well as numerous PCIe, Ethernet and other interfaces, system level interconnect can rapidly use a major fraction of the FPGA logic and routing resources. In addition, system-level interconnect tends to span large distances. The combination of very wide and physically long buses makes timing closure challenging

and usually requires deep pipelining of the soft bus, further increasing its resource use. The system-level interconnect challenge is becoming more difficult in advanced process nodes, as the number and speed of FPGA external interfaces increases, and the metal wire parasitics (and thus interconnect delay) scales poorly [92].

Abdelfattah and Betz [93]–[95] proposed embedding a hard, packet-switched network-on-chip (NoC) in the FPGA fabric to enable more efficient and easier-to-use system-level interconnect. Although a full-featured packet-switched NoC could be implemented using the soft logic and routing of an FPGA, an NoC with hardened routers and links is $23\times$ more area efficient, $6\times$ faster, and consumes $11\times$ less power compared to a *soft* NoC [95]. Designing a hard NoC for an FPGA is challenging since the FPGA architect must commit many choices to silicon (e.g. number of routers, link width, NoC topology) yet still maintain the flexibility of an FPGA to implement a wide variety of applications using many different external interfaces and communication endpoints. Work in [95] advocates for a mesh topology with a moderate number of routers (e.g. 16) and fairly wide (128 bit) links; these choices keep the area cost to less than 2% of the FPGA while ensuring the NoC is easier to lay out and a single NoC link can carry the entire bandwidth of a DDR channel. A hard NoC must also be able to flexibly connect to user logic implemented in the FPGA fabric; Abdelfattah et al. [96] introduced the *fabric port* which interfaces the hard NoC routers to the FPGA programmable fabric by performing width adaptation, clock domain crossing and voltage translation. This decouples the NoC from the FPGA fabric such that the NoC can run at a fixed (high) frequency, and still interface to FPGA logic and IO interfaces of different speeds and bandwidth requirements with very little glue logic. Hard NoCs also appear very well suited to FPGAs in datacenters. Datacenter FPGAs are normally configured in two parts: a *shell* provides system-level interconnect to the external interfaces, and a *role* implements the application acceleration functionality [9]. The resource use of the shell can be significant: it requires 23% of the device resources in the first generation of Microsoft’s Catapult systems [8]. Yazdanshenas et al. [97] showed that a hard NoC significantly improves resource utilization, operating frequency and routing congestion in such shell + role FPGA use cases. Other studies have proposed FPGA-specific optimizations to increase the area efficiency and performance of soft NoCs [98]–[100]. However, [101] shows that even optimized soft NoCs still trail hard NoCs in most respects (usable bandwidth, latency, area and routing congestion).

Recent Xilinx (Versal) and Achronix (Speedster7t) FPGAs integrate a hard NoC [102], [103] similar to the academic proposals discussed above. Versal uses

a hard NoC for system-level communication between various endpoints (Gigabit transceivers, processor, AI subsystems, soft fabric), and is in fact the only way for external memory interfaces to communicate with the rest of the device. It uses 128-bit wide links running at 1 GHz, matching a DDR channel’s bandwidth. Its topology is related to a mesh, but with all horizontal links pushed to the top and bottom of the device to make it easier to lay out within the FPGA floorplan. The Versal NoC contains multiple rows (i.e. chains of links and routers) at the top and bottom of the device, and a number of vertical NoC columns (similar to any other hard block columns such as DSPs) depending on the device size as shown in Fig. 18(a). The NoC has programmable routing tables that are configured at boot time and provides standard AXI interfaces [104] as its fabric ports. The Speedster7t NoC topology is optimized for external interface to

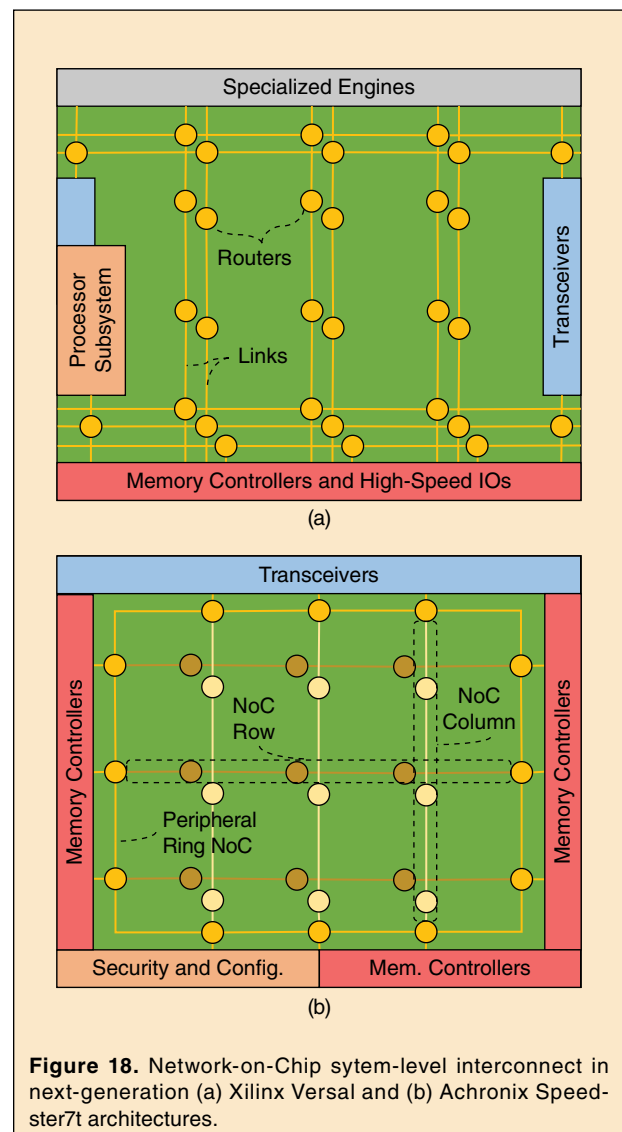


Figure 18. Network-on-Chip system-level interconnect in next-generation (a) Xilinx Versal and (b) Achronix Speedster7t architectures.

fabric transfers. It consists of a peripheral ring around the fabric with NoC rows and columns at regular intervals over the FPGA fabric as shown in Fig. 18(b). The peripheral ring NoC can operate independently without configuring the FPGA fabric to route the traffic between different external interfaces. There is no direct connectivity between the NoC rows and columns; the packets from a master block connecting to a NoC row will pass through the peripheral ring to reach a slave block connected to a NoC column.

G. Interposers

FPGAs have been early adopters of interposer technology that allows dense interconnection of multiple silicon dice. As shown in Fig. 19(a), a passive interposer is a silicon die (often in a trailing process technology to reduce cost) with conventional metal layers forming routing tracks and thousands of microbumps on its surface that connect to two or more dice flipped on top of it. One motivation for interposer-based FPGAs is achieving higher logic capacity at a reasonable cost. Both high-end systems and emulation platforms to validate ASIC designs before fabrication demand FPGAs with high logic capacity. However, large monolithic (i.e. single-silicon-die) devices have poor yield, especially early in the lifetime of a process technology (exactly when the FPGA is state-of-

the-art). Combining multiple smaller dice on a silicon interposer is an alternative approach that can have higher yield. A second motivation for 2.5 D systems is to enable integration of different specialized *chiplets* (possibly using different process technologies) into a single system. This approach is also attractive for FPGAs as the fabric's programmability can bridge disparate chiplet functionality and interface protocols.

Xilinx's largest Virtex-7 (28 nm) and Virtex Ultrascale (20 nm) FPGAs use passive silicon interposers to integrate three to four FPGA dice that each form a portion of the FPGA's rows. The largest interposer-based devices provide more than twice the logic elements of the largest monolithic FPGAs at the same process node. The FPGA programmable routing requires a large amount of interconnect, raising the question of whether the interposer microbumps (which are much larger and slower than conventional routing tracks) will limit the routability of the system. For example, in Virtex-7 interposer-based FPGAs, only 23% of the vertical routing tracks cross between dice through the interposer [105], with an estimated additional delay of ~1 ns [106]. The study in [105] showed that CAD tools that place the FPGA logic to minimize crossing of an interposer boundary combined with architecture changes that increase the switch flexibility to the interposer-crossing tracks can largely mitigate the impact of this reduced signal count. The entire vertical bandwidth of the NoC in the next-generation Xilinx Versal architecture (discussed in Section III-F) crosses between dice, helping to provide more interconnect bandwidth. An embedded NoC makes good use of the limited number of wires that can cross an interposer, as it runs its links at a high frequency and they can be shared by different communication streams as they are packet switched.

Intel FPGAs instead use smaller interposers called embedded multi-die interconnect bridges (EMIB) carved into the package substrate as shown in Fig. 19(b). Intel Stratix 10 devices use EMIB to integrate a large FPGA fabric die with smaller IO transceiver or HBM chiplets in the same package, decoupling the design and process technology choices of these two crucial elements of an FPGA. Some recent studies [107]–[109] used EMIB technology to tightly couple an FPGA fabric with specialized ASIC accelerator chiplets for DL applications. This approach offloads specific kernels of the computation (e.g. matrix-matrix or matrix-vector multiplications) to the more efficient specialized chiplets, while leveraging the FPGA fabric to interface to the outside world and to implement rapidly changing DL model components.

H. Other FPGA Components

Modern FPGA architectures contain other important components that we will not cover in detail.

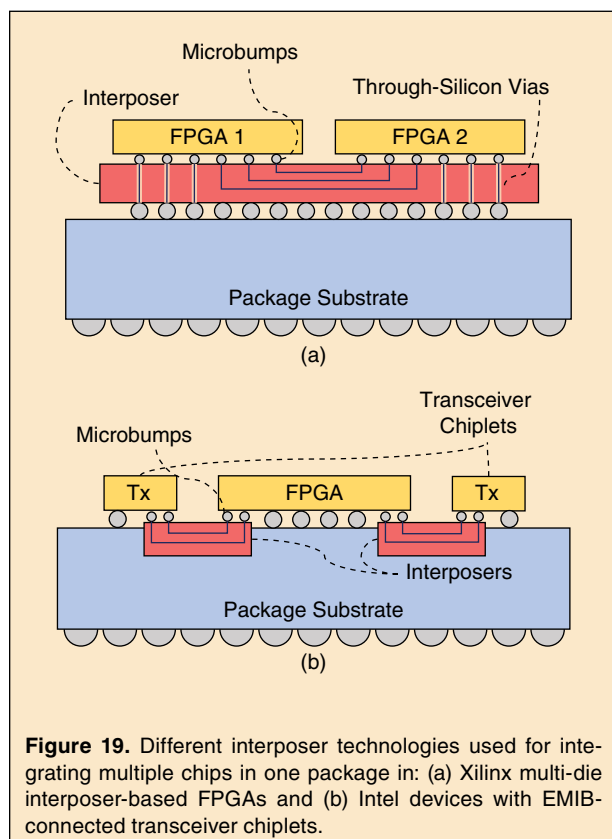


Figure 19. Different interposer technologies used for integrating multiple chips in one package in: (a) Xilinx multi-die interposer-based FPGAs and (b) Intel devices with EMIB-connected transceiver chiplets.

One such component is the *configuration circuitry* that loads the bitstream into the millions of SRAM cells that control the LUTs, routing switches and configuration bits in hard blocks. On power up, a configuration controller loads this bitstream serially from a source such as on-board flash or a hardened PCIe interface. When a sufficient group of configuration bits are buffered, they are written in parallel to a group of configuration SRAM cells, in a manner similar to writing a (very wide) word to an SRAM array. This configuration circuitry can also be accessed by the FPGA soft logic, allowing *partial re-configuration* of one part of the device while another portion continues processing. A complete FPGA application is very valuable intellectual property, and without *security measures* it could be cloned simply by copying the programming bitstream. To avoid this, FPGA CAD tools can optionally encrypt a bitstream, and FPGA devices can have a private decryption key programmed in by the manufacturer, making a bitstream usable only by a single customer who purchases FPGAs with the proper key.

Since FPGA applications are often communicating with many different devices at different speeds, they commonly include dozens of clocks. Most of these clocks are generated on-chip by programmable *phase-locked loops (PLLs)*, *delay-locked loops (DLLs)* and *clock data recovery (CDR) circuits*. Distributing many high frequency clocks in different ways for different applications is challenging, and leads to special interconnect networks for clocks. These *clock networks* are similar in principle to the programmable interconnect of Section III-B but use routing wire and switch topologies that allow construction of low-skew networks like H-trees, and are implemented using wider metal and shielding conductors to reduce crosstalk and hence jitter.

IV. Conclusion and Future Directions

FPGAs have evolved from simple arrays of programmable logic blocks and I/Os interconnected via programmable routing into more complex multi-die systems with many different embedded components such as BRAMs, DSPs, high-speed external interfaces, and system-level NoCs. The recent adoption of FPGAs in the HPC and datacenter domains, along with the emergence of new high-demand applications such as deep learning, is ushering in a new phase of FPGA architecture design. These new applications and the multi-user paradigm of the datacenter create opportunities for architectural innovation. At the same time, process technology scaling is changing in fundamental ways. Wire delay is scaling poorly which motivates rethinking programmable routing architecture. Interposers and 3D integration enable entirely new types of het-

erogeneous systems. Controlling power consumption is an overriding concern, and is likely to lead to FPGAs with more power-gating and more heterogeneous hard blocks. We do not claim to predict the future of FPGA architecture, except that it will be interesting and different from today!

Acknowledgments

The authors would like to thank Fynn Schwiegelshohn for valuable feedback, and the NSERC/Intel industrial research chair in programmable silicon and the Vector institute for funding support.



Andrew Boutros received his B.Sc. degree in electronics engineering from the German University in Cairo in 2016, and his M.A.Sc. degree in electrical and computer engineering from the University of Toronto in 2018. He was a research scientist at Intel's Accelerator Architecture Lab in Oregon before he returned to the University of Toronto where he is currently pursuing his Ph.D. degree. His research interests include FPGA architecture and CAD, deep learning acceleration, and domain-specific architectures. He is a post-graduate affiliate of the Vector Institute for Artificial Intelligence and the Center for Spatial Computational Learning. He received two best paper awards at Reconfig 2016 and FPL 2018.



Vaughn Betz received his B.Sc. degree in electrical engineering from the University of Manitoba in 1991, his M.S. degree in electrical and computer engineering from the University of Illinois at Urbana-Champaign in 1993, and his Ph.D. degree in electrical and computer engineering from the University of Toronto in 1998. He is the original developer of the widely used VPR FPGA placement, routing and architecture evaluation CAD flow, and a lead developer in the VTR project that has built upon VPR. He co-founded Right Track CAD to commercialize VPR, and joined Altera upon its acquisition of Right Track CAD. Dr. Betz spent 11 years at Altera, ultimately as Senior Director of software engineering, and is one of the architects of the Quartus CAD system and the first five generations of the Stratix and Cyclone FPGA families. He is currently a professor and the NSERC/Intel Industrial Research Chair in Programmable Silicon at the University of Toronto. He holds 101 US patents and has published over 100 technical articles in the FPGA area, thirteen of which have won best or most significant paper awards. Dr. Betz is a Fellow of the IEEE and the NAI, and Faculty Affiliate of the Vector Institute for Artificial Intelligence.

References

- [1] M. Hall and V. Betz, "HPIPE: Heterogeneous layer-pipelined and sparse-aware CNN inference for FPGAs," 2020, arXiv:2007.10451.
- [2] P. Yiannacouras et al., "Data parallel FPGA workloads: Software versus hardware," in *Proc. IEEE Int. Conf. Field-Programmable Logic Appl. (FPL)*, 2009, pp. 51–58.
- [3] M. Cummings and S. Haruyama, "FPGA in the software radio," *IEEE Commun. Mag.*, vol. 37, no. 2, pp. 108–112, 1999. doi: 10.1109/35.747258.
- [4] J. Rettkowski et al., "HW/SW co-design of the HOG algorithm on a Xilinx Zynq SoC," *J. Parallel Distrib. Comput.*, vol. 109, pp. 50–62, 2017. doi: 10.1016/j.jpdc.2017.05.005.
- [5] A. Bitar et al., "Bringing programmability to the data plane: Packet processing with a NoC-enhanced FPGA," in *Proc. IEEE Int. Conf. Field-Programmable Technol. (FPT)*, 2015, pp. 24–31.
- [6] H. Krupnova and G. Saucier, "FPGA-based Emulation: Industrial and Custom Prototyping Solutions," in *Proc. Int. Workshop on Field-Programmable Logic Appl. (FPL)*, Springer-Verlag, 2000, pp. 68–77.
- [7] A. Boutros et al., "Build fast, trade fast: FPGA-based high-frequency trading using high-level synthesis," in *Proc. IEEE Int. Conf. Reconfigurable Comput. FPGAs (ReConFig)*, 2017, pp. 1–6.
- [8] A. Putnam et al., "A reconfigurable fabric for accelerating large-scale datacenter services," in *Proc. ACM/IEEE Int. Symp. Comput. Architecture (ISCA)*, 2014, pp. 13–24.
- [9] A. M. Caulfield et al., "A cloud-scale acceleration architecture," in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2016, pp. 1–13.
- [10] J. Fowers et al., "A configurable cloud-scale DNN processor for real-time AI," in *Proc. ACM/IEEE Int. Symp. Comput. Architecture (ISCA)*, 2018, pp. 1–14.
- [11] I. Kuon and J. Rose, "Measuring the gap between FPGAs and ASICs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 26, no. 2, pp. 203–215, 2007. doi: 10.1109/TCAD.2006.884574.
- [12] A. Boutros et al., "You cannot improve what you do not measure: FPGA vs. ASIC efficiency gaps for convolutional neural network inference," *ACM Trans. Reconfigurable Technol. Syst. (TRETS)*, vol. 11, no. 3, pp. 1–23, 2018. doi: 10.1145/3242898.
- [13] S. Yang, *Logic Synthesis and Optimization Benchmarks User Guide: Version 3.0*. Microelectronics Center of North Carolina, 1991.
- [14] K. Murray et al., "VTR 8: High-performance CAD and customizable FPGA architecture modelling," *ACM Trans. Reconfigurable Technol. Syst. (TRETS)*, vol. 13, no. 2, pp. 1–55, June 2020. doi: 10.1145/3388617.
- [15] K. Murray et al., "Titan: enabling large and complex benchmarks in academic CAD," in *Proc. IEEE Int. Conf. Field-Programmable Logic Appl. (FPL)*, 2013, pp. 1–8.
- [16] H. Parandeh-Afshar et al., "Rethinking FPGAs: Elude the flexibility excess of LUTs with and-inverter cones," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2012, pp. 119–128.
- [17] H. Parandeh-Afshar et al., "Shadow AICs: Reaping the benefits of and-inverter cones with minimal architectural impact," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2013, pp. 279–279.
- [18] H. Parandeh-Afshar et al., "Shadow and-inverter cones," in *Proc. IEEE Int. Conf. Field-Programmable Logic Appl. (FPL)*, 2013, pp. 1–4.
- [19] G. Zgheib et al., "Revisiting and-inverter cones," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2014, pp. 45–54. doi: 10.1145/2554688.2554791.
- [20] V. Betz et al., *Architecture and CAD for Deep-Submicron FPGAs*. Springer Science & Business Media, 1999.
- [21] V. Betz and J. Rose, "How much logic should go in an FPGA logic block?" *IEEE Design Test Comput.*, vol. 15, no. 1, pp. 10–15, 1998. doi: 10.1109/54.655177.
- [22] G. Lemieux et al., "Generating highly-routable sparse crossbars for PLDs," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2000, pp. 155–164. doi: 10.1145/329166.329199.
- [23] C. Chiasson and V. Betz, "COFFE: Fully-automated transistor sizing for FPGAs," in *Proc. IEEE Int. Conf. Field-Programmable Technol. (FPT)*, 2013, pp. 34–41.
- [24] E. Ahmed and J. Rose, "The effect of LUT and cluster size on deep-submicron FPGA performance and density," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 12, no. 3, pp. 288–298, 2004. doi: 10.1109/TVLSI.2004.824300.
- [25] "Stratix II Device Handbook, Volume 1 (SI5V1.4.5)." Altera Corp., 2007.
- [26] D. Lewis et al., "The Stratix II logic and routing architecture," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2005, pp. 14–20.
- [27] T. Ahmed et al., "Packing techniques for Virtex-5 FPGAs," *ACM Trans. Reconfigurable Technol. Syst. (TRETS)*, vol. 2, no. 3, pp. 1–24, 2009. doi: 10.1145/1575774.1575777.
- [28] W. Feng et al., "Improving FPGA performance with a S44 LUT structure," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2018, pp. 61–66. doi: 10.1145/3174243.3174272.
- [29] "Versal ACAP Configurable Logic Block Architecture Manual (AM005 v1.0)," Xilinx Inc, 2020.
- [30] D. Lewis et al., "Architectural enhancements in Stratix V," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2013, pp. 147–156.
- [31] I. Ganusov and B. Devlin, "Time-borrowing platform in the Xilinx Ultrascale+ family of FPGAs and MPSoCs," in *Proc. IEEE Int. Conf. Field-Programmable Logic Appl. (FPL)*, 2016, pp. 1–9.
- [32] K. Murray et al., "Optimizing FPGA logic block architectures for arithmetic," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 6, pp. 1378–1391, 2020. doi: 10.1109/TVLSI.2020.2965772.
- [33] S. Yazdanehshenas and V. Betz, "Automatic circuit design and modeling for heterogeneous FPGAs," in *Proc. IEEE Int. Conf. Field-Programmable Technol. (ICFPT)*, 2017, pp. 9–16.
- [34] J. Chromczak et al., "Architectural enhancements in Intel Agilex FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2020, pp. 140–149.
- [35] S. Rasoulizadeh et al., "LUXOR: An FPGA logic cell architecture for efficient compressor tree implementations," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2020, pp. 161–171.
- [36] A. Boutros et al., "Math doesn't have to be hard: Logic block architectures to enhance low-precision multiply-accumulate on FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2019, pp. 94–103.
- [37] M. Eldafrawy et al., "FPGA logic block architectures for efficient deep learning inference," *ACM Trans. Reconfigurable Technol. Syst. (TRETS)*, vol. 13, no. 3, pp. 1–34, 2020. doi: 10.1145/3393668.
- [38] C. Chiasson and V. Betz, "Should FPGAs abandon the pass gate?" in *Proc. Int. Conf. Field-Programmable Logic Appl.*, 2013, pp. 1–8.
- [39] FlexLogix eFPGA. <https://flex-logix.com/efpga/>
- [40] V. Betz and J. Rose, "FPGA routing architecture: Segmentation and buffering to optimize speed and density," in *Proc. ACM Int. Symp. FPGAs*, 1999, pp. 59–68.
- [41] O. Petelin and V. Betz, "The speed of diversity: Exploring complex FPGA routing topologies for the global metal layer," in *Proc. IEEE Int. Conf. Field-Programmable Logic Appl. (FPL)*, 2016, pp. 1–10.
- [42] D. Lewis et al., "The Stratix routing and logic architecture," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2003, pp. 12–20.
- [43] X. Tang et al., "A study on switch block patterns for tileable FPGA routing architectures," in *Proc. IEEE Int. Conf. Field-Programmable Technol. (FPT)*, 2019, pp. 247–250.
- [44] G. Lemieux et al., "Directional and single-driver wires in FPGA interconnect," in *Proc. IEEE Int. Conf. Field-Programmable Technol. (FPT)*, 2004, pp. 41–48.
- [45] D. Lewis et al., "The Stratix 10 highly pipelined FPGA architecture," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2016, pp. 159–168.
- [46] B. Gaide et al., "Xilinx adaptive compute acceleration platform: Versal architecture," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2019, pp. 84–93. doi: 10.1145/3289602.3293906.
- [47] J. Tyhach et al., "A 90 nm FPGA I/O buffer design with 1.6 Gbps data rate for source-synchronous system and 300 MHz clock rate for external memory interface," in *Proc. IEEE Custom Integrated Circuits Conf.*, 2004, pp. 431–434.
- [48] N. Zhang et al., "Low-voltage and high-speed FPGA I/O cell design in 90nm CMOS," in *Proc. IEEE Int. Conf. ASIC*, 2009, pp. 533–536.
- [49] T. Qian et al., "A 1.25Gbps programmable FPGA I/O buffer with multi-standard support," in *Proc. IEEE Int. Conf. Integr. Circuits Microsyst.*, 2018, pp. 362–365.
- [50] P. Upadhyaya et al., "A fully-adaptive wideband 0.5–32.75Gb/s FPGA transceiver in 16nm FinFET CMOS technology," in *Proc. IEEE Symp. VLSI Circuits*, 2016, pp. 1–2.
- [51] "Implementing RAM functions in FLEX 10K Devices (A-AN-052-01)," Altera Corp., 1995.
- [52] K. Tatsumura et al., "High density, low energy, magnetic tunnel junction based block RAMs for memory-rich FPGAs," in *Proc. IEEE Int. Conf. Field-Programmable Technol. (FPT)*, 2016, pp. 4–11.

- [53] T. Ngai et al., "An SRAM-programmable field-configurable memory," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, 1995, pp. 499–502.
- [54] S. Wilton et al., "Architecture of centralized field-configurable memory," in *Proc. ACM Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 1995, pp. 97–103.
- [55] S. Yazdandshenas et al., "Don't forget the memory: Automatic block RAM modelling, optimization, and architecture exploration," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2017, pp. 115–124.
- [56] T. R. Halfhill, "Tabula's time machine," *Microprocessor Rep.*, vol. 131, 2010.
- [57] "Mercury programmable logic device family (DS-MERCURY-2.2)," Altera Corp., 2003.
- [58] C. E. LaForest and J. G. Steffan, "Efficient multi-ported memories for FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2010, pp. 41–50. doi: 10.1145/1723112.1723122.
- [59] C. E. LaForest et al., "Multi-ported memories for FPGAs via XOR," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2012, pp. 209–218. doi: 10.1145/2145694.2145730.
- [60] D. Lewis et al., "Architectural enhancements in Stratix-III and Stratix-IV," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2009, pp. 33–42.
- [61] R. Tessier, et al. "Power-efficient RAM mapping algorithms for FPGA embedded memory blocks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 26, no. 2, pp. 278–290, 2007. doi: 10.1109/TCAD.2006.887924.
- [62] B.-C. C. Lai and J.-L. Lin, "Efficient designs of multiported memory on FPGA," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 1, pp. 139–150, 2016. doi: 10.1109/TVLSI.2016.2568579.
- [63] H. Wong et al., "Comparing FPGA vs. custom CMOS and the impact on processor microarchitecture," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2011, pp. 5–14. doi: 10.1145/1950413.1950419.
- [64] E. Kadric et al., "Impact of memory architecture on FPGA energy consumption," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2015, pp. 146–155. doi: 10.1145/2684746.2689062.
- [65] L. Ju et al., "NVM-based FPGA block RAM with adaptive SLC-MLC conversion," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2661–2672, 2018. doi: 10.1109/TCAD.2018.2857261.
- [66] P. Longa and A. Miri, "Area-efficient FIR filter design on FPGAs using distributed arithmetic," in *Proc. IEEE Int. Symp. Signal Process. Inform. Technol.*, 2006, pp. 248–252.
- [67] P. K. Meher et al., "FPGA realization of FIR Filters by efficient and flexible systolization using distributed arithmetic," *IEEE Trans. Signal Process.*, vol. 56, no. 7, pp. 3009–3017, 2008. doi: 10.1109/TSP.2007.914926.
- [68] "Virtex-II platform FPGAs: Complete data sheet (DS031 v4.0)," Xilinx Inc, 2014.
- [69] C. R. Baugh and B. A. Wooley, "A two's complement parallel array multiplication algorithm," *IEEE Trans. Comput.*, vol. C-22, no. 12, pp. 1045–1047, 1973. doi: 10.1109/T-C.1973.223648.
- [70] "Using the DSP Blocks in Stratix & Stratix GX Devices (AN-214-3.0)," Altera Corp., 2002.
- [71] "XtremeDSP for Virtex-4 FPGAs (UG073 v2.7)," Xilinx Inc, 2008.
- [72] "DSP Blocks in Stratix III Devices (SIII51005-1.7)," Altera Corp., 2010.
- [73] "Stratix V Device Handbook Volume 1: Device Interfaces and Integration (SV-5V1)," Altera Corp., 2020.
- [74] "Virtex-5 FPGA XtremeDSP Design Considerations (UG193 v3.6)," Xilinx Inc, 2017.
- [75] "Virtex-6 FPGA DSP48E1 Slice (UG369 v1.3)," Xilinx Inc, 2011.
- [76] "UltraScale Architecture DSP Slice (UG579 v1.9)," Xilinx Inc, 2019.
- [77] H. Parandeh-Afshar and P. lenne, "Highly versatile DSP blocks for improved FPGA arithmetic performance," in *Proc. IEEE Int. Symp. Field-Programmable Custom Computing Mach. (FCCM)*, 2010, pp. 229–236.
- [78] A. Cevrero et al., "Field programmable compressor trees: Acceleration of multi-input addition on FPGAs," *ACM Trans. Reconfigurable Technol. Syst. (TRETSS)*, vol. 2, no. 2, pp. 1–36, 2009. doi: 10.1145/1534916.1534923.
- [79] M. Langhammer and B. Pasca, "Floating-point DSP block architecture for FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2015, pp. 117–125. doi: 10.1145/2684746.2689071.
- [80] S. Ahmad et al., "Xilinx First 7nm device: Versal AI core (VC1902)," in *Proc. Hot Chips Symp.*, 2019, pp. 1–28.
- [81] P. Gysel et al., "Hardware-oriented approximation of convolutional neural networks," 2016, arXiv:1604.03168.
- [82] N. Mellempudi et al., "Mixed low-precision deep learning inference using dynamic fixed point," 2017, arXiv:1701.08978.
- [83] A. Mishra et al., "WRPN: Wide reduced-precision networks," 2017, arXiv:1709.01134.
- [84] A. Boutros et al., "Embracing diversity: Enhanced DSP blocks for low-precision deep learning on FPGAs," in *Proc. IEEE Int. Conf. Field Programmable Logic Appl. (FPL)*, 2018, pp. 35–357.
- [85] S. Rasoulnezhad et al., "PIR-DSP: An FPGA DSP block architecture for multi-precision deep neural networks," in *Proc. IEEE Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM)*, 2019, pp. 35–44.
- [86] Intel Corp. "Intel Agilex variable precision DSP blocks user guide (UG-20213)," Intel Corp., 2020.
- [87] "Intel Stratix 10 NX FPGA: AI-optimized FPGA for high-bandwidth, low-latency AI acceleration (SS-1121-1.0)," Intel Corp., 2020.
- [88] L. Gwennapp, "Stratix 10 NX adds AI blocks," The Linley Group Newsletters, 2020.
- [89] A. Boutros et al., "Beyond peak performance: Comparing the real performance of AI-optimized FPGAs and GPUs," in *Proc. IEEE Int. Conf. Field-Programmable Technol. (FPT)*, 2020.
- [90] "Speedster7t machine learning processing user guide (UG088)," Achronix Corp., 2019.
- [91] "High Bandwidth Memory (HBM2) Interface Intel FPGA IP User Guide (UG-20031)," Intel Corp., 2020.
- [92] M. T. Bohr, "Interconnect scaling: The real limiter to high performance ULSI," in *Proc. Int. Electron Devices Meeting*, 1995, pp. 241–244.
- [93] M. S. Abdelfattah and V. Betz, "Design tradeoffs for hard and soft FPGA-based networks-on-chip," in *Proc. IEEE Int. Conf. Field-Programmable Technol. (FPT)*, 2012, pp. 95–103.
- [94] M. S. Abdelfattah and V. Betz, "The power of communication: Energy-efficient NoCs for FPGAs," in *Proc. IEEE Int. Conf. Field Programmable Logic Appl. (FPL)*, 2013, pp. 1–8.
- [95] S. Abdelfattah and V. Betz, "The case for embedded networks on chip on field-programmable gate arrays," *IEEE Micro*, vol. 34, no. 1, pp. 80–89, 2013.
- [96] M. S. Abdelfattah et al., "Take the highway: Design for embedded NoCs on FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2015, pp. 98–107. doi: 10.1145/2684746.2689074.
- [97] S. Yazdandshenas and V. Betz, "Quantifying and mitigating the costs of FPGA virtualization," in *Proc. IEEE Int. Conf. Field-Programmable Logic Appl. (FPL)*, 2017, pp. 1–7.
- [98] N. Kapre, and J. Gray, "Hoplite: A deflection-routed directional torus NoC for FPGAs," *ACM Trans. Reconfigurable Technol. Syst. (TRETSS)*, vol. 10, no. 2, pp. 1–24, 2017. doi: 10.1145/3027486.
- [99] M. K. Papamichael and J. C. Hoe, "CONNECT: Re-examining conventional wisdom for designing NoCs in the context of FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2012, pp. 37–46.
- [100] Y. Huan and A. DeHon, "FPGA optimized packet-switched NoC using split and merge primitives," in *Proc. IEEE Int. Conf. Field-Programmable Technol. (FPT)*, 2012, pp. 47–52.
- [101] S. Yazdandshenas and V. Betz, "Interconnect solutions for virtualized field-programmable gate arrays," *IEEE Access*, vol. 6, pp. 10,497–10,507, 2018. doi: 10.1109/ACCESS.2018.2806618.
- [102] I. Swarbrick et al., "Network-on-chip programmable platform in versal ACAP architecture," in *Proc. ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2019, pp. 212–221.
- [103] "Speedster7t network on chip user guide (UG089)," Achronix Corp., 2019.
- [104] "AMBA AXI and ACE protocol specification," ARM Holdings, Tech. Rep., 2013.
- [105] E. Nasiri et al., "Multiple dice working as one: CAD flows and routing architectures for silicon interposer FPGAs," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 24, no. 5, pp. 1821–1834, 2015. doi: 10.1109/TVLSI.2015.2478280.
- [106] R. Chaware et al., "Assembly and reliability challenges in 3D integration of 28nm FPGA die on a large high density 65nm passive interposer," in *Proc. IEEE Electronic Components Technol. Conf.*, 2012, pp. 279–283.
- [107] E. Nurvitadhi et al., "In-package domain-specific ASICs for intel Stratix 10 FPGAs: A case study of accelerating deep learning using tensorflow ASIC," in *Proc. IEEE Int. Conf. Field-Programmable Logic Appl. (FPL)*, 2018, pp. 106–1064.
- [108] E. Nurvitadhi et al., "Evaluating and Enhancing Intel Stratix 10 FPGAs for persistent real-time AI," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2019, pp. 119–119.
- [109] E. Nurvitadhi et al., "Why compete when you can work together: FPGA-ASIC integration for persistent RNNs," in *Proc. IEEE Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM)*, 2019, pp. 199–207.

FPGA Acceleration for Big Data Analytics: Challenges and Opportunities

Joost Hoozemans, Johan Peltenburg, Fabian Nonnenmacher,
Ákos Hadnagy, Zaid Al-Ars, and H. Peter Hofstee



©SHUTTERSTOCK.COM/WHITEMOCCA

Abstract

The big data revolution has ushered an era with ever increasing volumes and complexity of data requiring ever faster computa-

Digital Object Identifier 10.1109/MCAS.2021.3071608

Date of current version: 24 May 2021

tional analysis. During this very same era, CPU performance growth has been stagnating, pushing the industry to either scale their computation horizontally using multiple nodes in datacenters, or to scale vertically using heterogeneous components to reduce compute time. However, networking and storage continue to provide both higher throughput and lower latency, which allows for leveraging heterogeneous components, deployed in

data centers around the world. Still, the integration of big data analytics frameworks with heterogeneous hardware components such as GPGPUs and FPGAs is challenging, because there is an increasing gap in the level of abstraction between analytics solutions developed with big data analytics frameworks, and accelerated kernels developed with heterogeneous components. In this article, we focus on FPGA accelerators that have seen wide-scale deployment in large cloud infrastructures. FPGAs allow the implementation of highly optimized hardware architectures, tailored exactly to an application, and unburdened by the overhead associated with traditional general-purpose computer architectures. FPGAs implementing dataflow-oriented architectures with high levels of (pipeline) parallelism can provide high application throughput, often providing high energy efficiency. Latency-sensitive applications can leverage FPGA accelerators by directly connecting to the physical layer of a network, and perform data transformations without going through the software stacks of the host system. While these advantages of FPGA accelerators hold promise, difficulties associated with programming and integration limit their use. This article explores the existing practices in big data analytics frameworks, discusses the aforementioned gap in development abstractions, and provides some perspectives on how to address these challenges in the future.

I. Introduction

A lot has changed in the world of analytics since the first database management systems were popularized. Highly structured data was carefully administered, making it easy to distill information. Now, the field is confronted with a quintuple challenge: more data to process, more abstraction desired, new computationally intensive paradigms (like AI), and more latency sensitive (interactive) uses. At the same time, transistor performance improvements are slowing down and power efficiency becomes a greater and greater challenge.

Meanwhile, recent developments have also presented us with several new opportunities. Cloud infrastructure has opened up the use of large compute clusters to a far wider range of users by allowing more efficient sharing of hardware. Highly efficient compute units have been designed to target certain workloads in a highly specialized manner. The memory hierarchy has been extended with solid state drives and storage class memory, improving both throughput and latency for access to large datasets. Similarly, networking technology has been advancing rapidly, still keeping pace with Moore's law.

While the performance increase of general-purpose CPUs is not able to keep up with the advances in their periphery, reconfigurable logic found in FPGAs provides several unique strengths that may be able to cope better. For many applications, FPGAs have the ability to transform and filter data at the very high data rates that new storage systems can provide. When attached directly to the network, reconfigurable logic can provide similar functionality at line-rate with very low latency,

because data no longer needs to traverse various software layers of the networking and application stack. It allows users to specialize FPGA accelerators found in cloud instances to their applications, without the need to replace hardware. Tailoring the logic mapped on top of the FPGA fabric allows the construction of custom dataflow computers, where there is a minimal need to move data back and forth through the memory hierarchy. Even in cases where such FPGA designs provide only marginal gains in terms of throughput, the computations tend to be more power-efficient because of the reduced data movement and because the FPGA operates at relatively low clock frequencies.

However, FPGA acceleration still has several challenges. In spite of efforts to improve the ease of programming, FPGAs are currently not yet suitable to be used directly by an audience untrained in circuit design. Very few people with proper hardware design training are also active in the field of big data analytics, leading to a scarcity of designers. Furthermore, the use of FPGAs in cloud instances is currently still very explicit, requiring FPGA-aware code paths. Also, data coming from high-level languages running on interpreters and virtual machines needs to be transformed and copied sometimes several times during (de)serialization before it can be used in the accelerator. This often negates any advantage that can be gained by designing the accelerator in the first place. Last, there is still no systematic way to address the long synthesis time for datacenter-class FPGAs (where 12 hours is no exception) and the relatively long reprogramming delay (in the order of 100 ms).

In this article, we investigate these challenges and opportunities and summarize recent progress, and lay out a roadmap for further improvement. Ultimately, our goal is seamless integration of FPGAs in cloud infrastructure for big data analytics. Section II presents a more in-depth background of the related topics. Then, Section III discusses recent developments (opportunities) and missing links (challenges) for near-future big data analytics systems. Section IV presents a prototype system that features many of the components discussed. Section V provides an analysis and evaluation of our prototype implementation. Section VI provides a discussion of related work. Finally, Section VII concludes the article.

II. Background

A. Heterogeneous Computing

Increases in single-thread performance have dropped to less than 10% per year during the last decade [1] due in part to a slowdown in Moore's law. The additional transistors are not directly translating into raw performance anymore, but instead into higher core counts and, more

recently, more specialized compute units [2]. Modern processors often contain custom cores for compression, cryptography, media codecs, etc. These are better suited to meet the demands of higher performance at better energy efficiency. Graphics Processing Units (GPUs) have long been made available as a mainstream accelerator, targeting very specific (graphics processing) tasks at first but more recently being applied in a wider variety of workloads. More accelerators have been introduced, e.g. to target machine learning (Google's TPU [3]), or more irregular massively parallel applications (Graphcore's IPU).

In heterogeneous computing, parts of the workload are appropriately divided among multiple different but more specialized and efficient components than just the general-purpose CPU.

Not every application will have a compute fabric that provides a perfect match for it. FPGAs, however, create the opportunity to design a custom hardware implementation for a specific application with a fast time-to-market.

B. FPGA Computing

While FPGAs have been commonplace in certain application domains, and have been widely used for testing and prototyping ASIC designs, recent offerings from vendors (e.g. [4], [5]) are specifically targeting datacenters and cloud infrastructures.

There is roughly an order of magnitude overhead in both clock frequency and area compared to full-custom IC designs [6], since more transistors are required to implement the same function, due to the reconfigurable nature of the underlying FPGA fabric. In spite of this, FPGAs are becoming an important element in the datacenter strategies of several large cloud providers because often there is still potential for higher performance and/or higher energy-efficiency [7].

This potential comes from several factors:

- Custom datatypes (no need to use a 64-bit ALU for incrementing indexes that would only need a few bits)
- Custom memory hierarchies and connectivity (placing local memories in the most suitable location and providing a more optimal organization)
- No instructions (this means saving caches and memory bandwidth, and it also means no logic required for instruction decoding etc.)
- Very deep pipelining (dataflow designs can have pipeline depths of thousands of stages, all of which are performing useful work at all times)
- Abundant parallelism (pipelines can be duplicated as long as there is FPGA logic available)
- Low clock frequencies (resulting in significant reduction in power consumption)

These factors contrast sharply with a general-purpose processor, which spends the majority of energy

and logic moving data around between cores, caches and registers, decoding instructions and resolving dependencies. In comparison, a much larger fraction of FPGAs logic can be utilized for actual operations in a dataflow fashion, minimizing data movements between subsequent compute steps (performed by adjacent compute units on the chip).

However, FPGAs also have a number of drawbacks. Compared to CPUs and GPUs, the development cycles are much longer and require logic design expertise. Furthermore, synthesizing a design for an FPGA can take many hours. The benefits are further reduced when implementing control-heavy and highly complex algorithms.

1) Programming Practices

The adoption of this new form of compute fabric has been slow, mainly because of the difficulty associated with programming. The traditional way of programming an FPGA is by using an RTL language, such as VHDL or Verilog, developed to design ASICs. This is a tedious and time-consuming process that requires a very specialized set of skills and knowledge. High-Level Synthesis (HLS) aims to decrease both design time and the level of knowledge required (targeting more traditional software developers by using the C/C++ programming language). However, studies show that the decrease in time to achieve a first functional design is often paired with an increase in time required to tune for high performance. Additionally, unlike conventional compilers for CPUs that now typically outperform hand-written assembly code, HLS compilers still tend to achieve a lower quality result (in terms of performance and FPGA resource utilization) compared to a hand-written RTL version [8]. This could partially be attributed to the mismatch between the programming paradigm and the FPGA fabric. The imperative programming paradigm targets a single sequential machine with a large uniformly accessible memory and aims to provide it with an explicit instruction every cycle (*temporal* programming). The FPGA fabric, on the other hand, is by nature a *spatial* resource. Instead of instructions, programming an FPGA requires specifying the placement of the operations and connections between them. The 'program' is executed implicitly by simply flowing data through the circuit (hence the term 'dataflow' computing).

There exist efforts at various levels of abstraction aiming to provide a more suitable programming method for FPGAs, from high-level parallel constructs [9] to dataflow-oriented [10] and from adding a timing-agnostic layer to an RTL language [11] to using a functional language [12] for circuit generation.

2) Integration

There are two facets to discuss in terms of integration: 1) how to integrate newly created circuits into existing designs and provide re-usability between these designs, and 2) how the custom-designed FPGA circuits can be employed by software. Although it might seem to be just a matter of practicality, the amount of time a developer needs to spend on integration is alarmingly large. The time a designer needs for writing the actual compute kernel that will perform the operations is, even when considering an RTL design method, not that long. However, the amount of time needed to implement and debug the communication between all the components (both hardware and software) can be considerable.

Luckily, both vendors and the open source community have spent a great deal of effort to provide infrastructure aiming to alleviate this. For example, Gaisler's GRLIB provides open source IP cores for various components needed in a SoC environment, with a bus infrastructure that is easy to interface with. More recently, ARM's Advanced Xtensible Interface (AXI) has become the de-facto bus standard for creating or connecting to existing IP cores, and is supported by FPGA vendor tool-chains.

Although AXI has been a big help in standardizing hardware interfaces, it is still a primarily byte-oriented protocol developed for hardware designs, which means that 1) it is not very accessible for software developers and 2) custom glue logic is almost always necessary to integrate existing IP blocks into new designs or the other way around. As it turns out, the effort of creating glue logic, including for example arbitration and buffering (finding optimal buffer sizes in dataflow graphs is a research topic in itself [13]), is not to be underestimated.

In terms of software integration, the datacenter accelerator cards provide additional infrastructure in the form of a 'shell', which communicates with a standard driver so that designers only need to provide a set of interfaces on their hardware components and can subsequently communicate with them through a software API.

C. Performance Scaling

Figure 1 shows how performance of I/O technologies is scaling versus that of main memory (DRAM). It provides two insights that are necessary to illustrate the upcoming challenge in performance scaling: Bandwidths provided by networking and storage technologies are both scaling very rapidly, especially since the introduction of solid-state storage devices such as SSDs. DRAM, however, is not able to deliver. Where networking and storage performance is scaling at rates similar to Moore's law, the throughput delivered by DRAM has been doubling roughly every 26 months, and this rate is decreasing.

Adding more processor cores will not solve this problem. One could already argue that CPU core counts are unable to scale with Moore's law because of the additional logic needed for caches and inter-core communication. The more pressing matter is that all those cores will not be able to access data in main memory fast enough. The result is that "The poor processor is now getting sandwiched between these two exponential performance growth curves of flash and network bandwidth" [14].

In the upcoming years, this discrepancy can be mitigated by scaling out; distributing the growing data volumes over increasing numbers of servers. However, since there is a continuous difference in scaling, this solution will not be viable for long. It seems inevitable that high-performance datapaths with CPUs in them will have to be designed around better alternatives.

D. Big Data Analytics

As *vertical* scaling (adding compute resources to a single node) is not able to satisfy the growing demands for performance, *horizontal* scaling is becoming the new approach for tackling very large datasets. Various frameworks that hide the complexity of distributing storage and computations over large numbers of nodes (for example, Spark and Hadoop) quickly grew in popularity. One of the more challenging aspects that these frameworks take care of is resiliency that is needed for this distribution - in some cases this even made it possible to use commodity hardware as opposed to costly server-grade systems. The other catalyst fueling the adoption

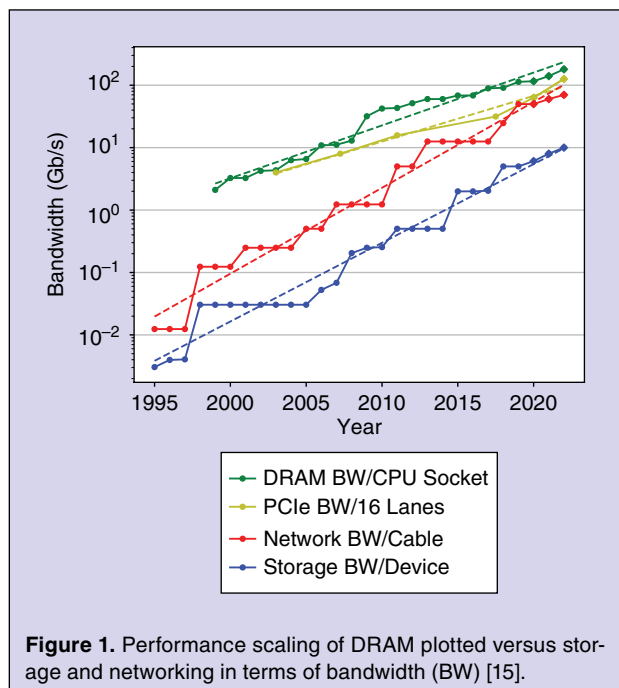


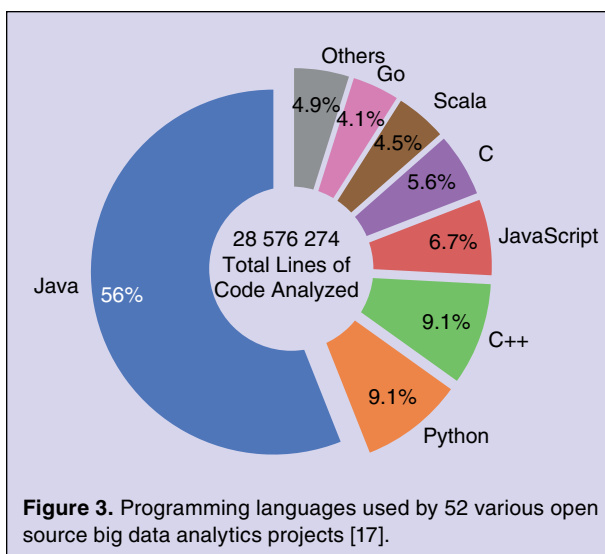
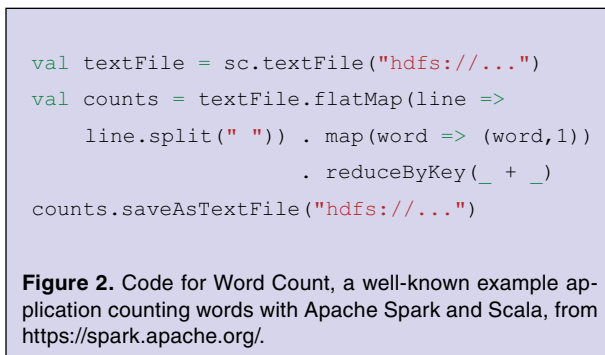
Figure 1. Performance scaling of DRAM plotted versus storage and networking in terms of bandwidth (BW) [15].

was the ease of programming. The popular frameworks provide very high-level programming interfaces to their users, making large compute clusters accessible to data scientists without needing lots of programming expertise in programming distributed systems.

1) Programming Practices

In contrast to the days of C programming, when very expensive computers were shared by many programmers who were writing highly optimized code, computing hardware has become relatively cheap and programmers relatively expensive. Big data frameworks have made it easy to increase performance by simply adding more nodes. In combination with the increasing importance of time-to-market, this has resulted in the emphasis being placed on productivity and ease of programming, while relying on adding hardware for achieving performance (instead of optimizing code).

Programming productivity is achieved by using modern languages employing increasingly high levels of abstraction. Figure 2 shows an example of a Spark program using its Resilient Distributed Dataset API. The input file can be TeraBytes in size, distributed across several



nodes, but this is all hidden from the user. These abstractions come at a cost; they move the computation further away from the bare metal. The consequence is twofold. First the often virtualized runtime environments of modern programming languages (for example, Spark uses Scala, which runs in a Java Virtual Machine) generally achieve less performance than languages compiled to native machine code (although concepts such as Just-In-Time compilation and the use of highly optimized libraries have at least partially addressed this). Second, the data itself is encoded in a way that is specific to the particular compute environment/programming language. While the execution performance limitations of interpreted languages can be largely mitigated, memory management presents a greater challenge, especially in the context of acceleration [16].

Figure 3 shows that about 80% of the frameworks rely on languages that use some form of automatic memory management. This means that many types of data that would have a very straightforward organization in C/C++ would be wrapped in an object (that would, for example, include reference counts for garbage collection). To send an array of these objects to a different analytics framework or to an accelerator, each one would need to be unwrapped and placed into an intermediate array with an encoding that the other party understands (serialization), followed by the reversed process on the other side (deserialization). This could be needed when using different frameworks or languages for certain parts of the computation.

2) Integration

The use of specific internal data representations makes integration of heterogeneous compute units difficult, because the data needs to be extracted and stored in a format that every accelerator can understand. This may be one of the reasons that adoption of heterogeneous accelerators in analytics has been slow in general, even for GPUs. Even though GPUs have been gaining popularity in datacenters for a variety of workloads including machine learning, there is only a handful of efforts in integrating them into analytics frameworks and supporting them in the mainline code repositories of popular open source frameworks such as Spark. Several research and non-upstream versions exist, but companies will (understandably!) be very cautious in deploying these in production.

E. Apache Arrow

A number of industry and academic efforts aim to address the overhead associated with data serialization and deserialization that is needed for interoperability. In this paper we focus on the Apache Arrow project, which proposes that no matter how you program your

application, the data should be represented the same way in memory. This way, data does not need to be (de)serialized when passed between different software technologies (e.g. Java and Python).

In addition, the format uses a *column*-oriented layout as opposed to the *row*-oriented layout used in traditional systems. Although the choice to organize tables in a row- or column-oriented fashion has pros and cons depending on the way it is accessed (e.g. retrieving a full row of data is very efficient using a row-oriented format, but requires several random memory accesses in a columnar format), in the context of data analytics storing the data in a columnar format where data of the same type is stored contiguously has a number of advantages [18]. The first advantage is that this allows computations on this data to take advantage of SIMD instructions that are supported in most contemporary hardware. Several values are loaded into wide SIMD registers in the CPU, after which operations can be performed on all of them simultaneously using a single instruction. The Gandiva execution engine that targets Arrow-formatted data makes use of the LLVM compiler framework to generate highly optimized SIMD code for CPUs using JIT compilation.

The second advantage of using a columnar format is that it allows to send buffers of data containing a column of values of the same type to accelerators such as GPUs and FPGAs. This allows these columns to be streamed into functional units in a straightforward fashion, without the need for decoding values from a row-oriented buffer of entries that can contain complex, nested and even variable-length datatypes. Whereas row-oriented formats continue to be commonplace in transactional systems (that need to process frequent record updates), analytics systems tend to work in a column-oriented fashion.

To summarize, Apache Arrow not only facilitates interoperability between software, but also between different types of compute hardware.

III. Opportunities and Challenges

In our view, the success of an analytics system in the near future will revolve around leveraging a number of key properties listed in Table I. In the following sections, we will discuss recent developments in the areas of Hardware, Programming and Runtime, that we believe will provide opportunities for building these systems. Then, we will provide views on current efforts to address challenges or missing links in the field.

A. Opportunities: Recent Developments in Big Data Analytics Systems

1) Hardware

In the area of computer hardware, recent developments have included several new components and interconnect technologies that will allow executing parts of the workload in the location and compute fabric that are most suitable for it.

Heterogeneous Compute Fabrics

In addition to the general-purpose GPU offerings that have become popular in recent years, FPGA-based datacenter accelerator cards have been introduced that can be integrated into existing servers [4]. In the near future, these accelerators will contain several types of compute fabric, including not only reconfigurable logic but also digital signal processing units, scalar processors and a grid of VLIW processors connected by a Network-on-Chip [19].

FPGA Overlays

As synthesis can take several hours especially for larger FPGAs, using a programmable overlay can provide a middle ground between performance and compilation time. An overlay is a fixed FPGA design containing compute elements that are highly optimized for a specific application or application domain. These elements can be programmed, and are typically supported by a compiler or other tool-chain that does not require

Table I.
Key system properties.

Hardware	• Heterogeneous (various types of compute fabric) • Uniform memory access • Accelerated storage and networking
Programming	• Spatial (functional, declarative, etc.) programming paradigms • Graph representation (DAG) of the application should be scheduled optimally for the (heterogeneous) hardware • Allow easy integration of user-designed accelerators • Allow acceleration using IP libraries • A single common representation of data in memory (no serialization)
Runtime	• Monitor performance of running tasks • Optimize performance by 1) synthesizing new components and storing them in the IP library, 2) updating parameters of accelerators according to the changing data characteristics

synthesis. Notable examples are the Catapult project [20] that accelerated the Bing search engine in 2012 and various machine learning accelerator overlays such as the Deep Learning Processing Unit (DPU) overlay available in Xilinx' Vitis toolset [21].

Accelerators Attached to Storage and Networking

The IBM Netezza system placed FPGAs between memory and storage to do (de)compression and pre-processing [22]. Similar hardware is now commonly available for custom use, in the form of storage-attached FPGA cards

such as the Nallatech 250SoC [23], and 'smart' SSDs available from Samsung [24]. On the networking side, SmartNICs provide custom programmable acceleration and virtualization without the need for the CPU to intervene [25]. These technologies will be key in helping the CPUs cope with the increasing throughput from storage and memory.

Interconnect

For some workloads, connecting an accelerator via PCIe is sufficient, if copying buffers can be overlapped with computation. However, many workloads that could otherwise be accelerated are bound by the limited bandwidth and the high latency. Additionally, if the CPU needs to orchestrate all transfers, latency can become very high for all but the most straightforward memory access patterns.

Next-generation interfaces such as CXL and OpenCAPI provide transparent, coherent access to main memory, making accelerators a peer of the CPU instead of a second class citizen. The current revision of OpenCAPI (3.0) provides throughput up to 25.6 GB/s at a latency of 378 ns [26, p40].

High-Bandwidth Memory (HBM)

As FPGAs have limited on-chip memory capacity, buffering enough data can be challenging. Current FPGAs have HBM integrated in the same package, providing several GBs of memory that can be used to buffer large chunks of data. HBM can be accessed by several ports simultaneously, allowing it to feed data into parallel kernels without using precious on-chip FPGA memory resources [27].

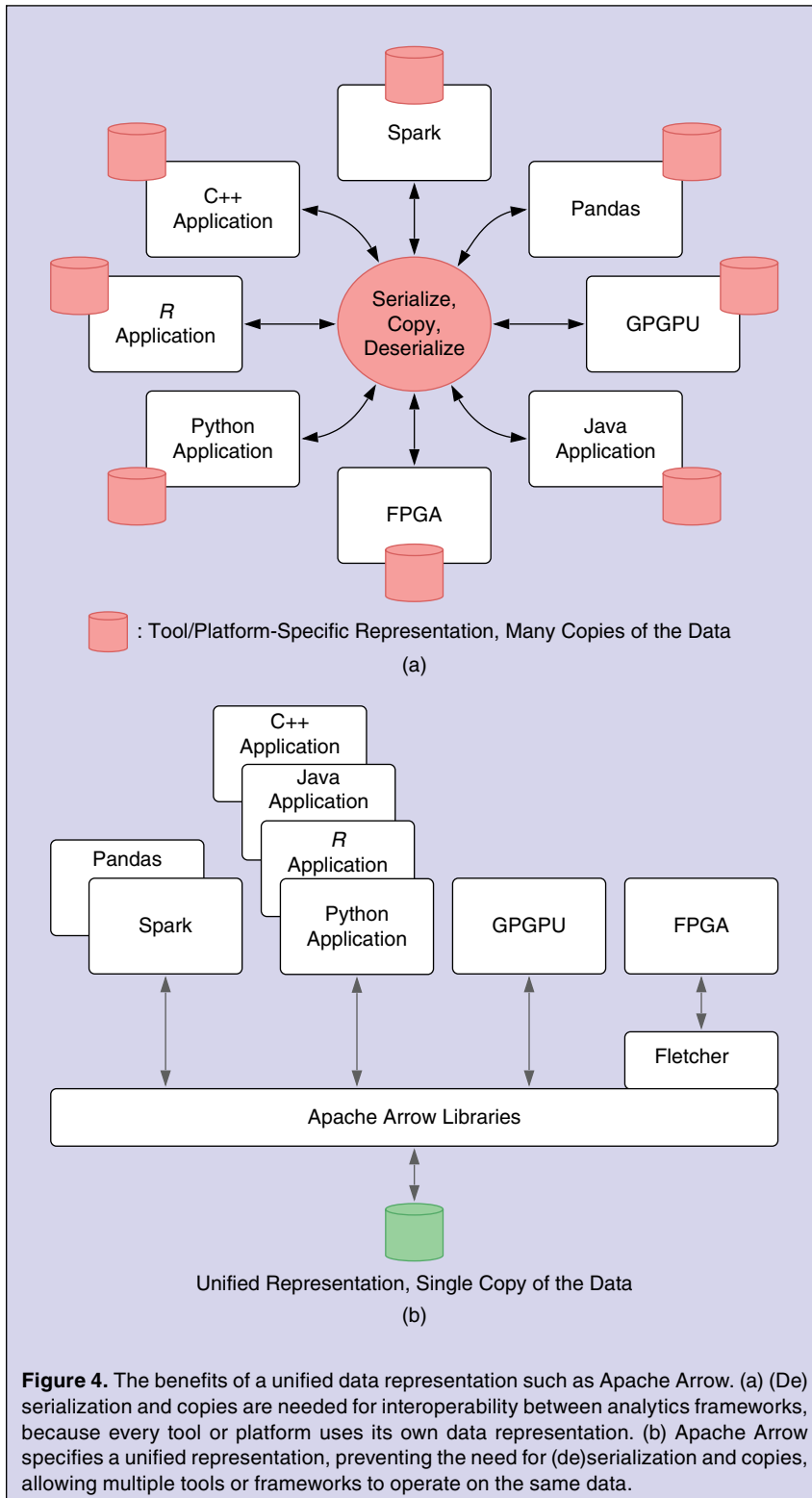


Figure 4. The benefits of a unified data representation such as Apache Arrow. (a) (De) serialization and copies are needed for interoperability between analytics frameworks, because every tool or platform uses its own data representation. (b) Apache Arrow specifies a unified representation, preventing the need for (de)serialization and copies, allowing multiple tools or frameworks to operate on the same data.

2) Programming

In spite of the gap between rising levels of abstraction and increased difficulty to design low-level accelerators, there are some developments that can aid in programming near-future analytics systems.

Parallel Programming

Although many efforts in the area of parallel programming have not yet proven to be a silver bullet, and likewise OpenCL did not turn out to be the fit-for-all solution for programming accelerators, we do believe that certain properties of the workloads and the approaches in programming in the big data world provide some opportunities for acceleration. First, we note that a cluster is inherently very spatial in nature, bringing it conceptually closer to the FPGA. This is reflected in the way they are being programmed; declarative (SQL) and functional aspects (Scala) are being used pervasively. MapReduce, a highly popular programming model, maps very naturally to a spatial resource such as a cluster but also to FPGAs [28].

In the meantime, optimization frameworks such as Halide [29], initially developed for image processing, and TVM [30] targeting machine learning, have taken the route of separating the specification of functionality from the implementation (the *schedule*). The actual code is generated for the target platform, allowing design-space exploration and keeping platform-specific optimizations outside of the base program. Taking this concept a step further, efforts such as LIFT & ELEVATE [31], Temporal to Spatial (T2S) [32] and HeteroCL [33] propose a separate mechanism to describe the mapping of the program onto the target platform and the optimizations.

Distributed Computing

Analytics frameworks distribute data over a large number of nodes in a resilient way, both in storage and computing [34].

Dataflow Representation

Analytics frameworks represent applications as a Directed Acyclic Graph (DAG), which can be used by a scheduler to analyze which parts of the program to execute where. The DAG-representation is a much closer match for the dataflow-oriented nature of FPGAs.

Unified Memory Layout

By using Apache Arrow, the memory of an analytics program can be accessed by all types of compute fabric without needing to copy or convert.

3) Runtime

Fletcher [35] allows FPGAs to be integrated with big data frameworks, as long as the Arrow unified memory layout is used. As depicted in Figure 5, it generates

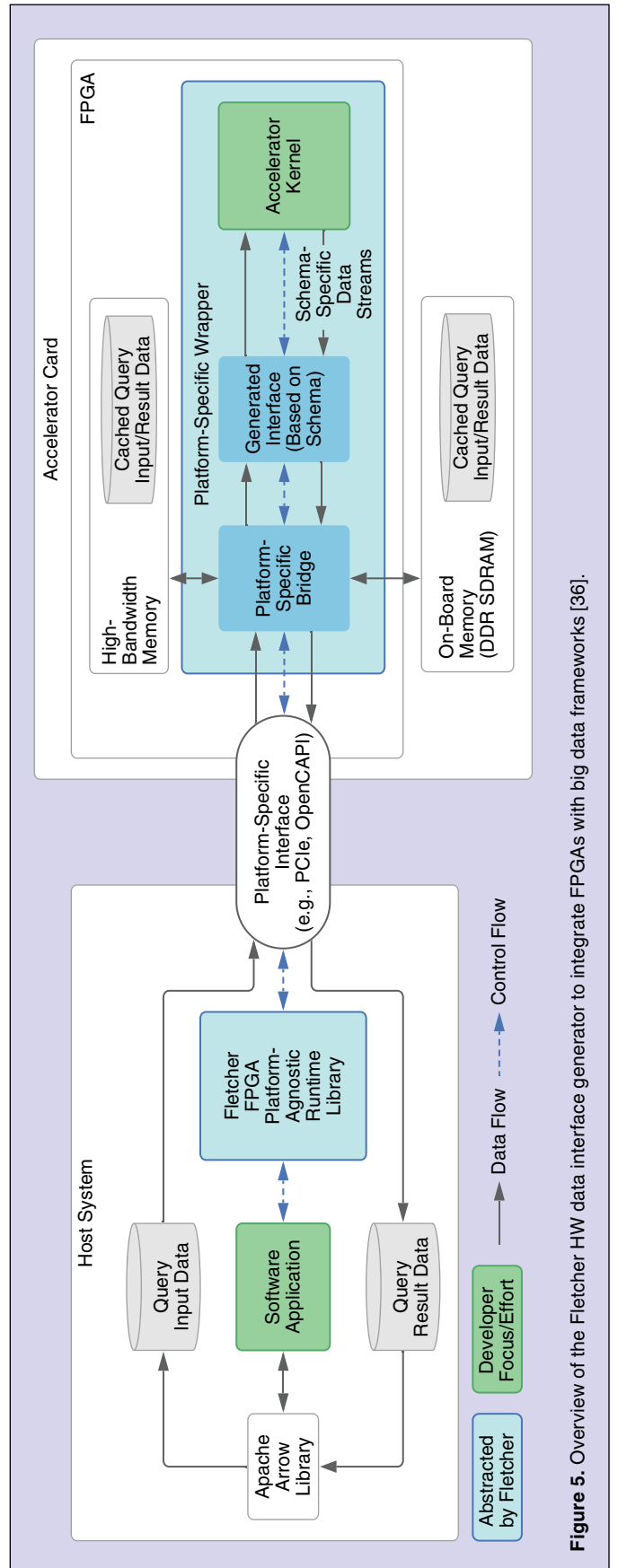


Figure 5. Overview of the Fletcher HW interface generator to integrate FPGAs with big data frameworks [36].

interfaces based on the *schemas* of the data. The schema describes the layout of a table, including for example the columns and datatypes.

During runtime, the data is organized in ‘chunks’ of tables named *RecordBatches* in Arrow terminology. Fletcher hides the complexity of having to translate the streams of elements from the *RecordBatches* to actual memory addresses in the host main memory (or worse, in DRAM on the FPGA board), and requesting the needed transfers. As a result, the kernel developer can work on the level of streams of datatypes such as strings and integers, instead of byte values.

B. Missing Links

1) Hardware

We recognize a large and rapidly increasing discrepancy between the high-level software and the low-level hardware. In software, complex datastructures are common with data stored in datatypes of possibly variable length (e.g. strings), using arrays that could even be nested. In contrast, AXI interfaces used in hardware provide a very simplistic way to transfer sequences of bytes, either in a memory-mapped or streaming fashion. It is up to the design to create a mapping between these, extracting the relevant data using custom glue logic that will likely change between applications.

A recently introduced specification called Tydi (Typed Dataflow Interfaces) [37] aims to address this discrepancy. With file formats specifying data layout on disk and Apache Arrow specifying data layout in memory, the Tydi standard specifies data layout ‘on the wire’ in digital hardware.

2) Programming

Ideally, accelerator kernels should be generated from the application’s execution graph. Alternatively, a designer should be able to develop and integrate such a component quickly. One could envision a library of such components, where the transformations that each is capable of is known to a scheduler. This scheduler can then recognize whether certain parts of the execution graph already have an accelerated implementation available in the library.

An open question remains on what granularity to generate and store these components, particularly regarding the synthesis times associated with large FPGAs. Although big data applications can exhibit extensive runtimes, a latency of several hours to generate a bitstream may still prove prohibitive. Design choices include storing complete bitstreams or making use of partial reconfigurability in combination with an on-FPGA Network-on-Chip structure.

3) Runtime

Partitioning and Mapping

With the ability to design kernels to accelerate parts of an analytics application, comes the need to detect which parts to accelerate. For a heterogeneous system to be truly holistic, the scheduler responsible for this mapping needs to take into account not only the application graph in terms of transformations to be applied to the data, but also the amount and location of this data. If accelerating a subgraph or operation on a hardware accelerator means that communication overhead will become a limiting factor, a different partitioning should be considered.

Design-Space Exploration (DSE)

Having more parameters means having more choices and in this context this means that the design space increases in size. Scheduling the DAG is only one aspect of the problem. When instantiating a kernel on an FPGA, there is a myriad of choices to be considered, such as the parallelism within each component versus the number of instances [38], the interface widths between them, tiling [39], etc.

Monitoring and Auto-Adaptation

Big data applications can run for extended amounts of time and the characteristics of the data can vary during this time. Based on these changing characteristics, the system parameters that were chosen by the DSE may no longer be optimal. Therefore, a runtime monitoring and adaptation loop can prevent operating in a mismatching configuration for extended periods of time. By sampling performance counters on the interfaces between kernel components, the runtime can identify which components and interfaces are over/underutilized and trigger a reconfiguration accordingly.

IV. FPGA-Accelerated Big Data System

For the purpose of demonstration, we have implemented a proof-of-concept that incorporates many of the key elements highlighted in the previous section. The system is based on the popular Apache Spark framework on the software side, and on an OpenCAPI-based POWER9 platform with Xilinx VU37P FPGA on the hardware side. First, we will provide a high-level description. Then, we will discuss the system in terms of how it can be programmed, and how its components were integrated.

A. Architecture

The goal of our envisioned system is to provide seamless integration of FPGA accelerated applications in scale-out frameworks for big data analytics, such as Apache Spark. So far, we have illustrated the various gaps, mostly in programming and integration, that need

to be closed to achieve this goal. A conceptual depiction of our proposed approach is shown in Figure 6.

On the left side of the figure, various computational steps of an analytics application are shown, represented as a DAG, consisting of multiple *stages*. Each stage performs a certain set of transformations on the data, until it is required to obtain data from partitions that may not be locally available (i.e. the data sits on other nodes). In the figure, the shuffle is required by the `ReduceByKey` transformation, which must gather all data items with the same key. As these items may be spread out over various partitions sitting on various cluster nodes, this transformation is called a *wide* transformation. Spark is especially good in taking care of many issues related to wide transformations, essentially dealing with general issues of scaling out computation over multiple nodes. Some related features are:

- Partitioning large data sets over multiple nodes.
- Distributing and optimizing descriptions of the desired computations.
- Scheduling the desired computations on all nodes.
- Recovering from node failures, through its resilience features.
- Offering an extensive set of libraries with algorithm implementations that perform better at scale.

Because such issues are dealt with by Spark itself, our system focuses only on improving transformations that are *narrow*, i.e. all data required for the computational steps is in a data partition local to the node that hosts the accelerator.

In our proposed system, the scheduler analyzes the DAG to see whether there are (parts of, or sub-) stages that can be efficiently accelerated on FPGA. It does so by searching a library that contains models of components that can implement the same parallel pattern and elemental function, depicted in the top-right corner of Figure 6. In the example, the Map pattern is applied, transforming each element in the dataset consisting of strings by applying the elemental function $x \rightarrow \{x, 1\}$. In other words, a string is transformed into a tuple of the string and the integer 1. There is a component model in the library that implements the Map pattern, with the elemental function ‘tuplicating’ the element with a constant.

Because the previous transformation (the `FlatMap`) also has a hardware implementation, the synthesis stage can compose components according to the longest possible acceleratable part of the DAG within a single stage, to end up with a hardware accelerator design such as illustrated in the bottom-right corner of Figure 6. If the FPGA is directly attached to storage, it can absorb the file loading step and perform the first parts of the computation before any data has touched main memory yet (similar to predicate pushdown in database systems).

Furthermore, OpenCAPI allows peripheral devices (I/O and accelerators) to transfer data between each other without going through main memory. That means that for example a network adapter could send data directly to an FPGA accelerator card.

It may happen that a certain design was previously synthesized to an FPGA bitstream, in which case the FPGA can be quickly reconfigured by loading the bitstream from a cluster-wide library or cache. If it was not previously synthesized, or if some synthesis-time parameters change, it must be resynthesized. We stipulate that in large clusters, the overhead of doing the full fledged FPGA synthesis is relatively low, because it only has to happen on one node out of potentially thousands. Furthermore, we also stipulate that workloads in big data clusters are often either long-running, or recurrent, and as such, the latency of bitstream synthesis may not pose as much of a problem as in other application domains.

After the correct bitstream has been obtained, the scheduler transforms the DAG into an FPGA-accelerated version, where the sub-stage that was implemented in the synthesized hardware accelerator is replaced by a software function that properly interfaces the data into the synthesized accelerator.

In a more advanced setup, the synthesized accelerator would be profiled to measure whether it actually speeds up the computation, such that it could be unloaded when the computation is not faster. In addition, the scheduler should take into account data movements and consider transforming the DAG to place accelerated transformations together. These efforts are left for future work.

We finally emphasize that describing the DAG (as a software programmer would), is closer to dataflow programming than traditional imperative programming, even though it is captured in languages that may also be used in the imperative style. This is useful for our system, because it allows many sorts of sub-stages to be easier to synthesize, as hardware dataflow designs can follow the structure of the DAG.

B. Programming Practices

For the high-level programming patterns from the Spark program to be synthesized to an FPGA, the following elements are needed:

- The datasets residing in Spark need to be accessible by the FPGA accelerator kernel. As discussed, this can be done by using the Apache Arrow format for the data sets (i.e. the `RecordBatches`) in memory, in turn leveraging Fletcher to generate streaming hardware interfaces for the hardware kernel implementation.
- Once on the FPGA, the data needs to flow between the components implementing the DAGs

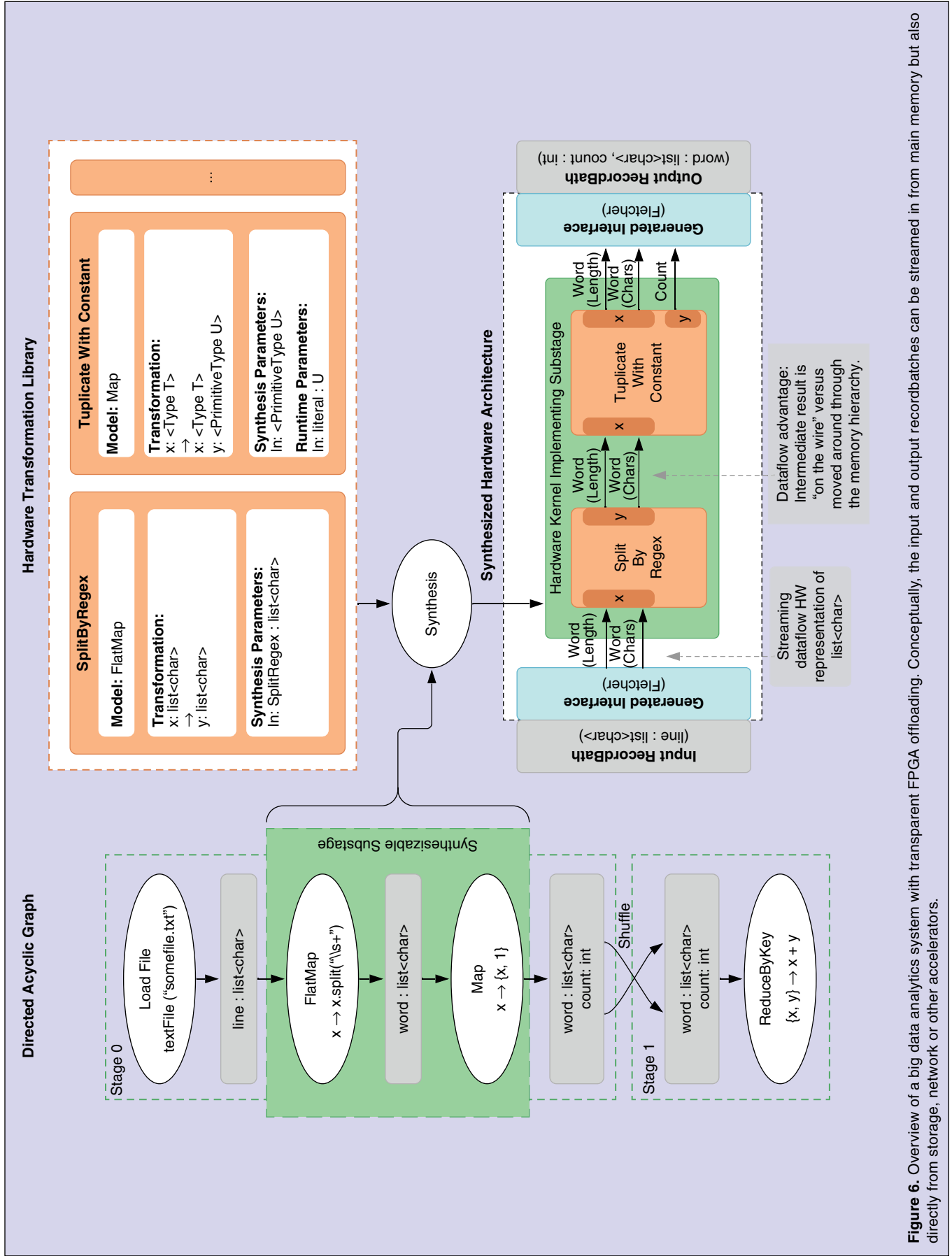


Figure 6. Overview of a big data analytics system with transparent FPGA offloading. Conceptually, the input and output recordbatches can be streamed in from main memory but also directly from storage, network or other accelerators.

transformations. Since the data may be complex and dynamically sized, a mechanism is required to consistently represent and transport such data structures over hardware streams. The recently introduced Tydi work aims to address this issue. It introduced an interface specification for streaming complex and dynamically sized data structures in hardware.

- The components that implement the parallel patterns (e.g. Map) and the elemental function that is to be performed on each element (e.g., the ‘tuplication’ function), need to be generated and properly connected according to the DAG.

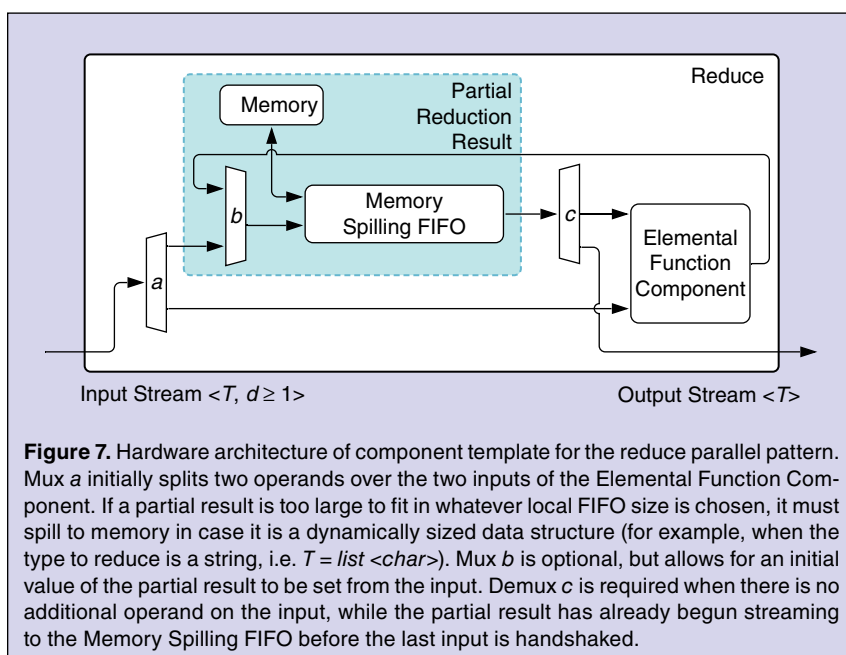
We implemented a hardware description language called Tydal [40], aimed towards structural hardware design with streaming components adhering to the Tydi specification. The Tydal language contains template components that implement the same types of parallel patterns found in many modern cluster computing frameworks such as Spark. Current implementations of template components include Map, FlatMap, Reduce, and Filter. Other elementary operations known from streaming dataflow designs are also included, for example clone (to duplicate a stream), split (to extract members of stream with a compound element type into separate streams), and (de)mux. All constructs support the transfer of multiple primitive elements per handshake over their interfaces to be able to scale throughput of the design. More coarse-grained parallelism, by instantiating multiple parallel units, is supported by the language, but automating this is left for future work.

The template components are re-used in the proposed approach to implement hardware structures similar to the Spark DAG to be accelerated. This is done by instantiating the template matching the parallel pattern used in the Spark DAG, together with the appropriate elemental function found in the hardware transformation library. This automated mechanism can re-use the back-end of the language, which is currently written in Rust, to construct the hardware structure necessary to implement the sub-stage of the DAG.

An example of how the Reduce parallel pattern hardware template can be implemented is shown in Figure 7. In Tydi, components operating on streams are called *streamlets*. Tydi specifies a type system that allows to express complex and dynamically sized data types (e.g. sum

or product types of dynamically-sized lists) flowing over streams, which form the interfaces of such streamlets. There is also a notion of dimensionality, denoted by d , to form dynamically-sized lists ($d = 1$), lists of lists ($d = 2$), etc. To perform the reduce transformation, it is required that the input dimensionality is at least one, such that the elements of the outermost dimension can be reduced using the elemental function component. The elemental function component only has to implement the reduction for two operands (e.g. an addition of two integers, with $d = 0$, a string concatenation where strings are a list of characters, i.e. $d = 1$). The whole parallel reduction can, for a single instance of the elemental function component, be implemented using the Reduction template component. In its most generic form, the Reduction template uses several multiplexers, allowing operands to stream in from the same interface, but split up to both elemental function component inputs. Because the first operand (essentially the first partial result) must be streamed in before the second one can be streamed in, it is required that it is temporarily stored in a FIFO, until the second operand has become available.

Since data can be dynamically sized and overflow the chosen dimensions of the FIFO holding the partial result, there must be some mechanism by which the dynamically sized data structure can spill to a larger memory, for example an on-board DRAM. Spilling could also be necessary if it is not possible to keep all needed intermediate data on-chip. The goal of Tydi is to stream data directly between components, to facilitate creating a fully pipelined dataflow design, but in certain complex cases it could be necessary to stream data to and from memory. In such cases, a DMA interface may be generated



by Fletcher, since in our setup, all data structures adhere to the Arrow type system. Making sure data fits in a node's memory is a task of the task distribution step in typical analytics frameworks. For a detailed description of the design and implementation of templates for other parallel patterns, we refer the reader to [40].

C. Integration

At the time of writing, Spark does not yet support Arrow directly, although this has been stated as a long-term development goal. However, limited support for columnar processing was added recently, by means of the *ColumnarBatch* data structure (that is equivalent to Arrow's *RecordBatch*). This allows column-oriented implementations to be defined for physical operators to operate on batches of columnar data. Not many of such operators currently exist, but one example is the Parquet reader. The scheduler has support to combine row-oriented and column-oriented operators, and can insert conversions if necessary. An example of this can be seen in Figure 8, showing a physical plan (which we will explain in more detail later) where the *FileSourceScanExec* uses the column-oriented Parquet reader, necessitating a conversion to the row-oriented format to allow Spark to perform all the other operations (for which no columnar implementations exist yet).

Spark has internal interfaces that can be extended, for example to implement custom implementations for certain

operations. For the proof-of-concept, the relevant components were extended with a custom version of the *columnar* (to prevent column-row conversion) execution engine of certain operations, that we are able to accelerate using one of our kernels. These custom *FletcherExec* operations call the Fletcher runtime to execute the corresponding operation on the FPGA instead of going through normal code generation and execution on the CPU. In addition, an optimization pass was added to the scheduler, that traverses the execution DAG in search of operations or subgraphs that have an accelerated equivalent available. For the proof-of-concept, there is only one such function. Left for future work is adding the automatic synthesis step where the scheduler can also trigger the creation of a new accelerator.

V. Performance Evaluation

A. Use-Case

To evaluate the proof-of-concept, we have implemented a practical use case in the form of a typical query on a tabular dataset. The publicly available Chicago Taxi Trips dataset [41] loaded from a Parquet file is used as input. Dictionary encoding is disabled, because Fletcher does not yet support dictionaries. The query involves filtering by taxi company names using a regular expression, and accumulating the duration of all of their trips. Figure 8 shows the query on the left side, the physical execution of this query

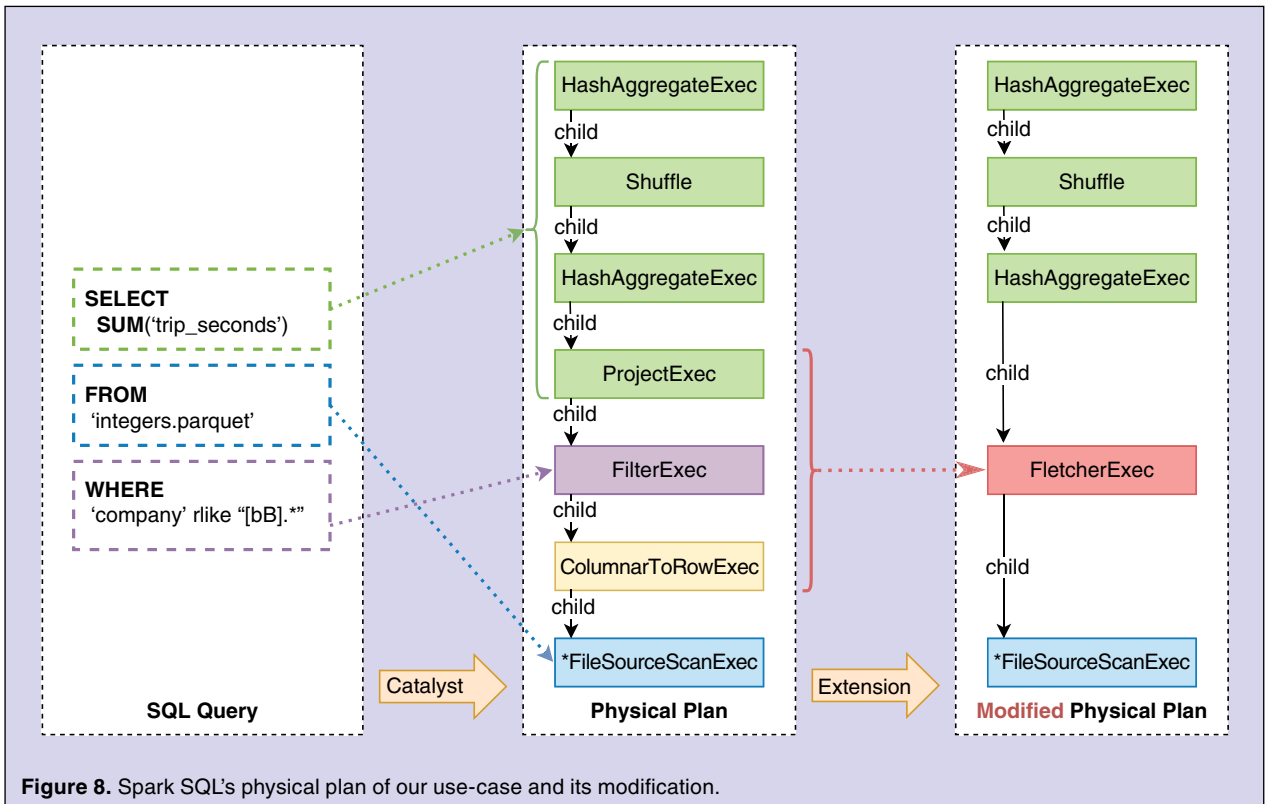


Figure 8. Spark SQL's physical plan of our use-case and its modification.

in Spark, and the nodes we have selected to accelerate on FPGA. The optimization pass that was added to the Spark scheduler transforms the DAG by replacing these nodes with a custom `FletcherExec` node that calls the accelerator. The row-column conversion step is not necessary through the use of Arrow; the file reader was replaced by the Parquet reader library function provided by the Arrow library, and the FPGA accelerator is based on Arrow and Fletcher, as discussed in previous sections.

The filter and projection step are composed into an accelerator design using the Tydal language associated with Tydi, shown schematically in Figure 9. Both the summation and regular expression are treated as a primitives. They can be either IP cores, manually implemented by the designer, and should be available in the component library. For the sum, a simple VHDL implementation is used. We use a tool that generates VHDL for the implementation of the regular expression matcher [42].

The regular expression matcher checks the predicate, producing a boolean that is in turn used by the `FilterStream` component. At an operating frequency of 200 MHz (matching the interface frequency of OpenCAPI), it can process up to 20 characters per cycle. As the company names in the schema have a maximum length of 40 characters, the matcher can process a string every 2 cycles or less. The other components are fully pipelined and can process an element every cycle. As a result, this accelerator is capable of processing between 100 and 200 Mrecords/s, depending on how many records have a name string longer than 20 characters. As the kernel interfaces are 4 bytes wide for the integer input and 20 bytes wide for the string input, the theoretical maximum throughput is 4.8 GB/s.

Important to note is that the presented Proof-of-Concept uses only a single instance of the accelerator. Because the query uses a commutative aggregation operator, it is straightforward to duplicate it, and distribute the input to them using an arbiter. This arbiter does need to keep the integer and string streams synchronized. When processing more complex queries, multiple instances of accelerators need to be managed by software. Development frameworks exist with the purpose of duplicating a kernel design provided by the developer, such as Fleet [43] and TaPaSCo [44]. As the tool-chain is currently in a very early prototyping stage, implementing parallel kernel instances is left as a future development.

B. Performance

Executing the query on the dataset, we measured the accelerator provides a throughput of 135 M records/s, for this particular dataset equaling 2.75 GB/s. The area utilization of the FPGA design is presented in Table II.

This kernel can be duplicated several times, especially considering a large fraction of the logic will be shared.

This design will in practice saturate the bandwidth of the OpenCAPI interface when using approximately 8 instances, which will likely fit in the available FPGA logic.

The most important result is that this performance was achieved from a very high-level implementation, even while the tool-chain used is in a very early stage of prototyping. This gives a reassuring perspective on the ability of the approach to generate high-performance FPGA circuits from these highly abstract big data applications.

In Figure 10, a comparison is plotted of running the query on a single instance of the FPGA accelerator versus a vanilla Spark installation running on a single thread. The Spark framework does not allow differentiating the Parquet reading from the actual execution (because Spark will interleave these steps and only perform reads from the file when blocks of data are being requested by lazy evaluation). However, for the FPGA-accelerated execution we can see that the actual execution takes only a small fraction of the total time.

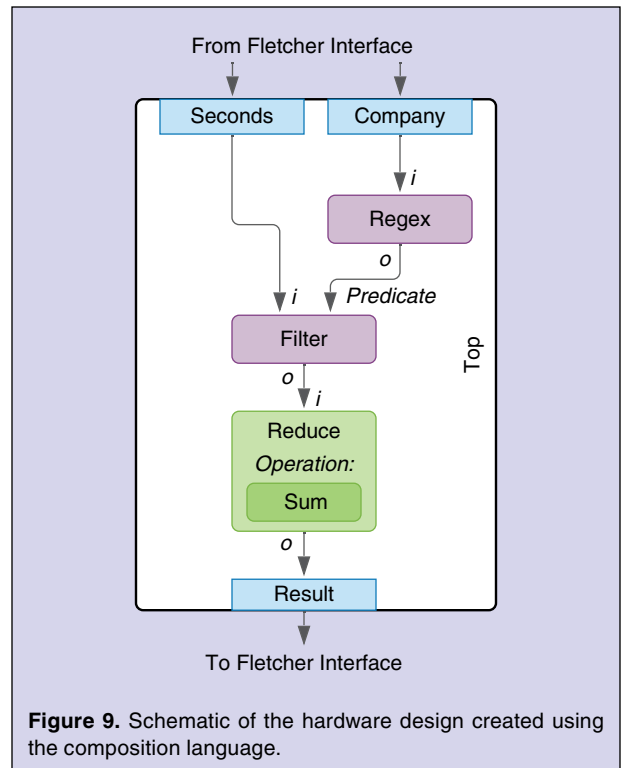


Figure 9. Schematic of the hardware design created using the composition language.

Table II.
FPGA resource utilization.

Resource	Used	Available	Utilization
CLBs	17322	162960	10.63%
LUTs	85193	1303680	6.53%
Registers	101305	2607360	3.89%
BRAM tiles	108	2016	5.36%

VI. Related Work

A. Hardware Description Languages

With very large FPGAs becoming commonplace and multi-wafer projects falling in costs, exploring highly customized architectures for applications becomes accessible to a broader range of developers. With the ability to create new computer architectures more quickly, comes the desire for tools that can facilitate this [45].

B. High-Level Synthesis

In the domain of FPGA acceleration, High-Level Synthesis (HLS) tools have been introduced by vendors to generate circuits from C/C++ and OpenCL code. Although this makes creating an initial solution on FPGA more accessible to developers with limited hardware knowledge, significant effort is required to achieve good quality of result. This includes tuning the source code and adding pragmas to guide the tools to make sensible design decisions. Recently, FPGA vendors have introduced their next generation development tools: Xilinx Vitis [21] and Intel's oneAPI [46]. They also lean heavily on a library of accelerated functions but on a different level than we propose in this work; Vitis and oneAPI both provide libraries that contain several examples of HLS implementations that can be easily modified by developers to suit their specific needs. OneAPI is accompanied with a new language called Data-Parallel C++, which aims to target various architectures (including CPUs, GPUs and FPGAs) from a single implementation.

C. Domain-Specific Hardware Generation Languages and Frameworks

Several researchers believe that somewhere between the low-level RTL design languages and the high-level descriptions, there is a middle ground for describing high-performance circuitry with reasonable design time. To achieve

this purpose, several languages have been developed such as C λ ASH [12], Pyrope [47], TLVerilog [11], and Chisel [48]. Some of these are embedded Domain-Specific Languages (eDSL), that extend an existing programming language such as Scala. eDSLs have the advantage that designs may use any constructs from the host language, often providing a high level of abstraction to generate component designs.

There are several frameworks that generate designs from a higher level description of an algorithm as input. These frameworks often build upon eDSLs, and usually rely on vendor synthesis tools (by generating old-fashioned RTL). Similar to the tools discussed in this work, they often support parallel patterns [49] such as Map, Fold (reduce), Groupby, etc. The DeLite tool [50] is a compiler architecture that can actually generate domain-specific hardware generation tools. It is built on top of MaxCompiler [10], which takes care of scheduling the dataflow graph. Spatial [9] is not only a hardware generation framework but also a full end-to-end acceleration framework including host runtime and infrastructure to communicate with FPGA. It includes design-space exploration functionality to guide the tiling of operations onto the FPGA fabric. Fleet [43] is another Scala eDSL offering a massively parallel streaming model for FPGAs.

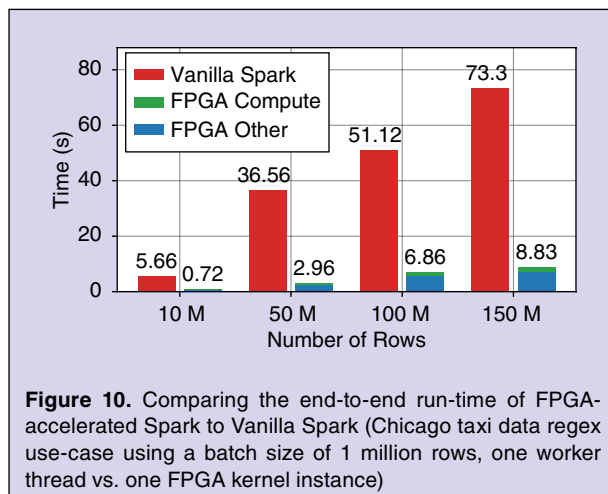
D. Functional or Declarative Languages

Functional languages have properties that are believed to provide a more natural mapping to logic circuits [12]. Besides C λ ASH, functional descriptions are used for the input for the SPIRAL [51] and Lift [52] frameworks. They apply rewrite rules to lower the high-level sources towards various compute fabrics, including FPGA. The Elevate language adds to this a functional DSL to also allow a performance engineer to steer these transformations in the right direction [31].

In the analytics context, the declarative SQL language has been prevalent since the heyday of database systems. It has been used as a source language for generating logic by, for example, the Glacier project [53], [54].

E. Database Acceleration

Accelerating SQL is only a part of the larger area of accelerating databases. Various types of accelerators specifically for databases have been proposed. For example, Netezza placed FPGAs between storage and main memory to do decompression and a number of pre-processing steps [22]. Swarm64 accelerates the open source PostgreSQL using FPGA [55]. Database workloads have several facets such as (de)compression, joining, filtering, and sorting, each having very different properties and suitability for FPGA. An overview of FPGA acceleration of in-memory databases is given in [56] and [57].



F. Big Data Frameworks Employing Accelerators

Although not widely supported yet, various academic and industrial efforts exist that aim to incorporate heterogeneous computing (e.g. Falcon computing targeting both GPUs and FPGAs [58]) or at least a certain type of accelerator into big data frameworks.

1) GPU

Compared to FPGAs, more work exists on supporting GPUs from big data frameworks. There are a number of reasons behind this. First, the GPU accelerator has been steadily building popularity for a longer period of time compared to FPGAs, and are more prevalent in datacenters and supercomputers. Second, they have proven to be very efficient in workloads related to machine learning. Third, analytics frameworks, in their turn, often provide very high-level libraries for machine learning due to its rapidly growing popularity (an example is MLlib for Apache Spark [59]). Targeting these high-level libraries directly provides a clear insertion point for GPUs. Although this approach will require serialization/deserialization as is discussed in this work, this approach is taken by several existing efforts [60]–[62].

The other approach used is to generate GPU code from the execution graph, similar to what is done in this work. The most well-known work is an industry project by Nvidia called RAPIDS [63]–[65]. Academic efforts include TensorFlow [66] and Spark-GPU [67].

2) FPGA

The open-source Blaze runtime system allows implementing FPGA-accelerated algorithms for a Spark data source [68]. Similar to the GPU work targeting high-level Spark libraries, this does not allow accelerating queries at the granularity of sub-stages in the execution graph, giving it less flexibility and only allowing to implement a whole algorithm on FPGA. The Kestrel Runtime from Falcon Computing [58] is built on this runtime.

Related efforts taking the same approach as this work include BigStream [69], which searches the Spark physical execution plan for FPGA acceleration candidates located in a bitstream store, and another novel project integrating Intel's "Spark Native SQL Engine" [70] with FPGAs [71].

As mentioned, FPGA usage in cloud instances is currently very explicit, with clients needing to acquire a full FPGA as opposed to CPU cycles that are much easier to share with other users. An overview of efforts aiming to virtualize FPGAs is presented in [72].

VII. Conclusion

In this article, we discussed our views on FPGA accelerators for near-future big data analytics systems,

their challenges and how to address them, as well as the opportunities and how to leverage these. We also presented a proposed architecture for a scalable but still fully programmable system that can be built using technologies that are currently becoming generally available. We discussed the current difficulties involved with designing custom accelerators for FPGAs and how to improve design re-use and IP core connectivity standardization by using Tydi and Tydal. We present an analysis and evaluation of an FPGA-accelerated big data prototype that makes use of several of the discussed components.

While highly experimental, the prototype is able to generate a high-performing FPGA circuit from very high-level code descriptions in Spark. This is made possible by specifically targeting the big data application domain. This allows taking advantage of certain properties, including the DAG representation that is used internally in analytics frameworks and the highly parallel programming language constructs such as Map, filter and Reduce. These patterns are very suitable for large compute clusters, but also map very naturally to a spatial compute resource such as reconfigurable logic. On a final note, within the context of Apache Spark and Apache Arrow, a structured enough environment has been contributed that allows for the automatic generation of infrastructure for FPGA accelerators, without losing potential performance advantage.

Acknowledgments

The authors would like to thank the reviewers and in particular Hongbo Rong for the thoughtful review and valuable comments. The authors also thank Patrick Ly-saght and Cathal McCabe from Xilinx for their support. This work is part of the FitOptiVis project funded by the ECSEL Joint Undertaking under grant number H2020-ECSEL-2017-2-783162. [73]



Joost Hoozemans Received his BSc in Computer Science from Utrecht University in 2011 and his MSc and PhD in Computer Engineering from Delft University of Technology in 2014 and 2018, respectively. His research interests include VLIW and TTA processors, reconfigurable computing and FPGA programmability, and dataflow computing.



Johan Peltenburg is a Ph.D. candidate with the Accelerated Big Data Systems Group, TU Delft. He focuses on FPGA accelerators for big data applications, working on the Fletcher accelerator framework. He is a member of IEEE.



Fabian Nonnemacher received his BEng in Computer Science from the DHBW Stuttgart in 2015, and his MSc in Computer Science from the Delft University of Technology in 2020. His research interests include distributed

systems, big data analytics and heterogeneous computer architectures.



Ákos Hadnagy received his BSc in Electrical Engineering from the Budapest University of Technology and Economics in 2018, and his MSc in Computer Engineering from the Delft University of Technology in 2020. His research inter-

ests include high-level FPGA design, hardware acceleration, and heterogeneous computer architectures.



Zaid Al-Ars is an Associate Professor with TU Delft, where he leads the Accelerated Big Data Systems Group. His work focuses on developing computing infrastructures for efficient processing of big data analytics applications. He is

co-founder of a couple of big data companies specialized in high performance analytics solutions and AI. He is also on the advisory board of a number of high-tech start-ups. He published more than 100 peer-reviewed publications and holds two patents. He is a member of IEEE.



H. Peter Hofstee is a distinguished research staff member at IBM and part-time Professor at TU Delft, best known for his contributions to heterogeneous computing as a chief architect of the Synergistic Processor Elements in the

Cell Broadband Engine used in PlayStation 3, and the first supercomputer to reach sustained petaflop operation. He currently focuses on optimizing the system roadmap for big data, analytics, and cloud, including the use of accelerated computation. Recent contributions include coherently attached reconfigurable acceleration on POWER7, paving the way for the new coherent attach processor interface on POWER8 through POWER10. He holds more than 100 issued patents. He is a member of IEEE. Contact him at hofstee@us.ibm.com.

References

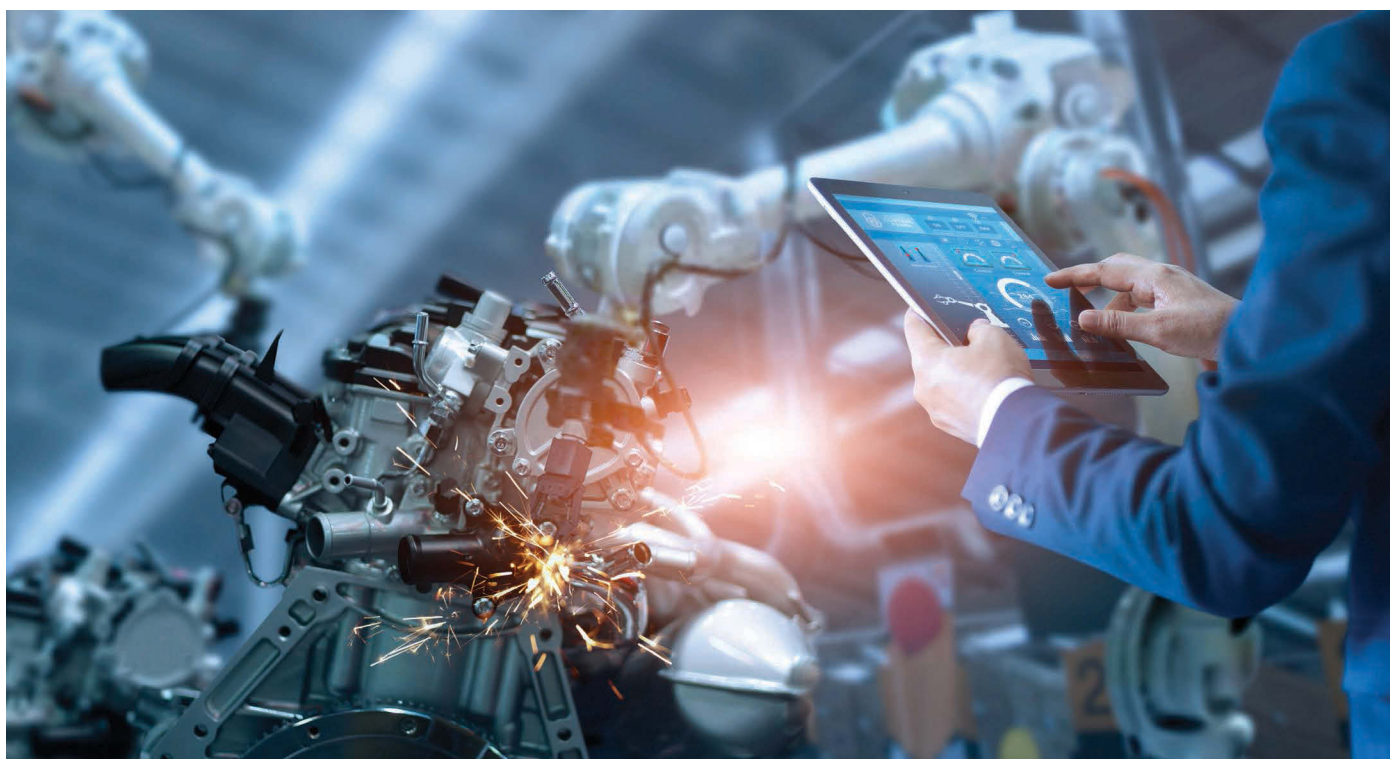
- [1] Based on data from spec.org, see the attached link for an example of a plot of this data [Online]. Available: <https://liberty.princeton.edu/Projects/AutoPar/>
- [2] M. D. Hill and M. R. Marty, "Retrospective on Amdahl's Law in the Multicore Era," *Computer*, vol. 50, no. 6, pp. 12–14, 2017. doi: 10.1109/MC.2017.164.

- [3] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Arch.*, 2017, pp. 1–12.
- [4] "Alveo," Xilinx. [Online]. Available: <https://www.xilinx.com/products/boards-and-kits/alveo.html>
- [5] "Intel accelerator cards," Intel. [Online]. Available: <https://www.intel.com/content/www/us/en/programmable/solutions/acceleration-hub/platforms.html>
- [6] H. Wong, V. Betz, and J. Rose, "Comparing FPGA vs. custom CMOS and the impact on processor microarchitecture," in *Proc. 19th ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, 2011, p. 5–14. doi: 10.1145/1950413.1950419.
- [7] "Brian Krzanich: Our strategy and the future of Intel," Intel Newsroom. [Online]. Available: <https://newsroom.intel.com/editorials/brian-krzanich-our-strategy-and-the-future-of-intel/>
- [8] S. Lahti, P. Sjövall, J. Vanne, and T. D. Hämäläinen, "Are we there yet? A study on the state of high-level synthesis," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 5, pp. 898–911, May 2019. doi: 10.1109/TCAD.2018.2834439.
- [9] D. Koeplinger et al., "Spatial: A language and compiler for application accelerators," in *Proc. 39th ACM SIGPLAN Conf. Programming Language Des. Implementation*, 2018, p. 296–311.
- [10] "MaxCompiler White Paper," Maxeler. [Online]. Available: <https://www.maxeler.com/media/documents/MaxelerWhitePaperMaxCompiler.pdf>
- [11] S. F. Hoover, "Timing-abstract circuit design in transaction-level verilog," in *Proc. IEEE Int. Conf. Comput. Des. (ICCD)*, 2017, pp. 525–532.
- [12] C. Baaij, M. Kooijman, J. Kuper, W. Boeijsink, and M. Gerards, "CFlash: Structural descriptions of synchronous hardware using Haskell," in *Proc. 13th EUROMICRO Conf. Digital Syst. Des.: Arch., Methods and Tools*, Sept. 2010, pp. 714–721.
- [13] M. Hendriks et al., "Monotonic optimization of dataflow buffer sizes," *J. Signal Process. Syst.*, vol. 91, no. 1, pp. 21–32, 2019. doi: 10.1007/s11265-018-1415-2.
- [14] F. Kruger, "CPU bandwidth—The worrisome 2020 trend," Mar. 2016. [Online]. Available: <https://blog.westerndigital.com/cpu-bandwidth-the-worrisome-2020-trend/>
- [15] J. Fang, J. Chen, Z. Al-Ars, P. Hofstee, and J. Hidders, "Work-in-progress: A high-bandwidth snappy decompressor in reconfigurable logic," in *Proc. Int. Conf. Hardw./Softw. Codes. Syst. Synthesis (CODES+ISSS)*, Sept. 2018, pp. 1–2. doi: 10.1109/CODES+ISSS.2018.8525953.
- [16] J. Peltenburg, A. Hesam, and Z. Al-Ars, "Pushing big data into accelerators: Can the JVM saturate our hardware?" in *Proc. Int. Conf. High Performance Comput.*, 2017, pp. 220–236.
- [17] J. Peltenburg, "Methods for efficient integration of FPGA accelerators with big data systems," Ph.D. dissertation, Delft Univ. of Technology, 2020.
- [18] D. J. Abadi, S. R. Madden, and N. Hachem, "Column-stores vs. row-stores: How different are they really?" in *Proc. ACM SIGMOD Int. Conf. Manag. of Data*, 2008, p. 967–980. doi: 10.1145/1376616.1376712.
- [19] B. Gaide, D. Gaitonde, C. Ravishankar, and T. Bauer, "Xilinx adaptive compute acceleration platform: Versal architecture," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, 2019, pp. 84–93.
- [20] A. Putnam et al., "A reconfigurable fabric for accelerating large-scale datacenter services," in *Proc. 41st Int. Symp. Comput. Arch. (ISCA)*, June 2014, pp. 13–24. doi: 10.1109/ISCA.2014.6853195.
- [21] V. Kathail, "Xilinx Vitis unified software platform," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, 2020, pp. 173–174.
- [22] P. Francisco et al., "The Netezza data appliance architecture: A platform for high performance data warehousing and analytics," 2011. [Online]. Available: https://www.ibmbigdatahub.com/sites/default/files/document/redguide_2011.pdf
- [23] "Nallatech 250SoC," Bittware. [Online]. Available: <https://www.bittware.com/fpga/250-soc/>
- [24] "Computational storage? We're the godfather, says Samsung," 451 Research. [Online]. Available: https://samsungsemiconductor-us.com/smartssd/wp-content/uploads/451_SmartSSD.pdf
- [25] M. Tork, L. Maudlej, and M. Silberstein, "Lynx: A SmartNIC-driven accelerator-centric architecture for network servers," in *Proc. 25th Int. Conf. Arch. Support for Programming Languages and Operating Syst.*, 2020, p. 117–131. doi: 10.1145/3373376.3378528.
- [26] B. Allison, "OpenCAPI: Next generation of acceleration for the cognitive era." [Online]. Available: <https://www.slideshare.net/ganesannarayanasamy/opencapi-next-generation-accelerator>

- [27] K. Kara, C. Hagleitner, D. Diamantopoulos, D. Syrivelis, and G. Alonso, "High bandwidth memory on FPGAs: A data analytics perspective," 2020. [Online]. Available: <https://arxiv.org/abs/2004.01635>
- [28] K. Neshatpour et al., "Energy-efficient acceleration of MapReduce applications using FPGAs," *J. Parallel Distrib. Comput.*, vol. 119, pp. 1–17, 2018. doi: 10.1016/j.jpdc.2018.02.004.
- [29] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," in *Proc. 34th ACM SIGPLAN Conf. Programming Language Des. Implementation*, 2013, p. 519–530. doi: 10.1145/2491956.2462176.
- [30] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th {USENIX} Symp. Operating Syst. Des. Implementation (OSDI) 18*, 2018, pp. 578–594.
- [31] B. Hagedorn, J. Lenfers, T. Koehler, S. Gorlatch, and M. Steuwer, "A language for describing optimization strategies," 2020. [Online]. Available: <https://arxiv.org/abs/2002.02268>
- [32] H. Rong, "Programmatic control of a compiler for generating high-performance spatial hardware," 2017. [Online]. Available: <https://arxiv.org/abs/1711.07606>
- [33] Y.-H. Lai et al., "HeteroCL: A multi-paradigm programming infrastructure for software-defined reconfigurable computing," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2019, pp. 242–251.
- [34] M. Zaharia, "An architecture for fast and general data processing on large clusters," Ph.D. dissertation, Electrical Engineering and Computer Sciences, Univ. of California at Berkeley, 2014.
- [35] J. Peltenburg, J. van Straten, L. Wijtemans, L. van Leeuwen, Z. Al-Ars, and P. Hofstee, "Fletcher: A framework to efficiently integrate FPGA accelerators with Apache Arrow," in *Proc. 29th Int. Conf. Field Programmable Logic and Appl. (FPL)*, Sept. 2019, pp. 270–277. doi: 10.1109/FPL.2019.00051.
- [36] J. Peltenburg, J. van Straten, M. Brobbel, H. P. Hofstee, and Z. Al-Ars, "Supporting columnar in-memory formats on FPGA: The hardware design of Fletcher for Apache Arrow," in *Applied Reconfigurable Computing*, C. Hochberger, B. Nelson, A. Koch, R. Woods, and P. Diniz, Eds. Cham: Springer International Publishing, 2019, pp. 32–47.
- [37] J. Peltenburg, J. van Straten, M. Brobbel, Z. Al-Ars, and H. P. Hofstee, "Tydi: An open specification for complex data structures over hardware streams," *IEEE Micro*, vol. 40, no. 4, July/August 2020. doi: 10.1109/MM.2020.2996373.
- [38] G. Zhong, A. Prakash, S. Wang, Y. Liang, T. Mitra, and S. Niar, "Design Space exploration of FPGA-based accelerators with multi-level parallelism," in *Proc. Des., Automat. Test Europe Conf. Exhib. (DATE)*, 2017, pp. 1141–1146.
- [39] G. Deest, N. Estivals, T. Yuki, S. Derrien, and S. Rajopadhye, "Towards Scalable and Efficient FPGA Stencil Accelerators," in *Proc. IMPACT'16—6th Int. Workshop on Polyhedral Compilation Techn., held with HIPEAC'16*, Prague, Czech Republic, Jan. 2016. [Online]. Available: <https://hal.inria.fr/hal-01425018>
- [40] A. Hadnagy, "Dataflow hardware design for big data acceleration using typed interfaces," Master's thesis, Delft Univ. of Technology, 2020. [Online]. Available: <http://resolver.tudelft.nl/uuid:38d9b35e-2d75-4cab-b6d1-723c2849badb>
- [41] "Chicago taxi trips," Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/chicago/chicago-taxi-trips-bq>
- [42] J. van Straten, "vhdre: a VHDL regex matcher generator," 2019. [Online]. Available: <https://github.com/abs-tudelft/vhdre>
- [43] J. Thomas, P. Hanrahan, and M. Zaharia, "Fleet: A framework for massively parallel streaming on FPGAs," in *Proc. 25th Int. Conf. Arch. Support Programming Languages and Operating Syst.*, 2020, p. 639–651. doi: 10.1145/3373376.3378495.
- [44] J. Korinth, J. Hofmann, C. Heinz, and A. Koch, "The TaPaSCo open-source toolflow for the automated composition of task-based parallel reconfigurable computing systems," in *Applied Reconfigurable Computing*, C. Hochberger, B. Nelson, A. Koch, R. Woods, and P. Diniz, Eds. Cham: Springer International Publishing, 2019, pp. 214–229.
- [45] L. Truong and P. Hanrahan, "A golden age of hardware description languages: Applying programming language techniques to improve design productivity," *Leibniz Int. Proc. Informatics, LIPIcs*, vol. 136, no. 7, pp. 1–7, 2019.
- [46] "oneAPI programming model," Intel. [Online]. Available: <https://www.oneapi.com/>
- [47] S.-H. Wang et al., "Pyrope, a modern HDL with a live flow." [Online]. Available: <https://masc.soe.ucsc.edu/pyrope.html>
- [48] J. Bachrach et al., "Chisel: Constructing hardware in a Scala embedded language," in *Proc. DAC Des. Automat. Conf.*, June 2012, pp. 1212–1221.
- [49] R. Prabhakar et al., "Generating configurable hardware from parallel patterns," in *Proc. Int. Conf. Arch. Support Programming Languages Operating Syst.—ASPLOS*, vol. 50, no. 2, pp. 651–665, 2016. doi: 10.1145/2954680.2872415.
- [50] A. K. Sujeeth et al., "Delite: A compiler architecture for performance-oriented embedded domain-specific languages," *ACM Trans. Embedded Comput. Syst. (TECS)*, vol. 13, no. 4s, pp. 1–25, 2014. doi: 10.1145/2584665.
- [51] F. Franchetti et al., "Spiral: Extreme performance portability," *Proc. IEEE*, vol. 106, no. 11, pp. 1935–1968, 2018. doi: 10.1109/JPROC.2018.2873289.
- [52] M. Kristien, B. Bodin, M. Steuwer, and C. Dubach, "High-level synthesis of functional patterns with lift," in *Proc. ACM SIGPLAN Conf. Programming Language Des. Implementation (PLDI)*, 2019, pp. 35–45.
- [53] R. Mueller, J. Teubner, and G. Alonso, "Glacier: A query-to-hardware compiler," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2010, p. 1159–1162. doi: 10.1145/1807167.1807307.
- [54] R. Mueller, J. Teubner, and G. Alonso, "Streams on wires: a query compiler for FPGAs," *Proc. VLDB Endowment*, vol. 2, no. 1, pp. 229–240, 2009.
- [55] "PostgreSQL database acceleration innovators," Swarm64. [Online]. Available: <https://swarm64.com/>
- [56] P. Papaphilippou and W. Luk, "Accelerating database systems using FPGAs: A survey," in *Proc. 28th Int. Conf. Field Programmable Logic and Appl. (FPL)*, Aug. 2018, pp. 125–1255. doi: 10.1109/FPL.2018.00030.
- [57] J. Fang, Y. T. Mulder, J. Hidders, J. Lee, and H. P. Hofstee, "In-memory database acceleration on FPGAs: A survey," *VLDB J.*, vol. 29, no. 1, pp. 33–59, 2020. doi: 10.1007/s00778-019-00581-w.
- [58] "Kestrel runtime—Big data application scheduling and load balancing," Falcon Computing. [Online]. Available: <https://www.falconcomputing.com/falcon-kestrel-runtime/>
- [59] "MLlib—Apache Spark," Apache Software Foundation. [Online]. Available: <https://spark.apache.org/mllib/>
- [60] P. Li, Y. Luo, N. Zhang, and Y. Cao, "HeteroSpark: A heterogeneous CPU/GPU Spark platform for machine learning algorithms," in *Proc. IEEE Int. Conf. Netw., Arch. and Storage (NAS)*, 2015, pp. 347–348.
- [61] R. Bordawekar, "Nvidia GTC Silicon Valley 2016: Accelerating Spark workloads using GPUs," 2016. [Online]. Available: <https://on-demand-gtc.gputechconf.com/gtcnew/sessionview.php?sessionName=s6280-accelerating+spark+workloads+using+gpus>
- [62] R. Bordawekar, "Accelerating Spark workloads using GPUs," Aug. 2016. [Online]. Available: <https://www.oreilly.com/content/accelerating-spark-workloads-using-gpus/>
- [63] J. L. Robert Evans, "Spark+AI Summit 2020: Deep dive into GPU support in Apache Spark 3.x," July 2020. [Online]. Available: https://databricks.com/de/session_na20/deep-dive-into-gpu-support-in-apache-spark-3-x
- [64] Spark RAPIDS plugin—Accelerate Apache Spark with GPUs," NVIDIA, 2020. [Online]. Available: <https://github.com/NVIDIA/spark-rapids>
- [65] "Open GPU data science," RAPIDS. [Online]. Available: <https://rapids.ai/about.html>
- [66] "TensorFrames: Experimental tensorflow binding for Scala and Apache Spark," Databricks, Apr. 2017. [Online]. Available: <https://github.com/databricks/tensorframes>
- [67] Y. Yuan, M. F. Salmi, Y. Huai, K. Wang, R. Lee, and X. Zhang, "Spark-GPU: An accelerated in-memory data processing engine on clusters," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2016, pp. 273–283. doi: 10.1109/BigData.2016.7840613.
- [68] M. Huang et al., "Programming and runtime support to blaze FPGA accelerator deployment at datacenter scale," in *Proc. 7th ACM Symp. Cloud Comput.*, 2016, pp. 456–469. doi: 10.1145/2987550.2987569.
- [69] "Hyperacceleration with Bigstream technology," Bigstream. [Online]. Available: <https://blog.bigstream.co/resources/hyper-acceleration-with-bigstream-technology>
- [70] "Spark Native SQL Engine—Optimized analytics package for Spark platform," Intel, 2020. [Online]. Available: <https://github.com/Intel-bigdata/OAP/tree/master/oap-native-sql>
- [71] W. C. Calvin Hung, "Spark+AI Summit 2020: Accelerating Spark SQL workloads to 50X performance with Apache Arrow-based FPGA accelerators," 2020. [Online]. Available: https://databricks.com/session_na20/accelerating-spark-sql-workloads-to-50x-performance-with-apache-arrow-based-fpga-accelerators
- [72] A. Vaishnav, K. D. Pham, and D. Koch, "A survey on FPGA virtualization," in *Proc. 28th Int. Conf. Field Programmable Logic and Appl. (FPL)*, 2018, pp. 131–1317. doi: 10.1109/FPL.2018.00031.
- [73] Z. Al-Ars et al., "The FitOptiVis ECSEL project: Highly efficient distributed embedded image/video processing in cyber-physical systems," in *Proc. 16th ACM Int. Conf. Comput. Front.*, 2019, pp. 333–338. [Online]. doi: 10.1145/3310273.3323437.

A Survey of FPGA-Based Robotic Computing

Zishen Wan,* Bo Yu,* Thomas Yuang Li, Jie Tang, Yuhao Zhu,
Yu Wang, Arijit Raychowdhury, and Shaoshan Liu



Abstract

Recent researches on robotics have shown significant improvement, spanning from algorithms, mechanics to hardware architectures. Robotics, including manipulators, legged robots, drones, and autonomous vehicles, are now widely applied in diverse scenarios. However, the high computation and data complexity of robotic algorithms pose great challenges to its applications. On the one hand, CPU platform is flexible to handle multiple robotic tasks. GPU platform has higher computational capacities and easy-to-use development frameworks, so they have been widely adopted in several applications. On the other hand, FPGA-based robotic accelerators are becoming increasingly competitive alternatives, especially in latency-critical and power-limited scenarios. With specialized designed hardware logic and algorithm kernels, FPGA-based accelerators can surpass CPU and GPU

in performance and energy efficiency. In this paper, we give an overview of previous work on FPGA-based robotic accelerators covering different stages of the robotic system pipeline. An analysis of software and hardware optimization techniques and main technical issues is presented, along with some commercial and space applications, to serve as a guide for future work.

I. Introduction

Over the last decade, we have seen significant progress in the development of robotics, spanning from algorithms, mechanics to hardware platforms. Various robotic systems, like manipulators, legged robots, unmanned aerial vehicles, self-driving cars

Digital Object Identifier 10.1109/MCAS.2021.3071609

Date of current version: 24 May 2021

* These authors contributed equally to this work.

Corresponding author: Shaoshan Liu (email: shaoshan.liu@perceptin.io).

have been designed for search and rescue [1], [2], exploration [3], [4], package delivery [5], entertainment [6], [7] and more applications and scenarios. These robots are on the rise of demonstrating their full potential. Take drones, a type of aerial robots, as an example, the number of drones has grown by 2.83x between 2015 and 2019 based on the U.S. Federal Aviation Administration (FAA) report [8]. The registered number has reached 1.32 million in 2019, and the FAA expects this number will come to 1.59 billion by 2024.

However, robotic systems are pretty complicated [9]–[11]. They tightly integrate many technologies and algorithms, including sensing, perception, mapping, localization, decision making, control, etc. This complexity poses many challenges for the design of robotic edge computing systems [12], [13]. On the one hand, the robotic system needs to process an enormous amount of data in real-time. The incoming data often comes from multiple sensors and is highly heterogeneous. However, the robotic system usually has limited on-board resources, such as memory storage, bandwidth, and compute capabilities, making it hard to meet the real-time requirements. On the other hand, the current state-of-the-art robotic system usually has strict power constraints on the edge that cannot support the amount of computation required for performing tasks, such as 3D sensing, localization, navigation, and path planning. Therefore, the computation and storage complexity, as well as real-time and power constraints of the robotic system, hinder its wide application in latency-critical or power-limited scenarios [14].

Therefore, it is essential to choose a proper compute platform for the robotic system. CPU and GPU are two widely used commercial compute platforms. CPU is designed to handle a wide range of tasks quickly and is often used to develop novel algorithms. A typical CPU can achieve 10-100 GFLOPS with below 1GOP/J power efficiency [15]. In contrast, GPU is designed with thousands of processor cores running simultaneously, which enable massive parallelism. A typical GPU can perform up to 10 TOPS performance and become a good candidate

for high-performance scenarios. Recently, benefiting in part from the better accessibility provided by CUDA/OpenCL, GPU has been predominantly used in many robotic applications. However, conventional CPU and GPUs usually consume 10 W to 100 W of power, which are orders of magnitude higher than what is available on the resource-limited robotic system.

Besides CPU and GPU, FPGAs are attracting attention and becoming a platform candidate to achieve energy-efficient robotics tasks processing. FPGAs require little power and are often built into small systems with less memory. They have the ability to parallel computations massively and makes use of the properties of perception (e.g., stereo matching), localization (e.g., SLAM), and planning (e.g., graph search) kernels to remove additional logic and simplify the implementation. Taking into account hardware characteristics, several algorithms are proposed which can be run in a hardware-friendly way and achieve similar software performance. Therefore, FPGAs are possible to meet real-time requirements while achieving high energy efficiency compared to CPUs and GPUs.

Unlike the ASIC counterparts, FPGA technology provides the flexibility of on-site programming and re-programming without going through re-fabrication with a modified design. Partial Reconfiguration (PR) takes this flexibility one step further, allowing the modification of an operating FPGA design by loading a partial configuration file. Using PR, part of the FPGA can be reconfigured at runtime without compromising the integrity of the applications running on those parts of the device that are not being reconfigured. As a result, PR can allow different robotic applications to time-share part of an FPGA, leading to energy and performance efficiency, and making FPGA a suitable computing platform for dynamic and complex robotic workloads.

FPGAs have been successfully utilized in commercial autonomous vehicles. Particularly, over the past three years, PerceptIn has built and commercialized autonomous vehicles for micromobility, and PerceptIn's products have been deployed in China, US, Japan and Switzerland. In this paper, we review how PerceptIn developed its computing system by relying heavily on FPGAs, which perform not only heterogeneous sensor synchronizations, but also the acceleration of software components on the critical path. In addition, FPGAs are used heavily in space robotic applications, for FPGAs offered unprecedented flexibility and significantly reduced the design



©SHUTTERSTOCK.COM/POPTIKA

Zishen Wan,* School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA 30332 USA and John A. Paulson School of Engineering and Applied Sciences, Harvard University, Cambridge, MA 02138 USA. Bo Yu,* Thomas Yuang Li and Shaoshan Liu, PerceptIn Inc, Fremont, CA 94539 USA. Jie Tang, School of Computer Science and Engineering, South China University of Technology, Guangzhou, Guangdong, China. Yuhao Zhu, Department of Computer Science, University of Rochester, Rochester, NY 14627 USA. Yu Wang, Department of Electronic Engineering, Tsinghua University, Beijing, China. Arijit Raychowdhury, School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA 30332 USA.

cycle and development cost. In this paper, we also delve into space-grade FPGAs for robotic applications.

The rest of paper is organized as follows: Section II introduces the basic workloads of the robotic system. Section III, IV and V reviews the various perception, localization and motion planning algorithms and their implementations on FPGA platforms. In section VI, we discuss about FPGA partial reconfiguration techniques. Section VII and VIII present robotics FPGA applications in commercial and space areas. Section IX concludes the paper.

II. Overview of Robotics workloads

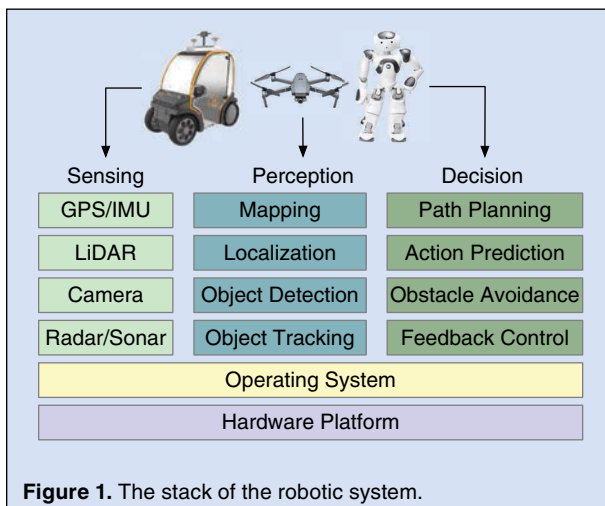
A. Overview

Robotics is not one technology but rather an integration of many technologies. As shown in Fig 1, the stack of the robotic system consists of three major components: application workloads, including sensing, perception, localization, motion planning, and control; a software edge subsystem, including operating system and runtime layer; and computing hardware, including both microcontrollers and companion computers.

We focus on the robotic application workloads in this section. The application subsystem contains multiple algorithms that are used by the robot to extract meaningful information from raw sensor data to understand the environment and dynamically make decisions about its actions.

B. Sensing

The sensing stage is responsible for extracting meaningful information from the sensor raw data. To enable intelligent actions and improve reliability, the robot platform usually supports a wide range of sensors. The number and type of sensors are heavily dependent on the specifications of the workload and the capability of the onboard compute platform. The sensors can include the following:



Cameras. Cameras are usually used for object recognition and object tracking, such as lane detection in autonomous vehicles and obstacle detection in drones, etc. RGB-D camera can also be utilized to determine object distances and positions. Take autonomous vehicle as an example, the current system usually mounts eight or more 1080p cameras around the vehicle to detect, recognize and track objects in different directions, which can greatly improve safety. Usually, these cameras run at 60 Hz, which will process multiple gigabytes of raw data per second when combined.

GNSS/IMU. The global navigation satellite system (GNSS) and inertial measurement unit (IMU) system help the robot localize itself by reporting both inertial updates and an estimate of the global location at a high rate. Different robots have different requirements for localization sensing. For instance, 10 Hz may be enough for low-speed mobile robots, but high-speed autonomous vehicles usually demand 30 Hz or higher for localization, and high-speed drones may need 100 Hz or more for localization, thus we are facing a broad spectrum of sensing speeds. Fortunately, different sensors have their own advantages and drawbacks. GNSS can enable fairly accurate localization, while it runs at only 10 Hz, thus unable to provide real-time updates. By contrast, both accelerometer and gyroscope in IMU can run at 100–200 Hz, which can satisfy the real-time requirement. However, IMU suffers bias wandering over time or perturbation by some thermo-mechanical noise, which may lead to an accuracy degradation in the position estimates. By combining GNSS and IMU, we can get accurate and real-time updates for robots.

LiDAR. Light detection and ranging (LiDAR) is used for evaluating distance by illuminating the obstacles with laser light and measuring the reflection time. These pulses, along with other recorded data, can generate precise and three-dimensional information about the surrounding characteristics. LiDAR plays an important role in localization, obstacle detection and avoidance. As indicated in [16], the choice of sensors dictates the algorithm and hardware design. Take autonomous driving as an instance, almost all the autonomous vehicle companies use LiDAR at the core of their technologies. Examples include Uber, Waymo, and Baidu. PerceptIn and Tesla are among the very few that do not use LiDAR and, instead, rely on cameras and vision-based systems, and in particular PerceptIn's data demonstrated that for the low-speed autonomous driving scenario, LiDAR processing is slower than camera-based vision processing, but increases the power consumption and cost.

Radar and Sonar. The Radio Detection and Ranging (Radar) and Sound Navigation and Ranging (Sonar)

system is used to determine the distance and speed to a certain object, which usually serves as the last line of defense to avoid obstacles. Take autonomous vehicle as an example, a danger of collision may occur when near obstacles are detected, then the vehicle will apply brakes or turn to avoid obstacles. Compared to LiDAR, the Radar and Sonar system is cheaper and smaller, and their raw data is usually fed to the control processor directly without going through the main compute pipeline, which can be used to implement some urgent functions as swerving or applying the brakes.

C. Perception

The sensor data is then fed into the perception layer to sense the static and dynamic objects, and build a reliable and detailed representation of the robot's environment using computer vision techniques (including deep learning).

The perception layer is responsible for object detection, segmentation and tracking. There are obstacles, lane dividers and other objects to detect. Traditionally, a detection pipeline starts with image pre-processing, followed by a region of interest detector and then a classifier that outputs detected objects. In 2005, Dalal and Triggs [17] proposed an algorithm based on histogram of orientation (HOG) and support vector machine (SVM) to model both the appearance and shape of the object under various condition. The goal of segmentation is to give the robot a structured understanding of its environment. Semantic segmentation is usually formulated as a graph labeling problem with vertices of the graph being pixels or super-pixels. Inference algorithms on graphical models such as conditional random field (CRF) [18], [19] are used. The goal of tracking is to estimate the trajectory of moving obstacles. Tracking can be formulated as a sequential Bayesian filtering problem by recursively running the prediction step and correction step. Tracking can also be formulated by tracking-by-detection handling with Markovian decision process (MDP) [20], where an object detector is applied to consecutive frames and detected objects are linked across frames.

In recent years, deep neural networks (DNN), also known as deep learning, have greatly affected computer vision and made significant progress in solving robot perception problems. Most state-of-the-art algorithms now apply one type of neural network based on convolution operation. Fast R-CNN [21], Faster R-CNN [22], SSD [23], YOLO [24], and YOLO9000 [25] were used to get much better speed and accuracy in object detection. Most CNN-based semantic segmentation work is based on Fully Convolutional Networks (FCN) [26], and there are some recent work in spatial pyramid pooling net-

work [27] and pyramid scene parsing network (PSPNet) [28] to combine global image-level information with the locally extracted feature. By using auxiliary natural images, a stacked autoencoder model can be trained offline to learn generic image features and then applied for online object tracking [29].

D. Localization

The localization layer is responsible for aggregating data from various sensors to locate the robot in the environment model.

GNSS/IMU system is used for localization. The GNSS consist of several satellite systems, such as GPS, Galileo and BeiDou, which can provide accurate localization results but with a slow update rate. In comparison, IMU can provide a fast update with less accurate rotation and acceleration results. A mathematical filter, such as Kalman Filter, can be used to combine the advantages of the two and minimize the localization error and latency. However, this sole system has some problems, such as the signal may bounce off obstacles, introduce more noise, and fail to work in closed environments.

LiDAR and High-Definition (HD) maps are used for localization. LiDAR can generate point clouds and provide a shape description of the environment, while it is hard to differentiate individual points. HD map has a higher resolution compared to digital maps and makes the route familiar to the robot, where the key is to fuse different sensor information to minimize the errors in each grid cell. Once the HD map is built, a particle filter method can be applied to localize the robot in real-time correlated with LiDAR measurement. However, the LiDAR performance may be severely affected by weather conditions (e.g., rain, snow) and bring localization error.

Cameras are used for localization as well. The pipeline of vision-based localization is simplified as follows: 1) by triangulating stereo image pairs, a disparity map is obtained and used to derive depth information for each point; 2) by matching salient features between successive stereo image frames in order to establish correlations between feature points in different frames, the motion between the past two frames is estimated; and 3) by comparing the salient features against those in the known map, the current position of the robot is derived [30].

Apart from these techniques, sensor fusion strategy is also often utilized to combine multiple sensors for localization, which can improve the reliability and robustness of robot [31], [32].

E. Planning and Control

The planning and control layer is responsible for generating trajectory plans and passing the control commands

based on the original and destination of the robot. Broadly, prediction and routing modules are also included here, where their outputs are fed into downstream planning and control layers as input. The prediction module is responsible for predicting the future behavior of surrounding objects identified by the perception layer. The routing module can be a lane-level routing based on lane segmentation of the HD maps for autonomous vehicles.

Planning and Control layers usually include behavioral decision, motion planning and feedback control. The mission of the behavioral decision module is to make effective and safe decisions by leveraging all various input data sources. Bayesian models are becoming more and more popular and have been applied in recent works [33], [34]. Among the Bayesian models, Markov Decision Process (MDP) and Partially Observable Markov Decision Process (POMDP) are the widely applied methods in modeling robot behavior. The task of motion planning is to generate a trajectory and send it to the feedback control for execution. The planned trajectory is usually specified and represented as a sequence of planned trajectory points, and each of these points contains attributes like location, time, speed, etc. Low-dimensional motion planning problems can be solved with grid-based algorithms (such as Dijkstra [35] or A* [36]) or geometric algorithms. High-dimensional motion planning problems can be dealt with sampling-based algorithms, such as Rapidly-exploring Random Tree (RRT) [37] and Probabilistic Roadmap (PRM) [38], which can avoid the problem of local minima. Reward-based algorithms, such as the Markov decision process (MDP), can also generate the optimal path by maximizing cumulative future rewards. The goal of feedback control is to track the difference between the actual pose and the pose on the predefined trajectory by continuous feedback. The most typical and widely used algorithm in robot feedback control is PID.

While optimization-based approaches enjoy mainstream appeal in solving motion planning and control problems, learning-based approaches [39]–[43] are becoming increasingly popular with recent developments in artificial intelligence. Learning-based methods, such as reinforcement learning, can naturally make full use of historical data and iteratively interact with the environment through actions to deal with complex scenarios. Some model the behavioral level decisions via reinforcement learning [41], [43], while other approaches directly work on motion planning trajectory output or even direct feedback control signals [40]. Q-learning [44], Actor-Critic learning [45], policy gradient [38] are some popular algorithms in reinforcement learning.

III. Perception on FPGA

A. Overview

Perception is related to many robotic applications where sensory data and artificial intelligence techniques are involved. Examples of such applications include stereo matching, object detection, scene understanding, semantic classification, etc. The recent developments in machine learning, especially deep learning, have exposed robotic perception systems to more tasks. In this section, we will focus on the recent algorithms and FPGA implementations in the stereo vision system, which is one of the key components in the robotic perception stage.

Real-time and robust stereo vision systems are increasingly popular and widely used in many perception applications, e.g., robotics navigation, obstacle avoidance [46] and scene reconstruction [47]–[49]. The purpose of stereo vision systems is to obtain 3 D structure information of the scene using stereoscopic ranging techniques. The system usually has two cameras to capture images from two points of view within the same scenario. The disparities between the corresponding pixels in two stereo images are searched using stereo matching algorithms. Then the depth information can be calculated from the inverse of this disparity.

Throughout the whole pipeline, stereo matching is the bottleneck and time-consuming stage. The stereo matching algorithms can be mainly classified into two categories: local algorithms [50]–[56] and global algorithms [57]–[61]. Local methods compute the disparities by only processing and matching the pixels around the points of interest within windows. They are fast and computationally-cheap, and the lack of pixel dependencies makes them suitable for parallel acceleration. However, they may suffer in textureless areas and occluded regions, which will result in incorrect disparities estimation.

In contrast, global methods compute the disparities by matching all other pixels and minimizing a global cost function. They can achieve much higher accuracy than local methods. However, they tend to come at high computation cost and require much more resources due to their large and irregular memory access as well as the sequential nature of algorithms, thus not suitable for real-time and low-power applications. Many research works in stereo systems focus on the speed and accuracy improvement of stereo matching algorithms, and some of the implementations are summarized in Tab. I

B. Local Stereo Matching on FPGA

Local algorithms are usually based on correlation, where the process involves finding matching pixels in the left and right image patches by aggregating costs within a

specific region. There are many ways for cost aggregation, such as the sum of absolute differences (SAD) [62], the sum of squared differences (SSD) [63], normalized cross-correlation (NCC) [64], and census transform (CT) [65]. Many FPGA implementations are based on these methods. Jin et al. [66] develop a real-time stereo vision system based on census rank transformation matching cost for 640×480 resolution images. Zhang et al. [67] propose a real-time high definition stereo matching design on FPGA based on mini-census transform and cross-based cost aggregation, which achieves 60 fps at 1024×768 pixel stereo images. The implementation of Honegger et al. [68] achieves 127 fps at 376×240 pixel resolution with 32 disparity levels based on block matching. Jin et al. [69] further achieve 507.9 fps for

640×480 resolution images by applying fast local consistent dense stereo functions and cost aggregation.

C. Global Stereo Matching on FPGA

Global algorithms can provide state-of-the-art accuracy and disparity map quality, however, they are usually processed through high computational-intensive optimization techniques or massive convolutional neural networks, making them difficult to be deployed on resource-limited embedded systems for real-time applications. However, some works have attempted to implement global algorithms on FPGA for better performance. Park et al. [70] present a trellis-based stereo matching system on FPGA with a low error rate and achieved 30 fps at 320×240 resolution with 128 disparity levels. Sabihuddin et al. [71]

Table I. Comparison of Stereo Vision Systems on FPGA platforms, across local stereo matching, global stereo matching, semi-global stereo matching (SGM) and efficient large-scale stereo matching (ELAS) algorithms. The results reported in each design are evaluated by frame rate (fps), image resolution (width $\times$ height), disparity levels, million disparity estimations per second (MDE/s), power (W), resource utilization (logic% and BRAM%) and hardware platforms, where $MDE/s = \text{width} \times \text{height} \times \text{fps} \times \text{disparity}$.

Algorithm	Reference	Frame Rate (FPS)	Image Resolution (Width $\times$ Height)	Disparity Level	MDE/s	Power (W)	Resource(%) Logic/BRAM	FPGA Platform
Local Stereo Matching	Jin et al. [66]	230	640×480	64	4522	–	34.0/95.0	Xilinx Virtex-4
	Zhang et al. [67]	60	1024×768	64	3020	1.56	61.8/67.0	XC4VLX200-10
	[67]	127	376×240	32	367	2.8	49.0/68.0	Altera EP3SL150
	Honegger et al. [68]	507.9	640×480	60	9362	3.35	81.0/39.7	Altera Cyclone III EP3C80
	Jin et al. [69]							Xilinx Vertex-6
Global Stereo Matching	Park et al. [70]	30	320×240	128	295	–	–/–	Xilinx Virtex II pro-100
	Sabihuddin et al. [71]	63.54	640×480	128	2498	–	23.0/58.0	Xilinx XC2VP100
		32	640×480	60	590	1.40	72.0/46.0	Xilinx XC4VLX160
	Jin et al. [72]	30	1920×1680	60	5806	–	84.8/91.9	Xilinx Kintex 7
	Zha et al. [59]	30	1024×768	64	1510	0.17	57.0/53.0	Xilinx Virtex-7
	Puglia et al. [60]							XC7Z020CLG484-1
Semi-Global Stereo Matching	Banz et al. [74]	37	640×480	128	1455	2.31	51.2/43.2	Xilinx Virtex-5
	Wang et al. [75]	42.61	1600×1200	128	10472	2.79	93.9/97.3	Altera 5SGSMD5K2
		127	1024×768	128	12784	–	–/–	Altera Cyclone IV
	Cambuim et al. [76]	72	1242×375	128	4292	3.94	75.7/30.7	Xilinx ZC706
		25	1024×768	256	5033	6.5	50.0/38.0	Altera Cyclone IV
	Rahnama et al. [77]	147	1242×375	64	4382	9.8	68.7/38.7	GX, Stratix IV GX
								Xilinx Ultrascale + ZCU102
	Cambuim et al. [78]							
	Zhao et al. [79]							
Efficient Large-Scale Stereo Matching	Rahnama et al. [80]	47	1242×375	–	–	2.91	11.9/15.7	Xilinx ZC706
	Rahnama et al. [81]	50	1242×375	–	–	5	70.7/8.7	Xilinx ZCU104

implement a dynamic programming maximum likelihood (DPML) based hardware architecture for dense binocular disparity estimation and achieved 63.54 fps at 640×480 pixel resolution with 128 disparity levels. The design in Jin et al. [72] uses a tree-structured dynamic programming method, and achieves 58.7 fps at 640×480 resolution as well as a low error rate. Recently, some other adaptations of global approaches for FPGA-implementation have been proposed, such as cross-trees [59], dynamic programming for DNA sequence alignment [60], and graph cuts [73], where all of these implementations achieve real-time processing.

D. Semi-Global Matching on FPGA

Semi-global matching (SGM) [82] bridges the gap between local and global methods, and achieves a notable improvement in accuracy. SGM calculates the initial matching disparities by comparing local pixels, and then approximates an image-wide smoothness constraint with global optimization, which can obtain more robust disparity maps through this combination. There are several critical challenges for implementing SGM on hardware, e.g., data dependence, high complexity, and large storage, so this is an active research field with recent works proposing FPGA-friendly variants of SGM [74], [75], [83]–[85].

Banz et al. [74] propose a systolic-array based hardware architecture for SGM disparity estimation along with a two-dimensional parallelization concept for SGM. This design achieves 30 fps performance at 640×480 pixel images with a 128-disparity range on the Xilinx Virtex-5 FPGA platform. Wang et al. [75] implement a complete real-time FPGA-based hardware system that supports both absolute difference-census cost initialization, cross-based cost aggregation and semi-global optimization. The system achieves 67 fps at 1024×768 resolution with 96 disparity levels on the Altera Stratix-IV FPGA platform, and 42 fps at 1600×1200 resolution with 128 disparity levels on the Altera Stratix-V FPGA platform. The design in Cambuim et al. [76] uses a scalable systolic-array based architecture for SGM based on the Cyclone IV FPGA platform, and it achieves a 127 fps image delivering rate in 1024×768 pixel HD resolution with 128 disparity levels. The key point of this design is the combination of disparity and multi-level parallelisms such as image line processing to deal with data dependency and irregular data access pattern problems in SGM. Later, to improve the robustness of SGM and achieve a more accurate stereo matching, Cambuim et al. [78] combine the sampling-insensitive absolute difference in the pre-processing phase, and propose a novel streaming architecture to detect noisy and occluded regions in the post-processing phase. The

design is evaluated in a full stereo vision system using two heterogeneous platforms, DE2i-150 and DE4, and achieves a 25 fps processing rate in 1024×768 HD maps with 256 disparity levels.

While most existing SGM designs on FPGA are implemented using the register-transfer level (RTL), some works leveraged the high-level synthesis (HLS) approach. Rahnema et al. [77] implement an SGM variation on FPGA using HLS, which achieves 72 fps speed at 1242×375 pixel size with 128 disparity levels. To reduce the design effort and achieve an appropriate balance among speed, accuracy and hardware cost, Zhao et al. [79] recently propose FP-Stereo for building high-performance SGM pipelines on FPGAs automatically. A series of optimization techniques are applied in this system to exploit parallelism and reduce resource consumption. Compared to GPU designs [86], it achieves the same accuracy at a competitive speed while consuming much less energy.

To compare these implementations, the depth quality of are evaluated on Middlebury Benchmark [87], with four image pairs *Tsukuba*, *Venus*, *Teddy*, *Cones*. As shown in Tab. II, there is a general trade-off between accuracy and processing speed. The stereo vision system designs in Tab. I are drawn as points in Fig. 2 (if both power and speed number are reported), using $\log_{10}(\text{power})$ as x -coordinate and $\log_{10}(\text{speed})$ as y -coordinate ($y - x = \log_{10}(\text{energy_efficiency})$). Besides FPGA-based implementations, we also plot GPU and CPU experimental results as a comparison to FPGA designs' performance. In general, local and semi-global stereo matching designs have achieved higher performance and energy efficiency than global stereo matching designs. As introduced in section III-C, global stereo matching algorithms usually involve massive computational-intensive optimization techniques. Even for the same design, varying design parameters (e.g., window size) may result in a 10x difference in energy efficiency. Compared to GPU and CPU-based designs, FPGA-based designs have achieved higher energy efficiency, and the speed of many FPGA implementations have surpassed general-purpose processors.

E. Efficient Large-Scale Stereo Matching on FPGA

Another popular stereo matching algorithm that offers a good trade-off between speed and accuracy is Efficient Large-Scale Stereo Matching (ELAS) [90], which is currently one of the fastest and accurate CPU algorithms concerning the resolution on Middlebury dataset. ELAS implements a slanted plane prior very effectively while its dense estimation of depth is completely decomposable over all pixels, which make it attractive for easily parallelized.

Rahnama et al. [80] first implement and evaluate an FPGA accelerated adaptation of the ELAS algorithm, which achieved a frame rate of 47 fps (up to 30× compared high-end CPU) while consuming under 4 W of power. By taking advantage of different components on the SoC, several elaboration blocks such as feature extraction and dense matching are executed on FPGA, while I/O and other conditional/sequential blocks are executed on ARM-core CPU. The authors also reveal the strategy to accelerate complex and computationally diverse algorithms for low power and real-time systems by collaboratively utilizing different compute components. Later, by leveraging and combining the best features of SGM and ELAS-based methods, Rahnama et al. [81] propose a sophisticated stereo approach and achieve an 8.7% error rate on the challenging KITTI 2015 dataset at over 50 fps, with a power consumption of only 4.5 W.

F. CNN-Based Stereo Vision System on FPGA

Convolutional neural networks (CNNs) have been demonstrated to perform very well on many vision tasks such as image classification, object detection, and semantic segmentation. Recently, CNN has also been utilized in stereo estimation [91], [92] and stereo matching [93]. CNN is applied to determine SGM penalties [94], estimate real-time optical flow disparity [95] and predict cost volume computation and aggregation [96].

CNN has been deployed on FPGA platforms in several works [97]–[100], with an example of lightweight YOLOv2 for object detection [101]. Nakahara et al. implement a pipelined-based architecture for lightweight YOLOv2 with a binarized CNN on Xilinx ZCU102 FPGA platform. This design achieves a 40.81 fps object detection speed, which is 177.4× faster than ARM Cortex-A57 and 27.5× faster than NVIDIA Pascal embedded GPU. Many FPGA-based CNN accelerator implementations have been summarized in [15].

IV. Localization on FPGA

A. Overview

For robots, one of the most critical tasks is localization and mapping. Simultaneous Localization and Mapping (SLAM) is an advanced robot navigation algorithm for constructing or updating a map of unknown surroundings while simultaneously keeping tracking the robot's location. Localization and mapping are two concurrent tasks and cannot be solved independently from each other. Localizing a robot requires a sufficiently detailed map, and constructing or updating or a map

Table II.
A comparison between different designs on performance (MDE/s) and accuracy results on Middlebury Benchmark.
(The lower of average bad pixel rate means the better stereo matching performance.)

Reference	MDE/s	Tsukuba			Venus			Teddy			Cones			Average Bad Pixel Rate
		nonocc ¹	all ²	disc ³	nonocc	all	disc	nonocc	all	disc	nonocc	all	disc	
Shan et al. [88]	15437	—	24.5	—	—	15.7	—	—	15.1	—	—	14.1	—	all = 17.3
Shan et al. [89]	13076	3.62	4.15	14.0	0.48	0.87	2.79	7.54	14.7	19.4	3.51	11.1	9.64	7.65
Wang et al. [75]	10472	2.39	3.27	8.87	0.38	0.89	1.92	6.08	12.1	15.4	2.12	7.74	6.19	5.61
Jin et al. [69]	9362	1.66	2.17	7.64	0.4	0.6	1.95	6.79	12.4	17.1	3.34	8.97	9.62	6.05
Jin et al. [66]	4522	9.79	11.6	20.3	3.59	5.27	36.8	12.5	21.5	30.6	7.34	17.6	21.0	17.2
Zhang et al. [67]	3020	3.84	4.34	14.2	1.2	1.68	5.62	7.17	12.6	17.4	5.41	11.0	13.9	8.2
Banz et al. [74]	1455	4.1	—	—	2.7	—	—	11.4	—	—	8.4	—	—	nonocc = 6.7
Jin et al. [72]	590	1.43	2.51	6.6	2.37	2.97	13.1	8.11	13.6	15.5	8.12	13.8	16.4	8.71

¹nonocc: average percentage of bad pixels in non-occluded regions.

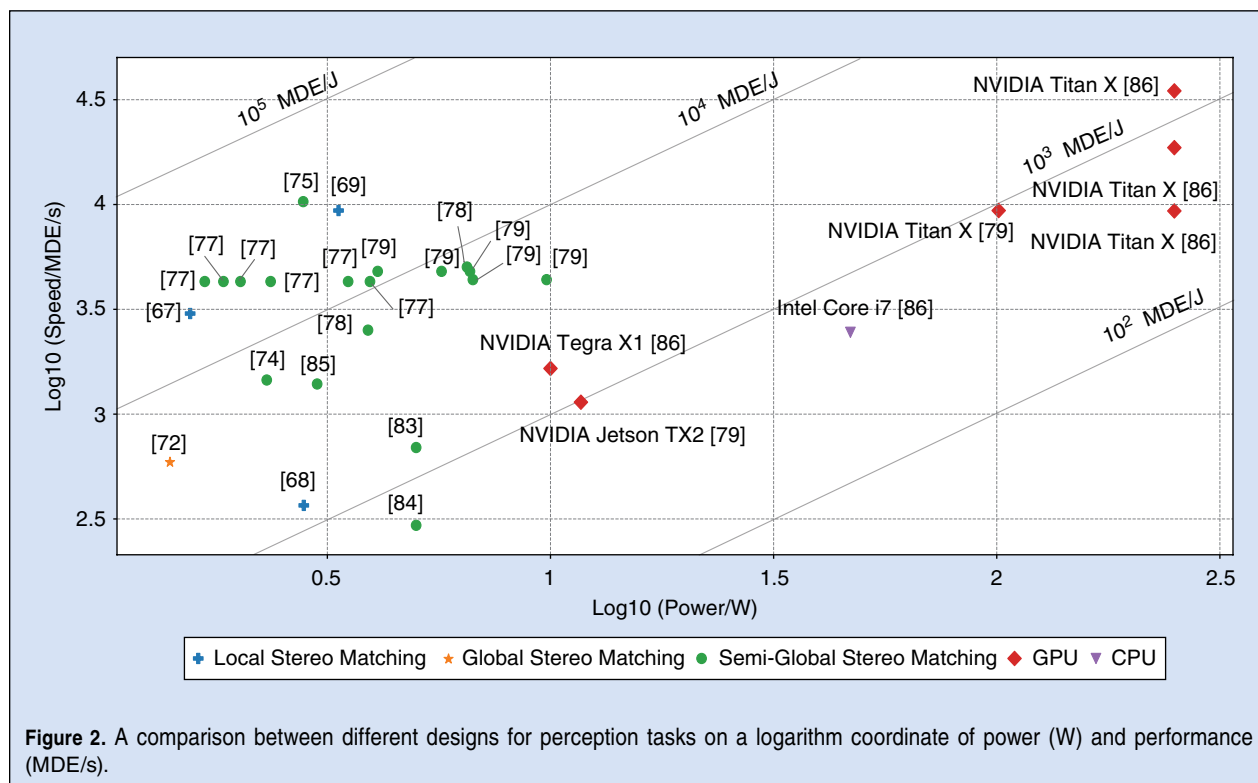
²all: average percentage of bad pixels in all regions.

³disc: average percentage of bad pixels in discontinuous regions.

Many SLAM algorithms have been developed in the last decades to improve the accuracy and robustness, and its implementation comes in a diverse set of sizes and shapes. One end of the spectrum is dense SLAM algorithms [102]–[105], which can generate high-quality maps of the environment with complex computations. Dense SLAM algorithms usually are executed on powerful and high-performance machines to ensure real-time performance. At the same time, the intensive computation characteristic makes dense SLAM hard to deploy on edge devices. The other end of the spectrum is sparse SLAM [106]–[109], which is computationally light by only selecting limited numbers of landmarks or features.

A typical SLAM system includes two components: the front-end and the back-end, which are with different computational characteristics. The front-end associates

A critical challenge to mobile robot localization is accuracy and efficiency under stringent power and resource constraints. To avoid losing tracked features due to large motions between consecutive frames, SLAM systems need to process sensory data at a high frame rate. For example, open data sets for evaluating localization algorithms [112], [113] for drones and vehicles provide images at 10 to 20 fps. Low power computing systems are always required to extend the battery life of mobile robots. Most SLAM algorithms are developed on CPU or GPU platforms, of which power consumption is hundreds of Watts. To execute SLAM efficiently on mobile robots and meet real-time and power constraints, specialized chips and accelerators have been developed. FPGA SoCs provide rich sensor interfaces, dedicated hardware logic and programmability, hence they have been explored in diverse ways in recent years.



We summarize and discuss FPGA-based accelerators for SLAMs in the following sections.

B. Dense SLAM on FPGA

Dense SLAM can construct high quality and complete models of the environment, and most of them are running in high-end hardware platforms (especially GPU). One of the representative real-time dense SLAM algorithms is KinectFusion [114], which was released by Microsoft in 2011. As a scene reconstruction algorithm, it continuously updates the global 3D map and tracks the location of depth cameras within the surrounding environment. KinectFusion is generally composed of three algorithms: ray-casting algorithm for generating graphics from surface information, iterative closest point (ICP) algorithm for camera-tracking and volumetric integration (VI) algorithm for integrating depth streams into the 3D surface. Several works have attempted to implement real-time dense SLAM algorithms on a heterogeneous system with FPGA embedded.

Several works implement computationally intensive components of dense SLAMs, such as ICP and VI, on FPGA to accelerate the critical path. Belshaw [102] presents an FPGA implementation of the ICP algorithm, which achieves over 200 fps tracking speed with low tracking errors. This design divides the ICP algorithm into filtering, nearest neighbor, transform recovery and transform application stages. It leverages fixed-point arithmetic and the power of two data points to utilize FPFA logic efficiently. Williams [103] notices that the nearest neighbor search takes up the majority of ICP runtime, and then proposes two hybrid CPU-FPGA architectures to accelerate the bottleneck of the ICP-SLAM algorithm. The implementation is performed with Vivado HLS, a high-level synthesis tool from Xilinx, and achieves a maximum 17.22× speedup over the ARM software implementation. Hoorick [104] presents an FPGA-based heterogeneous framework using a similar HLS method to accelerate the KinectFusion algorithm and explored various ways of dataflow and data management patterns. Gautier et al. [105] embed both ICP and VI algorithms on an Altera Stratix V FPGA by using the OpenCL language and the Altera OpenCL SDK. This design was a heterogeneous system with NVIDIA GTX 760 GPU and Altera Stratix V FPGA. By distributing different workloads on different parts of SoC, the entire system achieves up to 28 fps real-time speed.

C. Sparse SLAM on FPGA

Sparse SLAM algorithms usually use a small set of features to track and maintain a sparse map of surrounding environments. These algorithms exhibit lower power consumption but are limited to the localization accuracy.

1) EKF-SLAM

EKF-SLAM [106] is a class of algorithms that utilizes the extended Kalman Filter (EKF) for SLAM. EKF-SLAM algorithms are typically feature-based and use the maximum likelihood algorithm for data association. Several heterogeneous architectures using multi-core CPUs, GPUs, DSPs, and FPGAs are proposed to accelerate the complex computation in EKF-SLAM algorithms. Bonato et al. [115] presents the first FPGA-based architecture for the EKF-SLAM based algorithm that is capable of processing 2 D maps at up to 1800 features at real-time with a frequency of 14 Hz, compared to 572 features with Pentium CPU and 131 features with ARM. They analyze the computational complexity and memory bandwidth requirements for FPGA-based EKF-SLAM, and then propose an architecture with a parallel memory access pattern to accelerate the matrix multiplication. This design achieves two orders of magnitude more power-efficient than a general-purpose processor.

Similarly, Tertei et al. [116] propose an efficient FPGA-SoC hardware architecture for matrix multiplication with systolic arrays to accelerate EKF-SLAM algorithms. The setup of this design is a PLB peripheral to PPC440 hardcore embedded processor on a Virtex5 FPGA, and it achieves a 7.3× speedup with a processing frequency of 44 Hz compared to the pure software implementation. Later, taking into account the symmetry in cross-covariance matrix-related computations, Tertei et al. [117] improve the previous implementation to further reduce the computational time and on-chip memory storage on Zynq-7020 FPGA.

DSP is also leveraged in some works to accelerate EKF-SLAM algorithms. Vincke et al. [118] implement an efficient implementation of EKF-SLAM on a low-cost heterogeneous architecture system consisting of a single-core ARM processor with a SIMD coprocessor and a DSP core. The EKF-SLAM program is partitioned into different functional blocks based on the profiling characteristics results. Compared to a non-optimized ARM implementation, this design achieved 4.7× speed up from 12 fps to 57 fps. In a later work, Vincke et al. [119] replace the single-core ARM with a double-core ARM to optimize the non-optimized blocks using the OpenMP library. This design achieves a 2.75× speedup compared to non-optimized implementation.

2) ORB-SLAM

ORB-SLAM [107] is an accurate and widely-used sparse SLAM algorithm for monocular, stereo, and RGB-D cameras. Its framework usually consists of five main procedures: feature extraction, feature matching, pose estimation, pose optimization and map updating. Based on

the profiling results on a quad-core ARM v8 mobile SoC, feature extraction is the most computation-intensive stage in the ORB-SLAM system, which consumes more than half of CPU resources and energy budget [120].

ORB based feature extraction algorithm usually consists of two parts, namely Oriented Feature from Accelerated Segment Test (oFAST) [121] based feature detection and Binary Robust Independent Elementary (BRIEF) [122] based feature descriptors computation. To accelerate this bottleneck, Fang et al. [120] design and implement a hardware ORB feature extractor and achieved a great balance between performance and energy consumption, which outperforms ARM Krait by 51% and Intel Core i5 by 41% in computation latency as well as outperforms ARM Krait by 10% and Intel Core i5 by 83% in energy consumption. Liu et al. [123] propose an energy-efficient FPGA implementation eSLAM to accelerate both feature extraction and feature matching stages. This design achieves up to 3× and 31× speedup in frame rate, as well as up to 71× and 25× in energy efficiency improvement compared to Intel i7 and ARM Cortex-A9 CPUs, respectively. This eSLAM design utilizes a rotationally symmetric ORB descriptor pattern to make the algorithm more hardware-friendly, resulting in a 39% less latency compared to [120]. Rescheduling and parallelizing optimization techniques are also exploited to improve the computation throughput in eSLAM design.

Scale-invariant feature transform (SIFT) and Harris corner detector are also commonly-used feature extraction methods. SIFT is invariant to rotation and translation. Gu et al. [109] implement SIFT-feature based SLAM algorithm on FPGA and accelerate the matrix computation part to achieve speedup. Harris corner detector is used to extract corners and features of an image, and Schulz et al. [124] propose an implementation of Harris and Stephen corner detector optimized for an embedded SoC platform that integrates a multicore ARM processor with Zynq-7000 FPGA. Taking into account I/O requirements and the advantage of parallelization and pipeline, this design achieves a speedup of 1.77 compared to dual-core ARM processors.

3) Fast-SLAM

One of the key limitations of EKF-SLAM is its computational complexity since EKF-SLAM requires time quadratic in the number of landmarks to incorporate each sensor update. In 2002, Montemerlo et al. [108] propose an efficient SLAM algorithm called Fast-SLAM. Fast-SLAM decomposes the SLAM problem into a robot localization problem and a landmark estimation problem. It recursively estimates the full posterior distribution over landmark positions and robot path with a logarithmic scale.

Abouzahir et al. [125] implement Fast-SLAM 2.0 on a CPU-GPGPU-based SoC architecture. The algorithm is partitioned into function blocks, and each of them is implemented on the CPU or GPU accordingly. This optimized and efficient CPU-GPGPU partitioning enables accurate localization and a 37× execution speedup compared to non-optimized implementation on a single-core CPU. Further, Abouzahir et al. [126] perform a complete study of the processing time of different SLAM algorithms under popular embedded devices, and demonstrate that Fast-SLAM2.0 allowed a compromise between the consistency of localization results and computation time. This algorithm is then optimized and implemented on GPU and FPGA using HLS and parallel computing frameworks OpenCL and OpenGL. It is observed that the global processing time of FastSLAM2.0 on FPGA implementations achieves 7.5× acceleration compared to high-end GPU. The processing frequency achieves 102 fps and meets the real-time performance constraints of an operated robot.

4) VO-SLAM

The visual odometry based SLAM algorithm (VO-SLAM) also belongs to the Sparse SLAM class with low computational complexity. Gu et al. [109] implement the VO-SLAM algorithm on a DE3 board (Altera Stratix III) to perform drift-free pose estimation, resulting in localization results accurate to 1-2cm. A Nios II soft-core is used as a master processor. The authors design a dedicated matrix accelerator and propose a hierarchical matrix computing mechanism to support application requirements. This design achieves a processing speed of 31 fps with 30000 global map features, and 10× energy saving for each frame processing compared to Intel i7 CPU.

D Semi-Dense SLAM on FPGA

Semi-dense SLAM algorithms have emerged to provide a compromise between sparse SLAM and dense SLAM algorithms, which attempt to achieve improved efficiency and dense point clouds. However, they are still usually computationally intensive and require multicore CPUs for real-time processing.

Large-Scale Direct Monocular SLAM (LSD-SLAM) is one of the state-of-the-art and widely-used semi-dense SLAM algorithms, and it directly operates on image intensities for both tracking and mapping problems. The camera is tracked by direct image alignment, while geometry is estimated from semi-dense depth maps acquired by filtering over multiple stereo pixel-wise comparisons.

Several works have explored LSD-SLAM FPGA-SoC implementation. Boikos et al. [127] investigate the

performance and acceleration opportunities for LSD-SLAM in the SoC system. This design achieves an average framerate of more than 4 fps for a resolution of 320×240 with an estimated power of less than 1 W, which is a $2\times$ acceleration and more than $4.3\times$ energy efficiency compared to a software version running on embedded CPUs. The author also notes that the communication between two accelerators is via DDR memory since the produced intermediate data is too large to be fully cached on the FPGA. Hence, it is important to optimize the memory architecture (e.g., data movement and caching techniques) to ensure the scalability and compatibility of the design.

To further improve the performance of [127], Boikos et al. [128] re-implement the design using a dataflow architecture and distributed asynchronous blocks to allow the memory system and the custom hardware pipelines to function at peak efficiency. This implementation can process and track more than 22 fps with an embedded power budget and achieves a $5\times$ speedup over [127].

Furthermore, Boikos et al. [129] combine a scalable depth estimation with direct semi-dense SLAM architecture and propose a complete accelerator for semi-dense SLAM on FPGA. This architecture achieved more than 60 fps at the resolution of 640×480 and an order of magnitude power consumption improvement compared to Intel i7-4770 CPU. This implementation leverages multi-rate and multi-modal units to deal with LSD-SLAM's complex control flow. A new dataflow paradigm is also proposed where the kernel is linked with a single consumer and a single producer to achieve high efficiency.

E. CNN-Based SLAM on FPGA

Recently, CNNs have made significant progress in the perception and localization ability of the robots compared to handcrafted methods. Take one of the main SLAM components, feature extraction, as an example, the CNN-based approach SuperPoint [130] can achieve 10%-30% higher matching accuracy compared to handcrafted ORB. Other CNN-based methods, such as DeepDesc [131] and GeM [132], also present significant improvements in feature extraction and descriptor generation stage. However, CNN has a much higher computational complexity and requires more memory footprint.

Several works have explored to deploy CNN on FPGAs. Xilinx DPU [133] is one of the state-of-the-art programmable dedicated to CNN, which has a specialized instruction set and works efficiently across various CNN topologies. Xu et al. [134] propose a hardware architecture to accelerate CNN-based feature extraction SuperPoint on the Xilinx ZCU102 platform and achieve 20 fps in a real-time SLAM system. The key point of this design is an optimized software dataflow to deal with the ex-

tra post-processing operations within CNN-based feature extraction networks. 8-bit fixed-point numerics are leveraged in the post-processing operations and CNN backbone. Similar hardware-oriented model compression techniques (e.g., data quantization and weight reduction) have been widely adopted in robotics and CNN related designs [135]–[142].

Yu et al. [143] build a CNN-based monocular decentralized-SLAM (DSLAM) on the Xilinx ZCU102 MPSoC platform with DPU. DSLAM is usually used in multi-robot applications that can share environment information and locations between agents. To accelerate the main components in DSLAM, namely visual odometry (VO) and decentralized place recognition (DPR), the authors adopt CNN-based Depth-VO-Feat [144] and NetVLAD [145] to replace handcrafted approaches and propose a cross-component pipeline scheduling algorithm to improve the performance.

To enable multi-tasking processing in embedded robots on CNN accelerators, Yu et al. [146] further propose an Interruptible CNN accelerator (INCA) with a novel virtual-instruction-based interrupt method. Feature extraction and place recognition of DSLAM are deployed and accelerated on the same CNN accelerator of the embedded FPGA system, and the interrupt response latency is reduced by 1%.

F. Bundle Adjustment on FPGA

Besides the hardware implementation of the frontend of the SLAM system, several works investigate to accelerate the backend of the SLAM system, mainly Bundle Adjustment (BA). BA is heavily used in robot localization [107], [147], autonomous driving [148], space exploration missions [149] and some commercial products [150], where it is usually employed in the last stage of the processing pipeline to refine camera trajectories and 3D structures further.

Essentially, BA is a massive joint non-linear optimization problem that usually consumes a significant amount of power and processing time in both offline visual reconstruction and real-time localization applications.

Several works aim to accelerate BA on multi-core CPUs or GPUs using parallel or distributed computing techniques. Jeong et al. [151] exploit efficient memory handling and fast block-based linear solving, and propose a novel embedded point iterations method, which substantially improves the BA performance on CPU. Wu et al. [152] present a multi-core parallel processing solution running on CPUs and GPUs. The matrix-vector product is carefully restructured in this design to reduce memory requirements and compute latency substantially. Eriksson et al. [153] propose a distributed approach for very large scale global bundle adjustment

computation to achieve BA performance improvement. The authors present a consensus framework using the proximal splitting method to reduce the computational cost. Similarly, Zhang et al. [154] propose a distributed formulation to accelerate the global BA computation without much distributed computing communication overhead.

To better deploy BA in embedded systems with strict power and real-time constraints, recent works explore BA algorithm acceleration using specialized hardware. The design in [155] implements both the image frontend and BA backend of a VIO algorithm on a single-chip for nano-drone scale applications. Liu et al. [156] propose a hardware-software co-designed BA hardware accelerator and its implementation on an embedded FPGA-SoC to achieve higher performance and power efficiency simultaneously. Especially, a co-observation optimization technique and a hardware-friendly differentiation method are proposed to accelerate BA operations with optimized usage of memory and computation resources. Sun et al. [157] present a hardware architecture running local BA on FPGAs, which works without external memory access and refines both cameras poses and 3D map points simultaneously.

G. Discussion

We summarize FPGA based SLAM systems in Tab. III. It only includes works that implement the whole SLAM on an FPGA and provide overall performance and power evaluation. The works in the table adopt a similar FPGA-SoC architecture that accelerates computationally intensive components by FPGA fabrics and offloads others works to embedded processors on FPGAs. Compared with sparse method, the semi-dense implementation has lower frame rate, which is mainly due to the high resolution data processed in the pipeline. Due to the high frame rates and low power consumption, sparse SLAM FPGA have been used in drones and autonomous vehicles [16]. The two sparse SLAM implementations achieve similar performance in terms of frame rate. Compared with the ORB design, the VO SLAM design includes pre-processing and outliers removal hardware, such as image rectification and RANSAC, which lead to a more accurate but power inefficient implementation.

V. Planning and Control on FPGA

A. Overview

Planning and control are the modules that compute how the robot should maneuver itself. They usually include behavioral decision, motion planning and feedback control kernels. Without loss of generality, we focus on the motion planning algorithms and their FPGA implementations in this section.

As a fundamental problem in the robotic system, motion planning aims to find the optimal collision-free path from the current position to a goal position for a robot in complex surroundings. Generally, motion planning contains three steps, namely roadmap construction, collision detection and graph search [38], [158]. Motion planning will become a relatively complicated problem when robots work with a high degree of freedom (DOF) configurations since the search space will be exponentially increased. Typically, state-of-the-art CPU-based approaches take a few seconds to find a collision-free trajectory [159]–[161], making the existing motion planning algorithms too slow to meet the real-time requirement for complex robot tasks and environments. Several works have investigated approaches to speed up motion planning, either for each stage or whole pipeline.

B. Roadmap Construction

In the roadmap construction step, the planner generates a set of states in the robot's configuration space and then connects them with edges to construct a general-purpose roadmap in the obstacle-free space. Each state represents a robot's configuration, and each edge represents a possible robot movement. Conventional algorithms build the roadmap by randomly sampling poses from configuration space at runtime to navigate around the obstacles present at that time.

Several works explore roadmap construction acceleration. Yershova et al. [162] improve the nearest neighbor search to accelerate roadmap construction by orders of magnitude compared to the naive nearest-neighbor searching. Wang et al. [163] reduce the computation workload by trimming roadmap edges and keeping the roadmap to a reasonable size to achieve speedup. Different from online runtime approaches, Murray et al. [164]

completely remove the runtime latency by conducting the roadmap construction only once at the design time. A more general and much larger roadmap is pre-computed and allows for fast and successive

Table III.
Comparison of FPGA SLAM Systems.

	Method	Platform	Frame Rate	Power	Indoor Error
Boikos et al. [127]	Semi Dense	Xilinx Zynq 7020 SoC	4.5 fps	2.5 W	na
Liu et al. [123]	ORB	Xilinx Zynq 7000 SoC	31 fps	1.9 W	4.5 cm
Gu et al. [109]	VO	Altera Stratix III	31 fps	5.9 W	2 cm

queries in complex environments without reprogramming the accelerator during runtime.

C. Collision Detection

In the collision detection step, the planner determines whether there are potential collisions with the environment or the robot itself during movement. Specifically, collision detection is the primary challenge in motion planning, which often comprises 90% of the processing time [165].

Several works leverage data parallelization computing on GPUs to achieve speedup [165]–[167]. For example, Bialkowski et al. [165] divide the RRT* algorithm of collision detection tasks into three parallel dimensions and construct thread block grids to execute collision computations simultaneously. However, GPU can only provide a constant speedup factor due to the core limitations, which is still hard to achieve the real-time requirement.

Recently, [168]–[170] develop high-efficiency custom hardware implementations based on the FPGA system. Atay and Bayazit [168] focus on directly accelerating the PRM algorithm on FPGA by creating functional units to perform the random sampling, nearest neighbor search and parallelizing triangle-triangle testing. However, this design cannot be reconfigured at runtime, and the huge resource demands make it fail to support a large roadmap. Murray et al. [169] present a novel microarchitecture for an FPGA-based accelerator to speed up collision detection by creating a specialized circuit for each motion in the roadmap. This solution achieves sub-millisecond speed for motion planning query and improves the power consumption by more than one order of magnitude, which is sufficient to enable real-time robotics applications.

Besides real-time constraint, motion planning algorithms also have flexibility requirements to make the robots adapt to dynamic environments. Dadu-P [170] build a scalable motion planning accelerator to attain both high efficiency and flexibility, where a motion plan can be solved in around 300 microseconds in a dynamic environment. A hardware-friendly data structure representing roadmap edges is adopted to achieve flexibility, and a batched processing as well as a priority-rating method are proposed to achieve high efficiency. But this design comprises a 25× latency increase to make it retargetable to different robots and scenarios due to the external memory access. Murray et al. [164] develop a fully retargetable microarchitecture of collision detection and graph search accelerator that can perform motion planning in less than 3 ms with a modest power consumption of 35 W. This design divides the collision detection workflow into two stages. The

collision detection results for the discretized roadmap are precomputed in the first stage before runtime, and then the collision detection accelerator streams in the voxels of obstacles and the edges of flags which are in collision at runtime.

D. Graph Search

After collision detection, the planner will try to find the shortest and safe path from the start position to the target position based on the obtained collision-free roadmap through graph search. Several works explore graph search accelerations. Bondhugula et al. [171] employ a parallel FPGA-based design using a blocked algorithm to solve large instances of All-Pairs Shortest-Paths (APSP) problem, which achieves a 15× speedup over an optimized CPU-based implementation. Sridharan et al. [172] present an architecture-efficient solution based on Dijkstra's algorithm to accelerate the shortest path search, and Takei et al. [173] extend this for a high degree of parallelism and large-scale graph search. Recently, Murray et al. [164] accelerate graph search with the Bellman-Ford algorithm. By leveraging a precomputed roadmap and bounding specific robot quantities, this design enables a more compact and efficient storage structure, dataflows and a low-cost interconnection network.

VI. Partial Reconfiguration

FPGA technology provides the flexibility of on-site programming and re-programming without going through re-fabrication with a modified design. Partial Reconfiguration (PR) takes this flexibility one step further, allowing the modification of an operating FPGA design by loading a partial configuration file, usually a partial BIT file [174]. Using PR, after a full BIT file configures the FPGA, partial BIT files can be downloaded to modify reconfigurable regions in the FPGA without compromising the integrity of the applications running on those parts of the device that are not being reconfigured.

A major performance bottleneck for PR is the configuration overhead, which seriously limits the usefulness of PR. To address this problem, in [175], the authors propose a combination of two techniques to minimize the overhead. First, the authors design and implement fully streaming DMA engines to saturate the configuration throughput. Second, the authors exploit a simple form of data redundancy to compress the configuration bitstreams, and implement an intelligent internal configuration access port (ICAP) controller to perform decompression at runtime. This design achieves an effective configuration data transfer throughput of up to 1.2 Gbytes/s, which actually well surpasses the theoretical

upper bound of the data transfer throughput, 400 Mbytes/s. Specifically, the proposed fully streaming DMA engines reduce the configuration time from the range of seconds to the range of milliseconds, a more than 1000-fold improvement. In addition, the proposed compression scheme achieves up to a 75% reduction in bitstream size and results in a decompression circuit with negligible hardware overhead.

Another problem of PR is that it may incur additional energy consumption. In [176], the authors investigate whether PR can be used to reduce FPGA energy consumption. The core idea is that there are a number of independent circuits within a hardware design, and some can be idle for long periods of time. Idle circuits still consume power though, especially through clock oscillation and static leakage. Using PR, one can replace these circuits during their idle time with others that consume much less power. Since the reconfiguration process itself introduces energy overhead, it is unclear whether this approach actually leads to an overall energy saving or to a loss. This study identifies the precise conditions under which partial reconfiguration reduces the total energy consumption, and proposes solutions to minimize the configuration energy overhead. In this study, PR is compared against clock gating to evaluate its effectiveness. The authors apply these techniques to an existing embedded microprocessor design, and successfully demonstrate that FPGAs can be used to accelerate application performance while also reducing overall energy consumption.

Further, PerceptIn demonstrate in their commercial product that Runtime partial reconfiguration (RPR) is useful for robotic computing, especially computing for autonomous vehicles, because many on-vehicle tasks usually have multiple versions where each is used in a particular scenario [16]. For instance, in PerceptIn's design, the localization algorithm relies on salient features; features in key frames are extracted by a feature extraction algorithm (based on ORB features [177]), whereas features in non-key frames are tracked from previous frames (using optical flow [178]); the latter executes in 10 ms, 50% faster than the former. Spatially sharing the FPGA is not only area-inefficient, but also power-inefficient as the unused portion of the FPGA consumes non-trivial static power. In order to temporally share the FPGA and "hot-swap" different algorithms, PerceptIn develop a partial reconfiguration engine (PRE) that dynamically reconfigures part of the FPGA at runtime. The PRE achieves a 400 MB/sec reconfiguration throughput (i.e., bitstream programming rate). Both the feature extraction and tracking bitstreams are less than 4 MB. Thus, the reconfiguration delay is less than 1 ms.

VII. Commercial Applications of FPGAs in Autonomous Vehicles

Over the past three years, PerceptIn has built and commercialized autonomous vehicles for micromobility. Our products have been deployed in China, US, Japan and Switzerland. We summarize system design constraints, workloads and their performance characteristics from the real products. A custom computing system is developed by taking into account the inherent task-level parallelism, cost, safety and programmability [16], [179]. FPGA plays a critical role in our system, which synchronizes various sensors and accelerates the component on the critical path.

A. Computing system

Software pipeline. Fig. 3 shows the block diagram of the processing pipeline in our vehicle, which consists of three parts: sensing, perception and planning. The sensing module bridges sensors and computing system. It synchronizes various sensor samples for the downstream perception module, which performs two fundamental tasks: 1) locating the vehicle itself in a global map and 2) understanding the surroundings through depth estimation and object detection. The planning module uses the perception results to devise a driveable route, and then converts the planned path into a sequence of control commands, which will drive the vehicle along the path. The control commands are sent to the vehicle's Engine Control Unit (ECU) via the CAN bus interface.

Sensing, perception and planning are serialized. They are all on the critical path of the end-to-end latency. We pipeline the three modules to improve the throughput. Within perception, localization and scene understanding are independent and could execute in parallel. While there are multiple tasks within scene understanding, they are mostly independent with the only exception that object tracking must be serialized with object detection. The task-level parallelisms influence how the tasks are mapped to the hardware platform.

Algorithm. Our localization module is based on Visual Inertial Odometry algorithms [180], [181], which fuses camera images, IMU and GPS samples to estimate the vehicle pose in the global map. The depth estimation employs traditional stereo vision algorithms, which calculates depths according to the principal of triangulation [182]. In particular, our method is based on the classic ELAS algorithm, which uses hand-crafted features [183]. While DNN models for depth estimation exist, they are orders of magnitude more compute-intensive than non-DNN algorithms [184] while providing only marginal accuracy improvements to our use-cases. We detect objects using DNN models,

such as YOLO [24]. We use the Kernelized Correlation Filter (KCF) [185] to track detected objects. The planning algorithm is formulated as Model Predictive Control (MPC) [186].

Hardware architecture. Fig. 4 is the hardware system designed for our autonomous vehicles. The sensing hardware consists of stereo cameras, IMU and GPS. In particular, our system uses stereo cameras for depth estimation. One of the cameras is also used for semantic tasks such as object detection. The cameras along with the IMU and the GPS drive the VIO-based localization task.

Considering the cost, compute requirements and power budget, our computing platform is composed of a Xilinx Zynq Ultrascale+ FPGA and an on-vehicle PC equipped with an Intel Coffee Lake CPU and an Nvidia GTX 1060 GPU. The PC is the main computing platform, while the FPGA plays a critical role, which bridges sensors and the PC, and provides an acceleration platform. To optimize the end-to-end latency, explore the task level parallelism and ease practical development and deployment, planning and scene understanding are mapped onto the CPU and the GPU respectively, and sensing and localization are implemented on the FPGA platform.

B. Sensing on FPGA

We map sensing on the Zynq FPGA platform. The FPGA processes sensor data and transfer sensor data to the PC for subsequent processing. The reason that sensing is mapped to FPGA is three-fold. First, embedded FPGA platforms today are built with rich sensor interface (e.g. standard MIPI Camera Serial Interface) and sensor pre-processing hardware (e.g. ISP). Second, by having the FPGA directly process sensor data in situ, we allow accelerators on the FPGA to directly process sensor data without involving the power-hungry CPU for data movement and task coordination. Finally, processing sensor data on the FPGA naturally leads to a design of hardware-assisted multiple sensor synchronization mechanism.

Sensor Synchronization Sensor synchronization is critical to perception algorithms that fuse multiple sensors. Sensor fusion algorithms assume sensor samples have been well synchronized. For example, widely adopted datasets, such as KITTI, provide synchronized data so that researchers could focus on algorithmic development.

An ideal synchronization ensures that 1) various sensor samples have a unified timing system, and 2) timestamps of samples precisely record the time of events triggering the sensors. GPS synchronization is

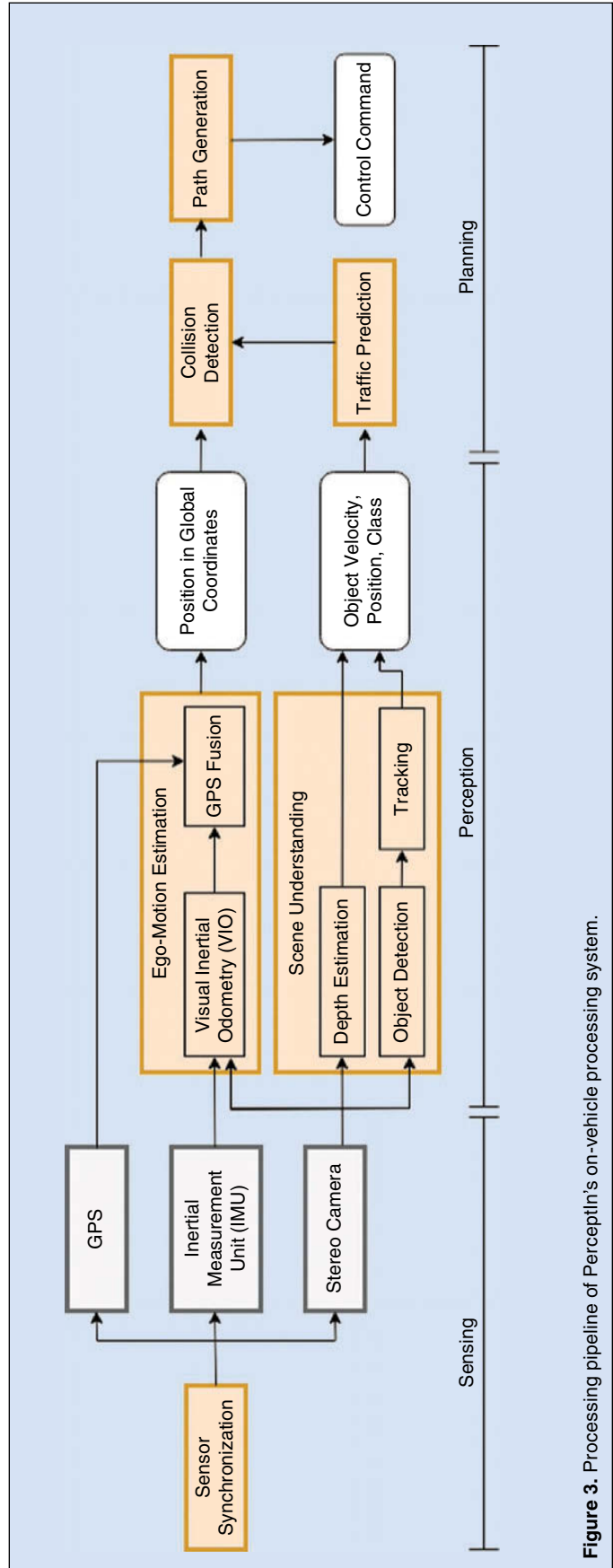
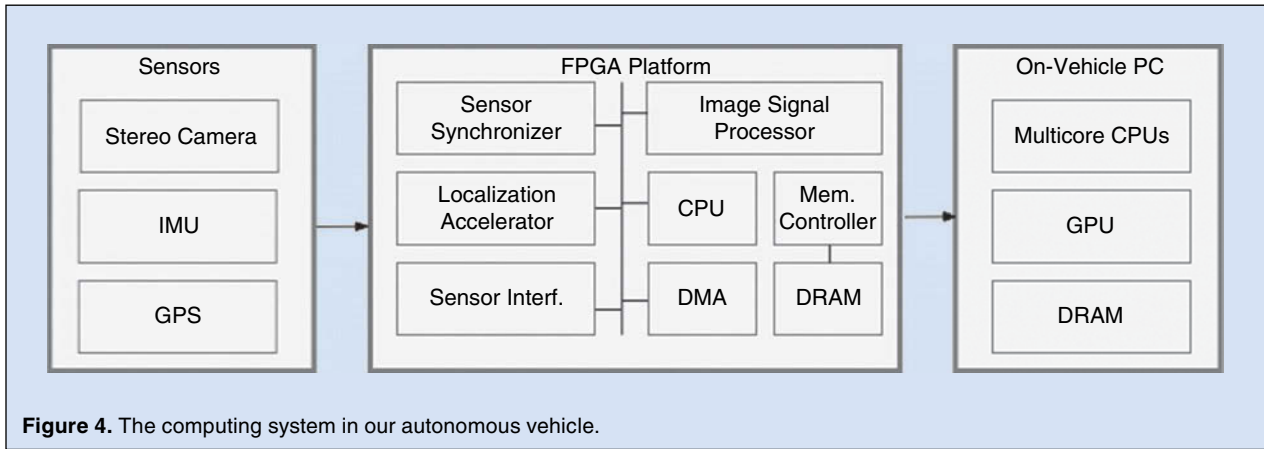


Figure 3. Processing pipeline of Perceptin's on-vehicle processing system.



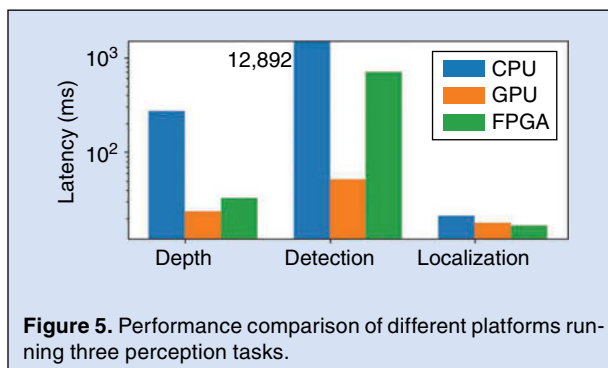
now widely adopted to unify various measurements in a global timing domain. Software-based synchronization associates samples with timestamps at the application or the driver layer. This approach is inaccurate due to the software processing before the timestamp stage. The software processing introduces variable latency that is non-deterministic.

To obtain more precise synchronization, we use a hardware synchronizer implemented by FPGA fabrics. The hardware synchronizer triggers the camera sensors and the IMU using a common timer initialized by the satellite atomic time provided by the GPS device. It records the triggering time of each sensor sample, and then pack the timestamp with the corresponding sensor data. In terms of costs, the synchronizer is extremely lightweight in design with only 1,443 LUTs and 1,587 registers and consumes 5mW of power.

C. Perception on FPGA

For our autonomous vehicles, the perception tasks include scene understanding (depth estimation and objection detection) and localization, which are independent. The slower one dictates the overall perception latency.

We evaluate our perception algorithms on the CPU, GPU and Zynq FPGA platform. Fig. 5 compares the la-



tency of each perception tasks on the FPGA platform with the GPU. Due to the available resources, the FPGA platform is faster than the GPU only for localization, which is more lightweight than other tasks. We offload localization to the FPGA while leaving other perception task on the GPU. This partitioning frees more GPU resources for depth estimation and object detection, which is benefit for reducing the perception pipeline's latency.

As with classic SLAM algorithms, our localization algorithm consists of a front-end and a back-end. The front-end uses the ORB features and descriptors for detecting and tracking key points [120], [187]. The back-end uses Levenberg-Marquardt's (LM) algorithm, a non-linear optimization algorithm, to optimize the position of 3 D key points and the pose of the camera [156], [188].

The ORB feature extraction/matching and the LM optimizer are the most time-consuming parts of our SLAM algorithm, which take up nearly all the execution time. We accelerate ORB feature extraction/matching and the non-linear optimizer on FPGA fabrics. The rest lightweight parts are implemented on the ARM core of the Zynq platform. We use independent hardware for each camera to extract features and compute descriptors. Hamming distance and Sum of Absolutated Difference (SAD) matching are implemented to obtain stable matching results. Compared with the CPU implementation, our FPGA implementation achieves a 2.2× speedup and 44 fps, and saves 83% energy.

We use LM algorithm to optimize features and poses over a fixe-size sliding window. To solve the non-linear optimization problem, the LM algorithm iteratively use Jacobbian to linearize the problem and solve the linear equation at each iteration. Schur elimination is used to reduce the dimension of the linear equation, thus reduce the complexity of solving the equation. Cholesky factorization is employed to solve the linear equation. For sliding-window based vSLAM, the Jacobian

and Schur elimination are the most time-consuming parts. By profiling our algorithm on datasets [189], Schur and Jacobian computations account for 29.8% and 48.27% of total time. We implemented Schur elimination and Jacobian updates on FPGA fabrics [156]. Compared with the CPU implementation, the FPGA achieves 4× and 27× speedup for Schur and Jacobian, and saves 76% energy.

VIII. Application of FPGAs in Space Robotics

In the 1980s, field-programmable gate arrays (FPGA) emerged as a result of increasing integration in electronics. Before the use of FPGA, glue-logic designs were based on individual boards with fixed components interconnected via a shared standard bus, which has various drawbacks, such as hindrance of high volume data processing and higher susceptibility to radiation-induced errors, in addition to inflexibility. The utilization of FPGAs in space applications began in 1992, for FPGAs offered unprecedented flexibility and significantly reduced the design cycle and development cost [190].

FPGAs can be categorized by the type of their programmable interconnection switches: antifuse, SRAM, and Flash. Each of the three technologies comes with trade-offs. Antifuse FPGAs are non-volatile and have minimal delay due to routing, resulting in a faster speed and lower power consumption. The drawback is evident as they have a relatively more complicated fabrication process and are only one time programmable. SRAM-based FPGAs are the most common type employed in space missions. They are field reprogrammable and use the standard fabrication process that foundries put in significant effort in optimizing, resulting in a faster rate of performance increase. However, based on SRAM, these FPGAs are volatile and may not hold configuration if a power glitch occurs. Also, they have more substantial routing delay, require more power, and have a higher susceptibility to bit errors. Flash-based FPGAs are non-volatile and reprogrammable, and also have low power consumption and route delay. The major drawback is that in-flight reconfiguration is not recommended for flash-based FPGAs due to the potentially destructive results if radiation effects occur during the reconfiguration process [191]. Also, the stability of stored charge on the floating gate is of concern: it is a function including factors such as operating temperature, the electric fields that might disturb the charge. As a result, flash-based FPGAs are not as frequently used in space missions [192].

A. Radiation Tolerance for Space Computing

For electronics intended to operate in space, the harsh space radiation present is an essential factor to con-

sider. Radiation has various effects on electronics, but the commonly focused two are total ionizing dose effect (TID) and single event effects (SEE). TID results from the accumulation of ionizing radiation over time, which causes permanent damage by creating electron-hole pairs in the silicon dioxide layers of MOS devices. The effect of TID is that electronics gradually degrade in their performance parameters and eventually fail to function. Electronics intended for application in space are tested for the total amount of radiation, measured in kRads, they can endure before failure. Usually, electronics that can withstand 100 kRads are sufficient for low earth orbit missions to use for several years [191].

SEE occurs when high-energy particles from space radiation strike electronics and leave behind an ionized trail. The results are various types of SEEs [193], which can be categorized as either soft errors, which usually do not cause permanent damage, or hard errors, which often cause permanent damage. Examples of soft error include single event upset (SEU), and single event transient (SET). In SEU, a radiation particle struck a memory element, causing a bit flip. Noteworthy is that as the cell density and clock rate of modern devices increases, multiple cell upset (MCU), corruption of two or more memory cells in a single particle strike, is increasingly becoming a concern. A special type of SEU is single event functional interrupt (SEFI), where the upset leads to loss of normal function of the device by affecting control registers or the clock. In SET, a radiation particle passes through a sensitive node, which generates a transient voltage pulse, causing wrong logic state at the combinatorial logic output. Depending on whether the impact occurs during an active clock edge or not, the error may or may not propagate. Some examples of hard error include single event latch-up (SEL), in which energized particle activates parasitic transistor and then cause a short across the device, and single event burn-out (SEB), in which radiation induces high local power dissipation, leading to device failure. In these hard error cases, radiation effects may cause the failure of an entire space mission.

Space-grade FPGAs can withstand considerable levels of TID and have been designed against most destructive SEEs [194]. However, SEU susceptibility is pervasive. For the most part, radiation effects on FPGA are not different from those of other CMOS based ICs. The primary anomaly stems from FPGAs' unique structure, involving programmable interconnections. Depending on their type, FPGAs have different susceptibility toward SEU in their configuration. SRAM FPGAs are designated by NASA as the most susceptible ones due to their volatile nature. Even after the radiation hardening

process, the configuration of SRAM FPGAs is only designated as “hardened” or simply having embedded SEE mitigation techniques rather than “hard,” which means close to immune [191]. Configuration SRAM is not used in the same way as the traditional SRAM. A bit flip in configuration causes an instantaneous effect without the need for a read-write cycle. Moreover, instead of producing one single error in the output, the bit flip shifts the user logic directly, changing the device’s behavior. Scrubbing is needed to rectify SRAM configuration. Antifuse and flash FPGAs are less susceptible to effects in configuration and are designated “hard” against SEEs in their configuration without applying radiation hardening techniques [191].

Design based SEU/fault mitigation techniques are commonly used, for, in contrast to fabrication level radiation hardening techniques, they can be readily applied to commercial off the shelf (COTS) FPGAs. These techniques can be classified into static and dynamic. Static techniques rely on fault-masking, toleration of error without requiring active fixing. One such example is passive redundancy with voting mechanisms. Dynamic techniques, in contrast, detect faults and act to correct them. The common SEU Mitigation Methods include [195], [196]:

- 1) Hardware Redundancy: functional blocks are replicated to detect/tolerate faults. Triple modular redundancy (TMR) is perhaps the most widely used mitigation technique. It can be applied to entire processors or parts of circuits. At a circuit level, registers are implemented using three or more flip flops or latches. Then, voters compare the values and output the majority, reducing the likelihood of error due to SEU. As internal voters are also susceptible to SEU, they are sometimes triplicated also. For mission-critical applications, global signals may be triplicated to mitigate SEUs further. TMR can be implemented at ease with the help supporting HDLs [197]. It is important to note that a limitation of TMR is that one fault, at most, can be tolerated per voter stage. As a result, TMR is often used with other techniques, such as scrubbing, to prevent error accumulation.
- 2) Scrubbing: The vast majority of memory cells in reprogrammable FPGAs contain configuration information. As discussed earlier, configuration memory upset may lead to alteration routing network, loss of function, and other critical effects. Scrubbing, refreshing and restoration of configuration memory to a known-good state, is therefore needed [196]. The reference configuration memory is usually stored in radiation-hardened memory cells either off or on the device. Scrubbers,

processors or configuration controllers, carry out scrubbing. Some advanced SRAM FPGAs, including ones made by Xilinx, support partial reconfiguration, which allows memory repairs to be made without interrupting the operation of the whole device. Scrubbing can be done in frame-level (partial) or device-level (full), which will inevitably lead to some downtime; some devices may not be able to tolerate such an interruption. Blind scrubbing is the most straightforward way of implementation: individual frames are scrubbed periodically without error detection. Blind scrubbing avoids the complexity required in error detection, but extra scrubbing may increase vulnerability to SEUs as errors may be written into frames during the scrubbing process. An alternative to blind scrubbing is readback scrubbing, where scrubbers actively detect errors in configuration through error-correcting code or cyclic redundancy check [195]. If an error is found, scrubber initiates frame-level scrubbing.

Currently, the majority of space-grade FPGA comes from Xilinx and Microsemi. Xilinx offers the Virtex family and Kintex. Both are SRAM based, which have high flexibility. Microsemi offers antifuse based RTAX and Flash-based RTG4, RT PolarFire, which have lower susceptibility against SEE and power consumption. 20 nm Kintex and 28 nm RT PolarFire are the latest generations. The European market is offered with Atmel devices and NanoXplore space-grade FPGAs [198]. Table IV shows the specifications of the above devices.

B. FPGAs in Space Missions

For space robotics, processing power is of particular importance, given the range of information required to accurately and efficiently process. Many of the current and previous space missions are packed with sophisticated algorithms that are mostly static. They serve to increase the efficiency of data transmission; nevertheless, data processing is done mainly on the ground. As the travel distance of missions increases, transmitting all data to, and processing it on the ground is no longer an efficient or even viable option due to transmission delay. As a result, space robots need to become more adaptable and autonomous. They will also need to preprocess on-board a large amount of data collected and compress it before sending it back to Earth [199].

The rapid development of new generation FPGAs may fill the need in space robotics. FPGAs enable robotic systems to be reconfigurable in real-time, making the systems more adaptable by allowing them to respond more efficiently to changes in environment and data. As a result, autonomous reconfiguration and performance

optimization can be achieved. Also, the FPGAs have a high capability for parallel processing, which is useful in boosting processing performance. The use of FPGA is present in various space robots. Some of the most prominent examples of the application are the NASA Mars rovers. Since the first pair of rovers were launched in 2003, the presence of FPGAs have steadily increased in the later rovers.

1) Mars Exploration Rover Missions

Beginning in the early 2000s, NASA have been using FPGAs in exploration rover control and lander control. In Opportunity and Spirit, the two Mars rovers launched in 2003, two Xilinx Virtex XQVR1000s were in the motor control board [200], which operates motors on instruments as well as rover wheels. In addition, an Actel RT 1280 FPGA was used in each of the 20 cameras on the rovers to receive and dispatch hardware commands. The camera electronics consist of clock driver that provides timing pulses through the charge-coupled device (CCD), an IC containing an array of linked or coupled capacitors. Also, there are signal chains that amplify the CCD output and convert it from analog to digital. The Actel FPGA provides the timing, logic, and control functions in the CCD signal chain and inserts a camera ID into camera telemetry to simplify processing [201].

Selected electronic parts have to undergo a multi-step flight consideration process before utilized in any space exploration mission [200], [202]. The first step is the general flight approval, during which the manufacturers perform additional space-grade verification tests beyond the normal commercial evaluation, and NASA meticulously examines the results. Additional device

parameters, such as temperature considerations and semiconductor characteristics are verified in these tests. What follows is flight-specific approval. In this step, NASA engineers examine the device compatibility with the mission. For instance, considerations of the operating environment including factors like temperature and radiation. Also included are a variety of mission-specific situations that the robot may encounter and the associated risk assessment. Depending on the specific application of the device, whether mission critical or not, and the expected mission lifetime, the risk standards varies. Finally, parts go through specific design consideration to ensure all the design requirements have been met. Parts are examined for their designs addressing issues such as SEL, SEU, SEFI. The Xilinx FPGAs used addressed some of the SEE through the following methods [201]:

- 1) Fabrication processes largely prevents SEL
- 2) TMR reduces SEU frequency
- 3) Scrubbing allows device recovery from single event functional interrupts

MER went successful and despite being designed for only 90 Martian days (1 Martian day = 24.6 hours), continued until 2019. The implementation of mitigation techniques was also proven to be effective as the observed error rate was very similar to that predicted [200].

2) Mars Science Laboratory Mission

Launched in 2011, Mars Science Lab (MSL) was the new Rover sent on to Mars. FPGAs were heavily used in its key components, mainly responsible for scientific instrument control, image processing, and communications.

Curiosity has 17 cameras on board: four navigation cameras, eight hazard cameras, the Mars Hand Lens

Table IV.
Specifications of Space-Grade FPGAs.

Device	Logic	Memory	DSPs	Technology	Rad. Tolerance
Xilinx Virtex-5QV	81.9 K LUT6	12.3 Mb	320	65 nm SRAM	SEE immune up to LET > 100 MeV/(mg · cm ²) and 1 Mrad TID
Xilinx RT Kintex UltraScale	331 K LUT6	38 Mb	2760	20 nm SRAM	SEE immune up to LET > 80 MeV/(mg · cm ²) and 100-120 Krads TID
Microsemi RTG4	150 K LE	5 Mb	462	65 nm Flash	SEE immune up to LET > 37 MeV/(mg · cm ²) and TID > 100 Krads
Microsemi RT PolarFire	481 K LE	33 Mb	1480	28 nm Flash	SEE immune up to LET > 63 MeV/(mg · cm ²) and 300 Krads
Microsemi RTAX	4 M gates	0.5 Mb	120	150 nm antifuse	SEE immune up to LET > 37 MeV/(mg · cm ²) and 300 Krads TID
Atmel ATFEE560	560 K gates	0.23 Mb	–	180 nm SRAM	SEL immune up to 95 MeV/(mg · cm ²) and 60 Krads TID
NanoXplore NG-LARGE	137 K LUT4	9.2 Mb	384	65 nm SRAM	SEL immune up to 60 MeV/(mg · cm ²) and 100 Krads TID

Imager (MAHLI), two Mast Cameras, the Mars Descent Imager (MARDI), and the ChemCam Remote Microscopic Imager [203]. MAHLI, the mast cameras, and MARDI share the same electronics design. Similar to the system used on MER, an Actel FPGA provides the timing, logic, and control functions in the CCD signal chain and transmits pixels to the digital electronics assembly (DEA), which interfaces the camera heads with the rover electronics, transmitting command to the camera heads and data back to the rover. There is one DEA dedicated to each of the imagers above. Each has a Virtex-II FPGA that contains a Microblaze soft-processor core. All of the core functionalities of the DEA, including timing, interface, and compression, are implemented in the FPGA as logic peripherals of the Microblaze. Specifically, the DEA provides an image processing pipeline that includes 12 to 8-bit commanding of input pixels, horizontal subframing, and lossless or JPEG image compression [203]. What runs on the Microblaze is the DEA flight software, which coordinates DEA hardware functions such as camera movements. It receives and executes commands, and transmits command from the Earth. The flight software also implements image acquisition algorithms, including autofocus and autoexposure, performs error correction of flash memory, and mechanism control fault protection [203]. In total, the flight software consists of 10,000 lines of ANSI C code, all implemented on the FPGA. Additionally, FPGAs power communication boxes (Electra-Lite) to provide critical communication to Earth from the rovers through a Mars relay network [204]. They are responsible for a variety of high speed bulk signal processing.

3) Mars 2020 Mission

Perseverance is NASA's latest launched Mars rover. The presence of FPGA continued and increased. FPGA was used in the autonomous driving system as a coprocessor for algorithm acceleration for the first time in NASA's planetary rovers. Perseverance runs on the GESTALT (grid-based estimation of surface traversability applied to local terrain) AutoNav algorithm same as Curiosity [205]. Added was the FPGA based accelerator, called Vision Compute Element (VCE). During landing, VCE serves to provide sufficient computing power for the Lander Vision System (LVS), which performs an intensive task of estimates the landing location in 10 seconds by fusing data from the designed landing location, IMU, and landmark matches. After landing, the connection between VCE and LVS is severed. Instead, VCE is repurposed for the GESTALT driving algorithm. The VCE has three cards plugged into a PCI backplane: a CPU card with BAE RAD750 processor, a Compute Element Power

Conditioning Unit (CEPCU), and a Computer Vision Acceleration Card (CVAC). While the former two parts were inherited from the MLS mission, the CVAC is new. It has two FPGAs. One is called the Vision Processor—a Xilinx Virtex 5QV that contains image processing modules for matching landmarks to estimate position. The other is called the Housekeeping FPGA—a Microsemi RTAX 2000 antifuse FPGA that handles tasks such as synchronization with the spacecraft, power management, Vision Processor configuration.

Through more than two decades of use in space, FPGAs have shown their reliability and applicability for space robotic missions. The properties of FPGAs make them good onboard processors, ones that have high reliability, adaptability, processing power, and power efficiency: FPGAs have been used for space robotic missions for decades and are proven in reliability; they have unrivaled adaptability and can even be reconfigured in run time; their capability for high degree parallel processing allow significant acceleration in executing many complex algorithms; hardware/software co-design method makes them potentially more power-efficient. They may finally help us close the two-decade performance gap between commercial processors and space-grade ASICs. As a direct result, the achievements that the world has made in fields such as deep learning and computer vision, which were often too computationally intense for space-grade processors to be used, may become applicable for robots in space in the near future. The implementation of those new technologies will be of great benefit for space robots, boosting their autonomy and capabilities and allowing us to explore farther and faster.

IX. Conclusion

In this paper, we review the state-of-the-art FPGA-based robotic computing accelerator designs and summarize their adopted optimized techniques. According to the results shown in Section III, IV and V, by co-designing both the software and hardware, FPGA can achieve more than 10× better performance and energy efficiency compared to the CPU and GPU implementations. We also review the partial reconfiguration methodology in FPGA implementation to further improve the design flexibility and reduce the overhead. Finally, by presenting some recent FPGA-based robotics applications in commercial and space areas, we demonstrate that FPGA has excellent potential and is a promising candidate for robotic computing acceleration due to its high reliability, adaptability and power efficiency.

The authors believe that FPGAs are the best compute substrate for robotic applications for several reasons: first, robotic algorithms are still evolving rapidly,

and thus any ASIC-based accelerators will be months or even years behind the state-of-the-art algorithms; on the other hand, FPGAs can be dynamically updated as needed. Second, robotic workloads are highly diverse, thus it is difficult for any ASIC-based robotic computing accelerator to reach economies of scale in the near future; on the other hand, FPGAs are a cost-effective and energy-effective alternative before one type of accelerator reaches economies of scale. Third, compared to SoCs that have reached economies of scale, e.g., mobile SoCs, FPGAs deliver a significant performance advantage. Fourth, partial reconfiguration allows multiple robotic workloads to time-share an FPGA, thus allowing one chip to serve multiple applications, leading to overall cost and energy reduction.

However, FPGAs are still not the mainstream computing substrate for robotic workloads, for several reasons: first, FPGA programming is still much more challenging than regular software programming, and the supply of FPGA engineers is still limited. Second, although there is significant progress in the past few years in the FPGA High-Level Synthesis (HLS) automation, such as [206], HLS is still not able to produce optimized code, and IP supports for robotic workloads are still extremely limited. Third, commercial software support for robotic workloads on FPGAs is still missing. For instance, there is no official ROS support on any commercial FPGA platform today. For robotic companies to fully exploit the power of FPGAs, these problems need to be first addressed, and the authors use these problems to motivate our future research work.



Zishen Wan (Student Member, IEEE) is currently a Ph.D. student in Electrical and Computer Engineering at Georgia Institute of Technology, Atlanta, GA, U.S.A. He received the M.S. degree from Harvard University, Cambridge, MA, in 2020, and the B.S. degree from Harbin Institute of Technology, Harbin, China, in 2018, both in Electrical Engineering. He has a broad research interest in VLSI design, computer architecture, and edge intelligence, with a focus on energy-efficient and robust hardware and system design for autonomous machines. He has received the Best Paper Award in DAC 2020 and CAL 2020.

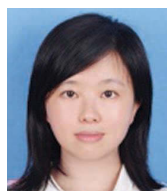


Bo Yu (Senior Member, IEEE) received the B.S. degree in electronic technology and science from Tianjin University, Tianjin, China, in 2006, and the Ph.D. degree from the Institute of Microelectronics, Tsinghua University, Beijing,

China, in 2013. He is currently the CTO of PerceptIn, Fremont, CA, U.S.A., a company focusing on providing visual perception solutions for robotics and autonomous driving. His current research interests include algorithm and systems for robotics and autonomous vehicles. Dr. Yu is also a Founding Member of the IEEE Special Technical Community on Autonomous Driving.



Thomas Yuang Li (Student Member, IEEE) is a research intern at PerceptIn, U.S.A. and a student member of the IEEE. His research interests include building autonomous space explorers for future commercial robotic space-exploration missions as well as space robotics and computing related topics.



Jie Tang (Senior Member, IEEE) is currently an associate professor in School of Computer Science and Engineering of South China University of Technology, Guangzhou, China. She received her B.E. degree from University of Defense Technology and Ph.D. degree from the Beijing Institute of Technology, both in Computer Science. She was previously a visiting researcher at the Embedded Systems Center at University of California, Irvine, USA, and a research scientist at Intel China Runtime Technology Lab. Dr. Tang is mainly doing research on Computing Systems for Autonomous Machines. She is a founding member and secretary of the IEEE Computer Society Special Technical Community on Autonomous Driving Technologies.

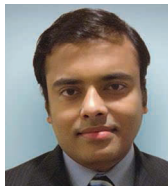


Yuhao Zhu (Member, IEEE) is an Assistant Professor of Computer Science at University of Rochester, U.S.A. His research group focuses on applications and computer systems for visual computing. His work is recognized by the Honorable Mention of the 2018 ACM SIGARCH/IEEE-CS TCCA Outstanding Dissertation Award and multiple IEEE Micro Top Picks designations. He is a recipient of the NSF CAREER Award in 2020.



Yu Wang (Senior Member, IEEE) received his B.S. degree in 2002 and Ph.D. degree (with honor) in 2007 from Tsinghua University, Beijing, China. He is currently a Tenured Professor and Chair with the Department of Electronic Engineering, Tsinghua University. His research interests include application specific hardware computing, parallel circuit analysis, and power/reliability aware system

design methodology. Dr. Wang has authored and coauthored over 250 papers in refereed journals and conferences. He has received Best Paper Award in ASPDAC 2019, FPGA 2017, NVMSA17, ISVLSI 2012, and Best Poster Award in HEART 2012 with 9 Best Paper Nominations. He is a recipient of DAC Under-40 Innovator Award in 2018. He served as TPC chair for ICFPT 2019, ISVLSI 2018, ICFPT 2011 and Finance Chair of ISLPED 2012-2016, and served as the program committee member for leading conferences in EDA/FPGA area. Currently he serves as Associate Editor for IEEE Trans on CAS for Video Technology, IEEE Transactions on CAD, and ACM TECS. He is an IEEE/ACM senior member. He is the co-founder of Deephi Tech (acquired by Xilinx in 2018), which is a leading deep learning computing platform provider



Arijit Raychowdhury (Senior Member, IEEE) is currently a Professor in the School of Electrical and Computer Engineering at the Georgia Institute of Technology, U.S.A., where he joined in January 2013. From 2013 to July 2019 he was an Associate Professor and held the ON Semiconductor Junior Professorship in the department. He received his Ph.D. degree in Electrical and Computer Engineering from Purdue University (2007) and his B.E. in Electrical and Telecommunication Engineering from Jadavpur University, India (2001). His industry experience includes five years as a Staff Scientist in the Circuits Research Lab, Intel Corporation, and a year as an Analog Circuit Researcher with Texas Instruments Inc. His research interests include low power digital and mixed-signal circuit design, design of power converters, sensors and exploring interactions of circuits with device technologies. Dr. Raychowdhury holds more than 25 U.S. and international patents and has published over 200 articles in journals and refereed conferences. He currently serves on the Technical Program Committees of ISSCC, VLSI Circuit Symposium, CICC, and DAC. He was the Associate Editor of the IEEE Transactions on Computer Aided Design from 2013-2018 and the Editor of the Microelectronics Journal, Elsevier Press from 2013 to 2017. He is the winner of Qualcomm Faculty Award, 2020; IEEE/ACM Innovator under 40 award; the NSF CISE Research Initiation Initiative Award (CRII), 2015; Intel Labs Technical Contribution Award, 2011; Dimitris N. Chorafas Award for outstanding doctoral research, 2007; the Best Thesis Award, College of Engineering, Purdue University, 2007; SRC Technical Excellence Award, 2005; Intel Foundation Fellowship, 2006; NASA INAC Fellowship, 2004; the Meissner Fellowship 2002. He and his stu-

dents have won several fellowships and eleven best paper awards over the years. Dr. Raychowdhury is a Senior Member of the IEEE.



Shaoshan Liu (Senior Member, IEEE) is Founder and CEO of PerceptIn (www.perceptin.io) U.S.A., a company focusing on providing visual perception solutions for autonomous robots and vehicles. Dr. Shaoshan Liu received his Ph.D. in Computer Engineering from University of California, Irvine and M.P.A. from Harvard University. His research focuses on Computing Systems for Autonomous Machines. Dr. Shaoshan Liu has published over 80 research papers and holds over 150 U.S. international patents on autonomous machines. Dr. Shaoshan Liu is an ACM Distinguished Speaker, and an IEEE Computer Society Distinguished Speaker.

References

- [1] A. Qiantori, A. B. Sutiono, H. Hariyanto, H. Suwa, and T. Ohta, "An emergency medical communications system by low altitude platform at the early stages of a natural disaster in Indonesia," *J. Med. Syst.*, vol. 36, no. 1, pp. 41–52, 2012. doi: 10.1007/s10916-010-9444-9.
- [2] A. Ryan and J. K. Hedrick, "A mode-switching path planner for uav-assisted search and rescue," in *Proc. 44th IEEE Conf. Decision and Control*, 2005, pp. 1471–1476.
- [3] N. Smolyanskiy, A. Kamenev, J. Smith, and S. Birchfield, "Toward low-flying autonomous MAV trail navigation using deep neural networks for environmental awareness," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots and Syst. (IROS)*, 2017, pp. 4241–4247.
- [4] A. Giusti et al., "A machine learning approach to visual perception of forest trails for mobile robots," *IEEE Robot. Automat. Lett.*, vol. 1, no. 2, pp. 661–667, 2015. doi: 10.1109/LRA.2015.2509024.
- [5] J. K. Stolaroff, C. Samaras, E. R. O'Neill, A. Lubers, A. S. Mitchell, and D. Ceperley, "Energy use and life cycle greenhouse gas emissions of drones for commercial package delivery," *Nature Commun.*, vol. 9, no. 1, pp. 1–13, 2018. doi: 10.1038/s41467-017-02411-5.
- [6] S. J. Kim, Y. Jeong, S. Park, K. Ryu, and G. Oh, "A survey of drone use for entertainment and AVR (augmented and virtual reality)," in *Augmented Reality and Virtual Reality*. Springer-Verlag, 2018, pp. 339–352.
- [7] S. Jung, S. Cho, D. Lee, H. Lee, and D. H. Shim, "A direct visual serving-based framework for the 2016 IROS autonomous drone racing challenge," *J. Field Robot.*, vol. 35, no. 1, pp. 146–166, 2018. doi: 10.1002/rob.21743.
- [8] "Fact sheet—The Federal Aviation Administration (FAA) aerospace forecast fiscal years (FY) 2020–2040," 2020. https://www.faa.gov/news/fact_sheets/news_story.cfm?newsId=24756
- [9] S. Liu, L. Li, J. Tang, S. Wu, and J.-L. Gaudiot, "Creating autonomous vehicle systems," *Synthesis Lectures Comput. Sci.*, vol. 6, no. 1, pp. 1–186, 2017. doi: 10.2200/S00787ED1V01Y201707CSL009.
- [10] S. Krishnan et al., "The sky is not the limit: A visual performance model for cyber-physical co-design in autonomous machines," *IEEE Comput. Arch. Lett.*, vol. 19, no. 1, pp. 38–42, 2020. doi: 10.1109/LCA.2020.2981022.
- [11] S. Krishnan et al., "Machine learning-based automated design space exploration for autonomous aerial robots," 2021, arXiv:2102.02988.
- [12] S. Liu and J.-L. Gaudiot, "Autonomous vehicles lite self-driving technologies should start small, go slow," *IEEE Spectr.*, vol. 57, no. 3, pp. 36–49, 2020. doi: 10.1109/MSPEC.2020.9014458.
- [13] S. Liu, L. Liu, J. Tang, B. Yu, Y. Wang, and W. Shi, "Edge computing for autonomous driving: Opportunities and challenges," *Proc. IEEE*, vol. 107, no. 8, pp. 1697–1716, 2019. doi: 10.1109/JPROC.2019.2915983.
- [14] S. Liu, J. Tang, Z. Zhang, and J.-L. Gaudiot, "Computer architectures for autonomous driving," *Computer*, vol. 50, no. 8, pp. 18–25, 2017. doi: 10.1109/MC.2017.3001256.

- [15] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, "[DL] a survey of FPGA-based neural network inference accelerators," *ACM Trans. Reconfigurable Technol. Syst. (TRETs)*, vol. 12, no. 1, pp. 1–26, 2019. doi: 10.1145/3289185.
- [16] B. Yu, W. Hu, L. Xu, J. Tang, S. Liu, and Y. Zhu, "Building the computing system for autonomous micromobility vehicles: Design constraints and architectural optimizations," in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarch. (MICRO)*, 2020. doi: 10.1109/MICRO50266.2020.00089.
- [17] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Comput. Soc. Conf. Comput. Vision and Pattern Recogn. (CVPR'05)*, 2005, vol. 1, pp. 886–893.
- [18] X. He, R. S. Zemel, and M. A. Carreira-Perpinan, "Multiscale conditional random fields for image labeling," in *Proc. IEEE Comput. Soc. Conf. Comput. Vision and Pattern Recogn. (CVPR 2004)*, 2004, vol. 2, p. II.
- [19] X. He, R. S. Zemel, and D. Ray, "Learning and incorporating top-down cues in image segmentation," in *Proc. Comput. Vision – ECCV 2006*, A. Leonardis, H. Bischof, and A. Pinz, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 338–351.
- [20] Y. Xiang, A. Alahi, and S. Savarese, "Learning to track: Online multi-object tracking by decision making," in *Proc. IEEE Int. Conf. Comput. Vision (ICCV)*, 2015, pp. 4705–4713. doi: 10.1109/ICCV.2015.534.
- [21] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vision (ICCV)*, Dec. 2015. doi: 10.1109/ICCV.2015.169.
- [22] S. Ren, K. He, R. B. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *CoRR*, vol. abs/1506.01497, 2015.
- [23] W. Liu et al., "SSD: Single shot multibox detector," *CoRR*, vol. abs/1512.02325, 2015.
- [24] J. Redmon, S. K. Divvala, R. B. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," *CoRR*, vol. abs/1506.02640, 2015.
- [25] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," *CoRR*, vol. abs/1612.08242, 2016.
- [26] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," *CoRR*, vol. abs/1411.4038, 2014.
- [27] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial pyramid pooling in deep convolutional networks for visual recognition," *CoRR*, vol. abs/1406.4729, 2014.
- [28] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid scene parsing network," *CoRR*, vol. abs/1612.01105, 2016.
- [29] L. Bertinetto, J. Valmadre, J. F. Henriques, A. Vedaldi, and P. H. S. Torr, "Fully-convolutional Siamese networks for object tracking," *CoRR*, vol. abs/1606.09549, 2016.
- [30] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: Part I," *IEEE Robot. Automat. Mag.*, vol. 13, no. 2, pp. 99–110, 2006. doi: 10.1109/MRA.2006.1638022.
- [31] M. Montemerlo et al., "Junior: The Stanford entry in the urban challenge," *J. Field Robot.*, vol. 25, no. 9, pp. 569–597, 2008. doi: 10.1002/rob.20258.
- [32] J. Ziegler et al., "Making bertha drive—an autonomous journey on a historic route," *IEEE Intell. Transp. Syst. Mag.*, vol. 6, no. 2, pp. 8–20, 2014. doi: 10.1109/MITS.2014.2306552.
- [33] C. Katrakazas, M. Qudus, W.-H. Chen, and L. Deka, "Real-time motion planning methods for autonomous on-road driving: State-of-the-art and future research directions," *Transp. Res. C, Emerg. Technol.*, vol. 60, pp. 416–442, 2015. doi: 10.1016/j.trc.2015.09.011.
- [34] B. Paden, M. Čáp, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *IEEE Trans. Intell. Veh.*, vol. 1, no. 1, pp. 33–55, 2016. doi: 10.1109/TIV.2016.2578706.
- [35] Y. Deng, Y. Chen, Y. Zhang, and S. Mahadevan, "Fuzzy dijkstra algorithm for shortest path problem under uncertain environment," *Appl. Soft Comput.*, vol. 12, no. 3, pp. 1231–1237, 2012. doi: 10.1016/j.asoc.2011.11.011.
- [36] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968. doi: 10.1109/TSSC.1968.300136.
- [37] S. M. LaValle and J. J. Kuffner Jr., "Randomized kinodynamic planning," *Int. J. Robot. Res.*, vol. 20, no. 5, pp. 378–400, 2001. doi: 10.1177/02783640122067453.
- [38] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Trans. Robot. Autom. (1989–June 2004)*, vol. 12, no. 4, pp. 566–580, 1996. doi: 10.1109/70.508439.
- [39] S. Shalev-Shwartz, N. Ben-Zrihem, A. Cohen, and A. Shashua, "Long-term planning by short-term prediction," 2016, arXiv:1602.01580.
- [40] M. Gómez, R. González, T. Martínez-Marín, D. Meziat, and S. Sánchez, "Optimal motion planning by reinforcement learning in autonomous mobile vehicles," *Robotica*, vol. 30, no. 2, pp. 159, 2012. doi: 10.1017/S0263574711000452.
- [41] S. Shalev-Shwartz, S. Shammah, and A. Shashua, "Safe, multi-agent, reinforcement learning for autonomous driving," 2016, arXiv:1610.03295.
- [42] M. Bojarski et al., "End to end learning for self-driving cars," 2016, arXiv:1604.07316.
- [43] X. Geng, H. Liang, B. Yu, P. Zhao, L. He, and R. Huang, "A scenario-adaptive driving behavior prediction approach to urban autonomous driving," *Appl. Sci.*, vol. 7, no. 4, p. 426, 2017. doi: 10.3390/app7040426.
- [44] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, no. 3–4, pp. 279–292, 1992. doi: 10.1023/A:1022676722315.
- [45] V. R. Konda and J. N. Tsitsiklis, "Actor-critic algorithms," in *Advances Neural Inf. Process. Syst.*, 2000, pp. 1008–1014.
- [46] S. L. Hicks, I. Wilson, L. Muhammed, J. Worsfold, S. M. Downes, and C. Kennard, "A depth-based head-mounted visual display to aid navigation in partially sighted individuals," *PLoS One*, vol. 8, no. 7, p. e67695, 2013. doi: 10.1371/journal.pone.0067695.
- [47] T. Whelan, R. F. Salas-Moreno, B. Glocker, A. J. Davison, and S. Leutenegger, "Elasticfusion: Real-time dense slam and light source estimation," *Int. J. Robot. Res.*, vol. 35, no. 14, pp. 1697–1716, 2016. doi: 10.1177/0278364916669237.
- [48] V. A. Prisacariu et al., "Infinitam v3: A framework for large-scale 3d reconstruction with loop closure," 2017, arXiv:1708.00783.
- [49] S. Golodetz, T. Cavallari, N. A. Lord, V. A. Prisacariu, D. W. Murray, and P. H. Torr, "Collaborative large-scale dense 3d reconstruction with online inter-agent pose optimisation," *IEEE Trans. Vis. Comput. Graphics*, vol. 24, no. 11, pp. 2895–2905, 2018. doi: 10.1109/TVCG.2018.2868533.
- [50] M. Pérez-Patricio and A. Aguilar-González, "FPGA implementation of an efficient similarity-based adaptive window algorithm for real-time stereo matching," *J. Real-Time Image Process.*, vol. 16, no. 2, pp. 271–287, 2019. doi: 10.1007/s11554-015-0530-6.
- [51] D.-W. Yang, L.-C. Chu, C.-W. Chen, J. Wang, and M.-D. Shieh, "Depth-reliability-based stereo-matching algorithm and its VLSI architecture design," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 25, no. 6, pp. 1038–1050, 2014. doi: 10.1109/TCSVT.2014.2361419.
- [52] A. Aguilar-González and M. Arias-Estrada, "An FPGA stereo matching processor based on the sum of hamming distances," in *Proc. Int. Symp. Appl. Reconfigurable Comput.*, 2016, pp. 66–77.
- [53] M. Pérez-Patricio, A. Aguilar-González, M. Arias-Estrada, H.-R. Hernandez-de Leon, J.-L. Camas-Anzueto, and J. de Jesús Osuna-Coutiño, "An FPGA stereo matching unit based on fuzzy logic," *Microprocessors Microsyst.*, vol. 42, pp. 87–99, 2016. doi: 10.1016/j.micpro.2015.10.011.
- [54] G. Cocorullo, P. Corsonello, F. Frustaci, and S. Perri, "An efficient hardware-oriented stereo matching algorithm," *Microprocessors Microsyst.*, vol. 46, pp. 21–33, 2016. doi: 10.1016/j.micpro.2016.09.010.
- [55] P. M. Santos, J. C. Ferreira, and J. S. Matos, "Scalable hardware architecture for disparity map computation and object location in real-time," *J. Real-Time Image Process.*, vol. 11, no. 3, pp. 473–485, 2016. doi: 10.1007/s11554-013-0338-1.
- [56] K. M. Ali, R. B. Atitallah, N. Fakhfakh, and J.-L. Dekeyser, "Exploring HLS optimizations for efficient stereo matching hardware implementation," in *Proc. Int. Symp. Appl. Reconfigurable Comput.*, 2017, pp. 168–176.
- [57] B. McCullagh, "Real-time disparity map computation using the cell broadband engine," *J. Real-Time Image Process.*, vol. 7, no. 2, pp. 87–93, 2012. doi: 10.1007/s11554-010-0155-8.
- [58] L. Li, X. Yu, S. Zhang, X. Zhao, and L. Zhang, "3d cost aggregation with multiple minimum spanning trees for stereo matching," *Appl. Opt.*, vol. 56, no. 12, pp. 3411–3420, 2017. doi: 10.1364/AO.56.003411.
- [59] D. Zha, X. Jin, and T. Xiang, "A real-time global stereo-matching on FPGA," *Microprocessors Microsyst.*, vol. 47, pp. 419–428, 2016. doi: 10.1016/j.micpro.2016.08.005.
- [60] L. Puglia, M. Vighiar, and G. Raiconi, "Real-time low-power FPGA architecture for stereo vision," *IEEE Trans. Circuits Syst. II, Express Briefs*, vol. 64, no. 11, pp. 1307–1311, 2017. doi: 10.1109/TCSII.2017.2691675.
- [61] A. Kjør-Nielsen et al., "A two-level real-time vision machine combining coarse- and fine-grained parallelism," *J. Real-Time Image Process.*, vol. 5, no. 4, pp. 291–304, 2010. doi: 10.1007/s11554-010-0159-4.
- [62] S. Wong, S. Vassiliadis, and S. Cotozana, "A sum of absolute differences implementation in FPGA hardware," in *Proc. 28th Euromicro Conf.*, 2002, pp. 183–188.

- [63] M. Hisham, S. N. Yaakob, R. A. Raof, A. A. Nazren, and N. W. Embedded, "Template matching using sum of squared difference and normalized cross correlation," in *Proc. IEEE Student Conf. Res. Development (SCoRED)*, 2015, pp. 100–104. doi: 10.1109/SCoRED.2015.7449303.
- [64] J.-C. Yoo and T. H. Han, "Fast normalized cross-correlation," *Circuits Syst. Signal Process.*, vol. 28, no. 6, p. 819, 2009. doi: 10.1007/s00034-009-9130-7.
- [65] B. Froba and A. Ernst, "Face detection with the modified census transform," in *Proc. 6th IEEE Int. Conf. Automatic Face and Gesture Recogn.*, 2004, pp. 91–96.
- [66] S. Jin et al., "FPGA design and implementation of a real-time stereo vision system," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 20, no. 1, pp. 15–26, 2009.
- [67] L. Zhang, K. Zhang, T. S. Chang, G. Lafruit, G. K. Kuzmanov, and D. Verkest, "Real-time high-definition stereo matching on FPGA," in *Proc. 19th ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, 2011, pp. 55–64. doi: 10.1145/1950413.1950428.
- [68] D. Honegger, P. Greisen, L. Meier, P. Tanskanen, and M. Pollefeys, "Real-time velocity estimation based on optical flow and disparity matching," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots and Syst.*, 2012, pp. 5177–5182.
- [69] M. Jin and T. Maruyama, "Fast and accurate stereo vision system on FPGA," *ACM Trans. Reconfigurable Technol. Syst. (TRETS)*, vol. 7, no. 1, pp. 1–24, 2014. doi: 10.1145/2567659.
- [70] S. Park and H. Jeong, "Real-time stereo vision FPGA chip with low error rate," in *Proc. Int. Conf. Multimedia and Ubiquitous Eng. (MUE'07)*, 2007, pp. 751–756.
- [71] S. Sabihuddin, J. Islam, and W. J. MacLean, "Dynamic programming approach to high frame-rate stereo correspondence: A pipelined architecture implemented on a field programmable gate array," in *Proc. Canadian Conf. Electr. Comput. Eng.*, pp. 1461–1466, 2008.
- [72] M. Jin and T. Maruyama, "A real-time stereo vision system using a tree-structured dynamic programming on FPGA," in *Proc. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, 2012, pp. 21–24. doi: 10.1145/2145694.2145698.
- [73] R. Kamasaka, Y. Shibata, and K. Oguri, "An FPGA-oriented graph cut algorithm for accelerating stereo vision," in *Proc. Int. Conf. ReConfigurable Comput. FPGAs (ReConFig)*, 2018, pp. 1–6. doi: 10.1109/RECONFIG.2018.8641737.
- [74] C. Banz, S. Hesselbarth, H. Flatt, H. Blume, and P. Pirsch, "Real-time stereo vision system using semi-global matching disparity estimation: Architecture and FPGA-implementation," in *Proc. Int. Conf. Embedded Comput. Syst., Arch., Model. Simulation*, 2010, pp. 93–101.
- [75] W. Wang, J. Yan, N. Xu, Y. Wang, and F.-H. Hsu, "Real-time high-quality stereo vision system in FPGA," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 25, no. 10, pp. 1696–1708, 2015. doi: 10.1109/TCSVT.2015.2397196.
- [76] L. F. Cambuim, J. P. Barbosa, and E. N. Barros, "Hardware module for low-resource and real-time stereo vision engine using semi-global matching approach," in *Proc. 30th Symp. Integrated Circuits Syst. Des., Chip Sands*, 2017, pp. 53–58. doi: 10.1145/3109984.3109992.
- [77] O. Rahnama, T. Cavalleri, S. Golodetz, S. Walker, and P. Torr, "R3sgm: Real-time raster-respecting semi-global matching for power-constrained systems," in *Proc. Int. Conf. Field-Programmable Technol. (FPT)*, 2018, pp. 102–109. doi: 10.1109/FPT.2018.00025.
- [78] L. F. Cambuim, L. A. Oliveira, E. N. Barros, and A. P. Ferreira, "An FPGA-based real-time occlusion robust stereo vision system using semi-global matching," *J. Real-Time Image Process.*, vol. 17, no. 5, pp. 1–22, 2019. doi: 10.1007/s11554-019-00902-w.
- [79] J. Zhao et al., "FP-stereo: Hardware-efficient stereo vision for embedded applications," 2020, arXiv:2006.03250.
- [80] O. Rahnama, D. Frost, O. Miksik, and P. H. Torr, "Real-time dense stereo matching with ELAS on FPGA-accelerated embedded devices," *IEEE Robot. Automat. Lett.*, vol. 3, no. 3, pp. 2008–2015, 2018. doi: 10.1109/LRA.2018.2800786.
- [81] O. Rahnama et al., "Real-time highly accurate dense depth on a power budget using an FPGA-CPU hybrid soc," *IEEE Trans. Circuits and Syst. II: Express Briefs*, vol. 66, no. 5, pp. 773–777, 2019. doi: 10.1109/TC-SII.2019.2909169.
- [82] H. Hirschmuller, "Accurate and efficient stereo processing by semi-global matching and mutual information," in *Proc. IEEE Comput. Soc. Conf. Comput. Vision and Pattern Recogn. (CVPR'05)*, 2005, vol. 2, pp. 807–814.
- [83] D. Honegger, H. Oleynikova, and M. Pollefeys, "Real-time and low latency embedded computer vision hardware based on a combination of FPGA and mobile CPU," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots and Syst.*, 2014, pp. 4930–4935.
- [84] S. Mattoccia and M. Poggi, "A passive RGBD sensor for accurate and real-time depth sensing self-contained into an FPGA," in *Proc. 9th Int. Conf. Distrib. Smart Cameras*, 2015, pp. 146–151. doi: 10.1145/2789116.2789148.
- [85] S. K. Gehrig, F. Eberli, and T. Meyer, "A real-time low-power stereo vision engine using semi-global matching," in *Proc. Int. Conf. Comput. Vision Syst.*, 2009, pp. 134–143.
- [86] D. Hernandez-Juarez, A. Chacón, A. Espinosa, D. Vázquez, J. C. Moure, and A. M. López, "Embedded real-time stereo estimation via semi-global matching on the GPU," *Procedia Comput. Sci.*, vol. 80, pp. 143–153, 2016. doi: 10.1016/j.procs.2016.05.305.
- [87] H. Hirschmuller and D. Scharstein, "Evaluation of cost functions for stereo matching," in *Proc. IEEE Conf. Comput. Vision and Pattern Recogn.*, 2007, pp. 1–8. doi: 10.1109/CVPR.2007.383248.
- [88] Y. Shan et al., "FPGA based memory efficient high resolution stereo vision system for video tolling," in *Proc. Int. Conf. Field-Programmable Technol.*, 2012, pp. 29–32.
- [89] Y. Shan et al., "Hardware acceleration for an accurate stereo vision system using mini-census adaptive support region," *ACM Trans. Embedded Comput. Syst. (TECS)*, vol. 13, no. 4s, pp. 1–24, 2014. doi: 10.1145/2584659.
- [90] A. Geiger, M. Roser, and R. Urtasun, "Efficient large-scale stereo matching," in *Proc. Asian Conf. Comput. Vision*, 2010, pp. 25–38.
- [91] S. Zagoruyko and N. Komodakis, "Learning to compare image patches via convolutional neural networks," in *Proc. IEEE Conf. Comput. Vision and Pattern Recogn.*, 2015, pp. 4353–4361.
- [92] J. Zbontar and Y. LeCun, "Stereo matching by training a convolutional neural network to compare image patches," *J. Mach. Learn. Res.*, vol. 17, no. 1, pp. 2287–2318, 2016.
- [93] W. Luo, A. G. Schwing, and R. Urtasun, "Efficient deep learning for stereo matching," in *Proc. IEEE Conf. Comput. Vision and Pattern Recogn.*, 2016, pp. 5695–5703.
- [94] A. Seki and M. Pollefeys, "SGM-Nets: Semi-global matching with neural networks," in *Proc. IEEE Conf. Comput. Vision and Pattern Recogn.*, 2017, pp. 231–240.
- [95] N. Mayer, E. Ilg, P. Hausser, P. Fischer, D. Cremers, A. Dosovitskiy, and T. Brox, "A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation," in *Proc. IEEE Conf. Comput. Vision and Pattern Recogn.*, 2016, pp. 4040–4048.
- [96] A. Kuzmin, D. Mikushin, and V. Lempitsky, "End-to-end learning of cost-volume aggregation for real-time dense stereo," in *Proc. IEEE 27th Int. Workshop on Mach. Learn. Signal Process. (MLSP)*, 2017, pp. 1–6. doi: 10.1109/MLSP.2017.8168183.
- [97] H. Li, X. Fan, L. Jiao, W. Cao, X. Zhou, and L. Wang, "A high performance FPGA-based accelerator for large-scale convolutional neural networks," in *Proc. 26th Int. Conf. Field Programmable Logic and Appl. (FPL)*, 2016, pp. 1–9.
- [98] J. Qiu et al., "Going deeper with embedded FPGA platform for convolutional neural network," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, 2016, pp. 26–35. doi: 10.1145/2847263.2847265.
- [99] K. Guo et al., "Angel-eye: A complete design flow for mapping CNN onto embedded FPGA," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 1, pp. 35–47, 2017. doi: 10.1109/TCAD.2017.2705069.
- [100] J. Yu et al., "Instruction driven cross-layer CNN accelerator for fast detection on FPGA," *ACM Trans. Reconfigurable Technol. Syst. (TRETS)*, vol. 11, no. 3, pp. 1–23, 2018. doi: 10.1145/3283452.
- [101] H. Nakahara, H. Yonekawa, T. Fujii, and S. Sato, "A lightweight YO-LOv2: A binarized CNN with a parallel support vector regression for an FPGA," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, 2018, pp. 31–40.
- [102] M. S. Belshaw, "A high-speed iterative closest point tracker on an FPGA platform," PhD thesis, 2008.
- [103] B. Williams, "Evaluation of a soc for real-time 3d slam," 2017.
- [104] B. Van Hoorick, "FPGA-based simultaneous localization and mapping (slam) using high-level synthesis," 2019.
- [105] Q. Gautier et al., "Real-time 3d reconstruction for FPGAs: A case study for evaluating the performance, area, and programmability trade-offs of the Altera OpenCL SDK," in *Proc. Int. Conf. Field-Programmable Technol. (FPT)*, 2014, pp. 326–329. doi: 10.1109/FPT.2014.7082810.

- [106] T. Bailey, J. Nieto, J. Guivant, M. Stevens, and E. Nebot, "Consistency of the EKF-SLAM algorithm," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots and Syst.*, 2006, pp. 3562–3568.
- [107] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A versatile and accurate monocular slam system," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, 2015. doi: 10.1109/TRO.2015.2463671.
- [108] M. Montemerlo et al., "FastSLAM: A factored solution to the simultaneous localization and mapping problem," *AAAI/IAAI*, vol. 593598, 2002.
- [109] M. Gu, K. Guo, W. Wang, Y. Wang, and H. Yang, "An FPGA-based real-time simultaneous localization and mapping system," in *Proc. Int. Conf. Field Programmable Technol. (FPT)*, 2015, pp. 200–203. doi: 10.1109/FPT.2015.7393150.
- [110] C. Cadena et al., "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1309–1332, 2016. doi: 10.1109/TRO.2016.2624754.
- [111] J. Engel, J. Sturm, and D. Cremers, "Semi-dense visual odometry for a monocular camera," in *Proc. IEEE Int. Conf. Comput. Vision*, 2013, pp. 1449–1456.
- [112] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1231–1237, 2013. doi: 10.1177/0278364913491297.
- [113] M. Burri et al., "The EuRoC micro-aerial vehicle datasets," *Int. J. Robot. Res.*, vol. 35, no. 10, pp. 1157–1163, 2016. doi: 10.1177/0278364915620033.
- [114] R. A. Newcombe et al., "KinectFusion: Real-time dense surface mapping and tracking," in *Proc. 10th IEEE Int. Symp. Mixed and Augmented Reality*, 2011, pp. 127–136.
- [115] V. Bonato, E. Marques, and G. A. Constantinides, "A floating-point extended Kalman filter implementation for autonomous mobile robots," *J. Signal Process. Syst.*, vol. 56, no. 1, pp. 41–50, 2009. doi: 10.1007/s11265-008-0257-8.
- [116] D. T. Tertei, J. Piat, and M. Devy, "FPGA design and implementation of a matrix multiplier based accelerator for 3d EKF SLAM," in *Proc. Int. Conf. ReConfigurable Comput. and FPGAs (ReConFig14)*, 2014, pp. 1–6.
- [117] D. T. Tertei, J. Piat, and M. Devy, "FPGA design of EKF block accelerator for 3d visual slam," *Comput. Electr. Eng.*, vol. 55, pp. 123–137, 2016. doi: 10.1016/j.compeleceng.2016.05.003.
- [118] B. Vincke, A. Elouardi, and A. Lambert, "Real time simultaneous localization and mapping: towards low-cost multiprocessor embedded systems," *EURASIP J. Embedded Syst.*, vol. 2012, no. 1, p. 5, 2012. doi: 10.1186/1687-3963-2012-5.
- [119] B. Vincke, A. Elouardi, A. Lambert, and A. Dine, "SIMD and OpenMP optimization of EKF-SLAM," in *Proc. Int. Conf. Multimedia Comput. Syst. (ICMCS)*, 2014, pp. 712–716. doi: 10.1109/ICMCS.2014.6911157.
- [120] W. Fang, Y. Zhang, B. Yu, and S. Liu, "FPGA-based ORB feature extraction for real-time visual slam," in *Proc. Int. Conf. Field Programmable Technol. (ICFPT)*, 2017, pp. 275–278. doi: 10.1109/FPT.2017.8280159.
- [121] Y. Biadgie and K.-A. Sohn, "Feature detector using adaptive accelerated segment test," in *Proc. Int. Conf. Inf. Sci. Appl. (ICISA)*, 2014, pp. 1–4.
- [122] M. Calonder, V. Lepetit, C. Strecha, and P. Fua, "Brief: Binary robust independent elementary features," in *Proc. European Conf. Comput. Vision*, 2010, pp. 778–792.
- [123] R. Liu, J. Yang, Y. Chen, and W. Zhao, "eSLAM: An energy-efficient accelerator for real-time ORB-slam on FPGA platform," in *Proc. 56th Annu. Des. Automat. Conf.*, 2019, pp. 1–6.
- [124] V. H. Schulz, F. G. Bombardelli, and E. Todt, "A Harris corner detector implementation in SoC-FPGA for visual slam," in *Robotics*. Springer-Verlag, 2016, pp. 57–71.
- [125] M. Abouzahir, A. Elouardi, S. Bouaziz, R. Latif, and A. Tajer, "Large-scale monocular FastSLAM2. 0 acceleration on an embedded heterogeneous architecture," *EURASIP J. Adv. Signal Process.*, vol. 2016, no. 1, p. 88, 2016. doi: 10.1186/s13634-016-0386-3.
- [126] M. Abouzahir, A. Elouardi, R. Latif, S. Bouaziz, and A. Tajer, "Embedding SLAM algorithms: Has it come of age?" *Robot. Autonom. Syst.*, vol. 100, pp. 14–26, 2018. doi: 10.1016/j.robot.2017.10.019.
- [127] K. Boikos and C.-S. Bouganis, "Semi-dense SLAM on an FPGA SoC," in *Proc. 26th Int. Conf. Field Programmable Logic Appl. (FPL)*, 2016, pp. 1–4. doi: 10.1109/FPL.2016.7577365.
- [128] K. Boikos and C.-S. Bouganis, "A high-performance system-on-chip architecture for direct tracking for slam," in *Proc. 27th Int. Conf. Field Programmable Logic Appl. (FPL)*, 2017, pp. 1–7. doi: 10.23919/FPL.2017.8056831.
- [129] K. Boikos and C.-S. Bouganis, "A scalable FPGA-based architecture for depth estimation in SLAM," in *Proc. Int. Symp. Appl. Reconfigurable Comput.*, 2019, pp. 181–196.
- [130] D. DeTone, T. Malisiewicz, and A. Rabinovich, "Superpoint: Self-supervised interest point detection and description," in *Proc. IEEE Conf. Comput. Vision and Pattern Recogn. Workshops*, 2018, pp. 224–236.
- [131] E. Simo-Serra, E. Trulls, L. Ferraz, I. Kokkinos, P. Fua, and F. Moreno-Noguer, "Discriminative learning of deep convolutional feature point descriptors," in *Proc. IEEE Int. Conf. Comput. Vision*, 2015, pp. 118–126.
- [132] F. Radenović, G. Tolas, and O. Chum, "Fine-tuning CNN image retrieval with no human annotation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 7, pp. 1655–1668, 2018. doi: 10.1109/TPAMI.2018.2846566.
- [133] Xilinx, "DPU for convolutional neural network."
- [134] Z. Xu, J. Yu, C. Yu, H. Shen, Y. Wang, and H. Yang, "CNN-based feature-point extraction for real-time visual slam on embedded FPGA," in *Proc. IEEE 28th Annu. Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM)*, 2020, pp. 33–37. doi: 10.1109/FCCM48280.2020.00014.
- [135] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2015, arXiv:1510.00149.
- [136] S. Krishnan, S. Chitlangia, M. Lam, Z. Wan, A. Faust, and V. J. Reddi, "Quantized reinforcement learning (QUARL)," 2019, arXiv:1910.01055.
- [137] H. F. Langroudi, V. Karia, J. L. Gustafson, and D. Kudithipudi, "Adaptive posit: Parameter aware numerical format for deep learning inference on the edge," in *Proc. IEEE/CVF Conf. Comput. Vision and Pattern Recogn. Workshops*, 2020, pp. 726–727.
- [138] T. Tambe et al., "Algorithm-hardware co-design of adaptive floating-point encodings for resilient deep learning inference," in *Proc. 57th ACM/IEEE Des. Automat. Conf. (DAC)*, 2020, pp. 1–6. doi: 10.1109/DAC18072.2020.9218516.
- [139] F. Li, B. Zhang, and B. Liu, "Ternary weight networks," 2016, arXiv:1605.04711.
- [140] J. Choi, S. Venkataramani, V. Srinivasan, K. Gopalakrishnan, Z. Wang, and P. Chuang, "Accurate and efficient 2-bit quantized neural networks," in *Proc. 2nd SysML Conf.*, 2019, vol. 2019.
- [141] J. Kim, K. Yoo, and N. Kwak, "Position-based scaled gradient for model quantization and pruning," *Adv. Neural Inform. Process. Syst.*, vol. 33, 2020.
- [142] T. Tambe et al., "Adaptivfloat: A floating-point based data type for resilient deep learning inference," 2019, arXiv:1909.13271.
- [143] J. Yu et al., "CNN-based monocular decentralized SLAM on embedded FPGA," 2020.
- [144] H. Zhan, R. Garg, C. Saroj Weerasekera, K. Li, H. Agarwal, and I. Reid, "Unsupervised learning of monocular depth estimation and visual odometry with deep feature reconstruction," in *Proc. IEEE Conf. Comput. Vision and Pattern Recogn.*, 2018, pp. 340–349.
- [145] R. Arandjelovic, P. Gronat, A. Torii, T. Pajdla, and J. Sivic, "NetVLAD: CNN architecture for weakly supervised place recognition," in *Proc. IEEE Conf. Comput. Vision and Pattern Recogn.*, 2016, pp. 5297–5307.
- [146] J. Yu et al., "INCA: Interruptible CNN accelerator for multi-tasking in embedded robots," in *Proc. 57th ACM/ESDA/IEEE Des. Automat. Conf. (DAC)*, 2020.
- [147] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source slam system for monocular, stereo, and RGB-D cameras," *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1255–1262, 2017. doi: 10.1109/TRO.2017.2705103.
- [148] S. Liu, *Engineering Autonomous Vehicles and Robots: The Dragon-Fly Modular-based Approach*, 1st ed. Wiley-IEEE Press, Mar. 2020.
- [149] M. Maimone, Y. Cheng, and L. Matthies, "Two years of visual odometry on the mars exploration rovers," *J. Field Robot.*, vol. 24, no. 3, pp. 169–186, 2007. doi: 10.1002/rob.20184.
- [150] B. Klingner, D. Martin, and J. Roseborough, "Street view motion-from-structure-from-motion," in *Proc. IEEE Int. Conf. Comput. Vision*, 2013, pp. 953–960.
- [151] Y. Jeong, D. Nister, R. Szeliski, and I.-S. Kweon, "Pushing the envelope of modern methods for bundle adjustment," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 8, pp. 1605–1617, 2011. doi: 10.1109/TPAMI.2011.256.
- [152] C. Wu, S. Agarwal, B. Curless, and S. M. Seitz, "Multicore bundle adjustment," in *Proc. CVPR 2011*, 2011, pp. 3057–3064.
- [153] A. Eriksson, J. Bastian, T.-J. Chin, and M. Isaksson, "A consensus-based framework for distributed bundle adjustment," in *Proc. IEEE Conf. Comput. Vision and Pattern Recogn.*, 2016, pp. 1754–1762.
- [154] R. Zhang, S. Zhu, T. Fang, and L. Quan, "Distributed very large scale bundle adjustment by global camera consensus," in *Proc. IEEE Int. Conf. Comput. Vision*, 2017, pp. 29–38.
- [155] A. Suleiman, Z. Zhang, L. Carlone, S. Karaman, and V. Sze, "Navion: A 2-mw fully integrated real-time visual-inertial odometry accelerator for autonomous navigation of nano drones," *IEEE J. Solid-State Circuits*, vol. 54, no. 4, pp. 1106–1119, 2019. doi: 10.1109/JSSC.2018.2886342.
- [156] Q. Liu, S. Qin, B. Yu, J. Tang, and S. Liu, "π-ba: Bundle adjustment hardware accelerator based on distribution of 3d-point observations," *IEEE Trans. Comput.*, 2020.

- [157] R. Sun, P. Liu, J. Xue, S. Yang, J. Qian, and R. Ying, "BAX: A bundle adjustment accelerator with decoupled access/execute architecture for visual odometry," *IEEE Access*, vol. 8, pp. 75,530–75,542, 2020. doi: 10.1109/ACCESS.2020.2988527.
- [158] P. Leven and S. Hutchinson, "A framework for real-time path planning in changing environments," *Int. J. Robot. Res.*, vol. 21, no. 12, pp. 999–1030, 2002. doi: 10.1177/027836490201012001.
- [159] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011. doi: 10.1177/0278364911406761.
- [160] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (bit*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, 2015, pp. 3067–3074.
- [161] K. Hauser, "Lazy collision checking in asymptotically-optimal motion planning," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, 2015, pp. 2951–2957.
- [162] A. Yershova and S. M. LaValle, "Improving motion-planning algorithms by efficient nearest-neighbor searching," *IEEE Trans. Robot.*, vol. 23, no. 1, pp. 151–157, 2007. doi: 10.1109/TRO.2006.886840.
- [163] W. Wang, D. Balkcom, and A. Chakrabarti, "A fast online spanner for roadmap construction," *Int. J. Robot. Res.*, vol. 34, no. 11, pp. 1418–1432, 2015. doi: 10.1177/0278364915576491.
- [164] S. Murray, W. Floyd-Jones, G. Konidaris, and D. J. Sorin, "A programmable architecture for robot motion planning acceleration," in *Proc. IEEE 30th Int. Conf. Appl.-Specific Syst., Arch. Process. (ASAP)*, 2019, vol. 2160, pp. 185–188.
- [165] J. Bialkowski, S. Karaman, and E. Frazzoli, "Massively parallelizing the RRT and the RRT," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots and Syst.*, 2011, pp. 3513–3518.
- [166] J. Pan and D. Manocha, "GPU-based parallel collision detection for fast motion planning," *Int. J. Robot. Res.*, vol. 31, no. 2, pp. 187–200, 2012. doi: 10.1177/0278364911429335.
- [167] J. Pan, C. Lauterbach, and D. Manocha, "G-planner: Real-time motion planning and global navigation using GPUs," in *AAAI*, 2010.
- [168] N. Atay and B. Bayazit, "A motion planning processor on reconfigurable hardware," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA 2006)*, 2006, pp. 125–132.
- [169] S. Murray, W. Floyd-Jones, Y. Qi, G. Konidaris, and D. J. Sorin, "The microarchitecture of a real-time robot motion planning accelerator," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarch. (MICRO)*, 2016, pp. 1–12. doi: 10.1109/MICRO.2016.7783748.
- [170] S. Lian, Y. Han, X. Chen, Y. Wang, and H. Xiao, "DADU-P: A scalable accelerator for robot motion planning in a dynamic environment," in *Proc. 55th ACM/ESDA/IEEE Design Automat. Conf. (DAC)*, 2018, pp. 1–6. doi: 10.1109/DAC.2018.8465785.
- [171] U. Bondhugula et al., "Hardware/software integration for FPGA-based all-pairs shortest-paths," in *Proc. 14th Annu. IEEE Symp. Field-Programmable Custom Comput. Mach.*, 2006, pp. 152–164. doi: 10.1109/FCCM.2006.48.
- [172] K. Sridharan, T. Priya, and P. R. Kumar, "Hardware architecture for finding shortest paths," in *Proc. TENCON 2009 IEEE Region 10 Conf.*, pp. 1–5.
- [173] Y. Takei, M. Hariyama, and M. Kameyama, "Evaluation of an FPGA-based shortest-path-search accelerator," in *Proc. Int. Conf. Parallel Distrib. Process. Techn. Appl. (PDPTA)*, The Steering Committee of The World Congress in Computer Science, Computer Engineering and Applied Computing (WorldComp), 2015, p. 613.
- [174] K. Vipin and S. A. Fahmy, "FPGA dynamic and partial reconfiguration: A survey of architectures, methods, and applications," *ACM Comput. Surveys (CSUR)*, vol. 51, no. 4, pp. 1–39, 2018. doi: 10.1145/3193827.
- [175] S. Liu, R. N. Pittman, and A. Forin, "Minimizing partial reconfiguration overhead with fully streaming DMA engines and intelligent ICAP controller," in *FPGA*, 2010, p. 292.
- [176] S. Liu, R. N. Pittman, A. Forin, and J.-L. Gaudiot, "Achieving energy efficiency through runtime partial reconfiguration on reconfigurable systems," *ACM Trans. Embedded Comput. Syst. (TECS)*, vol. 12, no. 3, p. 72, 2013. doi: 10.1145/2442116.2442122.
- [177] E. Rublee, V. Rabaud, K. Konolige, and G. R. Bradski, "Orb: An efficient alternative to sift or surf," in *ICCV*, vol. 11, p. 2, 2011.
- [178] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. 7th Int. Joint Conf. Artif. Intell.*, 1981.
- [179] W. Fang, Y. Zhang, B. Yu, and S. Liu, "Dragonfly+: FPGA-based quad-camera visual slam system for autonomous vehicles," *Proc. IEEE HotChips*, p. 1, 2018.
- [180] T. Qin, P. Li, and S. Shen, "VINS-MONO: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, 2018. doi: 10.1109/TRO.2018.2853729.
- [181] K. Sun et al., "Robust stereo visual inertial odometry for fast autonomous flight," *IEEE Robot. Automat. Lett.*, vol. 3, no. 2, pp. 965–972, 2018. doi: 10.1109/LRA.2018.2793349.
- [182] R. Szeliski, "Computer vision: Algorithms and applications," in *Texts in Computer Science*. London: Springer-Verlag, 2010.
- [183] A. Geiger, M. Roser, and R. Urtasun, "Efficient large-scale stereo matching," in *Proc. 10th Asian Conf. Comput. Vision*, 2010.
- [184] Y. Feng, P. Whatmough, and Y. Zhu, "ASV: Accelerated stereo vision system," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarch. (MICRO '52)*, 2019, p. 643–656.
- [185] J. F. Henriques, R. Caseiro, P. Martins, and J. Batista, "High-speed tracking with kernelized correlation filters," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 3, pp. 583–596, Mar. 2015. doi: 10.1109/TPAMI.2014.2345390.
- [186] A. Kelly, *Mobile Robotics: Mathematics, Models, and Methods*. Cambridge Univ. Press, 2013.
- [187] J. Tang, B. Yu, S. Liu, Z. Zhang, W. Fang, and Y. Zhang, "π-soc: Heterogeneous soc architecture for visual inertial slam applications," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots and Syst. (IROS)*, 2018, pp. 8302–8307. doi: 10.1109/IROS.2018.8594181.
- [188] S. Qin, Q. Liu, B. Yu, and S. Liu, "π-ba: Bundle adjustment acceleration on embedded FPGAs with co-observation optimization," in *Proc. 27th IEEE Annu. Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM 2019)*, San Diego, CA, Apr. 28–May 1, 2019, pp. 100–108.
- [189] <https://grail.cs.washington.edu/projects/bal/>
- [190] P. L. McKerracher, R. P. Cain, J. C. Barnett, W. S. Green, and J. D. Kinnison, "Design and test of field programmable gate arrays in space applications," 1992.
- [191] M. Berg, "FPGA mitigation strategies for critical applications," 2019.
- [192] D. Sheldon, "Flash-based FPGA NEPP FY12 summary report,"
- [193] R. Gaillard, "Single event effects: Mechanisms and classification," in *Soft Errors in Modern Electronic Systems*. Springer-Verlag, 2011, pp. 27–54.
- [194] M. Wirthlin, "FPGAs operating in a radiation environment: lessons learned from FPGAs in space," *J. Instrumentation*, vol. 8, no. 2, p. C02020, 2013. doi: 10.1088/1748-0221/8/02/C02020.
- [195] F. Brossier and E. Milh, "SEU mitigation techniques for advanced reprogrammable FPGA in space," Master's thesis, 2014.
- [196] B. Ahmed and C. Basha, "Fault mitigation strategies for reliable FPGA architectures," Ph.D. thesis, Rennes 1, 2016.
- [197] S. Habinc, "Suitability of reprogrammable FPGAs in space applications," Gaisler Research, Feasibility Rep., 2002.
- [198] G. Lentaris et al., "High-performance embedded computing in space: Evaluation of platforms for vision-based navigation," *J. Aerospace Inform. Syst.*, vol. 15, no. 4, pp. 178–192, 2018. doi: 10.2514/1.1010555.
- [199] T. Y. Li and S. Liu, "Enabling commercial autonomous robotic space explorers," *IEEE Potentials.*, vol. 39, no. 1, pp. 29–36, 2019. doi: 10.1109/MPOT.2019.2935338.
- [200] D. Ratter, "FPGAs on Mars," *Xcell J.*, vol. 50, pp. 8–11, 2004.
- [201] J. F. Bell III et al., "Mars exploration rover athena panoramic camera (pancam) investigation," *J. Geophys. Res. Planets*, vol. 108, 2003. doi: 10.1029/2003JE002070.
- [202] "Space flight system design and environmental test." <https://www.nasa.gov/sites/default/files/atoms/files/std8070.1.pdf> (accessed Sept. 1, 2020)
- [203] M. C. Malin et al., "The Mars Science Laboratory (MSL) mast cameras and descent imager: Investigation and instrument descriptions," *Earth Space Sci.*, vol. 4, no. 8, pp. 506–539, 2017. doi: 10.1002/2016EA000252.
- [204] C. D. Edwards, T. C. Jedrey, A. Devereaux, R. DePaula, and M. Dapore, "The electra proximity link payload for Mars relay telecommunications and navigation," 2003. doi: 10.2514/6.1AC-03-Q.3.a.06.
- [205] A. Johnson et al., "The lander vision system for Mars 2020 entry descent and landing," 2017.
- [206] "Vivado high-level synthesis." <https://www.xilinx.com/products/design-tools/vivado/integration/esl-design.html> (accessed Sept. 10, 2020)

The Evolution of Domain-Specific Computing for Deep Learning

Stephen Neuendorffer, Alireza Khodamoradi, Kristof Denolf,
Abhishek Kumar Jain, and Samuel Bayliss, *Xilinx Inc.*

Abstract

With the continued slowing of Moore's law and Dennard scaling, it has become more imperative that hardware designers make the best use of domain-specific information to improve designs. Gone are the days when we could rely primarily on silicon process technology improvements to provide faster and more ef-

ficient computation. Instead, architectural improvements are necessary to provide improved performance, power reduction, and/or reduced cost. Nowhere is this more apparent than when looking at Deep Learning workloads. Cutting-edge techniques achieving state-of-the-art training accuracy demand ever-larger training data-sets and more-complex network topologies, which results in longer training times. At the same time, after training



©SHUTTERSTOCK.COM/KKSSR

Digital Object Identifier 10.1109/MCAS.2021.3071629

Date of current version: 24 May 2021

these networks, we expect them to be deployed widely. As a result, executing large networks efficiently becomes critical, whether that execution is done in a data center or in an embedded system. In this article, we look at trends in deep learning research that present new opportunities for domain-specific hardware architectures and explore how next-generation compilation tools might support them.

I. Introduction

Recent advancements in Deep Learning (DL) methods have greatly expanded our ability to *train* data processing systems, rather than *design* them. Coupled to an exponential growth in available digital data like images and speech from various sources, including social media and scientific experiments, we've seen dramatic accuracy improvements in DL applications such as image classification and voice recognition [73]. These improvements have come about because of a virtuous cycle where readily available high-performance computing has made larger DL networks more feasible; the ability to train larger networks combined with the availability of huge data-sets has produced better results, and better results have drawn widespread adoption of these techniques and both the demand for, and investment in improved computing resources.

However both the growing cost of training large DL models on large data-sets, and deploying such models for inference in scale-out services and power-constrained systems, create significant pressure to increase the efficiency of DL computation. Training a single network can cost millions of dollars, and can have significant environmental effects [101]. Over 500 hours of video are uploaded to YouTube every minute, meaning that processing those videos requires significant computational resources [17] and the economic returns to incremental improvements in relative compute-efficiency are large. The use of DL networks in embedded systems, such as flying drones and self-driving cars, is only practical where application-driven throughput and latency requirements can be met using low-cost, power-constrained hardware.

While an obvious approach might be to reduce the complexity of the networks being used, this often has a direct impact on the quality or accuracy of the resulting network. As a result, significant effort has been applied to find ways to reduce the computational requirements of DL models without sacrificing accuracy through methods such as pruning, quantization, and through the use specialized model topologies and layer operations. Unfortunately, many of these methods make architectures for performing DL computation less efficient. This situation represents a significant opportunity for *domain-specific computing*: we can build device architectures tuned to match particular model

architectures, we can build networks that maximize the use of existing devices, or even co-design model topologies together with compute architectures.

Although architecture specialization can lead to increased efficiency [60], other trends naturally lead to more general architectures. DL model topologies are changing rapidly, so overly specialized compute architectures quickly become obsolete. Non-recurring engineering (NRE) costs to bring new devices to the market at the latest technology nodes continue to increase as device manufacturing becomes ever more complex. At the same time, new devices tend to be more heavily integrated, increasing the design time and commercial risk associated with bringing new devices to market. Hence, we desire devices that are not only specialized to particular markets and workloads, but also programmable and flexible enough to address emerging opportunities.

Although applications are closely related to domain-specific architectures, the key to leveraging new architectures ultimately comes down to the tools that exist to compile those applications. As architectures proliferate in the post-Moore's law era, we expect the cost of adapting applications to new architectures to become a significant barrier. Automating this process through tools will become increasingly critical. At the same time, we believe that it is impractical to build new tooling for every device and programming model. Just as retargetable compilers (e.g. LLVM) have become common when compiling sequential code onto microprocessors and synthesizing hardware description languages for programmable logic devices, we believe that with appropriate infrastructure, retargetable compilers will be able to target heterogeneous hardware as well.

In this paper, we take a forward look at some of these trends and attempt to draw some conclusions about how the intersection of domain-specific applications, architectures and tooling will evolve. First we investigate the trends in deep learning (Section II), followed by recent advances on HW architectures, focusing in section III on the Xilinx Versal platform. From there we discuss retargetable tooling in section V and explore in section IV how such tools can be applied to deep learning workloads.

II. Efficiency Trends in Deep Learning

Because of the significant computational requirements to train and deploy DL networks, efficient execution of these models is very important. However, the trade-offs available during training and deployment are actually quite different. During the training stage, the main focus is on reducing training time and performing more experiments in order to improve the

**We believe that with appropriate infrastructure,
retargetable compilers will be able to target
heterogeneous hardware as well.**

accuracy of the trained model. While the cost of training can be significant, the most limiting resource during training is usually the expertise of an DL-trained engineer to ensure that the training process makes progress. In order to minimize risks associated with training, it is common to use automated techniques like Network Architecture Search (NAS) [41] and scale compute performance by distributing computation across computer networks.

For deployment, however, a network configuration is often frozen and will be executed repeatedly. At this point, additional optimization can reduce the computational requirements of a network. Table I shows the memory size and compute requirements of some modern networks. Storing and executing these networks can become challenging, especially when multiple networks need to run concurrently. Embedded systems often have severe limitations on power consumption due to battery size and heat dissipation limitations, so using more memory or a faster processor may simply not be possible. Larger models also typically require more memory bandwidth, which can significantly limit achievable performance.

In the subsections that follow, we review many existing techniques for optimizing DL networks for efficient execution. We divide these into three main categories, focusing on leveraging quantization, sparsity, and co-design between models and architectures.

A. Quantization

One approach to model optimization is the *Quantization* of the weights and activations to use less expensive floating-point or reduced-precision fixed-point operations [55], [120]. During training, it is common to use 32-bit single-precision floating-point rather than 64-bit double-precision, since most networks can still be trained effectively at single precision [50]. In some cases, training can still be performed using 16-bit half-precision or ‘bfloat’ formats, although in others, convergence on an optimum may be slowed, and the training process may not achieve optimal accuracy if the loss-minimizing optimization process gets ‘stuck’ in a local minima.

During deployment, more aggressive quantization is often possible, since deep learning models are fundamentally designed with generalization and robustness in mind. As a result, it is often possible to reduce the precision of computation in a network without signifi-

cantly affecting the results. For instance, many image processing applications can be trained and then quantized to 8 bits for inference without much loss in accuracy [24]. Language processing applications are often deployed using floating-point, due to higher dynamic-range requirements. Quantization to very small bitwidths (i.e. 1 or 2 bits) [93], is also possible for some networks, but often requires a quantization-aware training process.

To incorporate quantization during training, a common technique is to add quantization operations in the forward path, but to compute the backward path using floating point. This allows the training process to continue without getting stuck in a local minimum. Specialized training frameworks, such as Brevitas [5], QPyTorch [13], and quantized TensorFlow [10], facilitate representing these quantization operations. Brevitas, in particular, is focused on low-bitwidth quantization and is integrated with the FINN framework [7], [28] to enable implementing heavily quantized networks in programmable logic.

One challenge in training quantized networks is identifying appropriate amounts of quantization for the different layers of a network. For instance, in quantized image processing it is common to compute the first layer of a network in a precision that takes advantage of the sensitivity and dynamic range of a connected sensor. Then later layers can be quantized more aggressively.

**Table I.
Memory and compute requirements of some modern
networks.**

Network	Model Size (MB)	GFLOPS
AlexNet [107]	233	0.7
VGG-16 [107]	528	15.5
VGG-19 [107]	548	19.6
ResNet-50 [107]	98	3.9
ResNet-101 [107]	170	7.6
ResNet-152 [107]	230	11.3
GoogLeNet [107]	27	1.6
InceptionV3 [21]	89	6
MobileNet [21]	38	0.58
SqueezeNet [21]	30	0.84

The Xilinx Versal architecture includes specialized on-chip network resources for data-distribution and an array of Vector-VLIW processor cores.

However, quantizing all layers of a network equally does not necessarily achieve the best result. In addition, it may be helpful to train a network initially with higher precision and then to increase quantization only after a network is mostly trained.

One approach to introducing quantization during training involves changing bit-precision based on a performance criterion. In [115], networks are trained with two different bit-widths. Each layer first applies its filters with low-precision parameters, and then a gating scheme compares the results against a learnable threshold and decides if an update stage has to be computed or skipped. Performing the update stage is equal to applying full-precision filters to the input.

Note that in some cases, it is also beneficial to quantize weights and activations differently. Since in most networks, weights must be stored off-chip while activations from a previous layer are stored on-chip, weight quantization has a much bigger effect on memory bandwidth than activation quantization [65], [95].

B. Pruning and Static Sparsity

Pruning is another optimization that can reduce the computational complexity of DL networks. Pruning le-

verages the redundancy in the representation of many networks by replacing small weight values with zero. Huang et al. [63] showed that more than 90% of the weights could be removed for some networks without a drop in accuracy. This reduces the storage capacity required to implement the network, and also allows the elimination of 80% of the associated compute operations. Pruning introduces *static sparsity* in networks, since this structure is fixed in the weights as part of the training of a network. This can enable significant architectural optimization during the inference process, since multiplications by zero will always have a zero output. As with quantization, pruning is often incorporated into the training process, enabling improved model accuracy. Pruning techniques are an active area of research [43] and new approaches are still emerging [44], [77], [105].

Early works such as Optimal Brain Damage [37] and Optimal Brain Surgeon [53] focus on iteratively retraining a network to encourage sparsity. At each iteration, these techniques compute the Hessian of the loss function to identify low importance weights, assigning them to zero. After multiple iterations, the introduced zeros tend to remain, while other weights adjust to

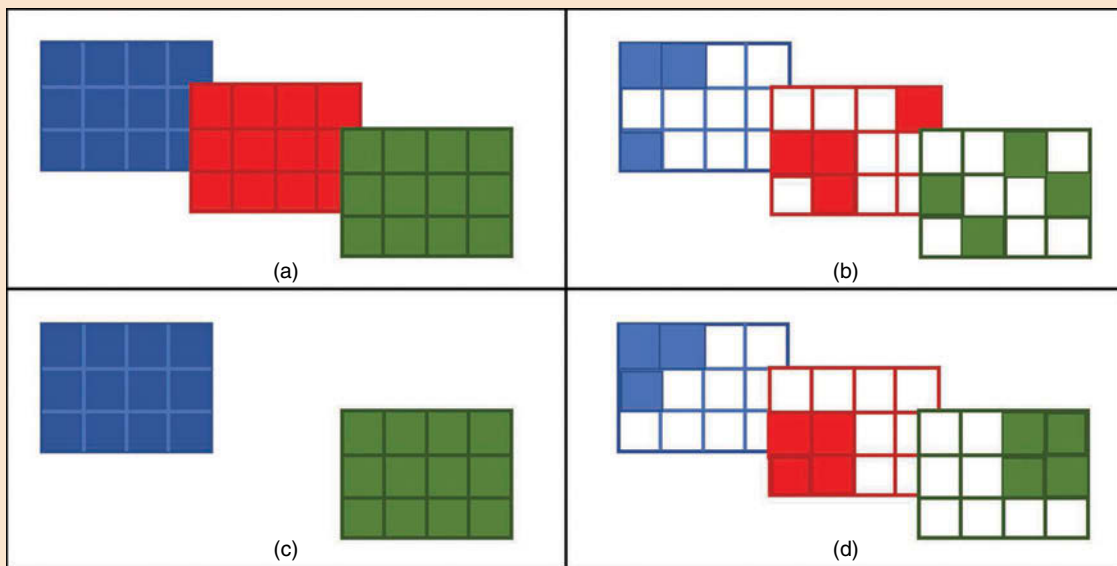


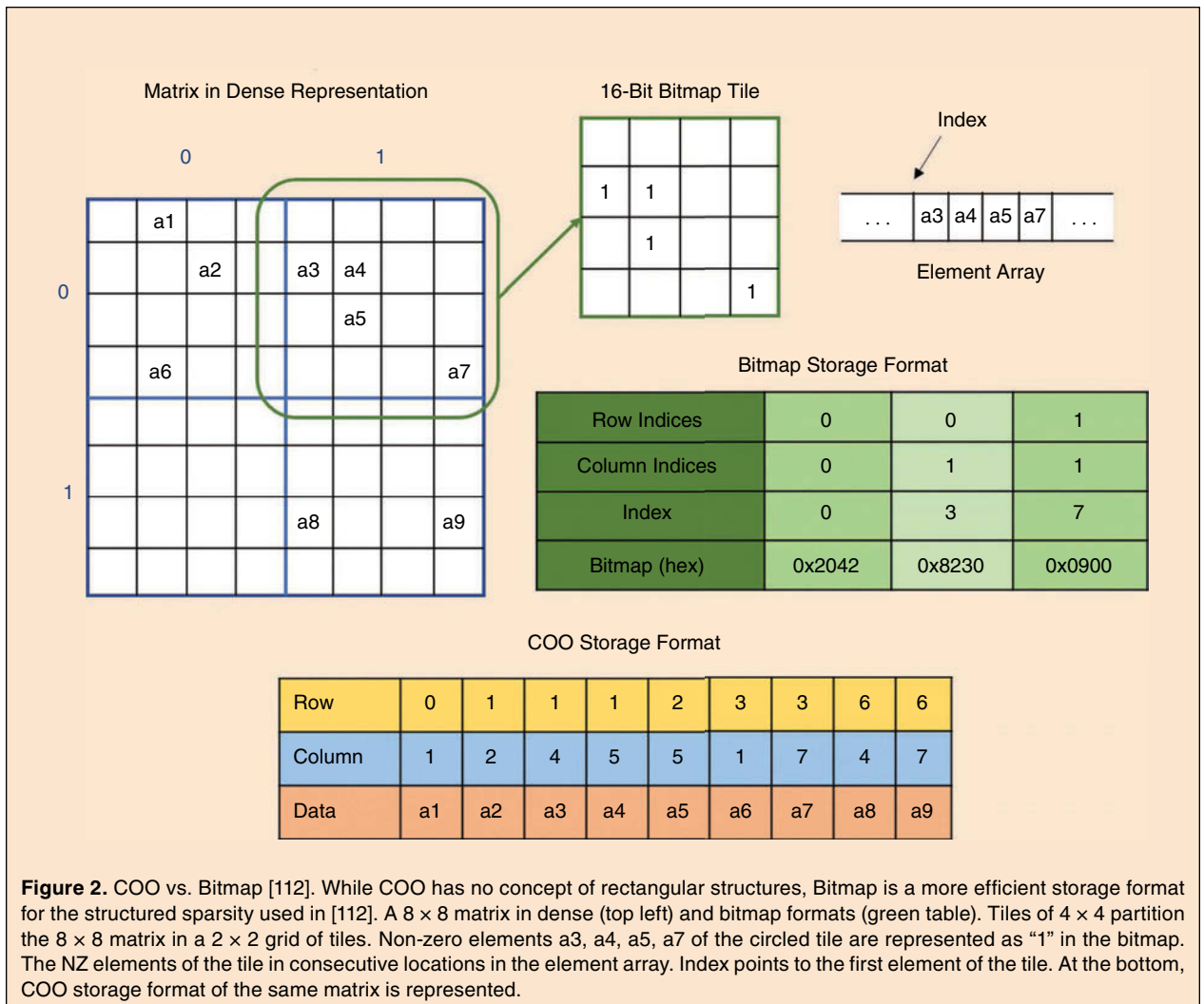
Figure 1. Static sparsity: weights in a neural network (a), can be sparsified by pruning. (b) shows a general and unstructured pruning of the parameters, (c) shows channel pruning, and (d) demonstrates an example of per-channel structured pruning.

accommodate for the pruned weights. One downside of these techniques is the increased training complexity, requiring additional training iterations and the calculation of second-order information.

Another pruning approach involves penalizing non-zero parameters in the training stage with additional regularization terms [52]. After training, small weights can again be pruned and the network retrained. This approach can reduce the iterations needed during retraining, reducing the overall training cost.

Although the usual goal is to preserve a network's accuracy during pruning, more aggressive pruning strategies can be considered that balance computational cost and network accuracy. In Slimmable Neural Networks [111], authors introduced a method to train multiple versions of a network with different channel depth. Then, at runtime, such networks can adjust their width on the fly based on the on-device resource constraints and preferred accuracy.

Several sparse network encoding schemes have been developed that are tuned to particular hardware platform characteristics. For instance, bitmap-based block shape storage format can be more suitable than a Coordinate format (COO) [112] when used to encode sparse matrices on GPU Tensor Core Units (TCU) [8], [9] (Figure 2). Efficient storage formats can also improve memory bandwidth. Tensaurus [100] co-designs DL hardware with a sparse storage format, enabling vectorized and streaming access of weight data and maximizing the utilization of the memory bandwidth. In [40], the authors argue that while the weight matrix is sparse, the activation matrix is still dense. Therefore, by storing layer outputs in CHW format instead of HWC (H: height, W: width, C: channels), vector loads from the activation matrix make it possible to process multiple spatial locations simultaneously. Their work showed that this technique could reduce the number of floating point operations required by MobileNet [61] and EfficientNet



MLIR [76] is a rapidly maturing compiler infrastructure focused on building and transforming abstractions. MLIR allows tools to be built with a wide variety of abstractions, called *dialects*.

[104] on an ARM processor by $2.4\times$ compared to their dense versions.

1) Dynamic Sparsity

Avoiding multiplication operation for zero operands in an efficient fashion requires carefully designed methods. ExTensor [57] suggests a hierarchical elimination of computation that relies on a rapid search for zero operands. Then a Skip Mechanism is used to leverage reusable data and efficiently load non-zero operands to the processing elements. Channel gating [62] is another technique for exploiting dynamic sparsity. In this technique, each channel's contribution to the final result is calculated, and by dynamic pruning based on a learnable threshold, all computations for a subset of input channels with low contributions are skipped.

Another optimization problem related to sparse parameters is related to memory allocation for the activations. While the memory allocation size for the sparse representation of weights can be determined when a network is designed, the size of output activations tends to be data-dependent and may change from one execution to the next. Liu et al. [81] presents different methods to determine the required memory allocation for the output. These methods represent different tradeoffs between memory utilization and the computational complexity of the algorithms used to calculate tight estimation bounds for the required memory allocation size. The varying quality of memory size estimations using these methods is important, since allocations that are too large will waste memory, while allocations that are too small will need to be dynamically reallocated: a costly operation.

As opposed to General Matrix Multiplication (GEMM), Sparse Matrix Multiply (SpMM) operations are quite irregular and often result in low arithmetic intensity. In

[27], the authors argue that decomposing SpMM operations into coarse-grained tasks is more suitable for processing cores with less parallelism. By contrast, for massively parallel computing, they argue that SpMM operations should be decomposed into fine-grained tasks. They follow this with the introduction of an algorithm that breaks down the computation into a set of data-parallel primitives such as gather, scatter, scan, and stable sort by key. The result is a robust, reliable, and efficient algorithm that achieves an average speedup of $5.7\times$ on a GPU. A high-level summary of this algorithm is shown in Algorithm 1. Bandwidth-saving optimizations of this algorithm can be found in [38]

2) Structured Sparsity

While previous pruning techniques are effective at introducing sparsity, this sparsity tends to be randomly distributed among the weights of a model. This *unstructured* sparsity (Figure 1.b) does not directly consider implementation constraints during the pruning process. While any pruned model can be represented in a compressed sparse format, the storage and computation can often be made more efficient by creating *structured* sparsity in the model. This cost reduction can be substantial when pruning is conducted in a fashion amenable to hardware [40], avoiding the irregular memory access patterns generated when computing sparse matrix-multiplications.

Exploitation of Block-sparse [32] (similar to Figure 1.d) and bank-balanced patterns increases the arithmetic efficiency. By grouping non-zero weights together in a block-sparse pattern, blocks of weights can be processed more efficiently using common dense computation techniques. Grouping weights in a bank-balanced pattern limits the number of non-zero weights in each vector. This technique allows the number of computation lanes to be reduced in a vector architecture, with a small amount of additional logic to route non-zero elements to the populated computation lanes.

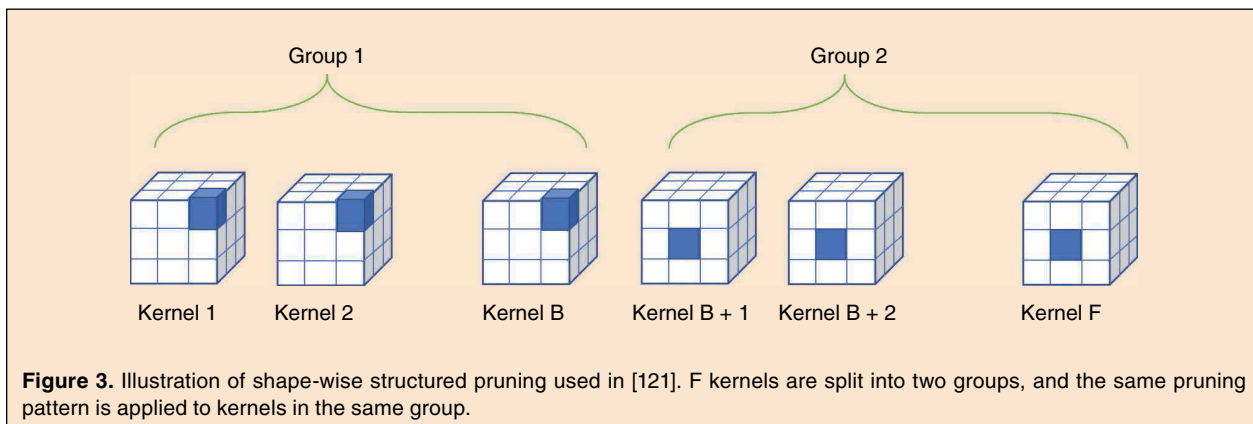
Heavily customizable devices such as FPGAs have been regarded as promising hardware platforms for convolutional neural networks. On these platforms, different sparsity patterns can be supported by adding a trivial control logic. Therefore, the DL application's weights can be pruned to more than one pattern, which provides

Algorithm 1: Data-parallel SpMM [27], [38]

```

Input: A, B
Output: C
1  $S \leftarrow \text{slice}(A)$  //decompose rows into slices
2 for  $k = 0 : S$  do
3    $\hat{C}_k \leftarrow \text{expand}(A_k, B)$  //expand intermediate matrix
4    $\hat{C}_k \leftarrow \text{sort}(\hat{C}_k)$  //sort by row and column
5    $\hat{C}_k \leftarrow \text{compress}(\hat{C}_k)$  //compress duplicate entries
6  $\hat{C} \leftarrow \text{construct}(\hat{C}_k)$  //concatenate slices

```



more flexibility and possible improvements in the training stage while keeping an efficient pipeline and data-flow design for inference hardware. For instance, in [121], filter kernels (multi-channel matrices) are split into multiple groups and each group is pruned on the same locations (Figure 3). Then a control logic skips the zero operands in different groups and reuses the processing units (PU) for implementing a dataflow design. This resulted in $6.7 \times$ speedup and $6.2 \times$ energy-efficiency for VGG-16 [80] on Xilinx ZCU102 FPGA [1].

Structural pruning can also be achieved through channel pruning (Figure 1.c). This is a hardware-agnostic technique and can be easily implemented on most hardware platforms. However, pruning a whole channel from a network requires extra care in the training stage. He et al. [56] introduced a sequential approach for channel pruning and showed a $2.5 \times$ improvement for VGG-16 on GPU time. Other hardware platforms also benefit from this technique, eliminating all the compute operations for the pruned channel.

C. Efficient Model Design

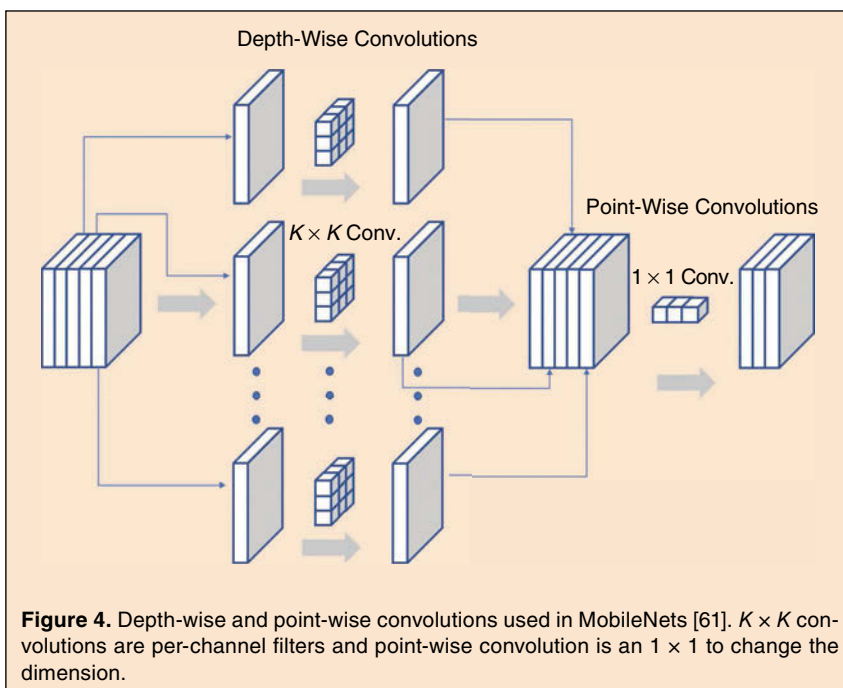
a) Network Topology

When designing a network, its architectures can be shaped to be more suitable for efficient hardware implementation. The MobileNet [61] family of classification networks does this by taking advantage of depth-wise separable convolutions. As shown in Figure 4, these networks first find spatial features within each channel filter and then combine these channels using point-wise (1×1 convolutions)

to form an output tensor. This technique results in fewer parameters and MAC operations.

ShuffleNet [114] introduces efficient convolutional networks for mobile devices by reducing the computation by point-wise convolutions (1×1 kernels) and shuffling channels (Figure 5.a-d). HadaNet [118] improves the accuracy of ShuffleNet by using Hadamard transforms to build efficient DNN blocks for mixing channel information (Figure 5.e).

Networks such as MobileNet and ShuffleNet are examples of networks specifically hand-crafted for deployments in mobile devices. In general, creating a network requires selecting a set of layer types and functions (i.e., fully-connected layer, convolutional layer, kernel sizes, activation functions, etc.) and deciding how to order and connect these layers (inter-layer and

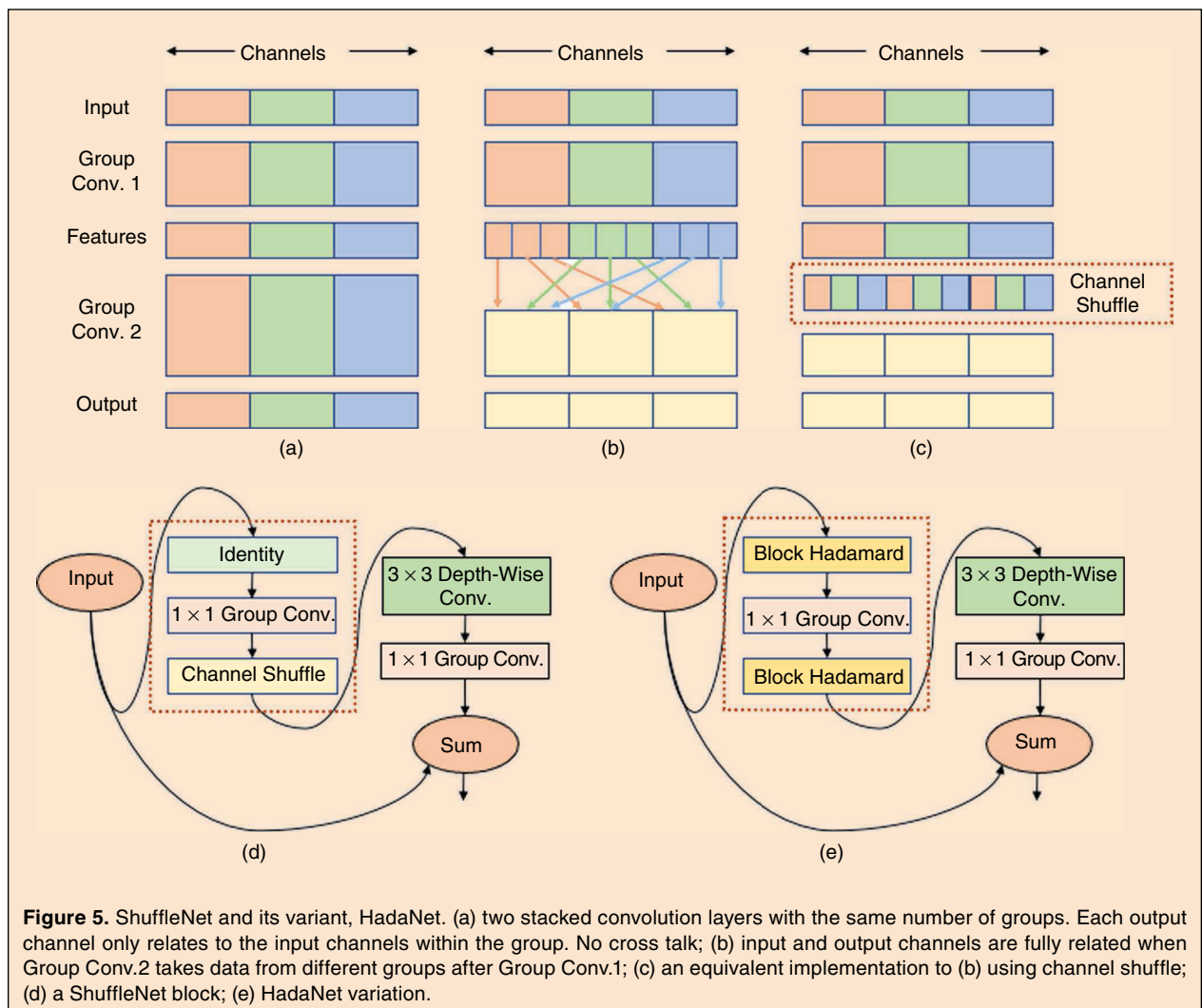


intra-layer connections). Therefore, manually searching for a *good* topology may not be a feasible task for many applications and designers, given the large combinatorial search space and the long training times associated with the evaluation of each design point.

Current Network Architecture Search (NAS) algorithms aim to resolve this problem by providing computationally feasible search algorithms. However, this search space can easily be expanded by including hardware-related parameters such as quantization, weight sparsification, and platform-specific parameters such as memory bandwidth. Recently, hardware-aware NAS algorithms try to include implementation factors as another constraint for efficiency in their search. For example, MnasNet [103] considers the model latency for producing mobile-friendly architectures. Yet, there is still a lot of room for improvement as currently, a single (or a limited number of) hardware platform(s) can be included in NAS [30], [99], [113].

b) Knowledge Distillation

In many cases, best performing supervised learning models have many parameters and are computationally expensive. However, smaller networks are more preferred for hardware realization. Knowledge Distillation is a training method for model compression that attempts to resolve this issue by training models with fewer parameters and competitive inference accuracy compared to the larger ones. In this method, a smaller network is trained to learn a larger network's behavior by replicating its output at every level [59], [109]. The larger network is trainable with current training methods and has more parameters, hidden layers, and architectural complexity. Although this method has its limitations, it is a promising technique to train smaller networks suitable for edge-computing. For example, in a modified knowledge distillation method, a teaching assistant network is used as an intermediate network to learn from a larger network and teach an even smaller network [83].



III. Architectures For Deep Learning

Given the computing needs associated with deep learning and slowing of technology scaling, many devices have been introduced that are tuned for deep learning [46]. For instance, Intel has introduced VNNI vector instructions tuned for deep learning [2] and GPU architectures have introduced specialized Tensor cores [112]. Some architectures, such as the Google TPU [67], are highly specialized for inference workloads, while other architectures are targeted at training of networks [87]. Some architectures focus on avoiding off-chip memory accesses and sparse computation [79]. While these architectures are focused on massively scaled-up workloads, other companies, such as Huawei [58], have developed architectures targeted at families of devices suitable for a variety of workloads. Much of this innovation has been driven by the presence of Machine-Learning benchmarks focused on different workloads, such as the MLPerf training benchmarks [82], the MLPerf inference benchmarks [94], and the sparse DNN graph challenge [70].

In this context, modern FPGAs fill an important role in domain-specific computing, where they typically perform better than general-purpose processors (CPUs), are more integrated and power-efficient than graphics-processing units (GPUs), and are more flexible and offer faster time-to-market than Application-Specific Integrated Circuits (ASICs). FPGA architectures also scale to a wide variety of device sizes, enabling efficient deployment in a range of throughput requirements. However, even though programmable logic architectures have incorporated many specialized blocks over time, there is still a significant cost to bit-level programmability [74] that might be narrowed by further specialization. The Xilinx Versal architecture is a further step in this direction. It includes specialized on-chip network resources for data-distribution and an array of Vector-VLIW processor cores suitable for many kinds of dense Multiply-Accumulate computation. In conjunction with fine-grained programmable logic, these new resources are capable of addressing many challenges in deep learning.

A. From Programmable Logic to Heterogeneous Computing

The earliest FPGAs, such as the Xilinx 2064, were capable of replacing a few fixed-function logic gates with a single device¹. Although not the only programmable device of their time, FPGAs typically excelled at more complicated logic functions that were difficult to express in

the less general structures of other devices. In an era when computers often consisted of many discrete logic chips and a few larger integrated circuits, the computing capabilities of early FPGAs were relatively limited. However, as devices became larger, building customized arithmetic circuits in FPGAs became possible and many people began to see the potential of FPGA-based logic to implement significant computation circuits. Later FPGAs, such as the Xilinx Virtex family include specialized DSP units implementing multiplication [12] and on-chip memory resources. These resources were often used to implement efficient customized computations. Over time, DSP units have expanded to support more complex DSP operations, including accumulation, pre-addition, and shifting, further increasing the available compute capability [14].

As FPGAs grew even more capable, it has become common to implement FPGA *overlays*, or designs in FPGAs that are themselves programmable in interesting ways. One aspect of this trend was the widespread use of ‘soft processors’ to implement complex control logic with relatively low performance requirements more efficiently. Soft processors have ranged from very simple micro-controllers [11] to complex vector processors [98], [110]. Soft processors often form the basis for programmable overlays, implementing unusual computing architectures that are tuned for particular applications or programming models [34], [49]. With careful design, programmable overlays can leverage many of the underlying FPGA structures, avoiding paying the overhead of programmability twice.

The need for control-oriented processors capable of running a regular operating system has led to most FPGA devices including hardened processor subsystems, which are often used in conjunction with more specialized soft processors implemented in programmable logic. These devices can be viewed as general-purpose processors extended through programmable logic with domain-specific computing capability.

B. Compute Efficiency

In practice, FPGA-based computing often requires a combination of DSP blocks to implement multiply-accumulate operations, on-chip Block RAM (BRAM) blocks to implement data storage, and programmable logic to implement pipeline control and unusual operations that don’t map well to DSP blocks. While some applications benefit greatly from the flexibility of bit-level programmable interconnect, other applications with high compute loads but lower sample rates, may not. In this case, even more specialized compute blocks and interconnect resources can be more efficient. The Xilinx Versal architecture [15], [45] shown in Figure 6 provides this specialization with

¹For a first-person account of the early history of FPGAs, see the Computer History Museum’s oral history interview with Bill Carter, an early Xilinx employee <https://www.youtube.com/watch?v=1oG-3XWLgog&list=UUHDR4RtxwA1KqKGwxgdK4Vg&index=65>.

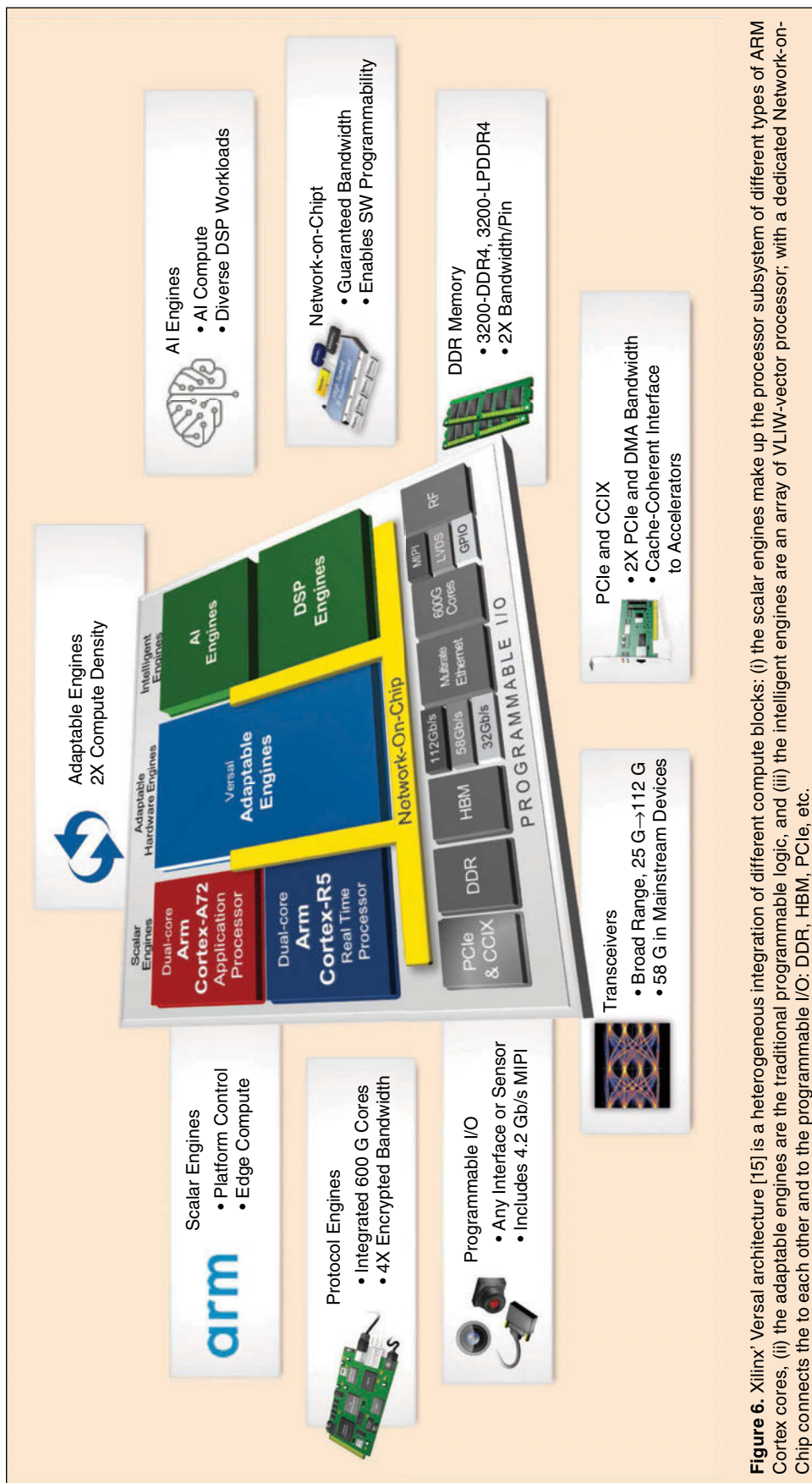


Figure 6. Xilinx' Versal architecture [15] is a heterogeneous integration of different compute blocks: (i) the scalar engines make up the processor subsystem of different types of ARM Cortex cores, (ii) the adaptable engines are the traditional programmable logic, and (iii) the intelligent engines are an array of VLIW-vector processor, with a dedicated Network-on-Chip connects the to each other and to the programmable I/O: DDR, HBM, PCIe, etc.

two new kinds of resources: an array of specialized AI Engine processors and a pervasive on-chip network interconnect.

Each AI Engine processor (Figure 7) is an independent processor with its own 16 kB program memory and 32 kB data memory. The AI Engine architecture is a highly parallel VLIW processor capable of executing up to 7 simultaneous, independent instructions scheduled to execute in the same clock cycle. These instructions can be scalar instructions, executing on the scalar datapath, or vector instructions executing on the vector datapath. For instance, a complex VLIW instruction could execute 2 vector loads, 1 vector store, 1 vector operation, 2 scalar operations. Each processor can be clocked at over 1 GHz and executes an independently compiled C/C++ program, avoiding concerns about timing closure common that can arise with high-speed programmable logic designs. Specialized signal processing operations including zero-overhead loop instructions and pointer-increment addressing modes are also supported to increase code density and efficiency.

The AI Engine vector unit implements one-dimensional 32-bit integer and fixed-point SIMD operations executing on each element of a 8 element vector independently. The output of the vector unit can be accumulated in a special wide vector accumulator register. After a final result is accumulated, it can be

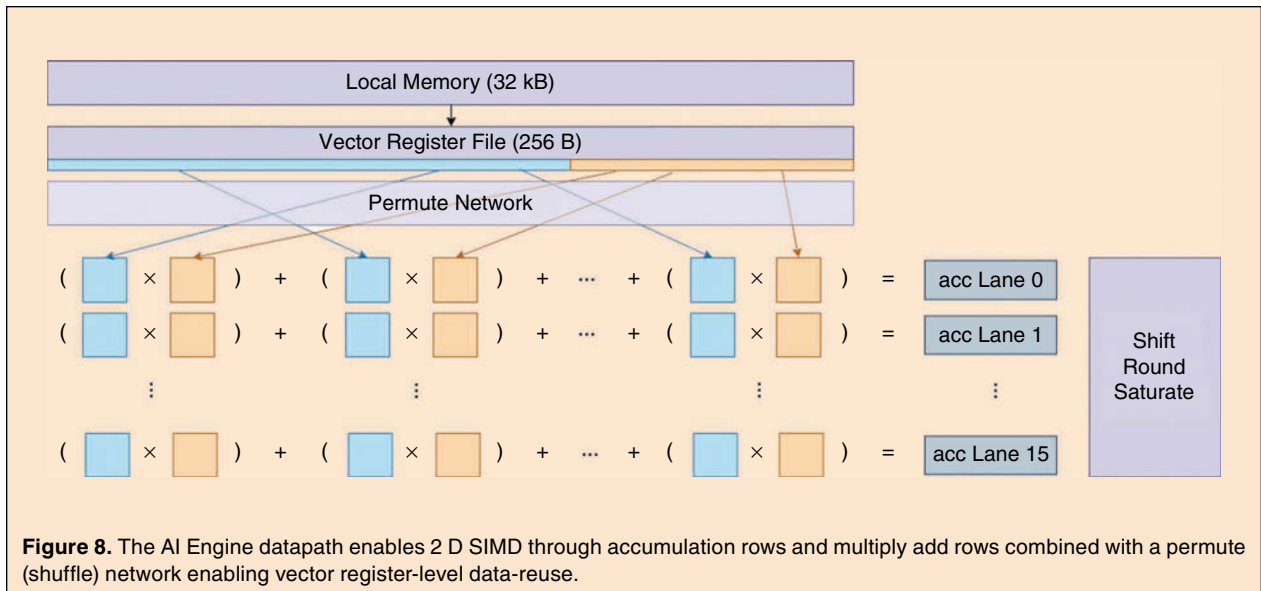
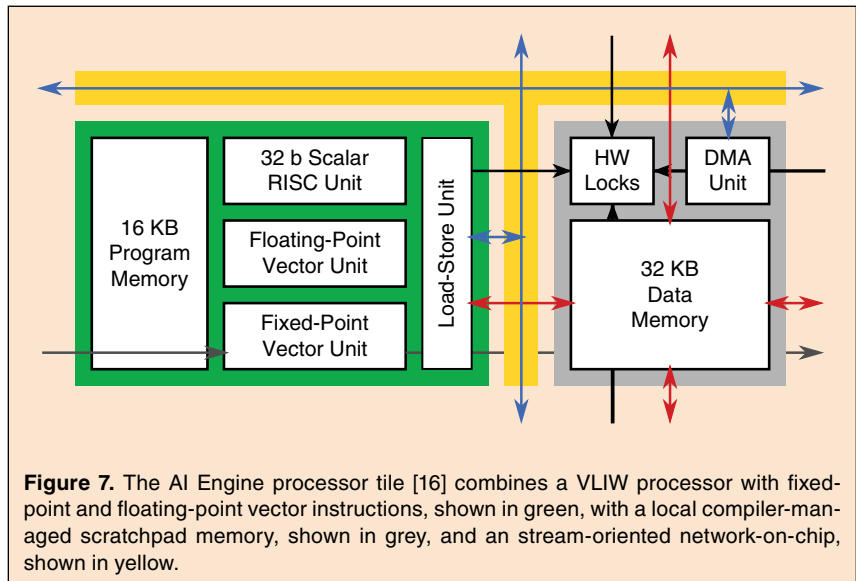
shifted, rounded or saturated before storing it in the local memory. 32-bit single-precision vector floating-point operations are also supported.

For 16-bit and 8-bit datatypes, the AI Engine vector unit implements two-dimensional SIMD operation (Figure 8) organized into rows and columns of multiplications. Each element of the output vector is computed from a row of multiplications which are added together, forming a single vector lane. The result of this reduction is accumulated in the wide vector accumulator register. The number of columns of a vector operation is dependent on the precision of the operand. 8-bit vector operations execute as 16 rows with 8 columns, yielding 128 8-bit MACs per clock. 16-bit vector operations are configured either as 8 lanes with 4 columns or 16 lanes with 2 columns, yielding 32 16-bit MAC per clock. Various combinations of complex data types are also natively supported [16].

For area-efficient implementation, the AI Engine architecture requires all vector memory operations to be on aligned addresses. To enable register-level data reuse and operations on unaligned data, the vector datapath includes a fused programmable permute network. The permute network enables flexible selection of vector register data between the vector register file and the SIMD datapath.

C. Communication and Memory Efficiency

In addition to arithmetic, the AI Engine architecture also has specialized communication and local storage capabilities, illustrated by arrows and dark grey blocks of Figure 7. These capabilities are designed to allow concurrent pipelined communication and compute, with each vector datapath executing efficiently from local memory, while the next set of data to be processed is being loaded into memory. Unlike most multi-processor architectures that rely on a cache hierarchy to reduce external memory bandwidth, the AI Engine architecture puts explicit data movement in the hands of programmers and tools. This approach avoids performance impacts from stalls, reduces redundant data



storage, and maximizes the utilization of relatively slow external memory. AI Engine processors have several communication mechanisms to support different application patterns, including shared memory and streaming communication.

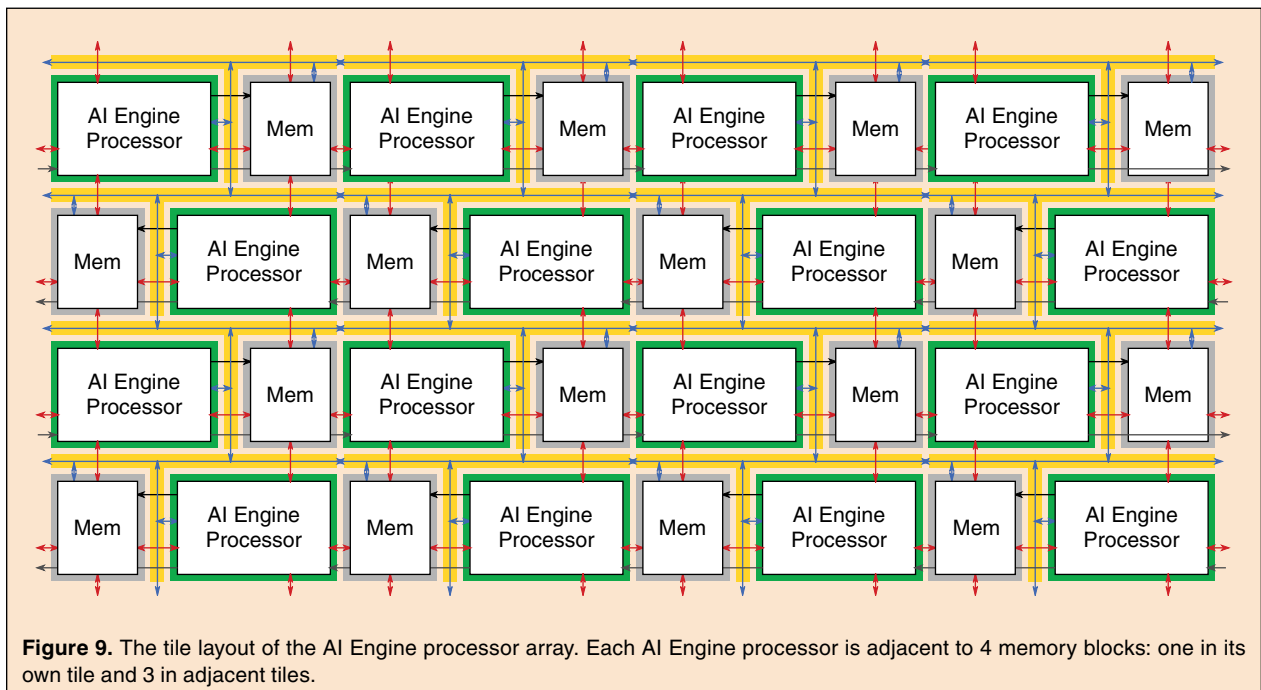
Shared memory communication allows direct communication between AI Engines, with processors reading and writing values in adjacent memories (represented by red arrows in Figure 7). This access is facilitated by the physical layout of AI Engine processors and memories in a checkerboard pattern, shown in Figure 9. To reduce synchronization costs, each data memory is also associated with a set of dedicated hardware locks that can be used to synchronize this communication with neighbours. Typically, communication happens through a pair of *ping-pong buffers*, where one processor is writing to one of the buffers while another processor is reading from another. When a processor is done accessing one buffer, it releases the lock associated with that buffer and attempts to acquire the lock associated with the other buffer. In this way, both processors can proceed in a pipelined fashion, with each processor operating concurrently on different blocks of data.

The stream-switched interconnect enables communication between non-neighboring processors, supported by the local DMA engines in each processor tile. These DMA engines access processor data memory and are capable of sending or receiving data over the AXI stream switched interconnect, represented by blue arrows of Figure 7. The DMA engines also synchronize

with AI Engine processors through the hardware locks. DMA-based communication can be overlapped with processor computation, requiring 2 independent sets of ping-pong buffers. The stream-switched interconnect is also able to broadcast data from one source to multiple destinations simultaneously.

A final communication mechanism is a direct cascade stream connection between horizontal neighbours, represented by grey arrows in Figure 7, enabling values stored in the wide vector accumulation registers to be communicated over a dedicated stream connection without loss of precision.

Communication between the AI Engine processors and the rest of the device can be performed in several ways. One mechanism allows stream switch connections directly into the programmable logic, enabling a wide, high bandwidth connection with programmable logic IP. It is also possible to leverage the hardened Network-on-Chip (NoC) [102], which connects the major building blocks of the Versal device (processor subsystem, programmable logic, the AIE-array) to each other and to the external memory and other I/O interfaces as shown in Figure 10. The Versal NoC is packet switched and supports both memory mapped and streaming connections with a deterministic routing flow. The NoC provides a persistent addressable interconnect that unifies all resources of the platform, enabling more of the programmable logic to be used for custom logic. The AI Engine stream switch interconnect can pass streaming data directly into the Noc, or hardened



DMAs can be used to convert streaming data into memory-mapped transactions.

While the Versal NoC provides enough bandwidth to saturate the available four 25 GBps DDR4 memory interfaces, applications leveraging large numbers of AI Engine processors may benefit from access to higher bandwidth on-chip memory. Application specific processing and data movement engines in programmable logic can complement the dense computational capabilities of the AI Engine processors. Deep Learning applications, in particular, can leverage the flexibility of programmable logic to build a memory hierarchy tuned to the needs of a particular type of network, or to implement layer types that don't map efficiently to the AI Engine architecture.

IV. Deep Learning on Versal

Several different mappings of DL applications are possible on Versal devices. These mappings tend to focus on leveraging specific characteristics of DL models that may be critical resources. For an overview of exploiting similar techniques on other HW architectures we refer to [46] and focused on FPGAs to [108].

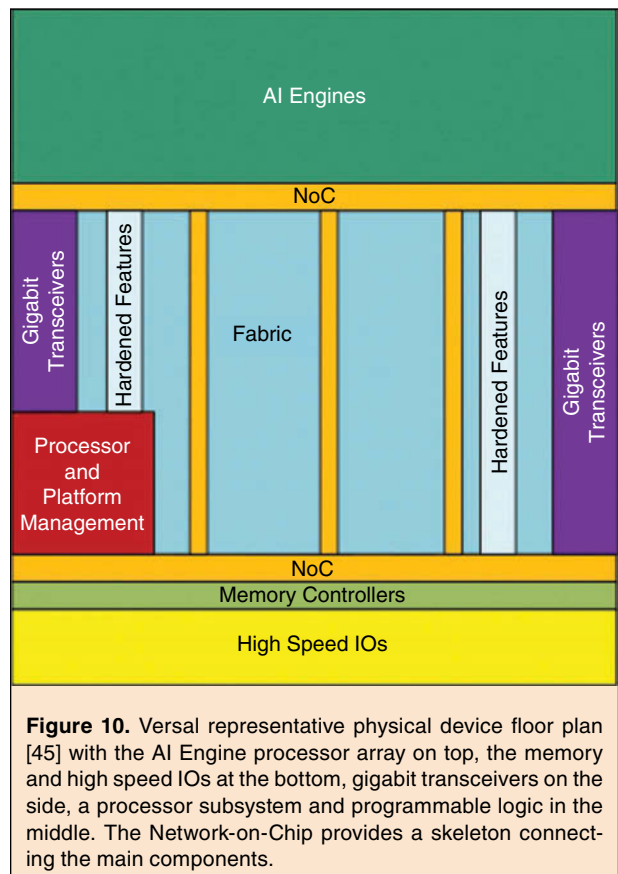
A. Dense Layer by Layer Processing

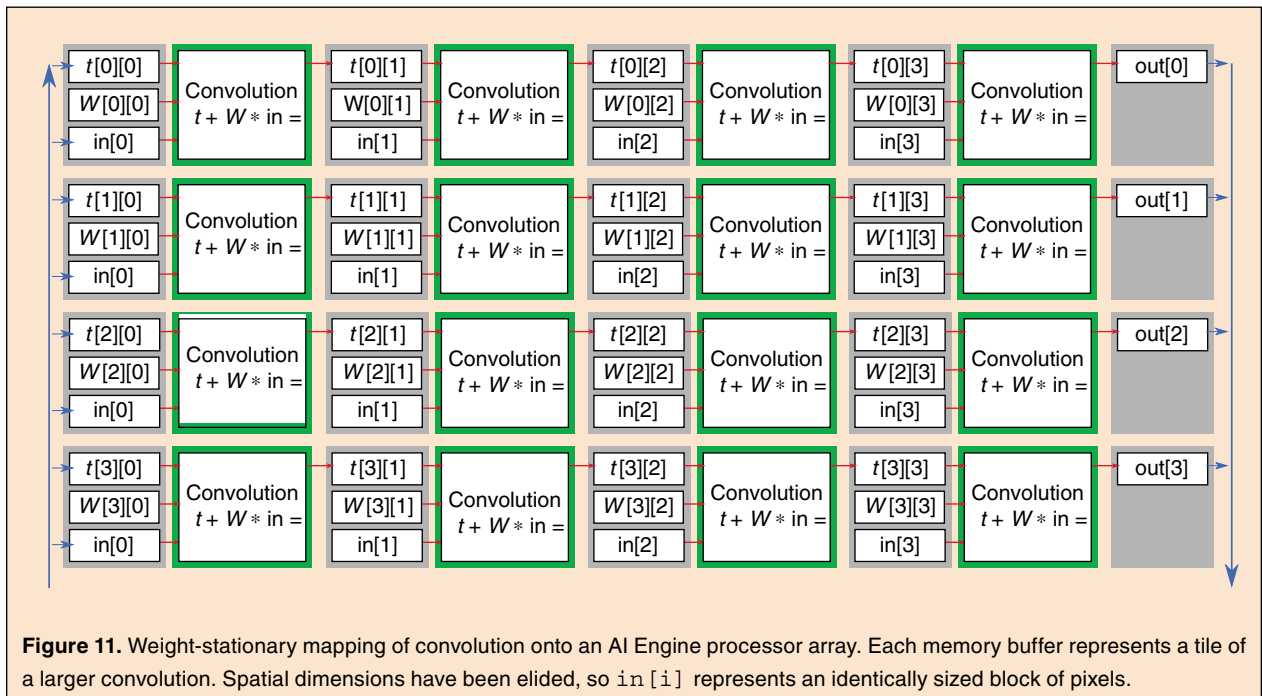
One possible approach is to implement layer-by-layer processing, similar to the approach of common GPU-oriented deep learning libraries, like cuDNN [35]. This approach typically involves large accelerators focused on accelerating dense linear algebra. Such accelerators leverage the data locality present in convolution and matrix multiplication operations to execute efficiently. In programmable logic, this approach has been implemented by the Xilinx DPU IP [18] and in a number of research projects. Similar techniques are also possible using arrays of AI Engine processors, as shown in Figure 11.

In this architecture, a single dense convolution layer is mapped onto a rectangular array of AI Engine processors. Each processor is configured with an independent set of weights where each row of processors contains the weights for the same output channel and different input channels. Each column of processors is designed to process the same portion of input data, enabling use of the stream broadcast mechanism to replicate input data to different processors. As each column of processors completes a block of partial results, it receives a set of partial results from a neighboring processor on the left and passes the accumulated result to its neighbor on the right. This results in an accumulated convolution result in the rightmost column of processors. To facilitate larger convolutions, partial outputs can also be presented to the input and accumulated during the convolution process.

B. Dataflow

An alternative approach is dataflow mapping (Figure 12), also called depth-first processing [47], resulting in a streaming implementation. Data flows through this pipelined computational graph implementing the neural network, with each layer corresponding to one pipeline stage. As with any pipelined architecture, balancing the execution time of different layers is important, since the pipeline will only execute as fast as the slowest layer. Hence, adapting the amount of resources per layer without impacting neighbouring layers is a key characteristic of dataflow frameworks. Channel deep line buffers (grey bars in Figure 12) only store the neighborhood data needed by the next layer's operation, making them better suited to reside in on-chip memory. Since tensor dimensions vary significantly throughout the neural network, tiling across the spatial dimension can be required to meet on-chip memory constraints [47] or data has to be spilled to the next level of the memory hierarchy. As a result, achieving an optimal dataflow implementation requires balancing memory and communication constraints to make sure the allocated compute (number of Processing Elements (PEs) in Figure 12) is not starved for data. On Versal, this dataflow approach can be implemented





both in the programming logic and in the AI Engine processor array.

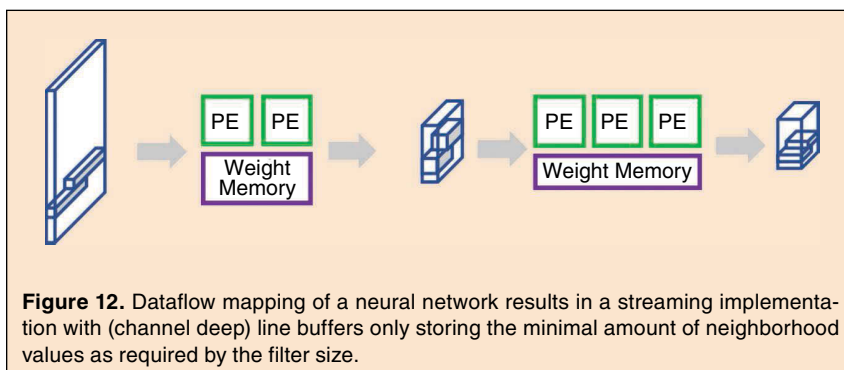
FINN is an open source Xilinx framework [7] for dataflow implementations on programmable logic. By focusing on heavily quantized neural networks, the size of both the weight memories and the line buffers can fit in on-chip memory, avoiding spilling to external memory. The main FINN compute block is the matrix-vector-threshold unit with a variable SIMD width and number of processing elements to support compute balancing [28].

On the AI Engine processor array, the ratio between local memory and compute is fixed. As a result, individual layers in an DL network must typically be partitioned across several AI Engine processors to keep both weights and activations on chip. In order to balance the computation and memory usage across the whole

network, we apply a broadcast-cascade design pattern across layers of the network, as shown in Figure 13.

In this simple example, all activation channels have the same size and all weight kernels are of equal dimensions. A single AI Engine local memory can store 4 activation channels and 4 weight channels, fitting all data of $conv2d_0$: 2 input activation channels, 2 weight sets (light and dark purple) each having 2 channels and 2 output activation channels. The 4 weight sets of $conv2d_1$ need to be distributed over 2 AI Engine processors. The broadcast feature of the AXI stream switch interconnect is used to duplicate the input activations. While $conv2d_2$ only has 2 weight sets, they are 4 channels deep. They are split over the channel dimension and distributed over 2 AI Engine processors that exchange the intermediate conv2d result of 2 channels over a cascade connection.

This dataflow approach can utilize sparsity effectively to reduce memory and compute footprint of the network implementation. It can also improve latency and throughput of the implementation as demonstrated in HPIPE [51]. Given a convolutional neural network, HPIPE generates network-specific dataflow RTL targeting FPGA devices. HPIPE demonstrates $4 \times$ the throughput of V100 GPU while achieving a throughput



of 4550 images/second for sparse Resnet-50 model (85% sparsity) at a batch size of one. A similar approach can be used to implement dataflow with sparse layers using Versal FPGA fabric, arrays of AI Engine processors or combination of both.

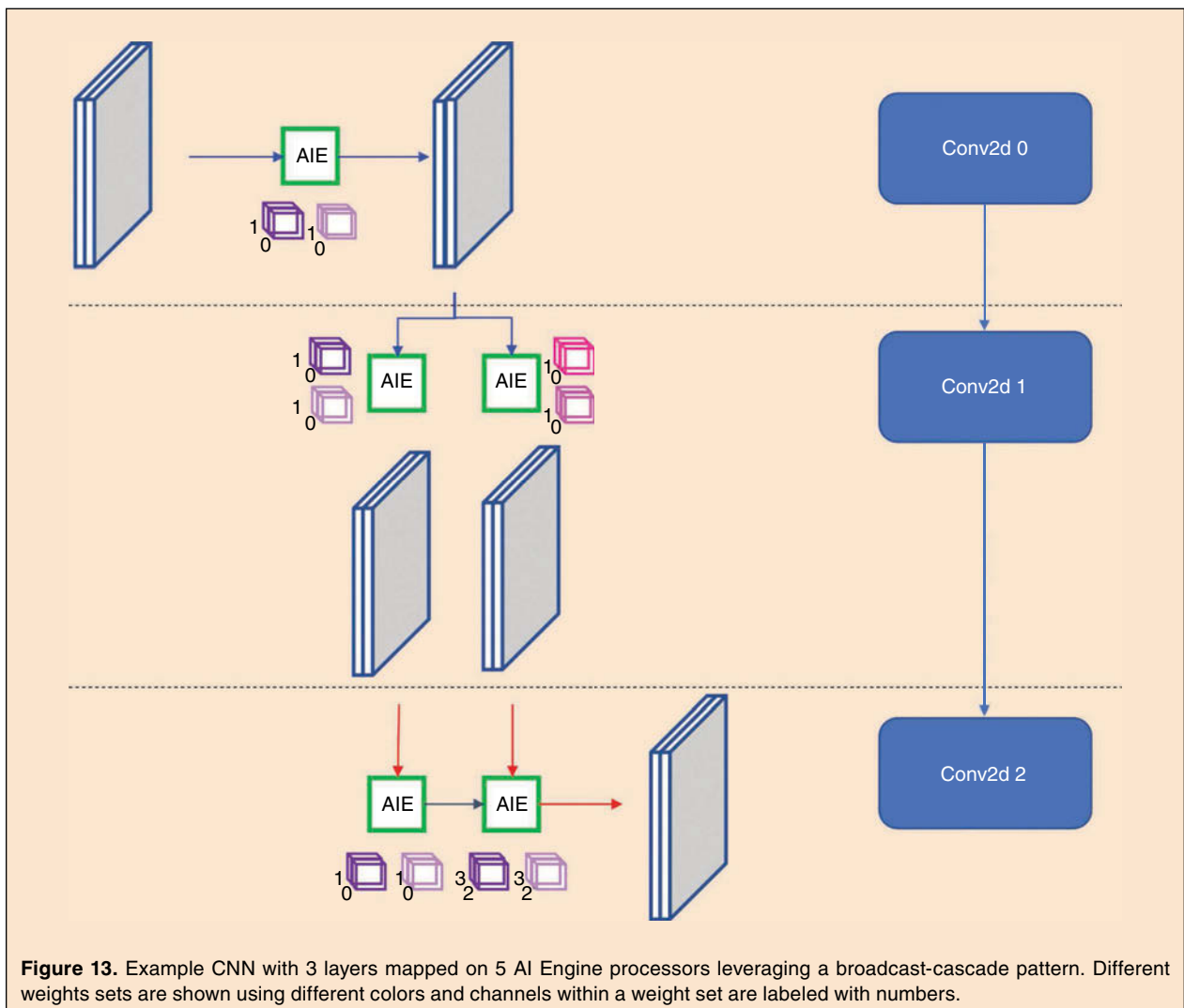
C. Sparse Code Generation

Many DL applications and algorithms are trying to exploit static sparsity by deploying sparse DNN models (pruned network parameters) on emerging DL hardware platforms. In the same way that dense convolutions can be expressed as dense matrix-multiply operations, a straightforward approach to implementing convolutions with sparse weights is to leverage existing libraries for sparse matrix-vector (SpMV) and sparse matrix-matrix (SpMM) multiplication or to generate code specialized for particular combinations of sparse and dense matrix formats [72]. Unfortunately, while dense computations typically exhibit significant data reuse enabling them to

scale easily, sparse computations often have less memory locality and are more likely to be I/O bound.

Another approach is to add hardware support for sparse [42], [54], [92], [117] or partially sparse matrices, as is present in some modern GPUs, such as NVidia's Ampere 100 [119] or Intel Movidius' Myriad [25]. This technique allows the weight matrix to be a sparse matrix that is specified only at runtime, as long as it meets strict structural constraints. In Versal, these techniques are well suited to implementation in programmable logic, leveraging the network-on-chip to provide access to external memory and the ability to specialize memory and compute architectures to the characteristics of a specific problem.

Although they are optimized for dense arithmetic operations, AI Engine processors are also flexible enough to implement some forms of sparse matrix processing. Sparse matrix computation can be efficiently implemented on these processors by generating code specialized



for a given set of pruned weights, as in [106]. This approach is possible for many inference use cases where weights are fixed at design time. Implementing this technique requires partitioning the weights for each layer into a number of partitions, where each partition is mapped onto a separate AI Engine processor. After partitioning, code is generated leveraging the vector permute network to select the correct data for each operation. Partitioning the weights across multiple cores also means that weights can often be stored entirely on-chip, avoiding the need to load weights from external memory during runtime. Larger networks can also be supported by loading new weights from off-chip or on-chip memory.

V. Tooling for Domain-Specific Computing

Appropriate tooling is also critical for the future success of domain-specific computing. Domain-specific embedded languages (DSEs) have enabled significant progress by leveraging existing languages and generic tooling. In deep learning, Python-based frameworks such as Pytorch [88] and TensorFlow [20] are widely used, however mapping these frameworks to execute on high performance hardware is a complex compilation task. These frameworks typically rely on highly optimized low-level libraries, such as cuDNN [35] or MIOpen [71] for GPU acceleration (see [85] for a complete overview on deep learning frameworks and libraries). While many functions in deep learning frameworks have corresponding low-level acceleration libraries, achieving the best performance often requires complex transformations to map a network description into an efficient implementation. For instance, it is common to merge layers in a network into a single low-level library function, in order to improve memory locality and avoid external memory access. It is also desirable to distribute computation of a network between multiple GPUs in a single compute node and across multiple compute nodes in a network. For instance, a *data-parallel* mapping might execute independent inputs on different devices, and a *model-parallel* mapping might execute different layers processing a single input on different devices. Depending on data set sizes, the structure of a particular deep learning model and the limitations of low-level libraries, different trade-offs might be preferred. Exploring these options to find the best solution is a complex problem, best handled by compilation and optimization tools.

In the future, we expect these design trade-offs to only become more complex. One driving factor is the increasing heterogeneity in our target architectures (see Section III). Tooling must typically support a wide variety of hardware resources, including processors, DMAs, interconnect, specialized accelerators, and compiler-managed

memories. While traditional compilers have largely succeeded in separating target-independent and target-specific considerations the scope of these optimizations is somewhat limited. For instance, the target-independent model of LLVM [75] uses a Static-Single Assignment (SSA) form where instructions execute sequentially and memory is homogeneous. Individual targets are responsible for more accurately modeling the details of a specific architecture, such as instruction parallelism or resource constraints, during code generation in order to improve performance. However, this model inherently limits the scope of potential higher-level optimizations. For instance, loop vectorization and transformation is done early in the compilation process on target-independent representations, but must have predictive model of target specific costs in order to be effective. More complex transformations, such as compiling OpenMP programs onto multiple cores require a complex interplay between target-specific information and runtime libraries. We believe that in next-generation domain-specific computing systems, these more complex transformations will become common and our approach is to build the compiler infrastructure to support these transformations.

A second driving factor is the need to support a wide range of domain-specific languages and frameworks. Each of these interfaces must faithfully represent the semantics of a particular framework, enabling the conversion from that framework to a wide variety of targets. The challenge here is to leverage domain-independent transformations whenever possible, otherwise we must reconstruct tools that are specific to each particular domain-specific framework without leveraging the commonality between them. In this case, it is common to transform from a domain-specific abstraction, through a sequence of domain-independent abstractions, before reaching a target-specific abstraction, as in Figure 14. While the abstractions themselves may be domain independent or target independent, it is possible that this sequence of transformations incorporates domain-specific or target-specific information. Fortunately, most computing frameworks fit into a relatively small number of patterns [22], suggesting that building infrastructure to capture these generic abstractions is a reasonable approach.

A number of systems have been built that attempt to address these challenges. Ptolemy II [91] focuses primarily on the heterogeneous composition of different abstractions in a single design. In Ptolemy II, each abstraction is called a *domain*. In order to provide understandable semantics for heterogeneous designs, these systems focus on hierarchical design, where each portion of a design is encapsulated in a well-defined actor-oriented component interface and a domain determines

how actor-oriented components interact. A composition of actors composed in a particular domain also defines an actor-oriented component interface, naturally leading to a hierarchical design methodology. Top-level components in the hierarchy represent significant portions of a design in a relatively abstract and coarse-grained way, while lower-level components typically represent smaller portions of a design in a more detailed fashion. MLIR [76] is a rapidly maturing compiler infrastructure focused on building and transforming abstractions. MLIR allows tools to be built with a wide variety of abstractions, called *dialects*. MLIR dialects can be composed in arbitrary ways, allowing the description of heterogeneous systems. MLIR also provides infrastructure to transform dialects in various ways, enabling compilation flows from domain-specific to target-specific abstractions [29]. The rest of this section provides an overview of how we are leveraging MLIR instruction to build next-generation domain-specific tooling for Xilinx Versal devices.

A. MLIR Programming of AI Engine Devices

We start at the lowest levels of abstraction with a target-specific abstraction for Xilinx Versal devices. Although we focus on device abstractions for the AI Engine portion of the device, there is nothing fundamentally preventing similar representation of the programmable logic portion of the device. The MLIR abstraction uses multiple MLIR operations to represent different modules in a device. For example, an `AIE.core` operation encapsulates the code that runs on a single AI Engine core. An `AIE.buffer` operation represents a buffer stored in tile memory. An `AIE.lock` operation represents a lock that can be accessed from a core or by a DMA. These operations all reference an `AIE.tile` operation, which represents a physical location in the device. Similar operations represent other aspects of the architecture, such as switchboxes and DMAs. A simple MLIR program representing communication between two cores is shown in Figure 15.

Code generation for the MLIR device dialect is relatively straight-

forward, since there is a direct correspondence between the structure of the MLIR code and the structure of the hardware. The code inside each core is lowered to the MLIR LLVM dialect using standard lowerings

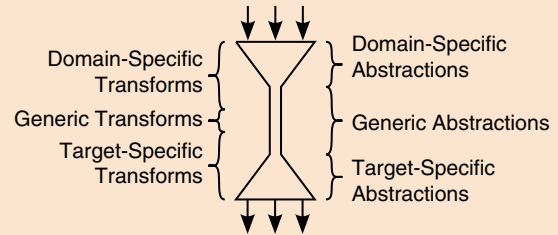


Figure 14. The hourglass model of compilers. Note that while we would prefer to maximize the usage of generic, domain-independent abstractions and transformations, in some cases it is useful to include domain-specific and target-specific transformations on domain-independent abstractions.

```
module @two_cores {
  %tile13 = AIE.tile(1, 3)
  %tile14 = AIE.tile(1, 4)

  %buf13_0 = AIE.buffer(%tile13)
    { sym_name = "a" } : memref<256xi32>
  %buf13_1 = AIE.buffer(%tile13)
    { sym_name = "b" } : memref<256xi32>
  %buf14_0 = AIE.buffer(%tile14)
    { sym_name = "c" } : memref<256xi32>

  %lock13_3 = AIE.lock(%tile13, 3)
  %lock13_5 = AIE.lock(%tile13, 5)
  %lock14_7 = AIE.lock(%tile14, 7)

  %core13 = AIE.core(%tile13) {
    AIE.useLock(%lock13_3, "Acquire", 1, 0)
    AIE.useLock(%lock13_5, "Acquire", 0, 0)
    %idx1 = constant 3 : index
    %val1 = load %buf13_0[%idx1] : memref<256xi32>
    %idx2 = constant 5 : index
    store %val1, %buf13_1[%idx2] : memref<256xi32>
    AIE.useLock(%lock13_3, "Release", 0, 0)
    AIE.useLock(%lock13_5, "Release", 1, 0)
    AIE.end
  }

  %core14 = AIE.core(%tile14) {
    AIE.useLock(%lock13_5, "Acquire", 1, 0)
    AIE.useLock(%lock14_7, "Acquire", 0, 0)
    %idx1 = constant 5 : index
    %val1 = load %buf13_1[%idx1] : memref<256xi32>
    %idx2 = constant 3 : index
    store %val1, %buf14_0[%idx2] : memref<256xi32>
    AIE.useLock(%lock13_5, "Release", 0, 0)
    AIE.useLock(%lock14_7, "Release", 1, 0)
    AIE.end
  }
}
```

Figure 15. Low-level code for an AI Engine design. The code inside each core first acquires locks associated with the buffers that will be accessed, then performs some computation that reads and writes from those buffers, and lastly releases the associated locks. Computation can be represented by any MLIR dialect that can be lowered to LLVM.

and can contain a mix of MLIR operations from different dialects. These operations can represent not only regular sequential code, but also high-level constructs such as affine loops from which vector code can be automatically generated [33]. Low-level device specific operations like `AIE.useLock` are also lowered to architecture-specific function intrinsics to enable backend code-generation.

B. Higher-Level Domain-Independent MLIR Abstractions

While the low-level MLIR for a device shown above can be written by a programmer, it is still somewhat cumbersome to write complex programs. Practical programs must be aware of many device details, such as which cores can access the buffers and locks that exist within each device tile and how locks and buffers must be coordinated to ensure that two cores can execute concurrently. However, with appropriate transformations these communication patterns can be generated from higher-level models. However, even with this level of programming, some aspects of the device are abstracted, such as the particular addresses of each buffer in the memory map of a core. Addressing details are generated automatically during the code generation process.

One possible higher-level model leverages dataflow process networks [39], [68] where computation is described as a concurrent graph of processes communicating by streams of data. Lowering a statically structured graph onto an array of processors is typically viewed as a transformation and optimization problem where *fusion* optimizations manipulate the dataflow graph to generate toplevel dataflow components where each component represents execution on a single processor [48]. In order to generate a balanced system, this is typically done with several goals. Primarily, this needs to be done considering locality of access, often merging dataflow components which share inputs or process data in a pipeline. Secondly, it is also important to balance the amount of processing done on each physical processor. This minimizes the amount of time where one processor is stalled waiting for data from another processor. In more complicated systems with feedback, unbalanced workloads may be preferable to ensure that feedback paths complete quickly. *Fission* transformations may also be important, where large granularity components exist in a dataflow program. Obviously data-parallel computations can be partitioned using pairs of Split/Join processes, while more complex components consisting of nested loops with recurrences require more complex analysis [96]. The Xilinx Vitis tools [19] implement a coarse-grained

graph-based dataflow programming model implemented as a C++ DSEL.

While dataflow programming is well-suited to many embedded processing systems where data is being processed as it is received [84], many other high-level programming models are also used that are more well suited to other kinds of applications. Large, regular computations, such as dense linear algebra, may be captured directly as stencil-based computations. The structure of this description can leverage to facilitate the mapping and implementation of these computations. On the other hand, some computations, such as sparse linear algebra are less regular and may be more amenable to other representations. For instance, the Legion model [26] leverages a dynamically created graph of computation intended for dynamic scheduling and dispatch.

C. Domain-Specific Abstractions in MLIR

At the highest levels of the toolchain, capturing a designer's intent becomes the primary goal. In MLIR-based systems it is common to build an MLIR abstraction which closely approximates the syntax and/or semantics of the language being used to capture a design. For deep learning, two major projects have worked to integrate with MLIR. Firstly, Tensorflow leverages MLIR internally to represent computational graphs [89]. These graphs, represented as an MLIR dialect can be lowered to other dialects corresponding to different execution frameworks, such as XLA or TensorflowLite. A similar structure is being built for Pytorch in the context of the MLIR-npcomp project [3]. In this case, the core `ATen` dialect is modeled after the core tensor computation library in Pytorch.

D. MLIR for Programmable Logic

Although programming domain-specific architectures is a challenging problem, developing domain-specific architectures is also a complex process. As devices have become more complex, it has become more common to integrate application-specific functional blocks containing complex computing algorithms, such as video codecs or processors tuned for deep learning computation into Systems-on-Chip (SOCs). In FPGA-based systems it is common for the bulk of the programmable logic to consist of domain-specific IP, since the domain-independent components, such as processors and memory controllers are implemented in ASIC technology. Ultimately, the cost to develop these domain-specific IP is a significant barrier to deployment of these designs. The use of high-level synthesis (HLS) technology has been adopted in order to improve designer productivity and quickly deliver new designs [36], [69].

We believe that the fundamental concepts of MLIR addressing this hardware design complexity are as applicable to the process of *building* accelerators as they are to the process of *programming* them. Furthermore, unifying the abstractions for accelerator design and accelerator programming is a key step towards enabling programming for arbitrary accelerators. At the simplest level, we expect to take the same high-level abstractions, such as dataflow process networks, and lower them to programmable logic as easily as the abstractions are lowered to an array of processors. This will likely require lowering that is aware of the target architecture in order to achieve the highest performance.

CIRCT [6] is a recent effort to implement this approach, with a focus on implementing hardware-oriented design abstractions in MLIR. The initial focus has been on providing a set of MLIR dialects with eventual lowering into Register-Transfer Level (RTL) designs represented in Verilog and SystemVerilog. One dialect in CIRCT is based on the LLHD framework [97]. LLHD explicitly takes a multi-level approach to capturing behavioral SystemVerilog designs and lowering them into more fundamental constructs which can be efficiently simulated or synthesized. A key aspect of this systems is the observation that useful input languages (such as SystemVerilog) provide a much wider set of constructs than can be easily synthesized, and managing this complexity in a structured way is key to building real systems. Another dialect in CIRCT focuses integration with the Chisel [23] design language through the FIRRTL [64], [78] intermediate representation.

CIRCT also includes basic dialects appropriate for high-level synthesis. In combination, these dialects form a structured IR for HLS, similar to other systems, such as LegUp [31] and Bambu [90]. By leveraging the multi-level capabilities of MLIR, we expect that structuring the lowering of these dialects into RTL should be simpler and more extensible than previous systems.

Dataflow models [39], [68] again form a key higher level abstraction, providing a notion of concurrency which is intrinsically decoupled, unlike synchronous models like RTL. This makes it easy to describe distributed systems with *elastic* or *latency-insensitive* properties, where the latency to communicate data from one process to another is guaranteed to not change the streams of data being computed. CIRCT implements dataflow models in the handshake dialect, along with the ability to extract a fine-grained dataflow model from sequential code represented in MLIR, using techniques described in [66]. The handshake dialect can then be lowered into an RTL representation implementing a basic HLS flow.

A common optimization in HLS tools is to statically schedule computations [116]. Static scheduling can enable smaller, higher performance circuits by reducing control logic and packing multiple operations into the same clock cycle. These scheduling decisions can be represented as a composition of a finite-state machine and the datapath used to implement the computation. Conceptually, in each state of the state machine, certain operations in a datapath are enabled. This abstraction facilitates reasoning about resource sharing and pipelined behavior, while leaving the specifics of how those mechanisms are implemented implicit. Lowering into an RTL-level description typically requires explicitly representing these mechanisms, converting both the control finite-state machine and the datapath into registers and combinational logic. This lowering also requires explicitly representing clocks and clock enables in the circuit in an efficient way. We anticipate modeling a finite-state machine + datapath model in CIRCT after Futil [4], which implements an intermediate representation leveraged by the Dahlia system [86].

VI. Conclusion

The scale of deep learning applications has created a huge opportunity for domain-specific computing. Architectures tuned for DL applications are going to be more efficient than general purpose architectures, enabling reduced cost and higher performance. At the same time, given the diversity of these applications, one device architecture is unlikely to dominate. Rapidly changing requirements as new DL network structures are invented, different precision requirements for different applications, radically different characteristics between training and inference workloads lead to the need for a variety of architectures and application mappings. Heterogeneous architecture can offer an advantage, enabling specialization to varying compute characteristics at different parts of a network. Although the design space for these architectures is huge, we believe that new approaches to tooling and compilers that embrace heterogeneity and multiple levels of abstraction, such as MLIR, will enable domain-specific computing opportunities not only for deep learning, but for other application areas as well.

Acknowledgments

As a survey paper, most of the technology described in this article was developed by other people. We greatly appreciate our colleagues inside and outside of Xilinx who have contributed to our understanding of the concepts expressed in this paper. We also appreciate the reviewers feedback, which helped us to clarify many of our ideas.



Stephen Neuendorffer is a Principal Engineer in the Xilinx Research Labs working on various aspects of system design for FPGAs. Previously, he was product architect of Xilinx Vivado HLS and co-authored a widely used textbook on HLS design for FPGAs. He received B.S. degrees in Electrical Engineering and Computer Science from the University of Maryland, College Park in 1998. He graduated with University Honors, Departmental Honors in Electrical Engineering, and was named the Outstanding Graduate in the Department of Computer Science. He received the Ph.D. degree from the University of California, Berkeley, in 2003, after being one of the key architects of Ptolemy II.



Alireza Khodamoradi is a Research Engineer at Xilinx Research Labs working on optimizing signal processing and deep learning applications for future Xilinx devices. He is also a Ph.D. candidate in the Department of Computer Science and Engineering at the University of California, San Diego. He received a master of advanced studies in wireless embedded systems from UC San Diego, a master of science in telecommunications from the University of Kerman, and a bachelor of science in electrical engineering from Azad University, Najafabad. His main research interests are signal processing and hardware acceleration for vision, RF, and deep learning applications.



Kristof Denolf is a Principal Engineer at Xilinx Research where he is working on video processing and computer vision applications to shape future Xilinx devices. He earned a M.Eng. in electronics from the Katholieke Hogeschool Brugge-Oostende (1998), now part of KULeuven, a M.Sc. in electronic system design from Leeds Beckett University (2000) and a Ph.D. from the Technical University Eindhoven (2007). He has over 20 years of combined research and industry experience at IMEC, Philips, Barco, Apple and Xilinx. His research interests include all aspects of the cost-efficient design of video, vision and graphics systems.



Abhishek Kumar Jain is a Staff Engineer at Xilinx responsible for the design, modelling, simulation/emulation and evaluation of domain-specific compute and interconnect architectures. After earning a Ph.D. degree in Comput-

er Engineering from NTU Singapore in 2016, he explored near memory accelerators and system-level emulation techniques while working as a Postdoc at Lawrence Livermore National Lab. For his Ph.D. dissertation, he developed techniques for building high-performance on-chip interconnect and processor array overlays for dataflow acceleration on FPGAs. He earned a B.Tech. degree in Electronics and Communication Engineering from Indian Institute of Information Technology, Allahabad, India in 2012. His research interests include reconfigurable systems, on-chip networks, domain-specific accelerators, sparse neural networks, and system-level interconnect architectures for FPGA/SoC platforms.



Samuel Bayliss is a Principal Engineer in the Xilinx Research labs in San Jose. In his research, he develops compilers, runtime systems and hardware infrastructure for efficient and accessible compute using heterogeneous architectures. He is a graduate of Imperial College London, where he earned a PhD in Electrical Engineering in 2013 for his research using Polyhedral compilation methods to optimize memory systems. This followed the earlier award in 2006, from the same university, of an MEng in Information Systems Engineering.

References

- [1] www.xilinx.com/products/boards-and-kits/ek-ul-zcu102-g.html
- [2] https://en.wikichip.org/wiki/x86/avx512_vnni (accessed Feb 5, 2021).
- [3] <https://github.com/llvm/mlir-npcomp>
- [4] <https://github.com/cucapra/futil>
- [5] "Brevitas: a Pytorch library for quantization-aware training." github.com/Xilinx/brevitas
- [6] "The CIRCT project." <https://github.com/llvm/circt>
- [7] "FINN: an experimental framework to explore deep neural network inference on FPGAs." xilinx.github.io/finn/
- [8] "Google cloud TPU." Google. <https://cloud.google.com/tpu>
- [9] "CUDA C programming guide 10.2." NVIDIA. <http://docs.nvidia.com/cuda/cuda-c-programmingguide/index.html>
- [10] "Quantization aware training in TensorFlow." tensorflow.org/model_optimization/guide/quantization/training
- [11] "Picoblaze 8-bit embedded microcontroller user guide. User Guide UG129." Xilinx, June 2011. https://www.xilinx.com/support/documentation/ip_documentation/ug129.pdf
- [12] "Embedded vision with INT8 optimization on Xilinx devices. White Paper WP490 (v1.0.1)," Xilinx, April 2017. https://www.xilinx.com/support/documentation/white_papers/wp490-embedded-vision-int8.pdf
- [13] "QPyTorch: A low-precision arithmetic simulation framework," 2019, arXiv:1910.04540.
- [14] "Versal ACAP DSP engine. Architecture manual AM004 (v1.1.1)," Xilinx, Dec. 2020. <https://www.xilinx.com/support/documentation/architecture-manuals/am004-versal-dsp-engine.pdf>
- [15] "Versal: The first adaptive compute acceleration platform (ACAP), WP505 (v1.1.1)," Xilinx, White Paper, Sept. 2020. [Online]. Available: https://www.xilinx.com/support/documentation/white_papers/wp505-versal-acap.pdf
- [16] "Xilinx AI engines and their applications, WP506 (v1.1)," Xilinx, White Paper, July 2020. https://www.xilinx.com/support/documentation/white_papers/wp506-ai-engine.pdf
- [17] "YouTube at 15: My personal journey and the road ahead," Feb. 2020. <https://blog.youtube/news-and-events/youtube-at-15-my-personal-journey> (accessed Feb 5, 2021).

- [18] "Zynq DPU v3.2. Product Guide PG338 (v3.2)," Xilinx, July 2020. https://www.xilinx.com/support/documentation/ip_documentation/dpu/v3_2/pg338-dpu.pdf
- [19] "Versal ACAP AI engine programming environment. User Guide UG1076," Xilinx, Jan. 2021. https://www.xilinx.com/support/documentation/sw_manuals/xilinx2020_2/ug1076-ai-engine-environment.pdf
- [20] M. Abadi et al., "Tensorflow: A system for large-scale machine learning," in *Proc. USENIX Symp. Oper. Syst. Design Implementation (OSDI)*, 2016, pp. 265–283.
- [21] S. Albanie, "Convenet-burden," <https://github.com/github/open-source-survey>
- [22] K. Asanovic et al., "The landscape of parallel computing research: A view from Berkeley," Department of Electrical Engineering and Computer Sciences, Univ. of California Berkeley, Tech. Memorandum UCB/EECS-2006-183, 2006.
- [23] J. Bachrach et al., "Chisel: constructing hardware in a Scala embedded language," in *Proc. Design Autom. Conf. (DAC)*, 2012.
- [24] R. Banner, I. Hubara, and E. Hoffer, "Daniel Soudry. Scalable methods for 8-bit training of neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, Curran Associates, Inc, 2018, vol. 31.
- [25] B. Barry et al., "Always-on vision processing unit for mobile applications," *IEEE Micro*, vol. 35, no. 2, pp. 56–66, 2015. doi: 10.1109/MM.2015.10.
- [26] M. Bauer, S. Treichler, E. Slaughter, and A. Aiken, "Legion: Expressing locality and independence with logical regions," in *Proc. Int. Conf. High Performance Comput., Netw., Storage and Analysis*, 2012, pp. 1–11.
- [27] N. Bell, S. Dalton, and L. N. Olson, "Exposing fine-grained parallelism in algebraic multigrid methods," *SIAM J. Sci. Comput.*, vol. 34, no. 4, pp. C123–C152, 2012. doi: 10.1137/110838844.
- [28] M. Blott et al., "FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 11, no. 3, Dec. 2018. doi: 10.1145/3242897.
- [29] U. Bondhugula, "High performance code generation in MLIR: An early case study with GEMM," 2020, arXiv:2003.00532.
- [30] H. Cai, L. Zhu, and S. Han, "ProxylessNAS: Direct neural architecture search on target task and hardware," 2018, arXiv:1812.00332.
- [31] A. Canis et al., "From software to accelerators with LegUp high-level synthesis," in *Proc. Int. Conf. Compilers, Architecture Synthesis Embedded Systems (CASES)*, 2013, pp. 1–9.
- [32] S. Cao et al., "Efficient and effective sparse LSTM on FPGA with bank-balanced sparsity," in *Proc. Int. Symp. Field Programmable Gate Arrays (FPGA)*, 2019, pp. 63–72. doi: 10.1145/3289602.3293898.
- [33] P. Chatarasi, S. Neuendorffer, S. Bayliss, K. Vissers, and V. Sarkar, "Vyasa: A high-performance vectorizing compiler for tensor convolutions on the Xilinx AI Engine," in *Proc. High Performance Extreme Comput. Conf. (HPEC)*, 2020, pp. 1–10.
- [34] H. Y. Cheah, F. Brossier, S. A. Fahmy, and D. L. Maskell, "The iDEA DSP block-based soft processor for FPGAs," *ACM Trans. Reconfigurable Technol. Syst. (TRETS)*, vol. 7, no. 3, pp. 19:1–19:23, 2014. doi: 10.1145/2629443.
- [35] S. Chetlur, C. Woolley, P. Vanderschuer, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cuDNN: Efficient primitives for deep learning," 2014, arXiv:1410.0759.
- [36] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, "High-level synthesis for FPGAs: From prototyping to deployment," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 30, no. 4, pp. 473–491, 2011.
- [37] Y. Le Cun, J. S. Denker, and S. A. Solla, *Optimal Brain Damage*. San Francisco: Morgan Kaufmann Publishers, 1990, pp. 598–605.
- [38] S. Dalton, L. Olson, and N. Bell, "Optimizing sparse matrix–matrix multiplication for the GPU," *ACM Trans. Math. Softw.*, vol. 41, no. 4, Oct. 2015. doi: 10.1145/2699470.
- [39] J. B. Dennis, "First version of a dataflow procedure language," in *Proc. Programming Symp., Colloque sur la Programmation*, Springer-Verlag, Apr. 1974, pp. 362–376.
- [40] E. Elsen, M. Dukhan, T. Gale, and K. Simonyan, "Fast sparse convnets," 2019, arXiv:1911.09723.
- [41] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *J. Mach. Learn. Res.*, vol. 20, no. 55, pp. 1–21, 2019.
- [42] J. Fowers, K. Ovtcharov, K. Strauss, E. S. Chung, and G. Stitt, "A high memory bandwidth fpga accelerator for sparse matrix-vector multiplication," in *Proc. Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM)*, 2014, pp. 36–43.
- [43] J. Frankle, G. Dziugaite, D. Roy, and M. Carbin, "Pruning neural networks at initialization: Why are we missing the mark?" 2020, arXiv:2009.08576.
- [44] J. Frankle, G. K. Dziugaite, D. Roy, and M. Carbin, "Linear mode connectivity and the lottery ticket hypothesis," in *Proc. Int. Conf. Mach. Learn.*, July 2020, vol. 119, pp. 3259–3269.
- [45] B. Gaide, D. Gaitonde, C. Ravishankar, and T. Bauer, "Xilinx adaptive compute acceleration platform: Versal™ architecture," in *Proc. Int. Symp. Field Programmable Gate Arrays (FPGA)*, 2019, pp. 84–93.
- [46] A. Ganguly, R. Muralidhar, and V. Singh, "Towards energy efficient non-von Neumann architectures for deep learning," in *Proc. 20th Int. Symp. Quality Electronic Design (ISQED)*, 2019, pp. 335–342.
- [47] K. Goetschalckx and M. Verhelst, "Breaking high-resolution CNN bandwidth barriers with enhanced depth-first execution," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 323–331, 2019. doi: 10.1109/JETCAS.2019.2905361.
- [48] M. I. Gordon et al., "A stream compiler for communication-exposed architectures," in *Proc. Architectural Support Programming Languages Oper. Syst. (ASPLOS)*, 2002, pp. 291–303.
- [49] J. Gray, "GRVI Phalanx: A massively parallel RISC-V FPGA accelerator accelerator," in *Proc. Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM)*, 2016, pp. 17–20.
- [50] Y. Guo, "A survey on methods and theories of quantized neural networks," 2018, arXiv:1808.04752.
- [51] M. Hall and V. Betz, "HPIPE: Heterogeneous layer-pipelined and sparse-aware CNN inference for FPGAs," 2020, arXiv:2007.10451.
- [52] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in *Proc. Adv. Neural Inform. Process. Syst.*, vol. 28, pp. 1135–1143, 2015.
- [53] B. Hassibi, D. G. Stork, G. Wolff, and T. Watanabe, "Optimal brain surgeon: Extensions and performance comparisons," in *Proc. Int. Conf. Neural Inform. Process. Syst.*, Morgan Kaufmann Publishers, 1993, pp. 263–270.
- [54] X. He et al., "Sparse-TPU: Adapting systolic arrays for sparse matrices," in *Proc. Int. Conf. Supercomput.*, ACM, 2020, pp. 1–12.
- [55] X. He, K. Zhao, and X. Chu, "AutoML: A survey of the state-of-the-art," *Knowl.-Based Syst.*, vol. 212, p. 106,622, 2021. doi: 10.1016/j.knsys.2020.106622.
- [56] Y. He, X. Zhang, and J. Sun, "Channel pruning for accelerating very deep neural networks," in *Proc. Int. Conf. Comput. Vision (ICCV)*, 2017, pp. 1398–1406.
- [57] K. Hegde et al., "Extensor: An accelerator for sparse tensor algebra," in *Proc. Int. Symp. Microarchitecture (MICRO)*, 2019, pp. 319–333.
- [58] L. Heng, "A scalable unified architecture for neural network computing from nano-level to high performance computing," in *Proc. HotChips*, 2019. [Online]. Available: https://old.hotchips.org/hc31/HC31_1.11_Huawei.Davinci.HengLiao_v4.0.pdf
- [59] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, arXiv:1503.02531.
- [60] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *Proc. Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2014, pp. 10–14.
- [61] A. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, arXiv:1906.07214.
- [62] W. Hua, Y. Zhou, C. D. Sa, Z. Zhang, and G. Edward Suh, "Channel gating neural networks," 2018, arXiv:1805.12549.
- [63] Q. Huang, K. Zhou, S. You, and U. Neumann, "Learning to prune filters in convolutional neural networks," in *Proc. Winter Conf. Appl. Comput. Vision (WACV)*, 2018, pp. 709–718.
- [64] A. Izraelevitz et al., "Reusability is FIRRTL ground: Hardware construction languages, compiler frameworks, and transformations," in *Proc. Int. Conf. Comput. Aided Design (ICCAD)*, Nov. 2017, pp. 209–216.
- [65] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. Conf. Comput. Vision Pattern Recognit. (CVPR)*, June 2018.
- [66] L. Josipović, R. Ghosal, and P. lenne, "Dynamically scheduled high-level synthesis," in *Proc. Int. Symp. Field Programmable Gate Arrays (FPGA)*, 2018, pp. 127–136.
- [67] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," *SIGARCH Comput. Archit. News*, vol. 45, no. 2, pp. 1–12, June 2017. doi: 10.1145/3140659.3080246.
- [68] G. Kahn, "The semantics of a simple language for parallel programming," in *Proc. IFIP Congr.*, North-Holland, 1974, pp. 471–475.
- [69] R. Kastner, J. Matai, and S. Neuendorffer, "Parallel programming for FPGAs," 2018, arXiv:1805.03648.

- [70] J. Kepner et al., "Sparse deep neural network graph challenge," in *Proc. HPEC*, 2019, pp. 1–7.
- [71] J. Khan et al., "MIOpen: An open source library for deep learning primitives," 2019, arXiv:1910.00078.
- [72] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. Amarasinghe, "The tensor algebra compiler," *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, pp. 77:1–77:29, Oct. 2017. doi: 10.1145/3133901.
- [73] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inform. Process. Syst.*, Curran Associates, vol. 25, pp. 1097–1105, 2012.
- [74] I. Kuon and J. Rose, "Measuring the gap between FPGAs and ASICs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst. (TCAD)*, vol. 26, no. 2, pp. 203–215, 2007. doi: 10.1109/TCAD.2006.884574.
- [75] C. Lattner and V. Adve, "LLVM: A compilation framework for life-long program analysis & transformation," in *Proc. Int. Symp. Code Generation Optimization (CGO)*, 2004, p. 75.
- [76] C. Lattner et al., "MLIR: A compiler infrastructure for the end of Moore's law," 2020, arXiv:2002.11054.
- [77] N. Lee, T. Ajanthan, and P. Torr, "Snip: Single-shot network pruning based on connection sensitivity," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2019.
- [78] P. Li et al., "Specification for the FIRRTL language," Department of Electrical Engineering and Computer Sciences, Univ. of California Berkeley, Tech. Memorandum UCB/EECS-2016-9, 2016.
- [79] S. Lie, "Wafer scale deep learning," in *Proc. HotChips*, 2019. [Online]. Available: https://old.hotchips.org/hc31/HC31_1.13_Cerebras.SeanLie.v02.pdf
- [80] S. Liu and W. Deng, "Very deep convolutional neural network based image classification using small training sample size," in *Proc. Asian Conf. Pattern Recognit. (ACPR)*, 2015, pp. 730–734.
- [81] W. Liu and B. Vinter, "A framework for general sparse matrix-matrix multiplication on GPUs and heterogeneous processors," *J. Parallel Distrib. Comput.*, vol. 85, pp. 47–61, 2015. doi: 10.1016/j.jpdc.2015.06.010.
- [82] P. Mattson et al., "MLPerf training benchmark," 2020, arXiv:1910.01500.
- [83] S.-I. Mirzadeh, M. Farajtabar, A. Li, N. Levine, A. Matsukawa, and H. Ghasemzadeh, "Improved knowledge distillation via teacher assistant," 2019, arXiv:1902.03393.
- [84] S. Neuendorffer and K. Visser, "Streaming systems in FPGAs," in *Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS)*, Springer Berlin Heidelberg, 2008, pp. 147–156.
- [85] G. Nguyen et al., "Machine learning and deep learning frameworks and libraries for large-scale data mining: A survey," *Artif. Intell. Rev.*, vol. 52, no. 1, pp. 77–124, 2019. doi: 10.1007/s10462-018-09679-z.
- [86] R. Nigam et al., "Predictable accelerator design with time-sensitive affine types," in *Proc. Conf. Programming Language Design Implementation (PLDI)*, 2020.
- [87] T. Norrie and N. Patil, "Google's training chips revealed: TPUv2 and TPUv3," in *Proc. HotChips*, 2020. [Online]. Available: https://hotchips.org/assets/program/conference/day2/HotChips2020_ML_Training_Google_Norrie_Patil.v01.pdf.
- [88] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inform. Process. Syst.*, Curran Associates, 2019, vol. 32, pp. 8026–8037.
- [89] J. Pienaar, "MLIR in TensorFlow ecosystem," in *Proc. Compilers Mach. Learn. (CAML)*, San Diego, 2020.
- [90] C. Pilato and F. Ferrandi, "Bambu: A modular framework for the high level synthesis of memory-intensive applications," in *Proc. Int. Field Programmable Logic Appl. Conf. (FPL)*, 2013, pp. 1–4.
- [91] C. Ptolemaeus, Ed., *System Design, Modeling, and Simulation using Ptolemy II*. Ptolemy.org, 2014.
- [92] E. Qin et al., "Sigma: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training," in *Proc. Int. Symp. High Performance Comput. Architecture (HPCA)*, 2020, pp. 58–70.
- [93] H. Qin, R. Gong, X. Liu, X. Bai, J. Song, and N. Sebe, "Binary neural networks: A survey," *Pattern Recognit.*, vol. 105, p. 107281, 2020. doi: 10.1016/j.patcog.2020.107281.
- [94] V. J. Reddi et al., "MLPerf inference benchmark," 2020, arXiv:1911.02549.
- [95] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, June 2017. doi: 10.1109/TPAMI.2016.2577031.
- [96] E. Rijpkema, E. Deprettere, and B. Kienhuis, "Deriving process networks from nested loop algorithms," *Parallel Process. Lett.*, vol. 10, no. 02n03, pp. 165–176, June 2000. doi: 10.1142/S0129626400000172.
- [97] F. Schuiki, A. Kurth, T. Grosser, and L. Benini, "LLHD: A multi-level intermediate representation for hardware description languages," 2020, arXiv:2004.03494.
- [98] A. Severance, J. Edwards, H. Omidian, and G. Lemieux, "Soft vector processors with streaming pipelines," in *Proc. Int. Symp. Field Programmable Gate Arrays (FPGA)*, 2014, pp. 117–126.
- [99] S. V. Kalluru Srinivas, H. Nair, and V. Vidyasagar, "Hardware aware neural network architectures using FbNet," 2019, arXiv:1906.07214.
- [100] N. Srivastava, H. Jin, S. Smith, H. Rong, D. Albonese, and Z. Zhang, "Tensaurus: A versatile accelerator for mixed sparse-dense tensor computations," in *Proc. Int. Symp. High Performance Comput. Architecture (HPCA)*, 2020, pp. 689–702.
- [101] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proc. Association Comput. Linguistics (ACL)*, July 2019, pp. 3645–3650.
- [102] I. Swarbrick, D. Gaitonde, S. Ahmad, B. Gaide, and Y. Arbel, "Network-on-chip programmable platform in VersalTM ACAP architecture," in *Proc. Int. Symp. Field Programmable Gate Arrays (FPGA)*, 2019, pp. 212–221.
- [103] M. Tan et al., "MnasNet: Platform-aware neural architecture search for mobile," in *Proc. IEEE/CVF Conf. Comput. Vision Pattern Recognit. (CVPR)*, June 2019.
- [104] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, June 2019, vol. 97, pp. 6105–6114.
- [105] H. Tanaka, D. Kunin, D. Yamins, and S. Ganguli, "Pruning neural networks without any data by iteratively conserving synaptic flow," in *Proc. 34th Conf. Neural Inform. Process. Syst.*, 2020.
- [106] X. Tang, T. Schneider, S. Kamil, A. Panda, J. Li, and D. Panozzo, "EGGS: Sparsity-specific code generation," *Comput. Graph. Forum*, vol. 39, no. 5, pp. 209–219, Aug. 2020. doi: 10.1111/cgf.14080.
- [107] N. Vassilieva, "Characterization and benchmarking of deep learning," hpcuserforum.com/presentations/Wisconsin2017/HPDLCookbook4HPCUserForum.pdf
- [108] S. I. Venieris, A. Kouris, and C.-S. Bouganis, "Toolflows for mapping convolutional neural networks on FPGAs: A survey and future directions," vol. 51, no. 3, 2018. doi: 10.1145/3186332.
- [109] J. Wang, L. Gou, W. Zhang, H. Yang, and H. Shen, "DeepVID: Deep visual interpretation and diagnosis for image classifiers via knowledge distillation," *IEEE Trans. Vis. Comput. Graph.*, vol. 25, no. 6, pp. 2168–2180, 2019. doi: 10.1109/TVCG.2019.2903943.
- [110] J. Yu, C. Eagleston, C. H.-Y. Chou, M. Perreault, and G. Lemieux, "Vector processing as a soft processor accelerator," *ACM Trans. Reconfigurable Technol. Syst. (TRET)*, vol. 2, no. 2, June 2009. doi: 10.1145/1534916.1534922.
- [111] J. Yu, L. Yang, N. Xu, and T. H. Jianchao Yang, "Slimmable neural networks," 2018, arXiv:1812.08928.
- [112] O. Zachariadis, N. Satpute, J. Gómez-Luna, and J. Olivares, "Accelerating sparse matrix-matrix multiplication with GPU tensor cores," 2020, arXiv:2009.14600.
- [113] L. L. Zhang, Y. Yang, Y. Jiang, W. Zhu, and Y. Liu, "Fast hardware-aware neural architecture search," 2019, arXiv:1910.11609.
- [114] X. Zhang, X. Zhou, M. Lin, and J. Sun, "Shufflenet: An extremely efficient convolutional neural network for mobile devices," in *Proc. Conf. Comput. Vision Pattern Recognit. (CVPR)*, 2018, pp. 6848–6856.
- [115] Y. Zhang, R. Zhao, W. Hua, N. Xu, G. Edward Suh, and Z. Zhang, "Precision gating: Improving neural network efficiency with dynamic dual-precision activations," 2020, arXiv:2002.07136.
- [116] Z. Zhang and B. Liu, "SDC-based modulo scheduling for pipeline synthesis," in *Proc. Int. Conf. Comput. Aided Design (ICCAD)*, 2013, pp. 211–218.
- [117] Z. Zhang, H. Wang, S. Han, and W. J. Dally, "SpArch: Efficient architecture for sparse matrix multiplication," 2020, arXiv:2002.08947.
- [118] R. Zhao, Y. Hu, J. Dotzel, C. D. Sa, and Z. Zhang, "Building efficient deep neural networks with unitary group convolutions," in *Proc. Conf. Comput. Vision Pattern Recognit. (CVPR)*, 2019, pp. 11,295–11,304.
- [119] A. Zhou et al., "Learning N:M fine-grained structured sparse neural networks from scratch," 2021, arXiv:2102.04010.
- [120] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, "Dorefa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients," 2016, arXiv:1606.06160.
- [121] C. Zhu, K. Huang, S. Yang, Z. Zhum, H. Zhang, and H. Shen, "An efficient hardware accelerator for structured sparse convolutional neural networks on FPGAs," 2020, arXiv:2001.01955.

Synthesizing General-Purpose Code Into Dynamically Scheduled Circuits

Lana Josipović, Andrea Guerrieri, and Paolo Ienne, *École Polytechnique Fédérale de Lausanne (EPFL), School of Computer and Communication Sciences, CH-1015 Lausanne, Switzerland.*
lana.josipovic@epfl.ch, andrea.guerrieri@epfl.ch, paolo.ienne@epfl.ch

Abstract

High-level synthesis (HLS) tools generate hardware designs from high-level programming languages and should liberate designers from the details of hardware description languages like VHDL and Verilog. HLS tools typically build datapaths that are controlled using a centralized controller, which relies on a compile-time schedule to determine the clock cycle when each operation executes. Such an approach results in high-throughput pipelined designs only in cases where memory accesses are provably independent and critical control decisions are determinable during code compilation. Unfortunately, when this is not the case, current tools must make pessimistic assumptions, yielding inferior schedules and lower performance. Recent advances in HLS have explored methods to overcome the conservatism in static scheduling and to remove the inability of HLS tools to handle dynamic events. Dataflow circuits play a significant role in this context: they are built out of units that communicate using point-to-point pairs of handshake control signals and this distributed control mechanism effectively implements a dynamic schedule, adapted at runtime to particular memory and control outcomes. Dataflow circuits can exploit the same optimization opportunities as standard HLS circuits (i.e., pipelining and resource sharing), but also introduce



©SHUTTERSTOCK.COM/CQ PHOTO JUJY

to HLS features similar to those of modern superscalar processors (i.e., out-of-order memory accesses and speculative execution), which are key for HLS to be successful in new contexts and broader application domains.

I. Introduction

Field Programmable Gate Arrays (FPGAs) are extremely versatile devices that can be configured into application-specific accelerators achieving high performance and energy efficiency. Recently,

Digital Object Identifier 10.1109/MCAS.2021.3071631

Date of current version: 24 May 2021

FPGAs have been integrated into datacenters [2], [9], [56], packaged with processors [13], and introduced to new application domains. However, their success on the global market critically depends on the ability of software application developers to build efficient FPGA designs [30], [31], [51] by extracting sufficient performance through modern programming paradigms. *High-level synthesis* (HLS) tools enable programmers to automatically generate hardware designs from high-level software abstractions instead of writing tedious and time-consuming low-level hardware descriptions. Despite their progress and some commercial success in the last decade, HLS tools still tend to be criticized for the difficulty of extracting the desired level of performance: generating good circuits from high-level languages still requires peculiar code restructuring, expert user interaction, and extensive experimentation with the tools [39]. Moreover, current HLS techniques face a fundamental issue when handling irregular applications: because they rely on static scheduling, i.e., the cycle in which each operation executes is fixed at compile time [26], they force worst-case assumptions on memory and control dependences. Therefore, HLS tools are primarily usable by designers with hardware expertise and acceptable only for some classes of applications, but they are still unable to meet the need to accelerate emerging applications such as graph processing, data analytics, and sparse linear algebra operations.

In this article, we provide an overview of standard HLS techniques and describe some classic HLS optimizations and their tradeoffs. We then detail a fundamentally different form of HLS which produces dynamically scheduled, dataflow circuits out of high-level code. These circuits use a distributed control system that makes local scheduling decisions dynamically during runtime; data is propagated from unit to unit as soon as memory and control dependences allow it and, otherwise, it is stalled until all conditions for execution are satisfied. However, a straightforward translation of high-level code into a dataflow circuit is not sufficient to obtain circuits that are truly competitive and useful in the HLS context. We detail the methodologies to exploit the same optimization opportunities that standard HLS relies on (i.e., pipelining and resource sharing). We then discuss methods to achieve characteristics that are beyond what classic HLS can do (i.e., out-of-order memory accesses and speculation); these features introduce to dataflow circuits characteristics similar to those of modern superscalar processors. The resulting circuits achieve solutions that are, in particular cases, superior to those obtained using standard HLS techniques: similarly to the tradeoff between VLIW processors and superscalars, the perfor-

mance of demanding applications is very significantly improved at an affordable cost.

II. How Does Classic HLS Work?

In this section, we describe typical high-level synthesis features and optimizations. We discuss standard scheduling techniques and illustrate the limitations they face in particular applications.

A. High-Level Synthesis

Hardware description languages (HDLs), such as VHDL and Verilog, have been used in the electronic design industry for decades to specify the details of hardware design in terms of low-level building blocks such as gates, registers, and multiplexers [3]. However, this description level requires hardware expertise and, typically, a longer time to develop the design. High-level synthesis tools allow designers to work at a higher level of abstraction by using a software language to specify the hardware functionality; this approach enables software engineers to program hardware and helps hardware engineers to speed up the design process as well as to efficiently explore the design space [51]. Although HLS can benefit both ASIC and FPGA designers, HLS tools are particularly gaining popularity in the FPGA domain, where the programming challenges of these devices are one of the biggest barriers to their mainstream adoption [3], [15].

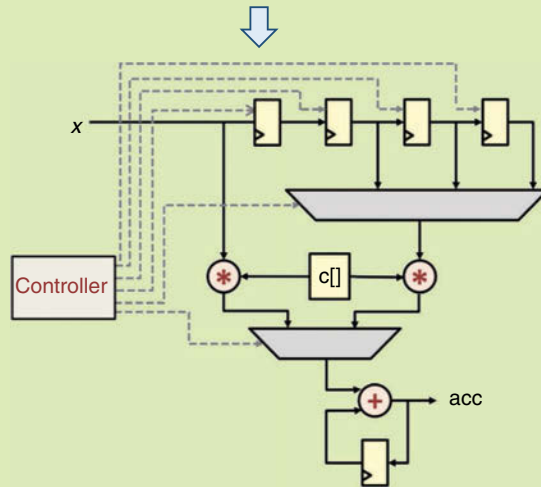
Different HLS tools rely on different high-level representations to describe the underlying hardware; the most popular ones use C/C++ as an input language [7], [69]. Generally speaking, the user provides the input functional specification and particular design constraints such as target device, desired clock frequency, and memory interface description; the tool then automatically analyzes concurrency, inserts registers to achieve the desired frequency, generates the control and datapath logic, and maps data onto storage elements to optimize the bandwidth and resource usage [45]. The user is typically required to restructure the code and annotate it with pragmas to guide the tool in reaching the desired design point.

Figure 1 illustrates several out of many possibilities to specify the functionality of a simple FIR filter in C code as well as the resulting circuits produced by an HLS tool [45]; apart from the different datapaths, as shown in the figure, each design has a kernel-specific controller which triggers the datapath components at appropriate clock cycles; we will discuss this functionality in the following section. The first circuit is obtained from a typical software representation, without any hardware-specific annotations or code restructuring. The second code is manually unrolled to explicitly express available

```

acc = 0;
for (i = 3; i >= 0; i--) {
    if (i == 0) {
        shift_reg[0] = x;
        acc += x * c[0];
    } else {
        shift_reg[i] = shift_reg[i-1];
        acc += shift_reg[i] * c[i];
    }
}
y = acc;

```



(a)

```

shift_reg[3] = shift_reg[2];
shift_reg[2] = shift_reg[1];
shift_reg[1] = shift_reg[0];
shift_reg[0] = x;

```

```

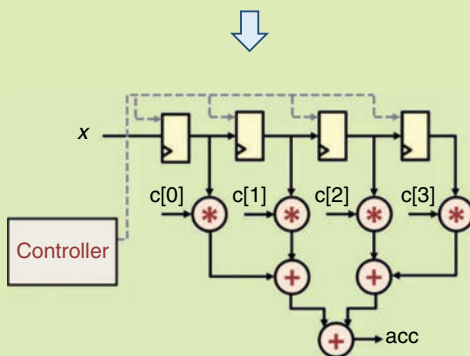
acc = shift_reg[3] * c[3];
acc += shift_reg[2] * c[2];
acc += shift_reg[1] * c[1];
acc += shift_reg[0] * c[0];

```

```

y = acc;

```



(b)

```

shift_reg[3] = shift_reg[2];
shift_reg[2] = shift_reg[1];
shift_reg[1] = shift_reg[0];
shift_reg[0] = x;

```

```

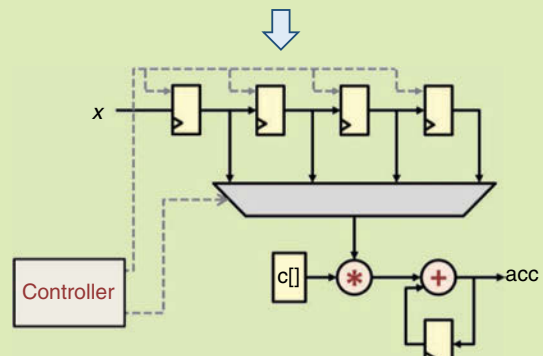
acc = 0;
for (i = 10; i >= 0; i--) {
    #pragma HLS pipeline
    acc += shift_reg[i] * c[i];
}

```

```

y = acc;

```



(c)

Figure 1. Design space exploration with static HLS [45]. All three codes in the figure describe the same functionality (i.e., an FIR filter); yet, the resulting HLS solutions differ in area and performance. (a) No optimization, (b) Unrolling, (c) Pipelining.

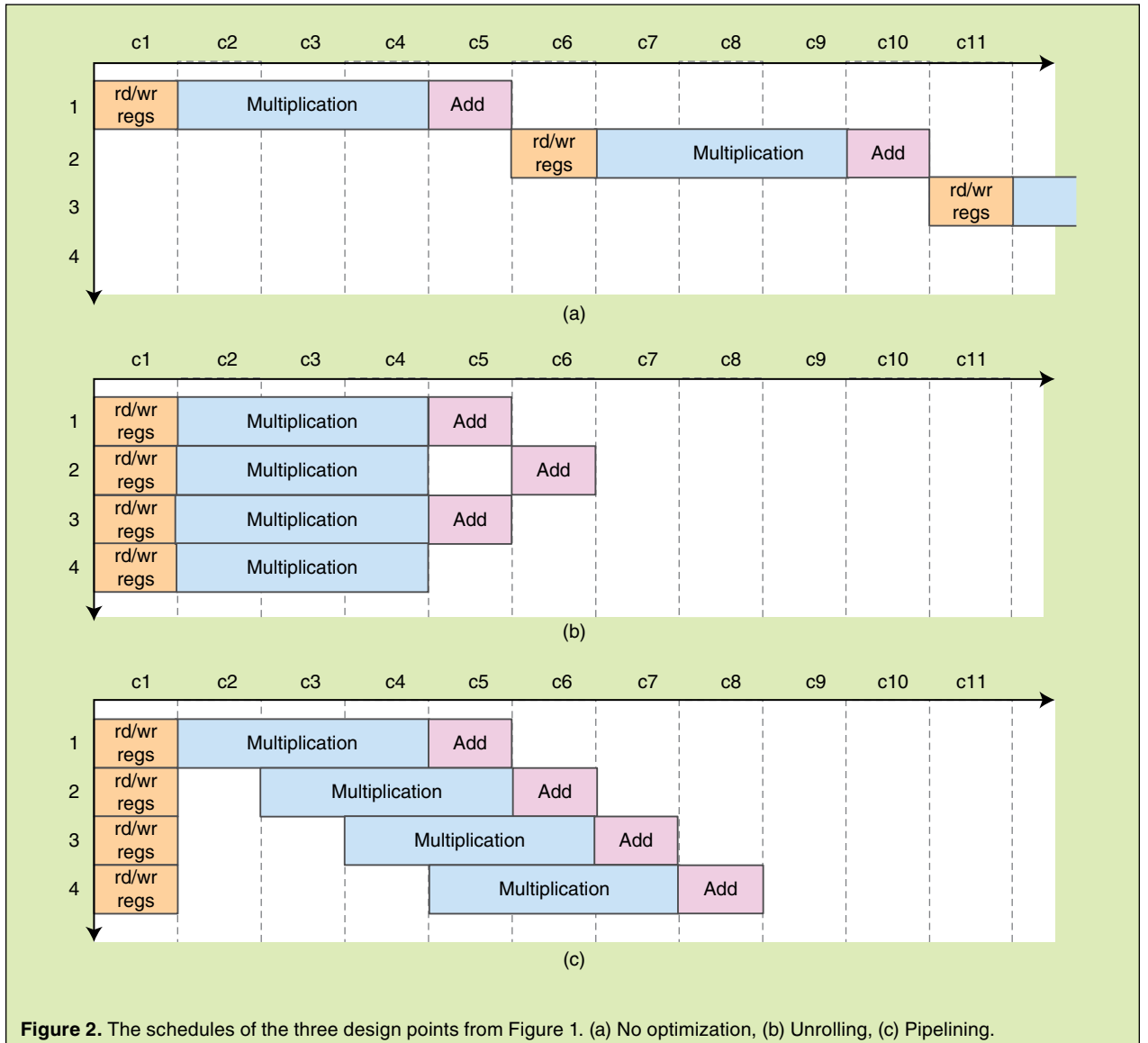
parallelism to the HLS tool—as the circuit below suggests, this design will employ multiple operators which can be used concurrently. The third design point uses a pragma to indicate to the tool that the code should be pipelined, i.e., the loop iterations should overlap for performance benefits. It is evident from the figure that the circuits differ in the number of employed resources (i.e., adders, multipliers, multiplexers); they also differ in performance, as we will discuss next.

B. Scheduling in HLS

HLS relies on a series of compiler optimizations to achieve performance- and area-efficient designs; some techniques are exploited by compilers in general (i.e., code motion, if conversion), whereas others are hardware-specific (i.e., bitwidth analysis, operation chaining). One of the

key algorithms in HLS synthesis is scheduling, i.e., deciding the clock cycle in which each operation will execute. It is typically achieved through *system of difference constraints* (SDC) modeling which incorporates a variety of constraints, such as resource usage, data dependences, control dependences, and clock frequency [6], [16].

The three schedules in Figure 2 correspond to the circuits in Figure 1. The schedule of each design is regulated by the controller; it implements a *finite state machine* which controls the behavior of the datapath by triggering operations, enabling registers, and multiplexing values in appropriate clock cycles. The first schedule corresponds to the sequential execution of the software code: one iteration starts after the previous one has completed. The code restructuring in the second figure enables operations to execute in parallel, hence



lowering the execution time, but with the investment of additional resources. The third circuit employs loop pipelining: a new loop iteration starts on every clock cycle and the resource requirements are minimal (i.e., new data is inserted into the single adder and the single multiplier on every cycle).

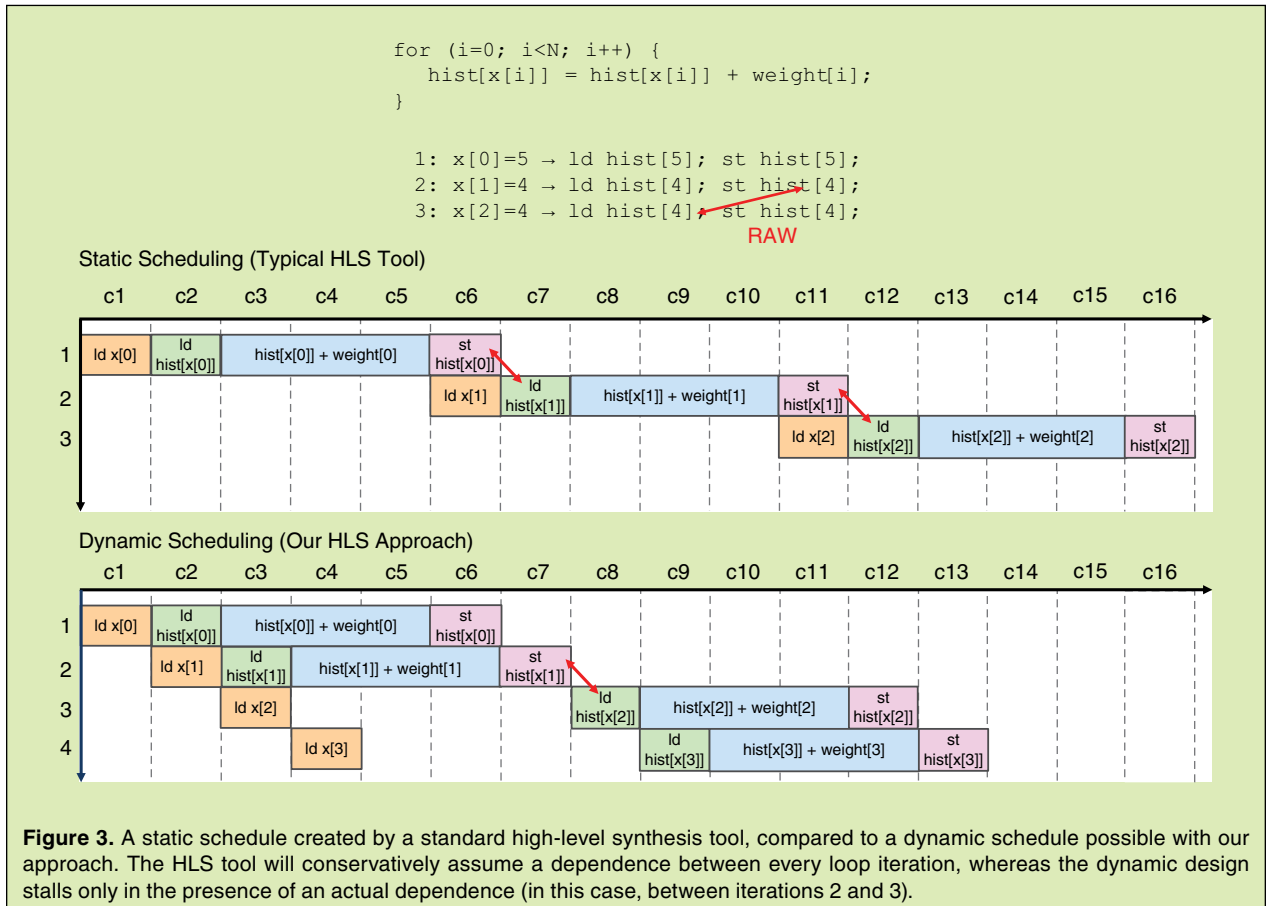
Loop pipelining is one of the key performance optimizations in HLS—as the example above suggests, it allows loop iterations to overlap in the best possible manner while honoring all data and control dependences of the program. The technique originates from software pipelining techniques for *Very Long Instruction Word* (VLIW) processors, which rely on modulo scheduling algorithms to exploit instruction-level parallelism among successive loop iterations [46], [58]. A pipeline is characterized by its *initiation interval* (II), i.e., the number of clock cycles between consecutive loop iterations; the ideal II is equal to 1, as is the case for the pipelined schedule in the figure.

C. Limitations of Standard Scheduling

As in the case of VLIWs, the ability of HLS pipelining to achieve a low II is limited in the presence of resource or memory port constraints as well as long-latency

loop-carried dependences. In addition, pipelining may be limited in cases where compile-time information is not sufficient to devise the best possible schedule. Examples of such situations include memory access patterns which cannot be determined at compile time, unpredictable control flow, and variable-latency operations. In such cases, standard tools must assume the presence of a dependence and produces a schedule with a conservative II.

To illustrate this limitation, consider the example in Figure 3. In this loop, there is a *possible* data dependence between the memory read of `hist[x[i+1]]` and the memory write to `hist[x[i]]` of the previous iteration. There is intrinsically no way a compiler or an HLS tool can ensure that such dependence does not exist, nor is it, in general, possible for a programmer to help the tools: in practice, the read may seldom or even never address the same value just written in memory, but there is no way to exclude a priori that this might happen. Ultimately, any HLS technique based on static scheduling hits the problem of potential dependences and needs to account for the worst-case scenario, irrespective of the actual data fetched from memory. The result is a conservative schedule valid for any possible input values, which assumes a dependence in every loop iteration



and postpones the read until the previous, possibly dependent write has been completed.

III. A Completely Different Way to Do HLS

The key to avoiding the limitations of static scheduling is to refrain from triggering the operations through a centralized pre-planned controller (as shown in Figure 4(a) and the examples in Figure 1), but to make scheduling decisions locally in the circuit as it runs: as soon as all conditions for execution are satisfied (e.g., the operands are available or critical control decisions are resolved), an operation starts. *Dataflow circuits* [18] are a natural method to realize such behavior. Such circuits are built out of units that implement *latency-insensitivity* by communicating with their predecessors and successors through point-to-point pairs of handshake control signals, as indicated in Figure 4(b). The data is propagated from unit to unit as soon as memory and control dependences allow it—otherwise, the handshaking mechanism stalls the data on-the-fly. This distributed control mechanism effectively implements a *dynamic schedule*, such as the bottom schedule in Figure 3: when a dependence exists (in the example, between the second and the third iteration) the dynamically scheduled circuit will stall the pipeline to prevent hazards. Otherwise, in the absence of an address collision, it will start a new iteration on every cycle and gain, in this case, up to a factor 6 in performance. Such dynamic behavior is beyond what classic static techniques can achieve [37].

In the rest of this article, after a brief digression on how the same issues have influenced computer architecture, we will describe our HLS methodology which produces dynamically scheduled dataflow circuits out of high-level code. We discuss our methodology to implement high-throughput pipelines and to identify resource sharing opportunities in the circuits we generate. We then detail the construction of a memory interface (i.e., a load-store queue) for dataflow circuits that can correctly handle memory accesses arriving out-of-order and show how to automatically customize this interface to a particular application. Further, we present a generic framework for handling speculative execution. All these features introduce to dataflow circuits characteristics similar to those of modern superscalar processors; we believe that such behavior is key for HLS to be successful in new contexts and broader application domains.

IV. Computer Architects Have Been There Already

The contrast between static and dynamic scheduling in HLS is in line with the experience in computer architecture.

Practically all high-end application processors in our computing devices and data centers are *superscalar out-of-order* processors [33]. Their architecture is usually similar to the one shown in Figure 5(a). The key idea is that *reservation stations* enable out-of-order execution: they hold back fetched and decoded instructions until

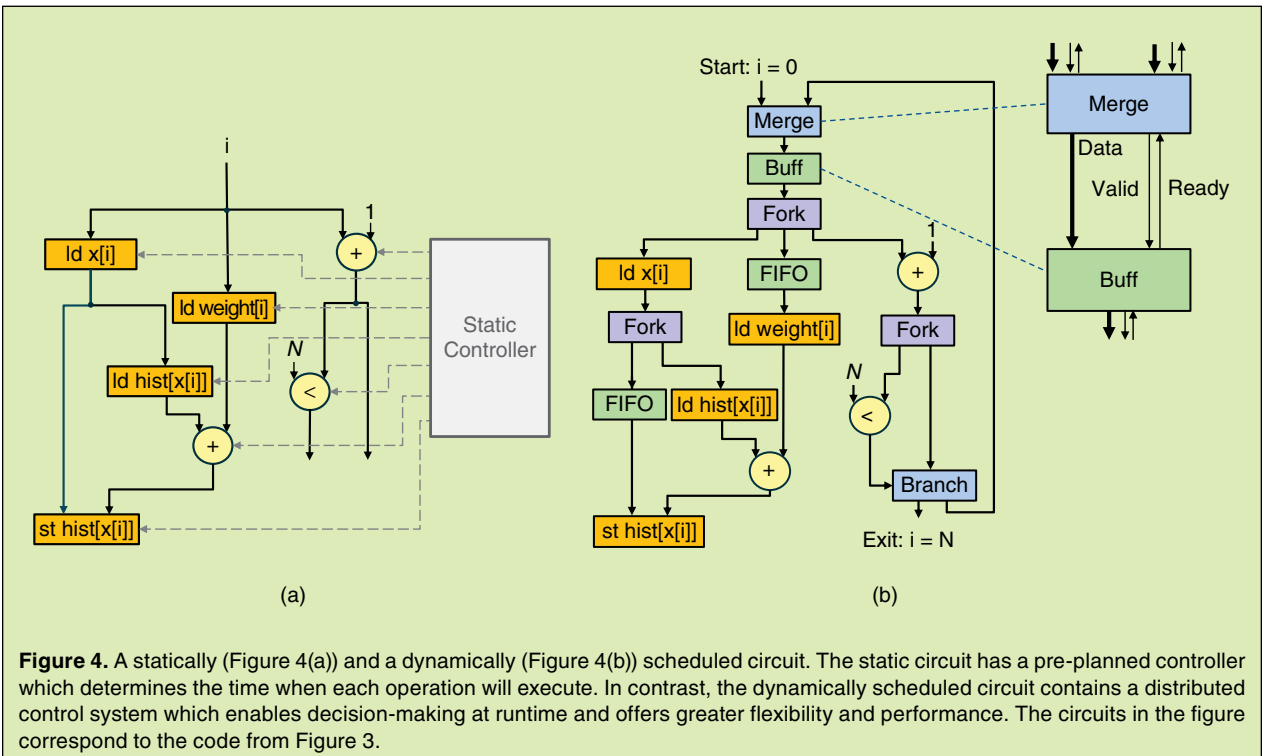


Figure 4. A statically (Figure 4(a)) and a dynamically (Figure 4(b)) scheduled circuit. The static circuit has a pre-planned controller which determines the time when each operation will execute. In contrast, the dynamically scheduled circuit contains a distributed control system which enables decision-making at runtime and offers greater flexibility and performance. The circuits in the figure correspond to the code from Figure 3.

all of their operands are available and let ready instructions advance; instructions are delayed if a load misses in the cache or a rare dependence through memory actually occurs. As with our HLS technique, there is no schedule planned in advance: the schedule develops dynamically as operands become available. Note that the *load-store queue* in the figure is essentially the same component that we employ in our circuits, as we will detail in Section VII-A. Modern processors can also execute instructions speculatively (e.g., even if the outcome of a preceding branch is yet unknown or if the existence of a dependence through memory has not yet been ascertained)—again, a feature that our HLS technique

also sports, as we will discuss in Section VII-B. In these processors, the *reorder buffer* is the key component to implement speculative execution and squash results of incorrectly executed instructions.

Another class of processors which exploit instruction-level parallelism are VLIW processors (a term introduced by Josh Fisher in 1983 [28]), illustrated in Figure 5(b). In VLIWs, the problem of deciding how early instructions can be executed and which ones can be issued in parallel is left completely to the compiler: the hardware simply fetches at once groups of operations which can be performed together (the *very long instructions*) and executes them without checking for operand availability.

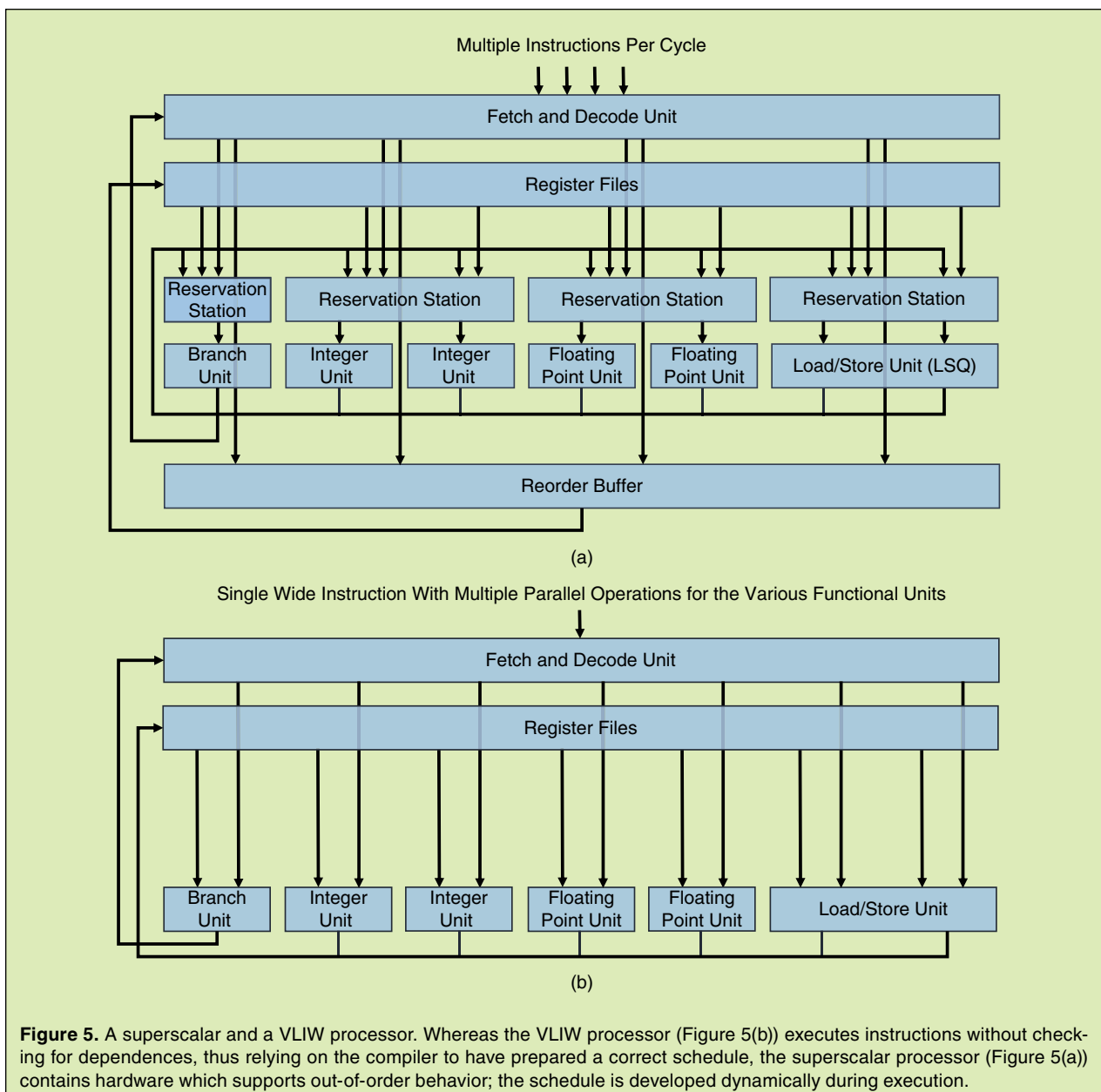


Figure 5. A superscalar and a VLIW processor. Whereas the VLIW processor (Figure 5(b)) executes instructions without checking for dependences, thus relying on the compiler to have prepared a correct schedule, the superscalar processor (Figure 5(a)) contains hardware which supports out-of-order behavior; the schedule is developed dynamically during execution.

The program is effectively a schedule computed statically by the compiler, exactly as in the case of statically-scheduled HLS. The appeal is clear: these processors do not need all the complex hardware that superscalar out-of-order processors require to make runtime decisions.

Researchers quickly figured out that VLIW processors need very sophisticated compilers. Today, a large body of literature exists on VLIW compilation techniques but such techniques often require either complex heuristics to drive the optimizations or pragmas from the programmers. Conversely, dynamically scheduled out-of-order processors achieve good levels of parallelism on-the-fly and without extensive code preparation, yet at the price of more complex hardware. Many of the key transformations to exploit fine-grain parallelism between operations in statically scheduled HLS derive directly from VLIW compilation techniques, such as trace scheduling, software pipelining, and modulo scheduling [27], [46], [58]. But, exactly as HLS tools producing statically scheduled circuits, VLIWs suffer when handling code with irregular memory or control dependences.

The dichotomy in computer architecture may tell us something about the future of dynamically scheduled HLS. In the mid-1990s, Hewlett-Packard and Intel partnered to develop the first (and, to date, only) general-purpose VLIW processor: Itanium. Servers using Itanium shipped mostly in the 2000s and were a major commercial failure [23]. Today, VLIWs thrive exclusively in markets with extremely regular and predictable applications, and where it is acceptable for skilled developers to tune code manually. They are, for instance, in our smartphones where they run complex digital signal processing applications. However, general-purpose, irregular, and control-dominated computing tasks require

the runtime flexibility of dynamic scheduling. Even in cost-sensitive devices such as smartphones, none of the processors which run operating systems (e.g., Android and iOS) are VLIWs.

Today, with FPGAs moving to data centers and facing broader application classes, HLS tools might have to satisfy the needs of general-purpose markets as well. Apart from the advantage of exploiting parallelism in cases where static scheduling cannot, the ability of dynamic scheduling to find an acceptable solution without the programmer's help may be critical in a future where HLS will not be driven by hardware designers (available to study the generated circuits and to restructure the input code) but by higher-level code generation tools (e.g., Delite [31]) and, ultimately, by software programmers.

V. From High-Level Code to a Dynamically Scheduled Circuit

In this section, we outline our HLS methodology which produces dynamically scheduled circuits out of C/C++ code. We first provide an overview of the latency-insensitive design paradigm; we then discuss the dataflow primitives we use and, finally, we describe our HLS conversion strategy.

A. Latency-Insensitive Protocols

Latency-insensitive protocols [8], [18] implement dynamically scheduled dataflow circuits. These circuits are built out of dataflow *units* which exchange pieces of data (referred to as *tokens* [50]) through *channels* composed of data lines and paired with handshake control signals: a *valid* signal indicates the availability of a piece of data and the *ready* signal indicates the readiness of a unit to accept new data. This distributed control system enables dataflow circuits to adapt the schedule at runtime to variable latencies of particular memory access patterns and control-flow decisions.

The latency-insensitive communication strategy originates from the asynchronous circuit domain. Figure 6(a) illustrates two commonly used asynchronous protocols which employ a pair of request and acknowledge signals to regulate data transfers. In the 4-phase protocol, a communication cycle involves four events (i.e., rising and falling edges) and the handshake signals return to zero at the end of each data transfer [29]. In the two-phase protocol, each cycle involves only two events (i.e., either rising or falling edges of the handshake signals) [61].

In the rest of this article, we consider a *synchronous* latency-insensitive protocol: the initiation and completion of data transfers are indicated by the value of the handshake signals at the rising clock edges [18], as illustrated in Figure 6(b). Our perfectly synchronous designs are therefore compatible with traditional VLSI

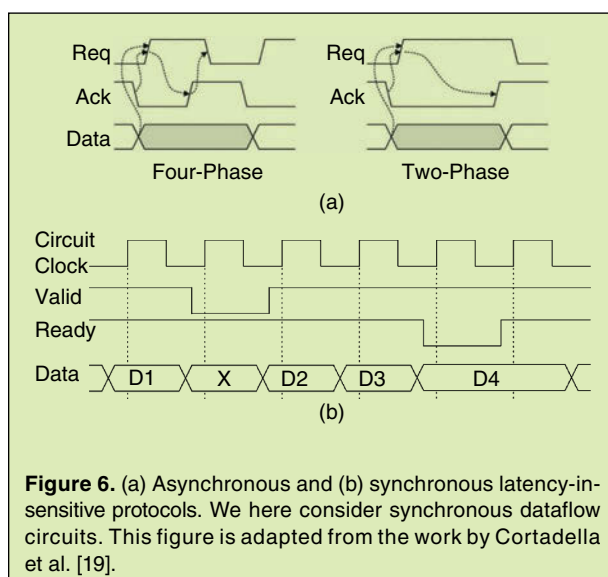


Figure 6. (a) Asynchronous and (b) synchronous latency-insensitive protocols. We here consider synchronous dataflow circuits. This figure is adapted from the work by Cortadella et al. [19].

and FPGA flows and directly comparable to standard HLS techniques.

B. Dataflow Units

In addition to standard functional units, dataflow circuits require specialized units which control the flow of data between units, as illustrated in Figure 7:

- An *eager fork (fork)* replicates every token received at the input to multiple outputs; as soon as one successor is ready to accept the token, the fork sends it to the successor; however, the fork can accept a new token only after all successors have accepted the previous one.
- A *lazy fork (lfork)* has the same functionality as the eager fork; however, it distributes a token to all successors at once (i.e., all successors must be ready for the lazy fork to send the token).
- A *join* acts as a synchronizer—its output is triggered only after all of its inputs become available.
- A *branch* implements program control-flow statements; it dispatches a token received at its single input to one of its multiple outputs based on a condition.
- A *merge* is a nondeterministic unit that propagates a token received on any of its input to its single output.
- A *control merge (cmerge)* is a merge that, apart from the data output, has an output which indicates which of the inputs was taken by the merge.
- A *mux* is a deterministic version of the merge; it propagates to its single output the input token selected by a control input.
- A *source* can always issue a valid token to its single successor (e.g., a constant).
- A *sink* is always ready to consume tokens from its single predecessor; tokens are simply discarded in the sink.

In addition, like any circuit produced by HLS, dataflow circuits employ any functional unit the code requires, such as integer and floating-point units. Units that require multiple operands contain a join to trigger the operation only when all inputs are available. Our circuits will require *buffers* which serve as registers in standard synchronous designs—we will discuss their properties and placement in Section VI-A. Finally, our circuits will interface to memory using read and write ports, yet, interfacing to memory is challenging due to the out-of-order nature of our system; we will address this issue in Section VII-A.

C. C-to-Dataflow Conversion

This section informally describes a way to transform a standard static single assignment (SSA) intermediate representation [64] into a functionally correct dataflow

circuit. We detail the formal correctness and liveness properties in our previous work [40].

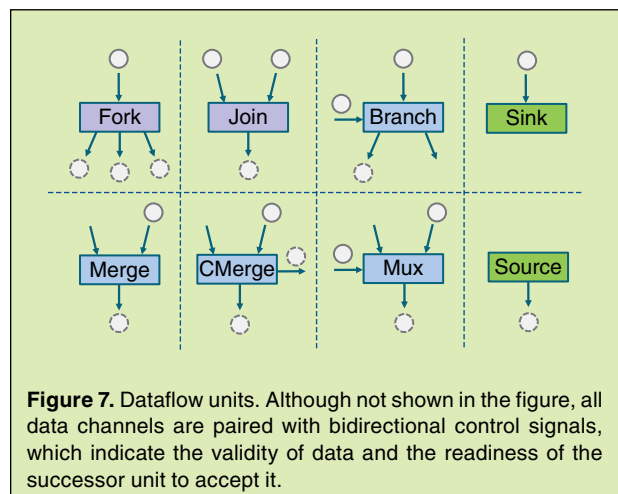
The programs we consider are organized into sections corresponding to basic blocks (BBs), i.e., pieces of code with no conditionals. All control flow statements are implemented between the BBs and each BB contains a dataflow graph (DFG) of program instructions. Each BB receives *live-in* variables from the predecessor BBs and produces *live-out* variables for the successor BBs. This typical compiler intermediate representation propagates a piece of data directly from a source operation to any number of its consumers. In a dataflow circuit, the data and control must be strictly coupled and the number of tokens must exactly match the number of distinct uses, i.e., an operation should be triggered exactly the number of times that it executes in the original program.

Implementing Control Flow

To guarantee that data is always accompanied by control, the following must hold: (1) every BB must provide data exclusively to its immediate successor BBs and (2) every BB must receive data exclusively from its immediate predecessor BBs. Hence, every BB liveout must be sent to the immediate successors using branch nodes; every BB livein must be injected into a BB through a mux node, with as many data inputs as there are predecessor BBs. This strategy guarantees that every piece of data is sent correctly from BB to BB, following the control flow of the program.

In-Order Control Network

Some operations do not have any inputs (e.g., constants); we must ensure that they are appropriately triggered and executed. Furthermore, a mux at the BB entry may receive multiple inputs at the same time; we need to ensure



that the inputs are accepted in order of program execution. To this end, we generate an in-order control path that follows the control flow of the program through the BBs—essentially, a data-less variable which is a live-in and live-out of each and every BB. The tokens on this path are used to trigger operations without inputs as many times as their BB becomes active. This path enters each BB through a cmerge, which connects to the muxes of the same BB and indicates the ordering of the inputs from which they will receive data, as illustrated in Figure 8. Whenever a mux is guaranteed to receive its inputs in order (e.g., when there is a single value propagating through the CFG and a token can only enter a BB from its single active predecessor), it may be disconnected from the cmerge and replaced with a simpler merge unit.

Constructing the Datapath

Once the control flow is correctly handled, the BB internals are straightforward to design—each instruction is literally converted into its dataflow unit (i.e., a functional unit with inputs and outputs accompanied by handshake signals). As every data exchange must be represented with an explicit token transfer (i.e., handshake exchange), units with multiple successors require a fork to replicate the output token into a token for each of the successors. Unused unit outputs (e.g., branch outputs without successors) connect to sinks which discard the unused tokens.

VI. Bringing HLS Optimizations to Dataflow Circuits

In this section, we present techniques that make the circuits produced with our HLS approach competitive to standard HLS solutions: we first describe how to pipe-

line dataflow circuits and then discuss how to save resources through time-multiplexing of functional units.

A. Pipelining

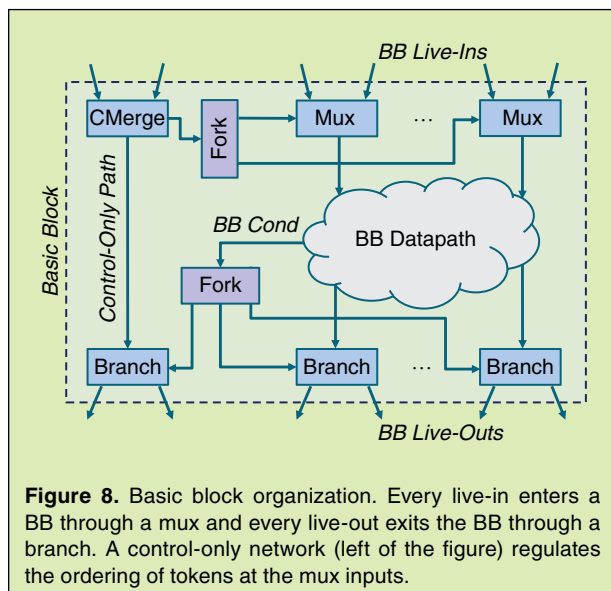
Dataflow circuits are naturally capable of pipelining, as the fine-grain handshake mechanism allows certain operations to run ahead and, consequently, enables executions of different operations to overlap. Yet, pipelining is not always possible due to *backpressure*: some paths take a longer time to consume a token and prevent potentially quick and independent paths from processing tokens at a high rate. This issue is illustrated on the left of Figure 9(a), showing the dataflow circuit implementing the code in the bottom of the figure: the fork could, in principle, issue tokens to the load on every cycle, but the path to the store stalls the first token until the multiplication completes, hence preventing new tokens from issuing to the load and limiting loop pipelining; the achieved schedule will, essentially, correspond to a nonpipelined schedule of a static HLS tool. Classic pipelining algorithms that standard tools exploit are not applicable in the absence of a static schedule; the solution here is to systematically identify and resolve backpressure to achieve the same pipelining effect.

Just like standard synchronous circuits, dataflow circuits require buffers, i.e., registers, which break combinational paths and, possibly, reduce the critical path of the circuit. Yet, in contrast to standard circuits, buffers can be placed on any channel (i.e., between any two dataflow units) without compromising the circuit functionality. This property can be exploited to mitigate backpressure by inserting buffers into the paths that create stalls and lower system throughput, as illustrated on the right of Figure 9(a). Buffers used for regulating throughput typically have a larger capacity (i.e., they are implemented as FIFOs with multiple data slots) to hold all tokens issued by the predecessor before the successor is ready to accept them—in this example, the buffer requires 3 slots to constantly consume tokens from the fork; without the backpressure on the fork, the iterator loop can issue a new token on every cycle and achieve a perfect pipeline with an II equal to 1.

To optimize the performance of dataflow circuits by strategically placing and sizing buffers, we have developed a mixed-integer linear programming model [43] based on Petri net theory [50]. This model allows for resource-optimal buffer placement and sizing, with the purpose of maximizing throughput of the performance-critical loops at the desired clock frequency.

B. Resource Sharing

Standard HLS tools perform scheduling in conjunction with resource allocation and sharing [70]; depending



on the optimization objective, they trade off area and performance by deciding the cycle in which each operation executes and allocating units accordingly. The top of Figure 9(b) shows two possible schedules for the code in the figure. The first schedule is unconstrained in resources; by scheduling both multiplications in the same cycle, it employs two multipliers to achieve the ideal loop pipeline with an initiation interval of 1. The second schedule enforces a resource constraint of one multiplier; each multiplication must be scheduled on every second cycle and causes an increase of the II to 2.

Dataflow circuits face the same optimization objectives and area-performance trade-offs. Yet, in the absence of a predetermined schedule, it is challenging to determine which operations can share a functional unit without a performance penalty. Intuitively, one could rely on statistical information on unit utilization to decide what to share, as illustrated in the bottom of Figure 9(b): if the two multipliers are fully utilized (i.e., fully occupied with tokens), sharing would damage throughput; if they are only half-utilized, one could employ a single multiplier instead that would always be filled with tokens. Yet, this approach on its own may still compromise performance because the execution of some operations may be delayed with respect to their execution in the original circuit. More

critically, although sharing seems to imply only some trivial circuitry, time-multiplexing units in dataflow circuits may cause deadlock by blocking certain data transfers and preventing operations from executing. Hence, a crucial step in making dataflow circuits resource-competitive with standard HLS is to systematically identify good sharing opportunities in an absence of a predetermined schedule, but also to build a sharing mechanism that always results in functional dataflow circuits.

The performance optimization model described in the previous section can be used directly to determine the flow of data through each functional unit and provides us with information on unit utilization; we can exploit this information to decide which units to share. Furthermore, the performance-limiting delays caused by sharing can be resolved through appropriate buffering, achieved by the same optimization technique. Employing a local scheduler at each unit input to regulate the multiplexing of the incoming tokens can avoid unit starvation and ensure the absence of deadlock [17], [32].

VII. Introducing Features of Superscalar Processors to HLS

The optimizations from the previous section enable dataflow circuits to achieve high-throughput pipelines and to share resources, just like standard HLS circuits

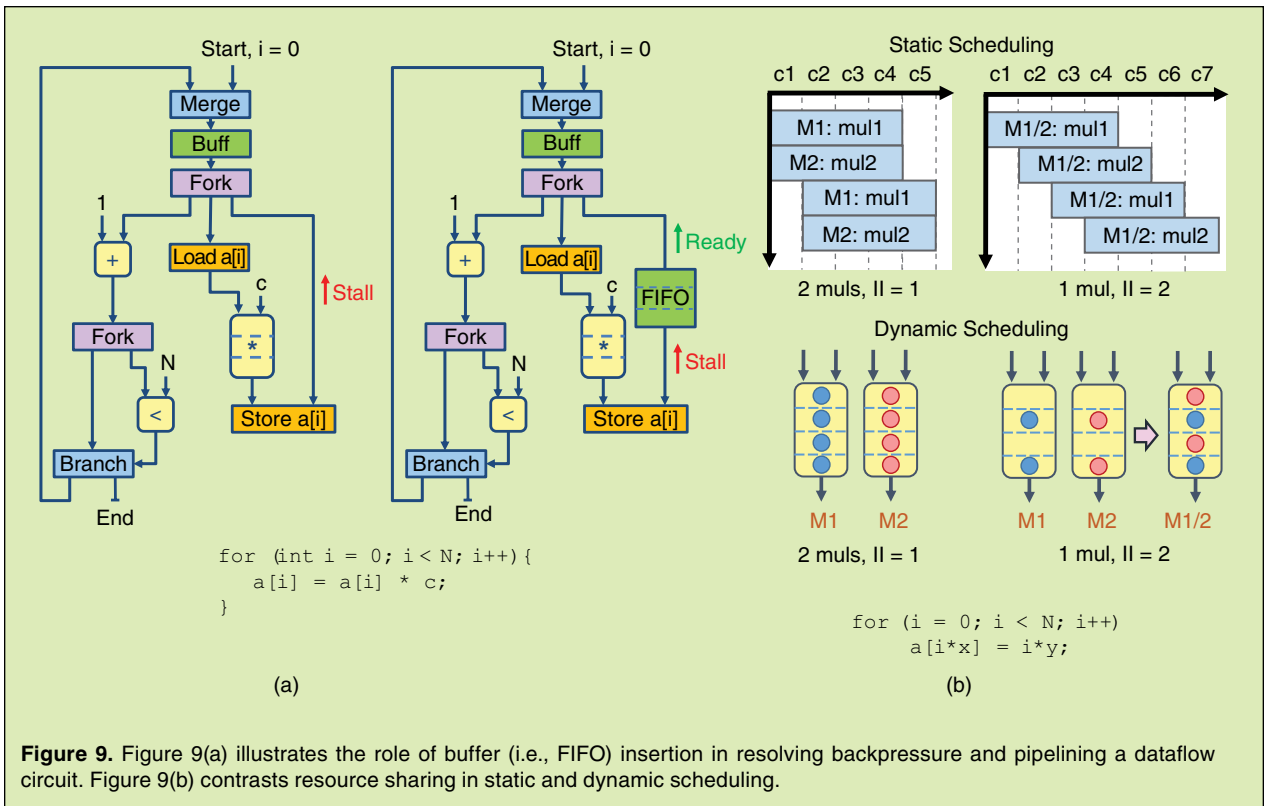


Figure 9. Figure 9(a) illustrates the role of buffer (i.e., FIFO) insertion in resolving backpressure and pipelining a dataflow circuit. Figure 9(b) contrasts resource sharing in static and dynamic scheduling.

can do. Yet, their ability to adapt the schedule at runtime allows them to support features which are regularly exploited by superscalar processors and beyond the capabilities of standard HLS, such as out-of-order memory interfaces and speculative execution—we describe these features in this section.

A. An Out-of-Order Memory Interface

One of the key enablers of dataflow pipelines lies in the ability to execute memory accesses in an order different than the one specified in the original program. As illustrated in Figure 3, pipelining in cases where memory dependences cannot be determined at compile time may be critical for good performance and is the key to the superiority of dataflow computation to statically scheduled HLS designs.

Out-of-order behavior has been exploited in out-of-order processors for decades [54], [55], [59]: load-store queues are used to ensure that all memory dependences are honored, while independent memory requests may execute out-of-order for performance benefits. Dataflow circuits require the same functionality, but a processor LSQ cannot be employed directly because of a fundamental difference between the two systems, as illustrated in Figure 10: In a processor, the notions of fetching and decoding instructions immediately convey the correct sequential order of requests at the memory inter-

face. In contrast, dataflow circuits lack such notions and the information of the original sequential program order is lost; an LSQ receiving memory requests out of order would not be able to decide which reorderings are legal. Therefore, to employ an LSQ and truly benefit from out-of-order execution, dataflow circuits require an alternative way to perform allocation and to convey the correct order of memory requests to the LSQ [38].

A way to provide this information is to send to the LSQ tokens which follow the actual order of execution of the basic blocks of the circuit. This ordering enables the LSQ to determine and resolve dependences as memory access arguments from different BBs arrive out-of-order. For this purpose, we use the in-order control path described in Section V-C—this path forks into the LSQ from each BB whose loads and stores are connected to the LSQ; whenever control flows into this BB (i.e., as soon as a decision has been made to enter a particular BB), it triggers the allocation of its memory accesses to the LSQ. This mechanism is illustrated in Figure 11; it ensures that the LSQ can correctly handle memory accesses arriving in arbitrary order while still respecting data dependences.

Although LSQs enable dataflow circuits to achieve high performance in situations that static scheduling cannot efficiently handle, they imply high resource requirements as well as power and clock degradation when implemented on an FPGA. Hence, it is beneficial to

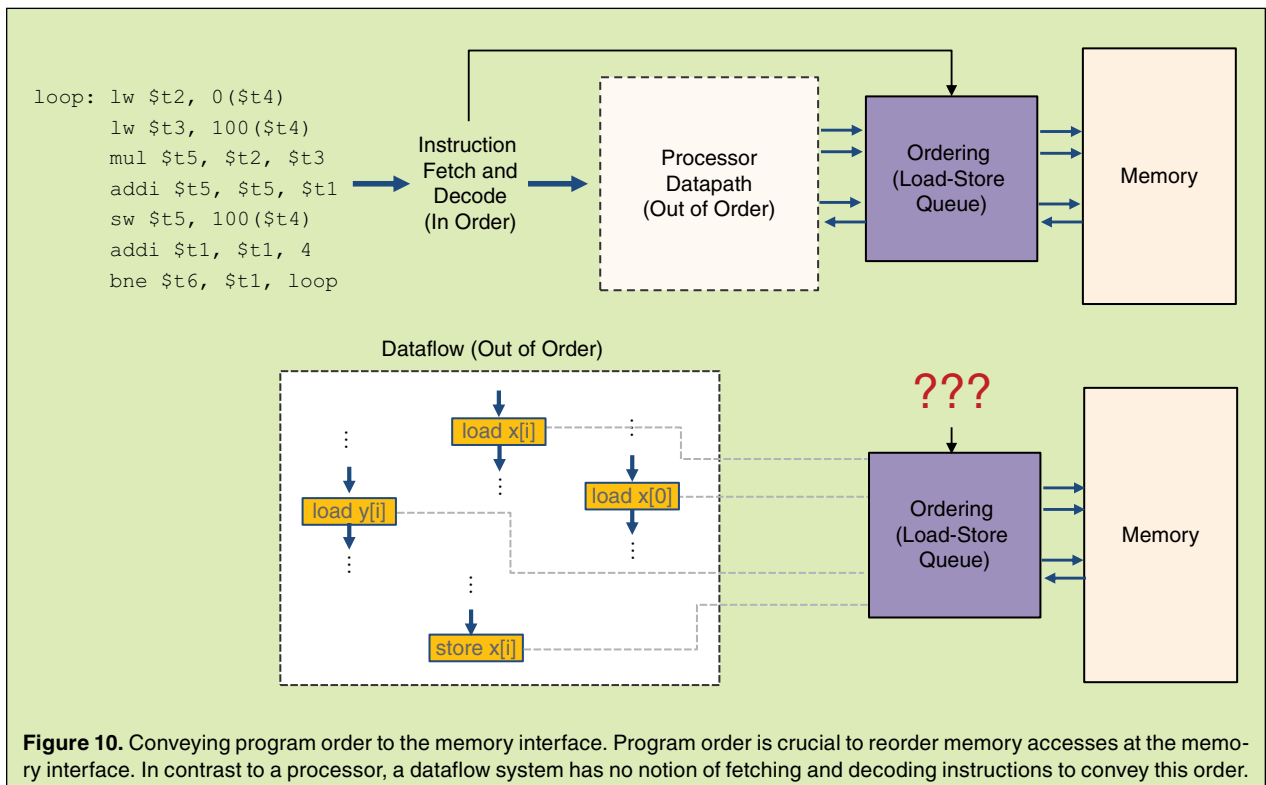


Figure 10. Conveying program order to the memory interface. Program order is crucial to reorder memory accesses at the memory interface. In contrast to a processor, a dataflow system has no notion of fetching and decoding instructions to convey this order.

leverage compiler analysis to simplify the memory interface whenever possible—whenever the compiler can disambiguate memory accesses, groups of accesses that cannot mutually conflict can use separate LSQs, while those which certainly have no dependences with any other access can connect to simple memory interfaces.

The code in Figure 12 shows a loop with multiple memory accesses which are analyzed and optimized using different memory analysis techniques. The information about the memory accesses available in each analysis step is illustrated on the top of the figure: the green dashed edges indicate a possible dependence among accesses which the memory interface must appropriately handle (i.e., if two accesses are certainly or possibly dependent, they require an LSQ). The memory interfaces obtained in different analysis steps are illustrated below. Without any memory analysis (case a) to reason about actual memory dependences, all accesses must connect to a single, large LSQ. By exploiting alias analysis (case b) and determining the memory access patterns using polyhedral techniques (case c), one can determine that some accesses (i.e., those accessing different arrays and those targeting different memory locations, respectively) cannot conflict (i.e., the presence of certain dependences is excluded); the memory interface can be simplified by employing multiple smaller LSQs and removing some LSQs altogether, as the second and the third memory configurations in the figure suggest. Finally, our specialized dataflow analysis [36] can determine that the load and the store to $x[i]$ naturally occur in the correct order as the load produces the data for the store—one can omit the LSQ completely without compromising correctness (case d).

Employing these memory analysis techniques enables us to obtain an optimized memory interface configuration in an out-of-order dataflow system: we profit from the LSQ whenever it is truly useful (such as the situation in Figure 3); otherwise, we remove or simplify it to save resources.

B. Speculation

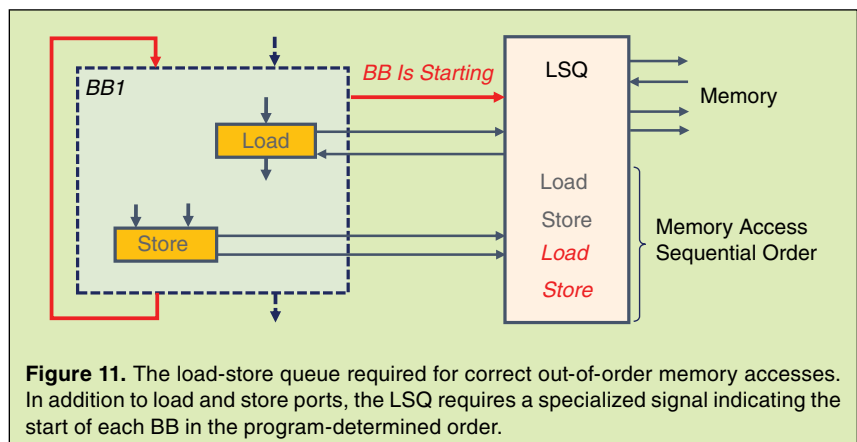
As in computer architecture, dynamic scheduling paves the way to one of the most powerful ideas in computing: executing some operations before one has the certitude that they are actually needed or that it is correct to execute them. Speculation can significantly improve the execution of loops where the condition on the loop continuation takes very long to compute by predicting very early whether

it makes sense to execute *tentatively* another iteration. Similarly, speculation can further improve the problem of memory dependences, not only by reordering accesses when there is no dependence detected, but also by assuming independence early on and reverting back if the prediction was wrong.

The example in Figure 13 illustrates the need to enable speculative execution in HLS. A standard, non-speculative schedule allows a new loop iteration to start only after the condition to exit the loop (which, in this example, takes multiple cycles to compute) has been checked; therefore, the loop cannot be pipelined. In contrast, a speculative system would achieve the lower schedule which tentatively starts a new loop iteration on every clock cycle, before the loop condition is known. The ability to implement speculation depends on reliable mechanisms to revert state changes due to wrongly executed operations and discard mispredicted values—in processors, this functionality is entrusted to reorder buffers and store queues [33]. In the example in Figure 13, the speculatively computed values from iterations 4 and 5 must be discarded and the result from the third iteration must be returned.

Figure 14 gives a sense of our strategy to implement generic forms of speculation in dataflow circuits [41]. The idea is that some units might be allowed to issue *speculative tokens*—pieces of data which might or might not prove correct and which will combine with other (nonspeculative) tokens, resulting in more speculative tokens traveling through a delimited region of the circuit. In other words, speculative tokens trigger some computations which might have to be squashed and possibly repeated with the correct nonspeculative tokens.

Speculation is triggered by a *speculator*, i.e., a special version of a regular dataflow unit which, besides its standard functionality, can also inject tokens into the circuit before receiving any at its input(s). The most



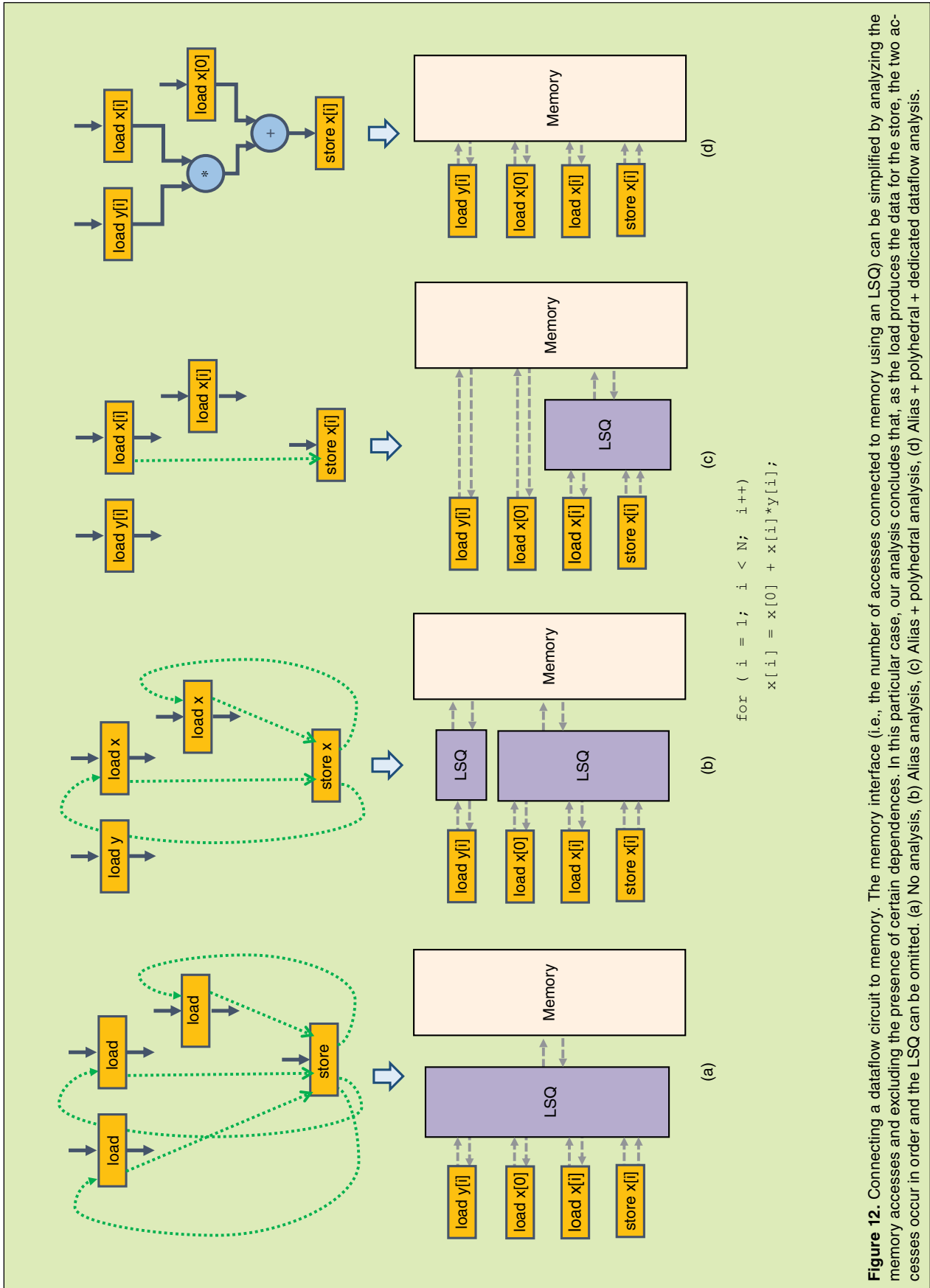


Figure 12. Connecting a dataflow circuit to memory. The memory interface (i.e., the number of accesses connected to memory using an LSQ) can be simplified by analyzing the memory accesses and excluding the presence of certain dependencies. In this particular case, our analysis concludes that, as the load produces the data for the store, the two accesses occur in order and the LSQ can be omitted. (a) No analysis, (b) Alias analysis, (c) Alias + polyhedral analysis, (d) Alias + polyhedral + dedicated dataflow analysis.

natural example is that of a branch node which receives the value to dispatch but not the condition; a branch speculator could predict the missing condition and send tentatively the value through one of its outputs. If, after issuing a speculative token, the speculator eventually receives the same data which it assumed speculatively (e.g., the condition it predicted), all is fine and execution was probably sped up; if, on the other hand, the data it eventually receives does not match the prediction, the speculative work must be discarded and the speculator must perform its function correctly (e.g., resend the value on the other branch output).

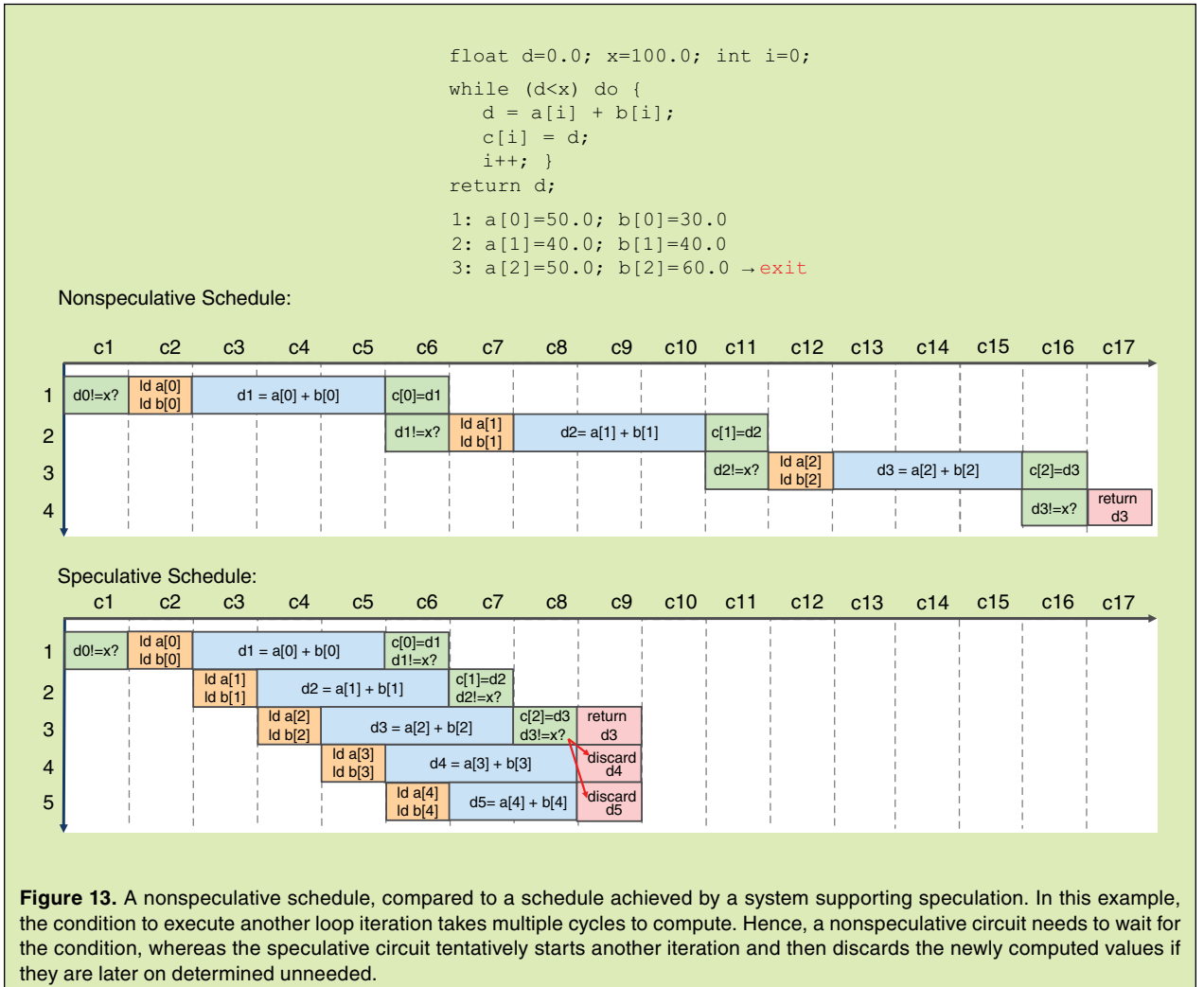
The speculative region in Figure 14 is bound on its input and output sides with specialized units which implement a squash-and-replay mechanism: *Save* units on the region inputs save a copy of all regular tokens which may combine with a speculative token and reissue them if the previous computation is squashed. *Commit* units at the region output let propagate further speculative results which turn out to be correct and simply squash

misspeculated values. Because commit units must differentiate speculative from nonspeculative tokens (the former ones need explicit commit information before propagating, while the latter ones can always go ahead), all channels between the speculator and the commit units must be enriched with a control signal which indicates the speculateness of the token being passed, as Figure 14 suggests.

Our generic framework can implement broad classes of speculation, which is beyond the capabilities of statically scheduled HLS; in the example of Figure 13, our technique achieves a perfect pipeline shown in the bottom schedule. As we will demonstrate in the following section, our speculation approach is of significant performance advantage in situations where waiting for a key execution decision is particularly time-consuming.

VIII. Evaluation

In this section, we compare our dynamically scheduled circuits with a commercial, statically-scheduled HLS tool.



Our complete HLS tool and the benchmarks we explore in this section are publicly available at dynamatic.epfl.ch.

A. Dynamatic HLS Compiler

The C-to-dataflow conversion and the optimizations we discussed in the previous sections are implemented in Dynamatic, our open-source HLS compiler [42]. Dynamatic takes as input C or C++ code and produces a synthesizable hardware description of the corresponding dataflow circuit. The synthesis step relies on the LLVM compiler framework [48]: the *clang* frontend parses the C/C++ program and produces a static single assignment intermediate representation (LLVM IR), which is then optimized using standard LLVM transformation and analysis passes. The optimized IR is then given as input to a set of our custom passes. The main pass adds dataflow units from Section V-B following the transformations described in Section V-C to produce a functionally correct dataflow circuit; other passes perform additional analysis and optimizations (e.g., buffer placement as described in Section VI-A and memory access analysis to create the memory interfaces described in Section VII-A). The result is a dataflow circuit netlist which can be directly converted

into a VHDL netlist of dataflow units; together with a predefined dataflow unit library, it can be synthesized into an FPGA bitstream.

B. Methodology and Benchmarks

To demonstrate the benefits of using dynamic scheduling in HLS, we compare our circuits with designs generated by Vivado HLS [69], a state-of-the-art commercial HLS tool. For a fair comparison, we employ the same arithmetic units and use the same RAMs as Vivado to connect to memory.

We simulate the designs in ModelSim [49] and use a set of test vectors for functional verification. We obtain the average loop initiation interval (II) from the simulation and the clock period (CP) from the post-routing timing analysis to calculate the total execution time. Placing and routing the designs using Vivado gives us the resource usage (i.e., the number of CLB slices, with the corresponding LUT and FF count, as well as the number of DSP units).

Our benchmarks are simple kernels which represent typical cases where static scheduling is known to run into its fundamental limits while dynamic scheduling should make a significant difference. We also consider two simple kernels where static scheduling is fully successful, to show that dynamically scheduling achieves virtually the same result with acceptable overheads.

- *Histogram and Matrix Power* have memory access patterns that cannot be determined at compile time—there may be read-after-write dependences between the stores and the loads from the following iterations.
- *If loop add* and *If loop mul* have a potential dependence across iterations which depends on the runtime-determined condition (i.e., the condition is determined based on data fetched from memory which is unknown during compilation).
- *Backtrack* and *Newton-Raphson* have long-latency, data-dependent conditions for starting a new loop iteration and could benefit from branch prediction.
- *FIR* and *MatVec* are regular kernels that do not have any memory or control dependences.

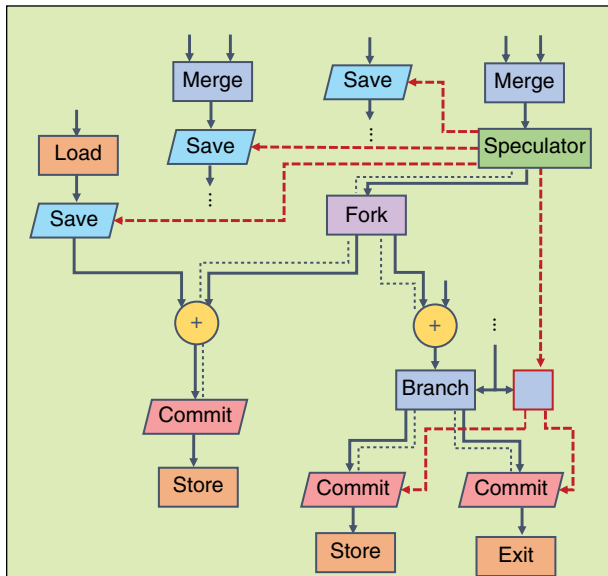


Figure 14. A region of a dataflow circuit implementing our speculative execution paradigm. The speculator initiates speculative execution by injecting tokens tentatively, save units capture required inputs of the region to enable a correct replay in case of misspeculation, and commit units prevent speculative tokens from affecting irreversibly the architectural state, such as memory. Speculative tokens are marked explicitly using an additional bit (represented by the dotted line). A dataflow control circuit (in red, dashed line) between the speculator and the save and commit units carries information about speculative events (start, commit, squash, etc.).

C. Results: Comparison with Static HLS

Figure 15 shows our results relative to those from Vivado HLS (results to the left or below the red square, which represents all Vivado designs, are better). Table I details our results.

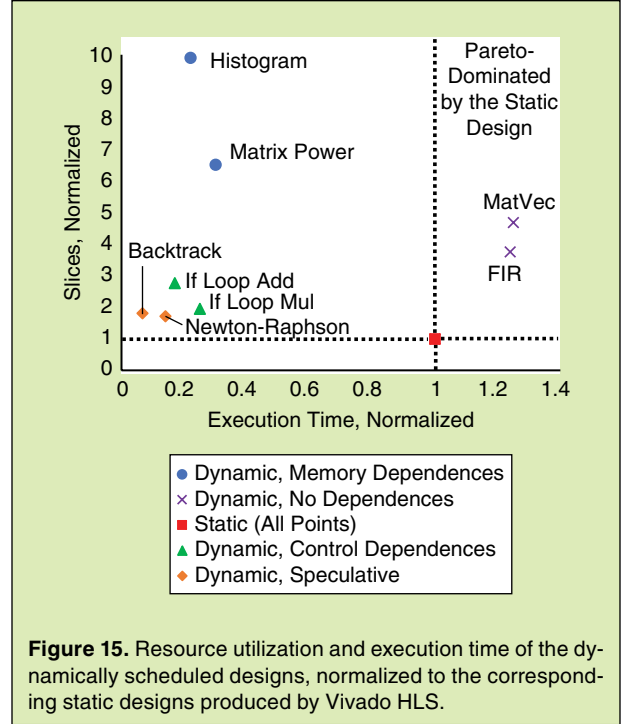
Avoiding conservative assumptions on memory and control dependences results in a significant improvement of the throughput and, consequently, execution

time in all of the corresponding benchmarks. On the *FIR* and *MatVec* benchmarks, static HLS techniques produce highly optimized pipelines because memory accesses can be disambiguated at compile time. The static HLS tool depends on techniques such as modulo scheduling [58] to restructure and pipeline the loop, whereas we effortlessly compile the LLVM IR into a dataflow circuit as-is: although both the static and dynamic design achieve the ideal II of 1, these are the only cases where our results are Pareto-dominated by the static results due to the increase in CP (caused by the additional dataflow logic that we insert into the circuit).

The overhead in slices of the dynamic designs, notable across all benchmarks, is partially due to the control logic that the dataflow circuits contain and which allows them to achieve the latency-insensitivity which we desire. It is immediately visible from Figure 15 that the circuits requiring an out-of-order memory interface demand significant additional resources. It should be emphasized that the resource and timing overhead could be minimized by implementing the LSQs as hard-macros, in the same way as other memory hierarchy components might be in the future (e.g., caches and TLBs). In contrast to the expensive memory interface, our speculation mechanism brings no significant area overhead, yet successfully accelerates all of the corresponding benchmarks by speculating on critical control decisions.

IX. Perspectives

In this section, we evoke some of the most important areas where dynamically scheduled HLS could improve in the future; we outline some research avenues beyond the scope of classic C-based HLS which may benefit from the techniques described in this work.



A. Partial Schedule Rigidification

One optimization aspect which is immediately manifest when looking at the circuits we generate is that we allow latency insensitivity through any unit and on any path. Although, in some cases, this is exactly the strength of our methodology and the reason for its superiority over standard HLS techniques, in many cases it is an expensive overkill: many computational paths may be constructed with fixed-latency components (ALUs, floating-point operators, etc.) and never really profit from the flexibility of dataflow computation. There may be

Table I.

Dynamically scheduled results (our dataflow circuits) contrasted to statically scheduled results (Vivado HLS). The slice count for the kernels with the LSQ is shown as slices of kernel + slices of LSQ.

Benchmark	II _{avg}		CP (ns)		Exec. Time (us)		Slices		LUTs		FFs		DSPs	
	STAT	DYN	STAT	DYN	STAT	DYN	STAT	DYN	STAT	DYN	STAT	DYN	STAT	DYN
Histogram	13.0	2.1	3.5	4.9	45.5	10.1	129	220 + 1073	254	4294	510	2033	2	2
Matrix power	13.0	2.7	3.4	4.9	16.8	5.0	200	295 + 1020	340	4463	735	2055	5	5
If loop add	10.0	1.1	3.2	5.0	32.0	5.5	141	393	315	960	525	1318	2	4
If loop mul	7.0	1.1	3.2	5.2	22.4	5.5	177	348	334	892	655	1127	5	5
FIR	1.0	1.0	2.9	3.6	2.9	3.6	47	178	83	463	176	526	3	3
MatVec	1.0	1.0	3.2	4.0	2.9	3.6	63	298	129	843	221	631	3	3
Backtrack	21.0	1.0	3.7	5.1	76.2	5.1	175	320	353	774	625	956	5	7
Newton-Raphson	8.0	1.0	5.4	5.5	4.3	0.6	201	348	585	1181	636	603	9	9

optimizations that rip-off, under some conditions, complex control paths from the corresponding datapaths and replace them with simpler, customized control structures. One could see this as a *selective rigidification* of the schedule where dynamism is not really needed.

The challenge in performing rigidification is to automatically identify which units and paths may be rigid, without compromising performance or circuit correctness. We already exploited Petri net theory to obtain information on the flow of tokens through the dataflow graph—this information may be critical to identify units through which data always flows at a constant rate. These units do not require handshake logic—instead,

they could be triggered using a local, predetermined scheduler which ensures that data is received and dispatched at appropriate time intervals; multiple independent schedulers could eventually be merged into a single finite-state machine which would control the entire rigid portion of the circuit, as illustrated in Figure 16. The result would be a hybrid statically and dynamically scheduled circuit which enables the programmer to exploit the ‘best of both worlds’ [11], depending on the properties of the code: in regular applications, the final result would qualitatively correspond to a statically scheduled circuit; dynamism would remain only in places where it is actually required for performance benefits and at a significantly reduced area overhead.

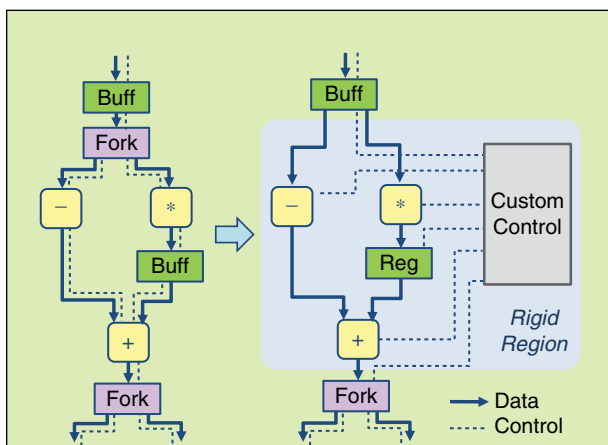


Figure 16. Dataflow circuit rigidification. To simplify the dataflow circuit, the distributed handshake control logic could be replaced with customized control structures whenever dynamism is not required.

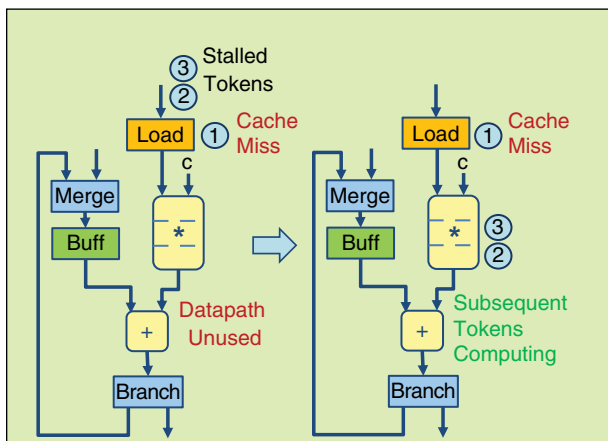


Figure 17. Multithreaded execution. Instead of a single thread which always issues tokens in order into noncyclic paths, one could devise a system where tagged tokens execute out-of-order to increase performance and hardware utilization.

B. Multithreaded Execution

Our current approach targets standard sequential C-based synthesis: there is a single execution thread, i.e., a single token enters through the starting point, propagates through the BBs following the control flow, and exits through the final BB; pipelining is achieved by repeatedly issuing tokens in order into noncyclic paths. Yet, this type of circuit construction may result in limited parallelism and datapath usage in cases where pipelining is not possible (e.g., loop-carried dependences) or when multiple tokens on a noncyclic path are stalled waiting for a long-latency event related to some preceding token (e.g., a cache miss).

Many standard HLS approaches support kernel replication to enable multiple parallel executions on independent copies of the datapath [14], hence fully exploiting the spatial parallelism of the device—the same is perfectly possible with dataflow circuits and could be achieved by high-level transformations (i.e., different input language or intermediate-level kernel replication) in dynamically scheduled HLS. In addition, some authors have looked into pipelining multiple threads on a single kernel (i.e., allowing one thread to execute on the datapath before the previous thread has completed) [62]. Such *multithreading*, analogous to simultaneous multithreading in superscalar processors [66], [67], allows for maximal hardware reuse as resources can be shared among multiple threads. Just like superscalars, dataflow circuits are naturally suited to accommodate such behavior; it could be implemented by inserting multiple *tagged tokens* into the circuit and allowing them to re-order on both cyclic and noncyclic paths, as illustrated in Figure 17. Enabling such multithreaded pipelines requires the creation of an efficient tagging mechanism which allows out-of-order execution wherever it is beneficial and reorders tokens appropriately when needed; similar mechanisms have been explored in dataflow architectures [22] and could be leveraged in this context

as well. Furthermore, such a system would require additional guarantees on the absence of deadlock and appropriate buffering to accommodate the desired number of tokens on each dataflow path. Enabling dataflow kernel replication as well as multithreaded execution on a single kernel has the potential to significantly improve parallelism and resource utilization, hence bringing a completely new optimization dimension to dataflow design.

C. Reconfigurable Dataflow Architectures

So far, we have only attempted to map our dataflow circuits to standard FPGAs—a natural alternative to explore are coarser *reconfigurable arrays*, whose limited flexibility as well as word-oriented nature promise efficiency in area, timing, and energy [35]. The absence of a centralized controller and the systematic pairing of data with handshake signals makes dataflow circuits particularly well-suited for such architectures: each array tile would be composed out of one or more dataflow primitives and the interconnect between tiles would carry data bundled with its control signals, as suggested in Figure 18.

One of the major challenges is to design an array that is structurally adequate for the computational patterns and interconnects which typically appear in dataflow circuits. Intuitively, circuits obtained from high-level code share some representative properties (e.g., BB organization with merges and branches at the inputs and outputs, respectively) which can be exploited to customize the array tiles. Our existing compilation flow can be used to translate a program into a functional netlist of hardware primitives; we would need to develop custom place-and-route techniques which exploit array-specific transformations and optimizations to map these netlists onto the underlying architecture and to enable efficient architectural exploration.

X. Related Work

In this section, we outline what others have done to circumvent some of the problems of statically scheduled HLS and we contrast our work with other dataflow-oriented approaches.

A. Towards Dynamic Scheduling

Recent advances in HLS have explored methods to overcome the conservatism in static scheduling and to remove the inability of HLS tools to handle dynamic events. Several techniques [1, 47] generate multiple schedules which are dynamically selected during runtime, once the values of all parameters are known; they rely on the capabilities of current HLS tools by replicating the source code and dynamically select-

ing which copy of the code needs to be executed. The drawback of these approaches is that they apply to only some very particular cases of dependences through memory; they are also affected by the area (or reconfiguration) overhead of synthesizing two or more versions of an accelerator and the cost of switching between them.

Tan et al. [63] describe an approach called ElasticFlow to apply loop pipelining on a particular class of irregular loop nests with no inter-iteration dependences in the outer loops. In their approach, multiple pipeline instances of a dynamic-bound inner loop are scheduled to execute in parallel. Dai et al. [20] propose methods for pipeline flushing by performing static scheduling for multiple initiation intervals of the pipeline to resolve different possible resource collisions; they later developed application-specific dynamic hazard detection circuitry [21] and have shown the ability of speculation but with stringent constraints (i.e., the approach lacks generality in the ability to revert arbitrarily the state after failed predictions). Nurvitadhi et al. [53] perform automatic pipelining, assuming that the datapath is already partitioned into pipeline stages. The underlying methodology in all these techniques is still based on static scheduling adapted to enable some level of dynamic behavior, which limits the achievable performance improvements only to some particular cases. We think that this body of recent work points to the importance of the ultimate

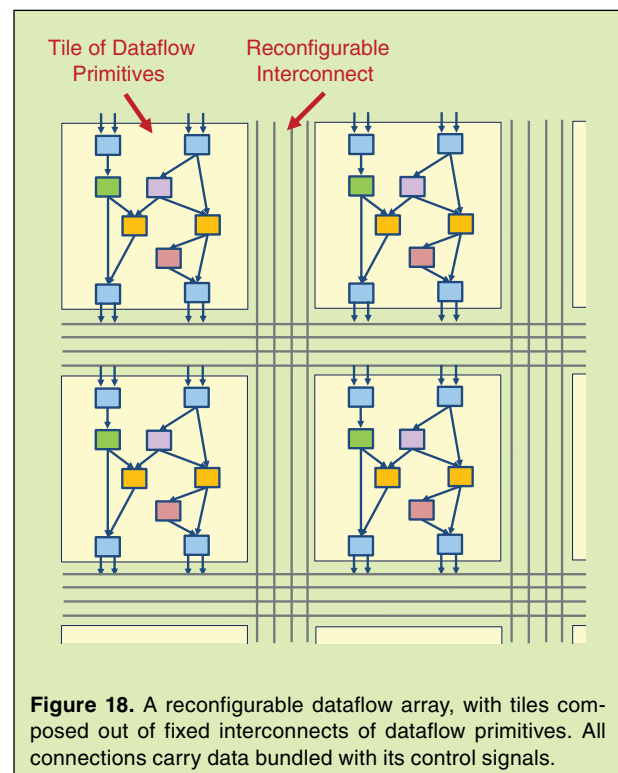


Figure 18. A reconfigurable dataflow array, with tiles composed out of fixed interconnects of dataflow primitives. All connections carry data bundled with its control signals.

solution to the limits of static scheduling: embracing general forms of dynamic scheduling.

B. Dynamic Scheduling in HLS

Different authors exploited latency-insensitive protocols [8], [18], [25] to construct synchronous and asynchronous dataflow circuits. Elastic circuits [18] are probably the best-studied form of latency insensitivity, but the original paradigm used in most of the papers by Cortadella et al. is too restrictive for HLS. Several approaches [10], [34] extended the SELF protocol [18] with constructs similar to the branch and merge which we use in this work. Kam et al. [44] show the ability of elastic circuits to create dynamic pipelines, but do not provide generic transformations to create such circuits out of high-level descriptions. Cheng et al. [12] describe circuits as networks of processes in which hardware accelerators exchange data via dynamic communication channels; similarly, standard HLS tools [69] can interconnect with handshakes various datapaths from nested loops and functions. We are interested in exploring dynamicity on a finer grain (i.e., the schedule of individual datapaths). Efforts in the asynchronous domain, such as Balsa [24] and Haste/TiDE [52], applied syntax-driven approaches for mapping a program into a structure of handshake components [60]; a synchronous backend for Haste/TiDE has later been developed. Putnam et al. [57] also explored synthesizing dataflow-like circuits from high-level specifications. Townsend et al. [65] used a functional programming intermediate representation as a starting point for synthesizing dataflow networks. Dataflow circuits, with their handshake signals, bring to mind Bluespec and its firing rules [68]. However, all these approaches provide little information on some critical conversion aspects and features which are at the heart of this work; to our best knowledge, these approaches have never been contrasted to modern HLS tools.

The efforts closest to ours are the work by Huang et al. [35] and Budiu et al. [4], [5]. Huang et al. generated dataflow circuits from C code, to be mapped to a coarse-grain reconfigurable array [35]. Their circuit generation approach differs from ours in two aspects: (1) They use a single branch node at the output of each basic block, which forces them to synchronize all the basic block outputs and, consequently, prevents loop iterations from overlapping (i.e., loops are not pipelined). (2) Their approach does not employ an LSQ at the memory interface and, thus, all memory accesses which cannot be disambiguated at compile time need to be conservatively sequentialized (“The memory dependence is implemented by creating a lockstep between the corresponding [...] memory ports” [35]). Budiu et al. described a compiler for generating *asynchronous circuits* from C code [4], [5]. Although their final circuits are

fundamentally different from ours (our circuits are *perfectly synchronous* and avoid the traditional difficulties associated with asynchronous designs), the generation strategy is similar to ours. Unfortunately, the exact methodology is never described in full detail and examples across different papers by the same authors do not seem perfectly consistent; although they also employ an LSQ to handle memory dependences, their allocation policy is more conservative than what we described in Section VII-A: they serialize memory accesses whose dependences cannot be resolved statically (“we insert a token edge between two instructions only if their points-to sets overlap and they do not commute” [5]). Both the approach by Budiu et al. and by Huang et al. largely limit the benefits of dynamic scheduling; although Budiu et al. maintain the LSQ, Huang et al. omit it, most likely due to its seemingly limited value. Our LSQ, with its group allocation policy, enables spatial architectures to fully exploit memory access parallelism; our results show that our strategy achieves highly optimized dynamic scheduling.

XI. Conclusions

High-level synthesis (HLS) tools enable hardware generation from high-level software code; because they provide a higher level of abstraction for accelerator design, their role in the future of reconfigurable computing is critical. Despite their recent commercial success and ability to successfully accelerate certain types of applications, standard HLS tools still heavily rely on manual code optimization, code restructuring, and extensive trial and error with configuration parameters and code annotations. In addition, these tools force worst-case assumptions in irregular applications, where data and control dependences cannot be statically resolved; they also provide only limited support for novel optimizations such as speculative execution. In this article, we described a dynamically scheduled form of HLS which produces dataflow circuits from imperative code. Compared to a commercial HLS tool, the result is a different trade-off between performance and circuit complexity, much as superscalar processors represent a different trade-off compared to VLIW processors: when static HLS exploits the maximum parallelism available, our technique achieves similar results with minimal degradation in cycle time and resources; when static HLS misses some key performance optimization opportunities, our circuits seize them by reordering memory accesses, dynamically resolving control dependences, and speculating on critical control decisions, achieving significant performance improvements with the investment of more resources. The ability of our approach to find these design points without requiring significant code restructuring by the programmer is likely to be extremely important in the future where HLS will

be used by software developers with limited hardware design expertise. We therefore believe that this avenue of HLS has the potential to open new doors for reconfigurable computing and its applications.



Lana Josipović (S'16) received a BSc (2013) and MSc (2015) in Electrical Engineering and Information Technology from the University of Zagreb, Croatia. In 2021, she received a PhD in Computer and Communication Sciences from EPFL, Switzerland. Her research interests include high-level synthesis, compilers, and reconfigurable computing. During her PhD, she developed Dynamatic, an open-source high-level synthesis tool that produces dynamically scheduled circuits from C/C++ code. She is a recipient of the Google PhD Fellowship in Systems and Networking, Google Women Techmakers Scholarship, Best Paper Award at FPGA'20, and Best Paper Award Nominations at FPGA'18 and CASES'17.

Country of residence: Switzerland



Andrea Guerrieri received his M.Sc. degree in Electronic Engineering from Politecnico di Torino, Italy, in 2015. In 2006, he started working on embedded systems in PB Elettronica, Italy, where he became Principal Engineer responsible for the development of the company's flagship products. In 2017, he joined the Processor Architecture Laboratory at Ecole Polytechnique Federale de Lausanne, Switzerland, where he leads and participates in research projects in collaboration with industry. Recent projects involve reconfigurable SoCs, exploiting dynamic partial reconfiguration of FPGAs for future space missions and planet observation. He is also a co-developer of Dynamatic, an open-source dynamically scheduled high-level synthesis tool.

Country of residence: Switzerland



Paolo Ienne (S'90, M'96, SM'10) received the *laurea* degree in Electrical Engineering from Politecnico di Milano, Italy, in 1991 and the Ph.D. degree in Computer Science from EPFL, Switzerland, in 1996. Since 2000, he is a Professor in the School of

Computer and Communication Sciences, EPFL. His research interests include computer and processor architecture, FPGAs and reconfigurable computing, electronic design automation, and computer arithmetic. Some of his articles have received the Best Paper Awards at prestigious venues (including at the FPGA, FPL, CASES, and DAC conferences) and several others have been nominated.

Ienne has served as general, programme, and topic chair of renowned international conferences, serves on the steering committee of the ARITH, FPL, and FPGA conferences, and is regularly member of several program committees. He was an associate editor of the ACM TODAES and is an associate editor of ACM CSUR and ACM TACO. Ienne has published over 200 articles in peer-reviewed journals and international conferences. He is a Senior Member of the IEEE and a Member of the ACM.

Country of residence: Switzerland

References

- [1] M. Alle, A. Morvan, and S. Derrien, "Runtime dependency analysis for loop pipelining in high-level synthesis," in *Proc. 50th Design Automation Conf.*, Austin, TX, June 2013, pp. 1–10. doi: 10.1145/2463209.2488796.
- [2] "Amazon EC2 F1 instances." Amazon.com. 2017.
- [3] D. F. Bacon, R. Rabbah, and S. Shukla, "FPGA programming for the masses," *Commun. ACM*, vol. 56, no. 4, pp. 56–63, Apr. 2013. doi: 10.1145/2436256.2436271.
- [4] M. Budiu, P. V. Artigas, and S. C. Goldstein, "Dataflow: A complement to superscalar," in *Proc. IEEE Int. Symp. Performance Analysis Syst. Softw.*, Austin, TX, Mar. 2005, pp. 177–186.
- [5] M. Budiu and S. C. Goldstein, "Pegasus: An efficient intermediate representation," Carnegie Mellon University, Tech. Rep. CMU-CS-02-107, May 2002.
- [6] A. Canis, S. D. Brown, and J. H. Anderson, "Modulo SDC scheduling with recurrence minimization in high-level synthesis," in *Proc. 23rd Int. Conf. Field-Programmable Logic Appl.*, Munich, Sept. 2014, pp. 1–8.
- [7] A. Canis et al., "LegUp: An open-source high-level synthesis tool for FPGA-based processor/accelerator systems," *ACM Trans. Embedded Comput. Syst. (TECS)*, vol. 13, no. 2, pp. 24:1–24:27, Sept. 2013. doi: 10.1145/2514740.
- [8] L. P. Carloni, K. L. McMillan, and A. L. Sangiovanni-Vincentelli, "Theory of latency-insensitive design," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 20, no. 9, pp. 1059–1076, Sept. 2001. doi: 10.1109/43.945302.
- [9] A. M. Caulfield et al., "A cloud-scale acceleration architecture," in *Proc. 49th Int. Symp. Microarchitecture*, Taipei, Taiwan, Oct. 2016, pp. 1–13.
- [10] S. Chatterjee, M. Kishinevsky, and U. Y. Ogras, "xMAS: Quick formal modeling of communication fabrics to enable verification," *IEEE Design Test Comput.*, vol. 29, no. 3, pp. 80–88, June 2012. doi: 10.1109/MDT.2012.2205998.
- [11] J. Cheng, L. Josipović, G. A. Constantinides, P. Ienne, and J. Wicks, "Combining dynamic & static scheduling in high-level synthesis," in *Proc. 28th ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, Seaside, CA, Feb. 2020, pp. 288–298.
- [12] S. Cheng and J. Wawrzyniak, "Synthesis of statically analyzable accelerator networks from sequential programs," in *Proc. Int. Conf. Comput.-Aided Design*, Austin, TX, Nov. 2016, pp. 126–133.
- [13] D. Chiou, "Intel acquires Altera: How will the world of FPGAs be affected?" in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, Feb. 2016, Monterey, p. 148.
- [14] J. Choi, S. Brown, and J. Anderson, "From software threads to parallel hardware in high-level synthesis for FPGAs," in *Proc. IEEE Int. Conf. Field Programmable Technol.*, Kyoto, Dec. 2013, pp. 270–277.
- [15] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, "High-level synthesis for FPGAs: From prototyping to deployment," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 30, no. 4, pp. 473–491, Apr. 2011. doi: 10.1109/TCAD.2011.2110592.
- [16] J. Cong and Z. Zhang, "An efficient and versatile scheduling algorithm based on SDC formulation," in *Proc. 43rd Design Automation Conf.*, San Francisco, July 2006, pp. 433–438.
- [17] J. Cortadella, M. Galceran-Oms, and M. Kishinevsky, "Elastic systems," in *Proc. 10th ACM/IEEE Int. Conf. Formal Methods Models Code-sign*, July 2010, pp. 149–158.
- [18] J. Cortadella, M. Kishinevsky, and B. Grundmann, "Synthesis of synchronous elastic architectures," in *Proc. 43rd Design Automation Conf.*, San Francisco, July 2006, pp. 657–662.

- [19] J. Cortadella, M. G. Oms, M. Kishinevsky, and S. S. Sapatnekar, "RTL synthesis: From logic synthesis to automatic pipelining," *Proc. IEEE*, vol. 103, no. 11, pp. 2061–2075, Nov. 2015. doi: 10.1109/JPROC.2015.2456189.
- [20] S. Dai, M. Tan, K. Hao, and Z. Zhang, "Flushing-enabled loop pipelining for high-level synthesis," in *Proc. 51st Design Automation Conf.*, San Francisco, June 2014, pp. 1–6.
- [21] S. Dai et al., "Dynamic hazard resolution for pipelining irregular loops in high-level synthesis," in *Proc. 25th ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, Monterey, Feb. 2017, pp. 189–194.
- [22] R. Desikan, S. Sethumadhavan, D. Burger, and S. W. Keckler, "Scalable selective re-execution for EDGE architectures," in *Proc. 11th Int. Conf. Architectural Support Program. Languages Oper. Syst.*, Boston, Oct. 2004, pp. 120–132.
- [23] J. C. Dvorak, "How the Itanium killed the computer industry," Jan. 2009.
- [24] D. Edwards and A. Bardsley, "Balsa: An asynchronous hardware synthesis language," *Comput. J.*, vol. 45, no. 1, pp. 12–18, Jan. 2002. doi: 10.1093/comjnl/45.1.12.
- [25] S. A. Edwards, R. Townsend, and M. A. Kim, "Compositional dataflow circuits," in *Proc. 15th ACM-IEEE Int. Conf. Formal Methods Models Syst. Design*, Vienna, Sept. 2017, pp. 175–184.
- [26] M. Fingeroff, *High-Level Synthesis Blue Book*, 1st ed. Xlibris Corp., 2010.
- [27] J. A. Fisher, "Trace scheduling: A technique for global microcode compaction," *IEEE Trans. Comput.*, vol. 30, no. 7, pp. 478–490, July 1981. doi: 10.1109/TC.1981.1675827.
- [28] J. A. Fisher, "Very long instruction word architectures and the ELI-512," in *Proc. 10th Annu. Int. Symp. Comput. Architecture*, Stockholm, June 1983, pp. 140–150. doi: 10.1145/800046.801649.
- [29] S. B. Furber and P. Day, "Four-phase micropipeline latch control circuits," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 4, no. 2, pp. 247–253, June 1996. doi: 10.1109/92.502196.
- [30] N. George et al., "Automatic support for multi-module parallelism from computational patterns," in *Proc. 24th Int. Conf. Field-Programmable Logic Appl.*, London, Sept. 2015, pp. 1–8.
- [31] N. George et al., "Hardware system synthesis from domain-specific languages," in *Proc. 23rd Int. Conf. Field-Programmable Logic Appl.*, Munich, Sept. 2014, pp. 1–8.
- [32] J. Hansen and M. Singh, "Multi-token resource sharing for pipelined asynchronous systems," in *Proc. Design, Automation Test Europe Conf. Exhib.*, Dresden, Mar. 2012, pp. 1191–1196.
- [33] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 5th ed. San Mateo, CA: Morgan Kaufmann, 2011.
- [34] G. Hoover and F. Brewer, "Synthesizing synchronous elastic flow networks," in *Proc. Design, Automation Test Europe Conf. Exhib.*, Munich, Mar. 2008, pp. 306–311.
- [35] Y. Huang, P. lenne, O. Temam, Y. Chen, and C. Wu, "Elastic CGRAs," in *Proc. 21st ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, Monterey, Feb. 2013, pp. 171–180.
- [36] L. Josipović, A. Bhattacharyya, A. Guerrieri, and P. lenne, "Shrink it or shed it! minimize the use of LSQs in dataflow designs," in *Proc. IEEE Int. Conf. Field Programmable Technol.*, Tianjin, Dec. 2019, pp. 197–205.
- [37] L. Josipović, P. Brisk, and P. lenne, "From C to elastic circuits," in *Proc. 51st Annu. Asilomar Conf. Signals, Syst., Comput.*, Pacific Grove, Nov. 2017, pp. 121–125.
- [38] L. Josipović, P. Brisk, and P. lenne, "An out-of-order load-store queue for spatial computing," *ACM Trans. Embedded Comput. Syst. (TECS)*, vol. 16, no. 5s, pp. 125:1–125:19, Sept. 2017. doi: 10.1145/3126525.
- [39] L. Josipovic, N. George, and P. lenne, "Enriching C-based high-level synthesis with parallel pattern templates," in *Proc. 26th IEEE Int. Conf. Field Programmable Technol.*, Xian, Dec. 2016, pp. 177–180.
- [40] L. Josipović, R. Ghosal, and P. lenne, "Dynamically scheduled high-level synthesis," in *Proc. 26th ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, Monterey, Feb. 2018, pp. 127–136.
- [41] L. Josipović, A. Guerrieri, and P. lenne, "Speculative dataflow circuits," in *Proc. 27th ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, Seaside, Feb. 2019, pp. 162–171.
- [42] L. Josipović, A. Guerrieri, and P. lenne, "Dynamatic: From C/C++ to dynamically scheduled circuits," in *Proc. 28th ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, Seaside, Feb. 2020, pp. 1–10.
- [43] L. Josipović, S. Sheikhha, A. Guerrieri, P. lenne, and J. Cortadella, "Buffer placement and sizing for high-performance dataflow circuits," in *Proc. 28th ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, Seaside, Feb. 2020, pp. 186–196.
- [44] T. Kam, M. Kishinevsky, J. Cortadella, and M. Galceran-Oms, "Correct-by-construction microarchitectural pipelining," in *Proc. 27th Int. Conf. Comput.-Aided Design*, Nov. 2008, pp. 434–441.
- [45] R. Kastner, J. Matai, and S. Neuendorffer, "Parallel programming for FPGAs," May 2018, arXiv:1805.03648.
- [46] M. S. Lam, "Software pipelining: An effective scheduling technique for VLIW machines," in *Proc. 1988 ACM Conf. Program. Language Design Implementation*, Atlanta, GA, June 1988, pp. 318–328.
- [47] J. Liu, S. Bayliss, and G. A. Constantinides, "Offline synthesis of online dependence testing: Parametric loop pipelining for HLS," in *Proc. 23rd IEEE Symp. Field-Programmable Custom Comput. Mach.*, Vancouver, May 2015, pp. 159–162.
- [48] The LLVM Compiler Infrastructure, 2018. <http://www.llvm.org>
- [49] "ModelSim." Mentor Graphics, 2016.
- [50] T. Murata, "Petri nets: Properties, analysis and applications," *Proc. IEEE*, vol. 77, no. 4, pp. 541–580, Apr. 1989. doi: 10.1109/5.24143.
- [51] R. Nane et al., "A survey and evaluation of FPGA high-level synthesis tools," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 10, pp. 1591–1604, Oct. 2016. doi: 10.1109/TCAD.2015.2513673.
- [52] S. F. Nielsen, J. Sparsø, J. B. Jensen, and J. S. R. Nielsen, "A behavioral synthesis frontend to the Haste/TiDE design flow," in *Proc. 15th Int. Symp. Asynchronous Circuits Syst.*, Chapel Hill, N.C., May 2009, pp. 185–194.
- [53] E. Nurvitadhi, J. C. Hoe, T. Kam, and S.-L. L. Lu, "Automatic pipelining from transactional datapath specifications," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 30, no. 3, pp. 441–454, Mar. 2011. doi: 10.1109/TCAD.2010.2088950.
- [54] I. Park, C. L. Ooi, and T. Vijaykumar, "Reducing design complexity of the load/store queue," in *Proc. 36th Int. Symp. Microarchitecture*, San Diego, Dec. 2003, pp. 411–422.
- [55] M. Pericàs et al., "A two-level load/store queue based on execution locality," in *Proc. 35th Int. Symp. Comput. Architecture*, Beijing, June 2008, pp. 25–36. doi: 10.1145/1394608.1382171.
- [56] A. Putnam et al., "A reconfigurable fabric for accelerating large-scale datacenter services," in *Proc. 41st Int. Symp. Comput. Architecture*, Minneapolis, June 2014, pp. 13–24.
- [57] A. R. Putnam, D. Bennett, E. Dellinger, J. Mason, and P. Sundararajan, "CHiMPS: A high-level compilation flow for hybrid CPU-FPGA architectures," in *Proc. 16th ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, Monterey, Feb. 2017, pp. 173–178.
- [58] B. R. Rau, "Iterative modulo scheduling," *Int. J. Parallel Program.*, vol. 24, no. 1, pp. 3–64, Feb. 1996. doi: 10.1007/BF03356742.
- [59] S. Sethumadhavan, F. Roesner, J. S. Emer, D. Burger, and S. W. Keckler, "Late-binding: Enabling unordered load-store queues," in *Proc. 34th Int. Symp. Comput. Architecture*, San Diego, June 2007, pp. 347–357.
- [60] J. Sparsø, "Current trends in high-level synthesis of asynchronous circuits," in *Proc. 16th IEEE Int. Conf. Electron., Circuits, Syst.*, Yasmine Hammamet, Dec. 2009, pp. 347–350.
- [61] I. E. Sutherland, "Micropipelines," *Commun. ACM*, vol. 32, no. 6, pp. 720–738, June 1989. doi: 10.1145/63526.63532.
- [62] M. Tan, B. Liu, S. Dai, and Z. Zhang, "Multithreaded pipeline synthesis for data-parallel kernels," in *Proc. Int. Conf. Comput.-Aided Design*, San Jose, Nov. 2014, pp. 718–725.
- [63] M. Tan, G. Liu, R. Zhao, S. Dai, and Z. Zhang, "ElasticFlow: A complexity-effective approach for pipelining irregular loop nests," in *Proc. 34th Int. Conf. Comput.-Aided Design*, Austin, TX, Nov. 2015, pp. 78–85.
- [64] L. Torczon and K. Cooper, *Engineering a Compiler*, 2nd ed. San Mateo, CA: Morgan Kaufmann, 2011.
- [65] R. Townsend, M. A. Kim, and S. A. Edwards, "From functional programs to pipelined dataflow circuits," in *Proc. 26th Int. Conf. Compiler Construction*, Austin, TX, Feb. 2017, pp. 76–86.
- [66] D. M. Tullsen, S. J. Eggers, J. S. Emer, H. M. Levy, J. L. Lo, and R. L. Stamm, "Exploiting choice: Instruction fetch and issue on an implementable simultaneous multithreading processor," in *Proc. 23rd Annu. Int. Symp. Comput. Architecture*, Philadelphia, May 1996, pp. 191–202.
- [67] D. M. Tullsen, S. J. Eggers, and H. M. Levy, "Simultaneous multithreading: Maximizing on-chip parallelism," in *Proc. 22nd Annu. Int. Symp. Comput. Architecture*, Santa Margherita Ligure, May 1995, pp. 392–403.
- [68] M. Vijayaraghavan and Arvind, "Bounded dataflow networks and latency-insensitive circuits," in *Proc. 9th Int. Conf. Formal Methods Models Codesign*, Cambridge, MA, July 2009, pp. 171–180.
- [69] "Vivado High-Level Synthesis," Xilinx Inc, 2018.
- [70] Z. Zhang and B. Liu, "SDC-based modulo scheduling for pipeline synthesis," in *Proc. 32nd Int. Conf. Comput.-Aided Design*, San Jose, CA, Nov. 2013, pp. 211–218.



Bright Minds. Bright Ideas.



Introducing IEEE Collabratec™

The premier networking and collaboration site for technology professionals around the world.

IEEE Collabratec is a new, integrated online community where IEEE members, researchers, authors, and technology professionals with similar fields of interest can **network** and **collaborate**, as well as **create** and manage content.

Featuring a suite of powerful online networking and collaboration tools, IEEE Collabratec allows you to connect according to geographic location, technical interests, or career pursuits.

You can also create and share a professional identity that showcases key accomplishments and participate in groups focused around mutual interests, actively learning from and contributing to knowledgeable communities. All in one place!

Network.
Collaborate.
Create.

Learn about IEEE Collabratec at
ieee-collabratec.ieee.org



By Cristina Meinhardt, José Luis Güntzel, and Mateus Grellert,
Departamento de Informática e Estatística, PPGCC, Universidade Federal de Santa Catarina-UFSC, Brazil and Ricardo Reis, *Instituto de Informática, PGMicro/PPGC, Universidade Federal do Rio Grande do Sul-UFRGS, Brazil*

EDAS2020: IEEE CASS/CEDA Seasonal School on Electronic Design Automation

Abstract

The first IEEE CASS/CEDA Seasonal School on Electronic Design Automation (EDAS2020) has offered twenty-two talks on key topics on electronic design automation for integrated circuits in modern and upcoming technologies, including current and future challenges faced by the industry and academia for implementing complex circuits and systems. The EDAS2020 was an event dedicated to students and professionals in Brazil and South America to promote discussions on hot topics and cover fundamental algorithms, computational methods, and machine learning techniques in the area, so that the attendees could leverage their comprehension and capabilities while attracting new students and researchers to the relevant problems. This first edition counted 215 participants, 11 posters, 22 engaged speakers, an interesting Hands-on Tutorial using Open Source EDA tools, and three panels. The event was held on a virtual platform, between December 7-11, 2020.

I. Introduction

The main goals of this CASS/CEDA seasonal school were:

To bring together the local community, fostering local interactions and stimulating cooperation among students, researchers, and professionals;

To bring the local community closer to leading researchers and professionals in their field of EDA;

To stimulate students to learn, research, and eventually follow a career in the field of EDA;

The success of this event can be assessed by considering the following factors:

- 1) Number of participants: The event had 215 participants.
- 2) Quality of the presentations: According to the feedback received from the participants, the presentations and the tutorial were considered very motivational, with excellent technical content and also bringing an overview on trend topics.

3) Networking between participants in a virtual environment, a big challenge. The panels and poster sessions proved to be a good alternative to effectively integrate the participants and stimulate the networking.

4) Number of new IEEE CASS memberships after the workshop: it was observed several new IEEE CASS memberships motivated by EDAS. To fulfill these objectives, an IEEE CASS and CEDA talk was included in the program, with institutional advertisement from IEEE and IEEE CASS broadcasted in the intervals of the courses, providing information to the general public on how to subscribe to IEEE and to CAS society, and explaining the membership benefits.

EDAS2020 was sponsored by IEEE Circuits and Systems Society (CASS), in the frame of the CASS Seasonal Schools Program. Therefore, a book with the slides of the courses is currently under preparation. EDAS was also sponsored by the IEEE Council on Electronic Design Automation (CEDA) and by Cadence, Synopsys and Chipus. Its organization was supported by the Brazilian Microelectronics Society (SBMicro), Brazilian Computer Society (SBC), Universidade Federal de Santa Catarina (UFSC), Universidade Federal do Rio Grande do Sul (UFRGS) and the Tutorial Program in Computer Science from UFSC (PET Computação). The staff was composed of fifteen volunteers (students) who helped in the social media advertisement, in the Zoom platform management, in session recording, in the interaction with the participants and in various other activities.

II. EDAS 2020 Program

EDAS 2020 program comprised 21 courses that covered the VLSI design flow in 30 hours, from the basic steps to the optimization techniques, as well as challenges for the advanced and future technology nodes.

All the speakers agreed to voluntarily prepare and give the live sessions of the courses (with an average

duration of 1 h 30 min each), dedicating their precious time to prepare and conduct the courses. The program started with an introductory course to provide an overview of the EDA area given by Ricardo Reis and José Luis Güntzel. After that, an **Introduction to the Modern Design Flow** was conducted by Mateus Fogaça (Cadence). The second part of the first day was dedicated to **Logic Synthesis** topics, with the basic concepts presented by Augusto Neutzling Silva (Cadence) and Jody Matos (Silvaco), and two courses about the **Challenges on Logic Synthesis** given by Vinícius Callegaro (Mentor) and Vinícius Possani (Synopsys). The second day main topic was the Placement step, with the **Global Placement** course given by Paulo Butzen (UFRGS), the **Placement Legalization** course by Jucemar Monteiro (Synopsys), the **Detailed Placement** course by Sheiny Fabre Almeida and Cristina Meinhardt (UFSC), and finalizing with the considerations about **Physically-aware synthesis** conducted by Guilherme Flach (Synopsys). The third day started with a course on **Automatic Cell Generation** by Ricardo Reis (UFRGS), followed by 3 courses dedicated to routing. The first one, given by Renato Hentschke (Synopsys), was dedicated to **Global Routing**, the second one, by Gracieli Posser (Cadence), was devoted to **Detailed Routing**, and the last one, given by Tiago Fontana (UFSC) and José Luis Güntzel (UFSC), on **Routing with Cell Movement**. In the fourth day, the following talks took place: **Timing Optimization**, by Vinícius Livramento (Flow Traders), **Gate Sizing**, by Marcelo Johann (UFRGS), **Visualization Tools**, by Ricardo Reis (UFRGS) and Clock Synthesis, by Gustavo Wilke (Synopsys). In the last day, December 11, the following talks were presented: **Synthesis of Quantum Circuits**, by Calebe Conceição (IFSul), **Synthesis of NML Circuits** by Omar Paranaíba Vilela Neto (UFMG) and José Augusto Nacif (UFV), **Synthesis**

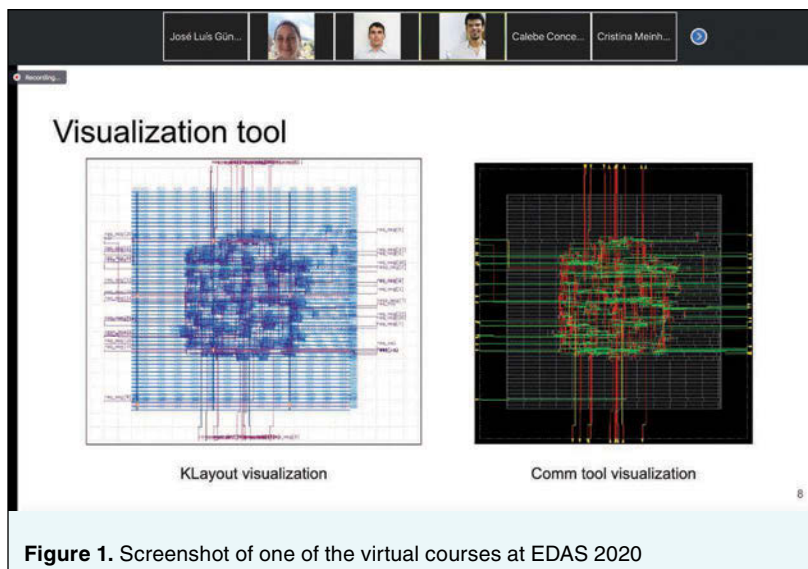


Figure 1. Screenshot of one of the virtual courses at EDAS 2020

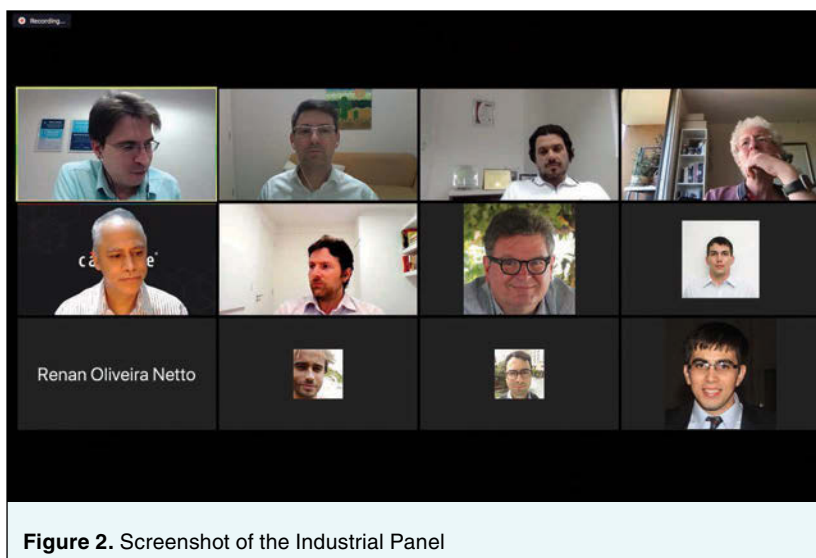


Figure 2. Screenshot of the Industrial Panel



Figure 3. Main flyer of EDAS 2020

of **3D Circuits**, by Carolina Metzler and Ricardo Reis (UFRGS) and the last one on **Machine Learning application on EDA**, by Renan Oliveira Netto (UFSC).

The program also included a 4-hour hands-on tutorial using the tools from the OpenRoad Project, given by Eder Monteiro, 3 poster sessions and 3 panels, and a talk about **the benefits of IEEE/CASS-CEDA membership** by José Rodrigo Azambuja (UFRGS/CASS) and José Luís Güntzel (UFSC/CEDA). The panels focused on topics that could motivate the participants, mostly undergraduate and graduate students, to target their future careers to the EDA area. Therefore, the first panel, chaired by Cristina Meinhardt (UFSC) and Leomar S. Rosa Jr. (UFPEl), counted with the participation of young researchers who reported their trajectories in EDA companies abroad. In the second panel, chaired by Gracieli Posser (Cadence) and Cristina Meinhardt (UFSC), the various Brazilian students who have participated in EDA contests reported how those experiences have leveraged their research and increased their skills in the area. Finally, in the third panel, chaired by Daniel Barcelos (CEITEC), professionals from IC Design and EDA companies talked about challenges and opportunities in next generation EDA tools, including career perspectives. The participants of this panel were Marcelo Silva (Cadence), Victor Grimblatt (Synopsys), Murilo Pessatti (Chipus), and Bernardo Culau (Silvaco).

With this program, we also achieved a surprising result: reconnecting us with former students formed in Brazilian institutions who are currently working in EDA companies around the world. EDAS2020 full program and more details are available at the EDAS website, including all posters and videos, as well the best posters: ecl.ufsc.br/edas and on our Facebook page: www.facebook.com/edas.ufsc/.

III. Conclusions

The first IEEE CASS/CEDA Seasonal School on Electronic Design Automation was successful, having 215 participants from several locations. Participants asked for a second edition of EDAS. In August 2022, a face-to-face edition of the IEEE/CEDA International Seasonal School on Physical Design Automation (PDA2021) is likely to occur in Florianópolis, Brazil. It will be the 4th edition of a series that began in 2017 at Porto Alegre (Brazil), followed to Chiayi (Taiwan) in 2018 and to Beijing (China) in 2019.

Acknowledgment

To Universidade Federal de Santa Catarina, for providing the infrastructure, to IEEE CASS, for the financial support, and to IEEE CEDA, for providing the Zoom platform, as well to the industrial sponsors Cadence, Synopsys and Chipus.

Women in Circuits and Systems (WiCAS) and Young Professionals (YP) at ICECS 2020

A “Women in Circuits and Systems” (WiCAS) event and a “Young Professionals” (YP) event took place during the 27th IEEE International Conference on Electronics Circuits and Systems (ICECS), the flagship conference for the Region 8 of the IEEE Circuits and Systems Society (CASS).

The WiCAS events traditionally aim to inspire and motivate both students and young professionals in the domain of circuits and systems to have efficient roles in their professions, by meeting successful female engineers and professors, through interesting technical and professional talks in fields of interest of CASS. The YP events usually include start-ups presentation, poster and demo sessions, aiming to provide a thrilling environment for early career researchers to present their work. Joining this event, young professionals have the opportunity to learn about state of the art and most

advanced activities in the area of circuits and systems, meet and interact with their peers, receive feedback from internationally well-known experts in the CASS domain from both academia and industry.

The ICECS 2020 WiCAS event was held on November 24th, 2020, organized by Dr. Erika Covi (NaMLab, Germany), Dr. Giulia Di Capua (University of Salerno, Italy), and Dr. Melika Payvand (ETHZ, Switzerland).

The WiCAS event included a panel discussion, driven by the theme “Inspiring stories of women’s success in STEM”. The event involved the participation of four recognized speakers, both from academia and industry: Prof. Teresa Serrano-Gotarredona (National Microelectronics Institute, Seville, Spain), Dr. Angeliki Pantazi (Research Manager at IBM, Zurich, Switzerland), Dr. Chiara Bartolozzi (Italian Institute of Technology, Genoa, Italy), and Dr. Elham Ebrahimi (Comms Technical Lead at Spire, Edinburgh, United Kingdom). As women in leadership positions with proven excellent record of accomplishment in fields of interest of the

IEEE CASS, the WiCAS panelists shared precious experiences and advices with the conference attendees. The speakers not only shared their personal journey and the path to their success, but also their research interest. Their research and experience spanned from analog/mixed-signal design and neuromorphic engi-

neering, all the way to emerging technologies and even space applications. The panel discussion was a lively session on the impact of gender gap and cognitive biases in women's career.

The ICECS 2020 YP event was held on November 25th, 2020, organized by Dr. Nazila Fough (Robert Gordon

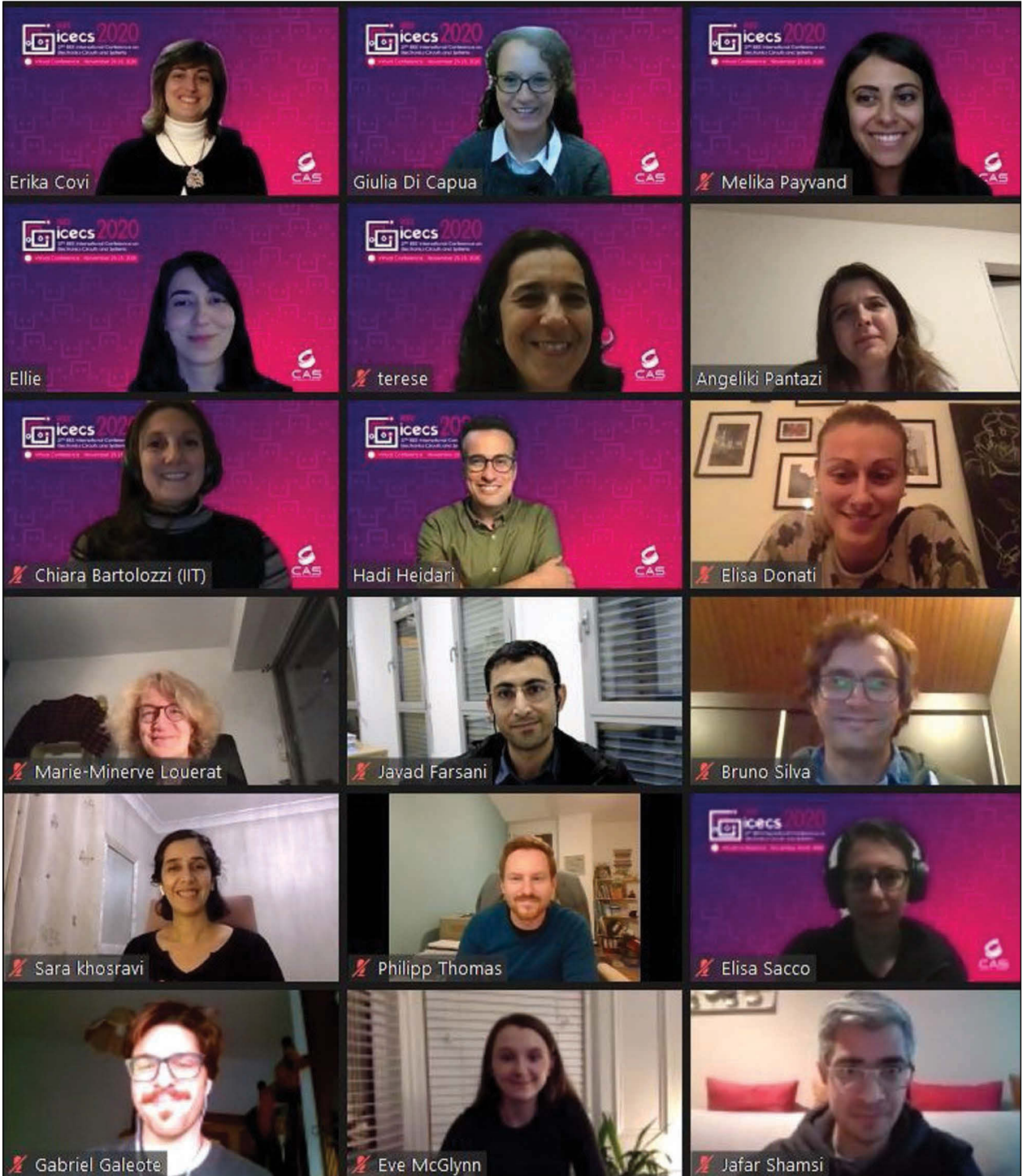


Figure 1. Some attendees of the WiCAS session.

WiCAS events aim to inspire and motivate both students and young professionals in the domain of circuits and systems to have efficient roles in their professions.

University, United Kingdom) and Dr. Giulia Di Capua (University of Salerno, Italy).

The YP event included welcoming talks about IEEE, IEEE YP and IEEE CAS, generously provided by Dr. Mona Ghassemian (IEEE UK & Ireland section chair), Dr. Vinko Lešić (IEEE YP Region 8 chair), Prof. Izzet Kale (IEEE UK & Ireland vice-chair and CAS Chapter-chair) and Prof. Tony Davies (IEEE R8 History Activities coordinator). A discussion session on how to get research works successfully published was also organized, thanks to the amazing contributions of Dr. Hakim Meskine (Advance Electronics Materials Editor in Chief, Wiley) and Dr. Charles Glaser (Springer Nature Editorial Director). After these talks, a YP demo session and a YP poster session took place. The

YP track was open to students, new lecturers and early career researchers, who had the opportunity to present their research work, stay technically up-to-date and learn about future challenges in CASS domain.

During the conference, awards were assigned for the best contributions presented in the WiCAS and YP competitions. All WiCAS and YP winners were announced during the ICECS 2020 awarding ceremony.

Twenty-seven papers were accepted for the WiCAS competition, and three awards were assigned. A “1st Runner-Up Paper Award” was assigned to Dr. Saman Fatima, a Doctoral Researcher at the University of Paris Saclay (France), for the paper titled “A CMOS Readout Circuit for a Low Shunt Resistance IR photo-Detector.”

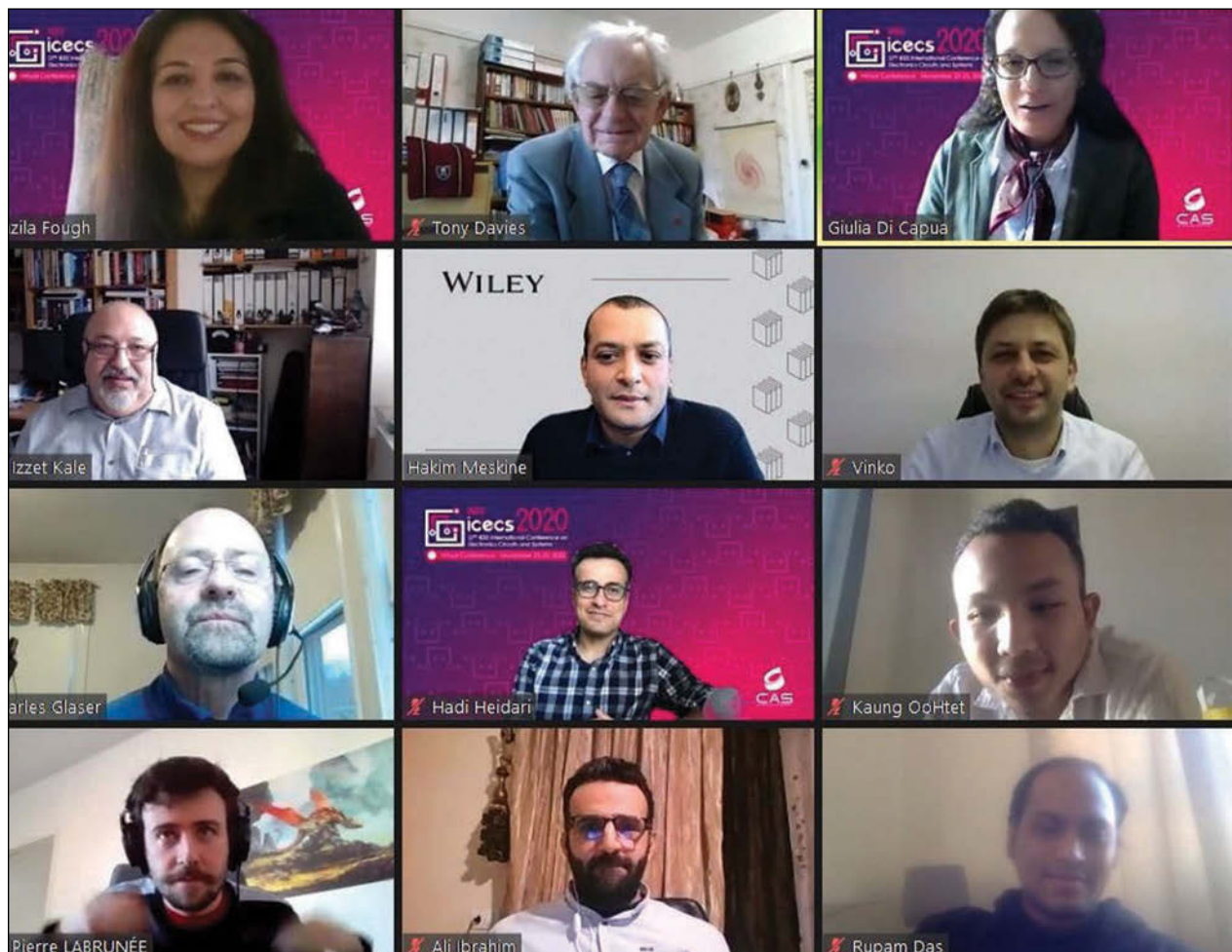


Figure 2. Some attendees of the YP session.

YP events include start-ups presentation, poster and demo sessions, aiming to provide a thrilling environment for early career researchers to present their work.

A “2nd Runner-Up Paper Award” was assigned to Ms. Eve McGlynn, a PhD Student at the University of Glasgow (United Kingdom), for the paper titled “Encapsulated Magnetolectric Composites for Wirelessly Powered Brain Implantable Devices.” A “Best Paper Award” was eventually assigned to Dr. Fanny Spagnolo, a Postdoctoral Researcher at the University of Calabria (Italy), for the paper titled “A High-Performance and Power-Efficient SIMD Convolution Engine for FPGAs.”

Twelve papers and four demo presentations were accepted for the YP competition, and three YP awards were eventually assigned. A “Best Demo Award” was assigned to Dr. Jason Eshraghian, a Post-Doctoral Fellow at the University of Michigan (U.S.A.), for the live demonstration titled “Prosthesis Control Using a Real-Time Retina Cell Network Simulator.” A “Best Presentation Award” was assigned to Ms. Elena-Diana Șandru, a PhD student at the University Polytechnic of Bucharest (Romania), for the poster titled “Machine Learning-Based Local Sensitivity Analysis of Integrated Circuits to Process Variations.” Finally, a “Best Paper Award” was assigned to Mr. Kosuke Uchiyama, a PhD student at the Nagoya University (Japan),

for the poster titled “Design of Fully-Integrated Self-Powered FM Transmitter Using On-Chip Photodiodes in 65-nm CMOS.”

The WiCAS and YP winners received a one-year IEEE plus IEEE CASS membership, thanks to the generous support provided by the IEEE CASS. Moreover, as a sign of appreciation to all 2020 WiCAS and YP participants, the WiCAS co-chairs and the YP co-Chairs were pleased to grant a 25 USD voucher for CAS-related technical books to all speakers who gave live presentations during the conference days.

The WiCAS and YP co-chairs express their gratitude to the ICECS 2020 general co-chairs, Prof. Hadi Heidari and Prof. Elena Blockina, and to the conference organizing committee, for the technical support received in the definition of the WiCAS and YP sessions. The WiCAS and YP co-chairs would also like to thank the IEEE Circuits and Systems Society for the financial support that helped making these events a great moment of professional and personal enrichment.

Many thanks to Giulia Di Capua, Melika Payvand, Erika Covi and Nazila Fough who contributed to this report.

Ricardo Reis, *Senior Member, IEEE*

2021 IEEE CASS Rio Grande do Sul Webinars

Last year, the IEEE CASS Rio Grande do Sul Chapter and the Graduate Program on Microelectronics (PGMICRO) of the Universidade Federal do Rio Grande do Sul (UFRGS) transformed Local Face-to-Face Seminars into Global Webinars. A report about this action was published in the IEEE CASS Magazine, 4th quarter, 2020 [1]. All 2020 webinars were recorded and they are available at the IEEE CASS Rio Grande do Sul YouTube channel (www.youtube.com/cassriograndedosul)

In February 5th, it was started the season 2021 of the IEEE CASS Rio Grande do Sul webinars, with a talk by prof. **Subhasish Mitra** (Stanford University, USA), with the title **21st-Century NanoSystems for Abundant-Data Computing: The N3XT 1,000X** [Fig. 1].

Then, the following invited talks were organized in 2021 or are scheduled:

Prof. Hai “Helen” Li (Duke University, USA), Efficient Deep Learning at Scale, February 12

Prof. Ricardo Reis (UFRGS, Brazil), Challenges in the Design of Integrated Systems for IoT, Feb. 19

Prof. Rajesh K. Gupta (UC San Diego, USA), Programming Human Spaces, February 26

IEEE CASS RS Talks 2021
UFRGS, Porto Alegre, Brazil

1:30 PM (Brasilia Time, GMT-3), February 5, 2021

Prof. Subhasish Mitra, Dept. of EE and Dept. of CS, Stanford University, USA
21st-Century NanoSystems for Abundant-Data Computing: The N3XT 1,000X

Abstract:
The world's needs for analyzing massive amounts of data is growing dramatically. The computation demands of these abundant-data applications, such as AI machine learning, far exceed the capabilities of today's computing systems. For example, a Dream AI Chip would co-locate all memory on a single chip together with computing elements, quickly accessible at low energy. Such Dream Chips aren't realizable today. Computing systems instead use large off-chip memory and spend enormous time and energy shuttling data back-and-forth. This memory wall gets worse with growing problem sizes, especially as conventional transistor miniaturization gets increasingly difficult. This talk will present the N3XT (Nano-Engineered Computing Systems Technology) approach to build transformative NanoSystems by exploiting unique properties of underlying nanotechnologies. N3XT creates new chip architectures for computation immersed in memory through ultra-dense (monolithic) 3D integration of new logic devices (such as carbon nanotube transistors for high-speed and low-energy circuits) and new non-volatile memory (such as dense resistive RAM that can store multiple bits inside each memory cell). To scale with growing problem sizes, new Illusion systems orchestrate multiple N3XT chips to create an illusion of a Dream Chip with near-Dream energy and throughput.
A wide variety of N3XT hardware prototypes, built in commercial US foundry as part of DARPA's 3DSoc program as well as research facilities, represent leading examples of transforming scientifically-interesting nanomaterials and nanodevices into actual 21st-century NanoSystems. N3XT NanoSystems target 1,000X system-level energy-delay-product benefits especially for abundant-data applications. Such massive benefits enable coming generations of applications that push new frontiers, from deeply-embedded computing systems all the way to the cloud.

Short CV:
Subhasish Mitra is Professor of EE and of CS at Stanford University. He directs the Stanford Robust Systems Group, leads the Computation Focus Area of the Stanford SystemX Alliance, and is a member of the Wu Tsai Neurosciences Institute. Prof. Mitra also holds the Carnot Chair of Excellence in NanoSystems at CEA LETI in France. His research ranges across Robust Computing, NanoSystems, Electronic Design Automation (EDA), and Neurosciences. Results from his research group have influenced almost every contemporary electronic system, and have inspired significant government and research initiatives in multiple countries. Prof. Mitra also has consulted for major technology companies including Cisco, Google, Intel, Samsung, and Xilinx. Prof. Mitra's honors include the Newton Technical Impact Award in EDA (test of time honor by ACM SIGDA and IEEE CEDA), the Semiconductor Research Corporation's Technical Excellence Award, the Intel Achievement Award (Intel's highest honor), and the US Presidential Early Career Award. He has published award-winning papers at major venues. He is an ACM Fellow and an IEEE Fellow.

YouTube Live @ IEEE CASS Rio Grande do Sul Chapter
<https://www.youtube.com/cassriograndedosul>

UFRGS
PGMICRO

CAS
SOCIETY OF ELECTRONIC DESIGNERS










Figure 1. Flyer of the first 2021 Webinar.

Dr. Rodrigo Calderón Rico (Apple, Cupertino, USA), Missing Hardware Functions to Apply Intelligence in Communication Systems, March 5

Prof. Yuanqing Cheng (Beihang University, China), Reliable and Low Power Design for STT-MRAM, March 12

Dr. Gustavo Wilke (Synopsys, USA), Analysis and Optimization of Mesh-based Clock Distribution Architectures, March 19

Dr. Luciano Ost (Loughborough University, England), Early Soft Error Assessment of Embedded Deep Inference Networks, March 26.

Dr. Rob Aitken (Arm Research, San Jose, USA), The Challenges of Software-Defined Hardware, April 9

Prof. Ricardo Jacobi (Universidade de Brasília, Brazil), Algorithms and Architectures for Morphological Image Processing, April 16

Prof. Eby G. Friedman (University of Rochester, USA), Single Flux Quantum Circuits, Algorithms, and Design Methodologies, April 23

Prof. Sachin S. Sapatnekar (University of Minnesota, USA), Automating Analog Layout: Why This Time is Different, May 7

Prof. Bruno Zatt (UFPEL, Brazil), Immersive Media Technology: Challenges, Solutions and Trends, May 14

Krishnakumar Nair (Facebook, USA), Accelerators for Deep Learning, May 21

Prof. Michael Hubner (Brandenburg University of Technology, Germany), Going New Ways in Embedded System Design, June 4

Prof. Ibrahim Elfadel (Khalifa University, UAE), Machine Learning Methods in VLSI CAD, June 11

Dr. Lou Scheffer (Howard Hughes Medical Institute, USA), The Electrical Design of Biological Systems—What can we Learn from the Brain of a Fly?, June 18

Please, go to the CASS Rio Grande do Sul YouTube channel to see the titles and abstracts of next talks, and to subscribe to the YouTube channel. It is also possible to find, in the YouTube channel, the recorded versions of past 2021 Talks, as well other talks related to other events organized by the CASS Chapter. CASS Talks keep reaching a broad set of countries, all around the world. Please, don't miss next talks.

Reference

[1] R. Reis, R. Brum, and J. R. Azambuja, "How CASS-RS transformed local face-to-face seminars into global webinars [CAS Society News]," *IEEE Circuits Syst. Mag.*, vol. 20, no. 4, pp. 78–87, 2020, doi: 10.1109/MCAS.2020.3027224.

IEEE Foundation



We have 30 million reasons to be proud.

Thanks to our donors, supporters and volunteers who answered the call of the ***Realize the Full Potential of IEEE Campaign***, helping impact lives around the world through the power of technology and education.



Illuminate



Educate



Engage



Energize

Realize Your Impact

Learn how: ieeefoundation.org/campaign

Correction

The March 2021 issue's "CAS Society News" column regarding CAS Society awards for 2020 [1] omitted the following information:

- The Region 9 Chapter of the Year Award was given to CASS Rio Grande do Sul Chapter.

- The Region 10 Chapter of the Year Award was given to CASS Taipei Chapter.

Reference

[1] "CAS society awards 2020," *IEEE Circuits Syst. Mag.*, vol. 21, no. 1, pp. 78–79, 2021.

Digital Object Identifier 10.1109/MCAS.2021.3071636

Date of current version: 24 May 2021

TAP. CONNECT. NETWORK. SHARE.



Connect to IEEE—no matter where you are—with the IEEE App.



Stay up-to-date
with the latest news



Schedule, manage, or
join meetups virtually



Get geo and interest-based
recommendations



Read and download
your IEEE magazines



Create a personalized
experience



Locate IEEE members by location,
interests, and affiliations

Download Today!



Harness the publishing power of IEEE Access[®]

IEEE Access is a multidisciplinary open access journal offering high-quality peer review, with an expedited, binary review process of 4 to 6 weeks. As a journal published by IEEE, IEEE Access offers a trusted solution for authors like you to gain maximum exposure for your important research.



Explore the many benefits of IEEE Access:

- **Receive high-quality, rigorous peer review** in only 4 to 6 weeks
- **Reach millions of global users** through the IEEE Xplore[®] digital library by publishing open access
- **Submit multidisciplinary articles** that may not fit in narrowly focused journals
- **Obtain detailed feedback** on your research from highly experienced editors
- **Establish yourself as an industry pioneer** by contributing to trending, interdisciplinary topics in one of the many topical sections IEEE Access hosts
- **Present your research to the world quickly** since technological advancement is ever-changing
- **Take advantage of features** such as multimedia integration, usage and citation tracking, and more
- **Publish without a page limit** for \$1,750 per article



What + If = IEEE

420,000+ members in 160 countries.
Embrace the largest, global, technical community.

People Driving Technological Innovation.

ieeep.org/membership

#IEEEmember



KNOWLEDGE

COMMUNITY

PROFESSIONAL DEVELOPMENT

CAREER ADVANCEMENT